微机原理及接口技术

WEIJI YUANLI JI JIEKOU JISHU

盛殉华　张凡　编著



中国铁道出版社

微机原理及接口技术

张　凡　盛珣华　编著

中　国　铁　道　出　版　社

(京)新登字 063 号

内 容 简 介

本书以 PC486 为主要背景，对 32 位微型计算机的系统结构、指令系统、汇编语言、存储器、中断与中断控制、I/O 接口进行了系统论述。全书内容由浅入深，循序渐进，读者学习后能够掌握 32 位微机和接口技术的基本内容。

图书在版编目(CIP)数据

微机原理及接口技术/盛珣华编. —北京：铁道出版社，2000.4
ISBN 7-113-03695-3

Ⅰ.微… Ⅱ.盛… Ⅲ.①微型计算机-理论②微型计算机-接口 Ⅳ.TP364.7

中国版本图书馆 CIP 数据核字(2000)第 03673 号

书　　名：微机原理及接口技术
作　　者：张　凡　盛珣华
出版发行：中国铁道出版社(100054，北京市宣武区右安门西街 8 号)
责任编辑：刘　波　郭　宇
封面设计：李艳阳
印　　刷：河北遵化胶印厂
开　　本：787×1092　1/16　印张：15.75　字数：391 千
版　　本：2000 年 4 月第 1 版　2000 年 4 月第 1 次印刷
印　　数：1～3000 册
书　　号：ISBN 7-113-03695-3/TP·435
定　　价：20.10 元

版权所有　盗印必究
凡购买铁道版的图书，如有缺页、倒页、脱页者，请与本社发行部调换。

前　言

本书是为贯彻国家教委“面向21世纪教学内容改革”的精神，适应微型计算机迅速发展的需要，结合多年讲授“微型计算机原理与接口技术”课程的实践，加以整理和编写的。

全书以世界上的主流微机（80X86系列微机）中具有代表性的PC486为主要背景，对32位微型计算机的系统结构，指令系统与汇编语言程序设计，存储器，中断与中断控制，I/O接口进行了系统的论述。全书内容由浅入深，循序渐进，书中例题丰富，各章附有思考题与习题。读者学习本书后能够掌握32位微型计算机与接口技术的基本内容，再加上适当学时的实验配合，可培养运用微型计算机解决科技和工程中的问题的能力。

本书参考学时48～80，适用于非计算机专业本科生作为学习计算机的基础教材，也可供工程技术人员作为参考和自学用。

全书第1～6章，第10章为微机原理部分，由盛珣华老师编写；7～9章为接口技术和应用部分，由张凡老师编写。本书承蒙北方交通大学通信与控制工程系安维蓉教授审核，提出许多宝贵意见，在此表示衷心的感谢。

由于编者水平有限，书中有不当之处，恳请读者批评指正。

作　者

1999年11月于北京

目　录

第一章　微型计算机基础 …… 1

第一节　微型计算机发展概况 …… 1
第二节　微型计算机的硬件结构 …… 1
第三节　微型计算机的运算基础 …… 2
一、进位计数制 …… 2
二、数的符号与小数点的表示 …… 3
三、数的编码方法 …… 5
第四节　IBM PC 系列微机系统简介 …… 6
一、机箱结构型式 …… 6
二、主板结构 …… 7
三、常用 I/O 适配器 …… 7
四、华硕 PXI-486SP3 主板简介 …… 8
第五节　微型计算机系统的主要性能指标 …… 8
第六节　当前微机中应用的先进计算机技术 …… 9
一、流水线技术 …… 9
二、高速缓冲存储器技术 …… 10
三、虚拟存储器技术 …… 10
四、RISC 技术 …… 10
思考题与习题 …… 11

第二章　32 位微处理器结构 …… 12

第一节　80486 CPU 的结构 …… 12
一、80486 CPU 的内部结构 …… 12
二、内部寄存器 …… 15
三、存 储 器 …… 19
四、堆栈(stack) …… 21
第二节　80486 CPU 芯片的引脚信号 …… 22
第三节　微机系统中的总线结构 …… 24
一、总线的操作 …… 25
二、时钟周期、总线周期和指令周期 …… 25
三、80486 的基本总线操作时序 …… 26
四、80486 系列微机系统的标准总线 …… 28
思考题与习题 …… 40

第三章　80486的指令系统和寻址方式 …… 42

第一节　80486的数据类型 …… 42
第二节　80486的寻址方式 …… 43
第三节　80486的指令系统 …… 47
一、80486的指令格式 …… 48
二、80486的指令系统 …… 48
思考题与习题 …… 65

第四章　汇编语言 …… 68

第一节　MASM汇编语句结构 …… 68
一、指令语句 …… 68
二、指示语句 …… 69
第二节　MASM汇编语言数据、表达式和运算符 …… 69
一、常　　数 …… 69
二、变　　量 …… 69
三、标　　号 …… 70
四、表 达 式 …… 71
五、运 算 符 …… 71
第三节　伪指令(指示语句) …… 73
一、变量定义语句 …… 73
二、符号定义语句 …… 74
三、段定义伪指令 …… 75
四、过程定义伪指令(PROC/ENDP) …… 81
五、其它伪指令 …… 82
思考题与习题 …… 86

第五章　汇编语言程序设计 …… 88

第一节　程序设计概述 …… 88
第二节　顺序结构程序 …… 90
第三节　分支程序 …… 93
第四节　循环程序 …… 96
一、循环的基本结构 …… 98
二、循环控制方式 …… 99
三、多重循环 …… 101
第五节　子 程 序 …… 102
第六节　算术运算、代码转换和串操作 …… 114
第七节　实用程序设计举例 …… 120
思考题与习题 …… 133

第六章　存储器 …… 135

第一节　概　　述 …… 135
第二节　存储器芯片及其选择 …… 135
第三节　内存储器的构成原理 …… 140
一、存储器结构的确定 …… 140
二、存储器片选控制方法 …… 141
三、存储器地址分配与设置 …… 142
四、读写控制线与数据总线的连接 …… 145
第四节　80486 的存储器结构与存储管理 …… 147
一、内存配置及结构 …… 147
二、80486 与 32 位、16 位、8 位内存的接口 …… 148
三、存储器工作方式及管理 …… 149
思考题与习题 …… 152

第七章　中断控制 …… 154

第一节　中断概述 …… 154
一、中断与中断功能 …… 154
二、中断控制 …… 154
三、中断处理过程 …… 155
第二节　IBM PC 机的中断系统 …… 156
一、中断类型 …… 156
二、中断矢量表 …… 157
三、中断响应时序 …… 158
第三节　中断控制器 82C59A …… 159
一、82C59A 的结构与引脚信号 …… 159
二、82C59A 的工作方式 …… 161
三、82C59A 的连接方式 …… 163
四、82C59A 的编程 …… 163
五、82C59A 的应用 …… 168
思考题与习题 …… 170

第八章　输入输出接口技术 …… 171

第一节　概　　述 …… 171
一、I/O 接口的构成及功能 …… 171
二、I/O 接口的编址方式 …… 172
三、I/O 数据传送的控制方式 …… 172
第二节　并行接口 8255A …… 173
一、8255A 的结构与引脚信号 …… 173
二、8255A 的工作方式 …… 175

三、8255A 的编程 …… 177
四、8255A 的应用 …… 178
第三节 定时/计数器 8253 …… 181
一、8253 的结构与引脚信号 …… 181
二、8253 的工作方式 …… 183
三、8253 的编程 …… 185
四、8253 的应用 …… 186
第四节 串行接口 8250 …… 187
一、串行通信 …… 187
二、串行接口 8250 …… 191
第五节 DMA 控制器 8237A …… 197
一、8237A 的结构与引脚信号 …… 197
二、8237A 的工作方式 …… 200
三、8237A 的工作时序 …… 201
四、8237A 的内部寄存器 …… 202
五、8237A 的编程 …… 206
第六节 多功能 I/O 接口 82380 …… 206
一、82380 的结构与功能 …… 207
二、82380 与微处理器的连接 …… 211
思考题与习题 …… 213

第九章 数/模和模/数转换接口技术 …… 214

第一节 模拟输入输出通道 …… 214
第二节 D/A 转换器 …… 215
一、D/A 转换原理 …… 215
二、D/A 转换器的特性 …… 217
三、典型的 D/A 转换器 …… 217
四、DAC 芯片的应用 …… 220
第三节 A/D 转换器 …… 223
一、A/D 转换原理 …… 223
二、A/D 转换器的特性 …… 224
三、典型的 A/D 转换器 …… 224
四、ADC 芯片的应用 …… 227
第四节 模拟通道的抗干扰措施 …… 229
思考题与习题 …… 231

第十章 Pentium 微处理器 …… 232

第一节 Pentium 微处理器的芯片引脚 …… 232
第二节 Pentium 微处理器的存储器系统和管理 …… 236
一、Pentium 微处理器的存储器系统 …… 236

二、Pentium 存储器的管理 …… 236
第三节 Pentium 对 80486 的几项重大改进 …… 237
一、超标量体系结构 …… 237
二、Cache 结构 …… 238
三、分支预测逻辑 …… 238
第四节 Pentium 的寄存器和新指令 …… 238
一、寄存器 …… 238
二、Pentium 新增加的指令 …… 238
附录 A 80486 指令系统表 …… 240
附录 B 数字处理指令一览表 …… 242
附录 C ASC Ⅱ 码表 …… 243

第一章　微型计算机基础

第一节　微型计算机发展概况

计算机是一种自动、高速、精确地进行信息处理的现代化电子设备。自从1946年第一台计算机诞生，至今仅50多年，计算机已经由电子管时代，晶体管时代，中小规模集成电路时代，发展到大规模、超大规模集成电路时代。现正在向第五代计算机发展。从70年代初，由大规模集成电路组成的微型计算机问世，到现在仅30年左右时间，微处理器已经推出了四代产品，四位微处理器，八位微处理器，16位微处理器和32位以上微处理器。在微型计算机发展过程中，最成功也最具有影响力的是IBM PC系列微机，又称PC机(PERSONAL COMPUTER)。1982年，IBM公司推出了以Intel 8086 CPU为处理器的IBM PC机。1983年又推出了采用Intel 8088 CPU为处理器的IBM PC/XT。这两种机型的内存为1MB，支持单任务的操作系统。1984年，以Intel 80286为CPU的16位增强型PC机IBM PC/AT上市，其内存可达到8MB，并支持多任务多用户操作系统。继IBM PC/AT之后，Intel公司推出了32位微处理器80386和80486，由80386和80486 CPU构成的PC/386和PC/486内存物理地址空间可达4GB，支持多任务多用户操作系统，并增加了高速缓冲存储器Cache。486以后的CPU，名称上改为奔腾(Pentium)系列，Pentium 586简称P5，也称586，其字长已达到64位，运算速度和功能、性能比PC/486机又有很大提高。

在微型机硬件发展的同时，软件系统也得到了迅速的发展。以操作系统为例，PC机最初使用的是DOS操作系统。随着PC机的升级，DOS操作系统已由开始的DOS 1.0版本升级到DOS 6.2版本。DOS操作系统虽然有很大的改进，但它仍是一种采用命令行接口的单任务、单用户操作系统。和DOS操作系统相比，MS Windows操作系统有更大的优越性。MS Windows提供了一个具有图形功能的用户界面操作环境，使用户使用更方便，因此得到了广泛的应用。MS Windows也由Windows 3.1发展到Windows 95，Windows NT和Windows 98。

在微机家族中，单片微机的发展同样引人注目。单片机是把CPU、一定容量的存储器和I/O接口电路集成到一片芯片上，构成具有计算机完整功能的一种微机。单片机的字长已由4位、8位发展到目前的16位，存储器可以扩充到64KB，有的还含有ADC和DAC，并且有功能很强的指令系统。单片机在工业控制和智能仪表中得到广泛的应用。

第二节　微型计算机的硬件结构

虽然微型计算机发展迅速，但至今为止微机的硬件体系结构仍采用冯·诺依曼建立的经典结构。这种结构的主要特点是：微型计算机系统的硬件由五大部分组成，如图1.1所示。这五部分是：运算器、控制器、存储器、输入设备和输出设备。其中运算器和控制器合称微处理器MPU(或CPU)。输入/输出设备由多个I/O接口和外部设备组成。微机的各组成部分通过总线连接起来。总线包括地址总线AB(ADDRESS BUS)、数据总线DB(DATA BUS)和控制总

线CB(CONTROL BUS)。我们将微型计算机的这种系统结构称为三总线结构，简称为总线结构。

微处理器(MPU)是微机的核心部分，MPU 中的运算器用于对信息进行处理和运算；控制器根据程序的要求发出各种控制命令，协调各部件之间的工作。存储器的作用是存储程序、数据和运算的结果。输入和输出设备用于微机与外部交换信息。

总线是一组导线，用于上述各部分之间的信息传输。数据总线 DB 用来传送数据信息，由于数据信息可在 MPU 和存储器、I/O 设备之间相互传送，故为双向总线。地址总线 AB 用于传送 MPU 发出的地址信息，是单向总线。微机中存储单元和 I/O 设备都有地址，在交换信息前，MPU 先通过传送地址信息找寻所需交换信息的存储器和 I/O 设备，然后经数据总线进行信息的交换。控制总线 CB 用来传送控制信号、时序信号和状态信号，由于有的信号由 MPU 发向存储器和 I/O 设备，有的信号由存储器和 I/O 设备发向 MPU，故控制总线从整体上看是双向的。

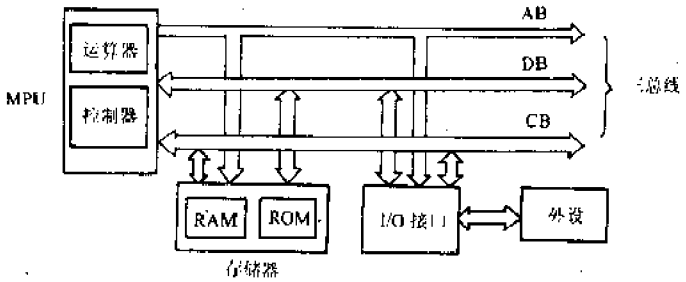


图 1.1　微型计算机结构框图

第三节　微型计算机的运算基础

一、进位计数制

进位计数制是最常用的数值表示方法，一个数由一定数目的数码排列在一起组成，每个数码的位置规定了该数码所具有的数值——权，该位置称为数位，数码的个数称为基值。该计数制又称为以基值为进位的计数制，数位的权是基值的幂，运算中，某一数位累计到基值以后向高数位进一，高数位的一，相当于低数位的基值大小，日常生活中，常见的计数制基值如十进制、八进制、十六进制、二进制等。基值为 r 的数值 N 的表示方法为：

$$N=(d_{n-1}d_{n-2}\cdots d_1d_0d_{-1}\cdots d_{-m})_r \tag{2.1.1}$$

或

$$N=d_{n-1}r^{n-1}+d_{n-2}r^{n-2}+\cdots+d_1r^1+d_0r^0+d_{-1}r^{-1}+\cdots d_{-m}r^{-m}$$

$$=\sum_{i=-m}^{n-1}d_ir^i \tag{2.1.2}$$

式中　m、n 是正整数，n 为整数的位数，m 为小数的位数，d_i 是 r 个数码 0,1,…,(r−1)中的任意一个。r^i 为数位 i 的权。

1. 十进制

十进制数码为 0、1、2、3、4、5、6、7、8、9 之一，权为 10 的幂，逢十进一，借一当十。例如：

$$(245.25)_{十}=2\times10^2+4\times10^1+5\times10^0+2\times10^{-1}+5\times10^{-2}$$

2. 二进制

二进制数码为 0 和 1；权为 2 的幂，逢二进一，借一当二。例如：

$$(11110101.01)_{二}=1\times2^7+1\times2^6+1\times2^5+1\times2^4+0\times2^3+1\times2^2+0\times2^1$$
$$+1\times2^0+0\times2^{-1}+1\times2^{-2}=(245.25)_{十}$$

3. 八进制

八进制数码有 0、1、2、3、4、5、6、7 八个，权为 8 的幂，逢八进一，借一当八。例如：

$$(365.2)_{八}=3\times8^2+6\times8^1+5\times8^0+2\times8^{-1}=(245.25)_{十}$$

4. 十六进制

十六进制有 0、1、2、3、4、5、6、7、8、9、A、B、C、D、E、F 十六个数码，其中 A、B、C、D、E、F 分别表示十进制的 10、11、12、13、14、15；权为 16 的幂，即逢十六进一，借一当十六，例如：

$$(F5.4)_{十六}=F\times16^1+5\times16^0+4\times16^{-1}=(245.25)_{十}$$

二进制由于实现容易（只需两种状态），算术运算规则简单（运算规则只有六种）即 0+0=0，0+1=1+0=1，1+1=10，0×0=0，0×1=1×0=0，1×1=1，易应用于逻辑代数（真假判断处理同二进制运算相结合，0 表“假”1 表“真”），节省设备等，使其广泛应用于计算机。由于其书写复杂，不容易看出数值大小等，使得人们考虑用八进制、十六进制来缩写，以利阅读。数的表示上通常省略十进制的基值表示符号 D(decimal)，而二进制则以 B(binary)、八进制以 O(octad)、十六进制以 H(hexa decimal)来表示其进位基值。例如：

$$245.25=245.25D=11110101.01B=365.2O=F5.4H$$

二、数的符号与小数点的表示

(一)机器数与真值

计算机中数是用二进制表示的。数码“0”和“1”，可由二值器件的两个不同稳态来表示，数的符号也只能用这两种不同稳态来表示。一般情况，是在数的最高位之前增设一位符号位 χ_0，χ_0 为 0 表示正数，为 1 表示负数。例如：$N_1=+0101011$ 和 $N_2=-0101011$ 在机器中表示为

$$\begin{array}{c} \overset{x_0}{\uparrow} \\ 0,0101011 \text{ 和 } 1,0101011 \\ \underset{x_0}{\uparrow} \end{array}$$

这种符号数码化的数，叫机器数；机器数所代表的数的本身叫真值。机器数具有如下特点：

①机器字长是有限的，字长可决定数的表示范围。机器字长是指以多少位二进制位来表示一个数，一个字长一般是字节(8 位)的整数倍。如 8 位、16 位、32 位和 64 位字长等等。

②符号数值化，参与运算。

③小数点按约定方式标出，而不是以专门器件表示。

(二)带符号数与不带符号数

计算机中字长是一定的，因此在表示带符号数与不带符号数时数值范围是有区别的。如果所表示的是不带符号数，则机器字长的所有位都参与表示数值。如表示的是带符号数，则要留出机器字长的最高位做符号位，其余位表示数值。例如，机器字长为 8 位的带符号数与不带符号数表示范围如图 1.2。

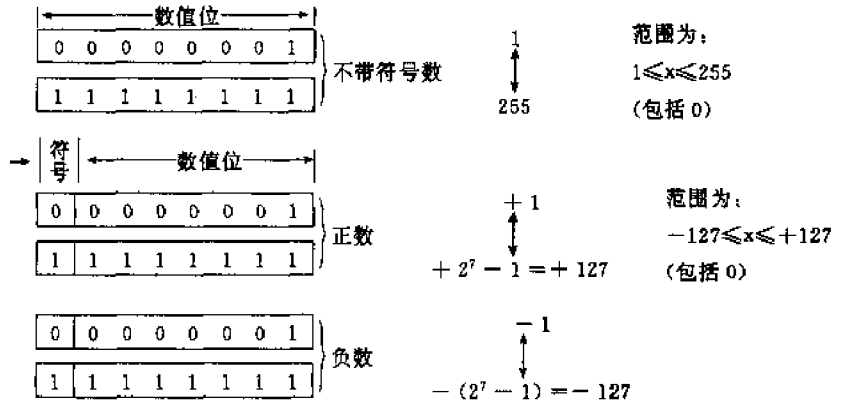


图 1.2　带符号整数与不带符号整数的区别

(三)定点与浮点表示

在计算机中，对小数点的处理有两种，分别称为定点数和浮点数，下面作简单介绍：

1. 定点数

定点数是指小数点位置固定不变。常用的定点数有下列两种：

①纯小数：小数点固定在符号位之后，如 1.1010111。此时机器中所有数均为小数。

②纯整数：小数点固定在最低位之后，如 11010111。此时机器中所有数均为整数。

机器字长为 8 位的带符号数，整数与小数表示范围如图 1.3 所示。

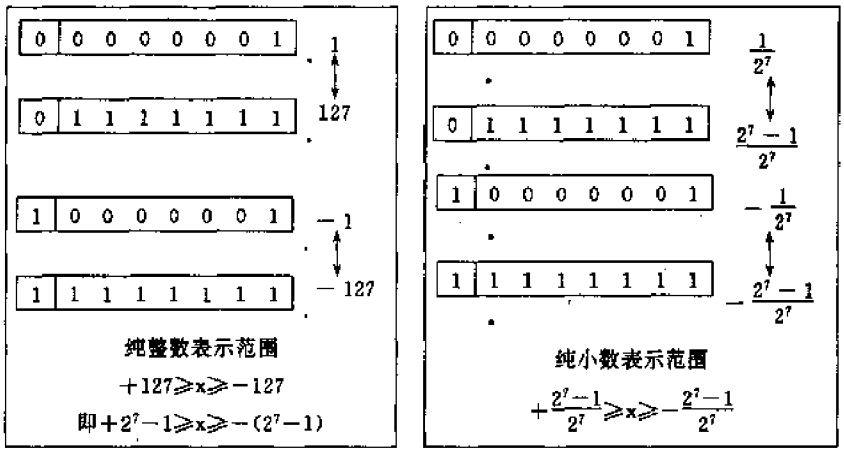


图 1.3　八位带符号整数与小数比较

2. 浮点数

浮点数由阶码和尾数两部分组成。对任意一个带符号的二进制数 N 的普遍形式可表示为

$$N = \pm S \times 2^{\pm J}$$

S——二进制小数，称尾数(尾符包括在内)；

J——数的阶码，二进制整数表示(阶符包括在内)；

2——阶码的底。

一个浮点数 $-0.101011010\times2^{-00010}$ 的表示形式如图 1.4 所示。

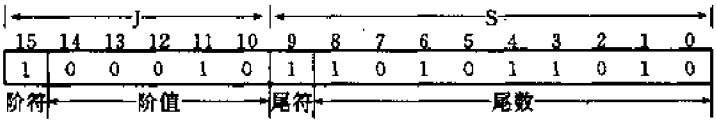


图 1.4　浮点数表示

在浮点数表示中，阶码和尾数的符号同样以“0”表示正，“1”表负，因此一个数要两组数码表示，其中一组表示带符号的阶码。阶的正负实际上指出了数的小数点的实际位置，当阶码是正 n 阶时，数 N 的实际小数点位置是尾数小数点向右移 n 位；当阶码是负 n 阶时，则尾数小数点向左移 n 位；尾数的正负表示整个数 N 的正负。

可以看出，尾数位数（有效数字位）的多少表示数的精度，位数越多，精度越高。阶码位数多少表示了机器能表示的数的范围，位数越多，范围越大，例如 3 位数的范围为 $2^{-7}\sim2^{+7}$，4 位数的阶码范围则为 $2^{-15}\sim2^{+15}$（不包含阶符值）。

3. 规格化数与“溢出”

为了提高浮点数的表示精度，数采用规格化表示法。

规格化数：如果尾数的第一位有效数字是 1 时，该数即是规格化的数。例如 1.1010111、0.1010001、0.1000000。

非规格化数：当尾数的第一位有效数字是 0 时，例如，1.0100000、1.0111111、0.0111111 等等，该数即为非规格化的数。

即当 $\frac{1}{2}\leqslant S<1$ 时为已规格化的数。显然只要移动浮点数的小数点，改变阶码，即可实现对数的规格化。

当超出机器可能表示数的范围时，我们称为“溢出”。对规格化的浮点数，若阶码超出机器所能表示的最小数时，称为“下溢”，此时机器将把此数作“0”处理；若阶码超出机器所能表示的最大的绝对值时，称为“上溢”，机器要停止运算，进行处理。

三、数的编码方法

计算机采用的是二进制数，因此，在计算机中表示的数、字母、符号等都以特定的二进制码来表示，这就是二进制编码，即以若干位二进制位的不同组合来表示一组数、字母及符号的方法。

（一）BCD——十进制数的二进制编码

BCD 码是以四位二进制的不同组合表示十进制数十个数码的方法。

常用的 BCD 码为 8421 BCD 码，即每位 10 进制数码用四位二进制数来表示，四位二进制数从高到低的权值分别为 2^3、2^2、2^1、2^0 即 8421，由于它们与二进制数位权一样，故又称为自然的 BCD 码（NBCD），如表 1.1。

表 1.1　NBCD 码表示

十进制数	8421 BCD	十进制数	8421 BCD
0	0000	10	0001 0000
1	0001	12	0001 0010
2	0010	13	0001 0011
3	0011	24	0010 0100
4	0100	25	0010 0101
5	0101	26	0010 0110
6	0110	47	0100 0111
7	0111	48	0100 1000
8	1000	49	0100 1001
9	1001	50	0101 0000

BCD 码的用途：

①在以二进制工作的机器内，数从十进制到二进制或从二进制到十进制转换时，BCD 码用作中间表示。

②它可以表示数，并可在计算机内进行直接运算。

(二)ASC Ⅱ码

ASC Ⅰ码(American Standard Code for Information Interchange)又称美国信息交换标准码，是 7 位二进制编码。它总共可表示 128 个符号，包括 26 个英文大写字母，26 个英文小写字母，0～9 共 10 个数字，32 个通用控制字符和 34 个专用字符，见附录 A。查附录 A 可看到，字母 B 的 ASC Ⅰ码为 $b_7b_6b_5b_4b_3b_2b_1$=1000010，若用 16 进制表示，则 B 的 ASC Ⅱ码为 42H。同理 $ 符号的 ASC Ⅱ码为 0100100，24H。常用英文大写字母 A～Z 的 ASC Ⅰ码为 41H～5AH。小写英文字母 a～z 的 ASC Ⅱ码为 61H～7AH。

ASC Ⅱ码表中有一些符号是作为计算机控制字符使用的，这些控制符号有专门用途。例如：回车符 CR 的 ACS Ⅱ码为 ODH，换行符 LF 的 ASC Ⅱ码为 OAH。

(三)汉字编码

我国是使用汉字的国家，要在我国推广使用计算机，汉字的使用和处理成为十分重要的问题。1981 年公布的国家标准《信息交换用汉字编码》(GB2312－80)规定了汉字的编码，即国际码。该标准编码字符集共收录汉字 6763 个，其中一级汉字 3775 个，二级汉字 3008 个，另外还定义了 700 多个西文字母、数字和图形符号。

国际码中，每个汉字由两个字节表示，每个字节用 7 位二进制码，最高位为 0，例如：字符"大"的国际码为 00110100 01110011 两个字节。为了使汉字编码和常用的 ASC Ⅱ码相区别，汉字编码在机器内的表示与国际码不同，形成汉字的内码。一种机器常用若干种汉字输入方法，但其内码是统一的。通常内码由国际码的两个字节最高位置"1"构成的。如汉字"大"的内码为 10110100 11110011，即 B4H，F3H。

第四节　IBM PC 系列微机系统简介

PC 系列微机又称 80X86(或 X86)系列微机，是指以 Intel 公司的 CPU 系列芯片为微处理器的微机系统，包括 IBMPC、PC/XT，PC/AT，386、486、586 微机等。这类微机结构上大体相同，经过不断发展，其功能不断增强，性能不断提高。

PC 系列微机系统从外部看由主机箱、键盘、CRT 显示器和打印机等几个主要部分构成。其中主机箱是微机的核心部分。主机箱内包括：

- 主板(Main board)，又称系统板、母板
- 显示适配器卡(又称视频卡)
- 软、硬盘驱动器及其适配器卡
- 其它接口电路
- 电源

下面对各部分作一个简单介绍。

一、机箱结构型式

为了适应不同的环境和应用，有几种不同的机箱结构

1. 台式机箱，也叫卧式机箱。这种机箱目前最流行。主板平装在机箱的底部，磁盘和光盘驱动器装在右前部，电源装在右后部，各种扩展卡竖直地插在母板的扩展槽中，机箱上板可开启，显示器常放在机箱上方，以便节省空间。

2. 塔式机箱，也叫立式机箱。这种机箱更节省空间，机箱可立放在桌子下面的隔板上或地板上，主板放在机箱的壁上，扩展卡是水平插入母板内。为了修理或扩展方便，机箱的壁可拆卸。

3. 便携式。包括笔记本型、膝上型和掌上型三种。这种计算机体积很小，在商务中已得到广泛应用，可以随身携带，使用十分方便。

二、主板结构

主板是一块多层印刷电路板，上面集成了 CPU、存储器、系统时钟发生器、键盘适配器、开关、跳线器等电子器件及总线和总线扩展槽。主板按所配用的 CPU 不同，分为 XT 主板、AT 主板、386 主板、486 主板和 Pentium 主板；按使用总线的不同可分为 ISA 总线主板、EISA 总线主板、VESA 局部总线和 PCI 局部总线主板等。有些 PC 机将软、硬件驱动适配器、显示器适配器和串行口、并行口也集成在主板上，构成一体化结构的主板，可提高主机的可靠性和稳定性。

三、常用 I/O 适配器

微型计算机的外设需要通过适配器(又叫控制器)才能与 CPU 进行数据交换。常用的适配器有显示器适配器、软/硬盘驱动适配器、打印机适配器和串行接口等。

1. 显示适配器

PC 机的显示能力与显示器和显示适配器有关，目前常用的显示适配器如下：

(1)EGA(增强图形适配器) 这种显示适配器是 IBM 公司于 1984 年随 AT 机推出的一种功能较强的显示适配器。它具有较好的字符和图形显示质量。

(2)VGA(视频图形陈列适配器) 这种显示适配器是 IBM 公司 1987 年推出的视频控制器卡。VGA 显示模式为模拟信号彩色显示器，具有丰富的色彩。它显示的字符和图形质量比 EGA 好。

(3)SVGA(超级视频图形陈列适配器) 它比 VGA 功能更强，性能更好，可支持 1024×768 的显示分辨率。

2. 软/硬盘驱动器适配器

早期的软硬盘适配器功能单一，各自独立。发展到如今，已将软硬盘驱动器适配器集成在一起，成为多功能卡。目前使用较多的软硬盘接口卡除了有软硬盘适配器外，还集成了两个串行口，一个并行口和一个游戏口接口卡。与主板 I/O 扩展槽相对应，多功能卡也有许多种类。目前 PC 机大多采用 IDE 接口卡，采用的是 ISA 总线接口标准，具有体积小，芯线少(40 芯扁平电缆连接)，多磁头、大容量、小型化硬盘的优点。另一种 SCSI 接口，通过 50 芯扁平电缆连接，可以同硬盘驱动器、光盘驱动器、打印机等多种外设连接，并通过命令与它们进行通信，故越来越多的被采用。

3. 其它接口电路

(1)打印机适配器

打印机是重要的输出设备，通常连接在并行端口或 RS—232 串行口上。目前一般把打印机接口集成在多功能卡上。打印机接口可以有多个，DOS 操作系统称第一个打印机接口为 LPT1，其它依次为 LPT2 和 LPT3 等。

(2)串行接口

串行口又称RS—232端口，用来连接打印机，Modem和鼠标器等。目前串行口一般集成在多功能卡或者主板上。DOS系统中有串行口1(COM1)和串行口2(COM2)。DOS 3.3以后的版本允许有COM3和COM4，但它们的用途受到限制。

除上述接口电路外，486微机系统中还常常配置一些其它的接口电路，以扩展系统的功能。为了节省空间，简化安装过程，提高系统的可靠性，常将不同的I/O适配器电路做在一块电路板上，构成特殊功能卡。例如网卡，扫描仪接口卡、多媒体功能卡等。

4. 电源

PC的电源将220V交流电转换成±5V，±12V四种DC电压。486微机的电源台式机为150～220W，立式机为220～400W。电源中有风扇提供整个系统的冷却。电源应满足最低安全标准，不产生干扰TV或无线电接收的电磁辐射。

四、华硕PVI-486SP3主板简介

华硕PVI-486SP3主板布局图如图1.5所示。该主板是一块典型的PCI总线486主板，其特点如下：

1. 支持486全系列CPU以及5X86(包括Cyrix 5X86/100/133，AMD 5X86/133/P—75)

2. 带4个16位的ISA总线扩展槽和一个VESA总线扩展槽以及3个PCI总线扩展槽，可以使用ISA、VESA、PCI总线接口卡，适应性强。

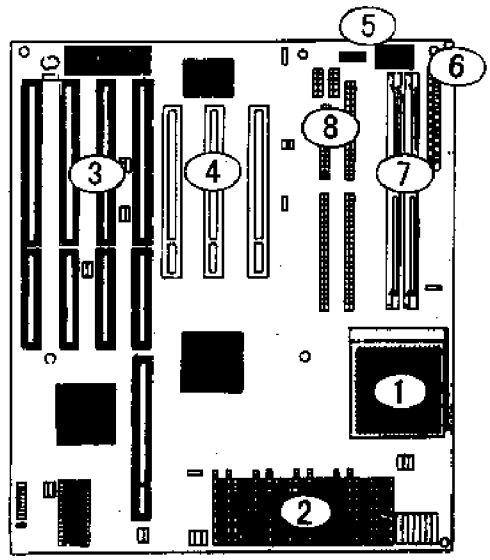


图1.5 PVI-486SP3主板元件布局图

3. 提供两个72线的SIMM内存条插座槽，可以使用容量为1MB、2MB、4MB、8MB、16MB、32MB或64MB的单/双面SIMM内存条，最多可以安装128MB的动态RAM。要求安装的动态RAM的存取速度为70ns或更快，当安装两条存储条时，要求内存条的存取速度相同。

4. 主板上集成了32位局部总线的EIDE接口卡，包含软盘接口、硬盘接口及一并两串游戏接口电路。两个硬盘接口最多可以连接4个IDE接口硬盘或CD-ROM驱动器。

5. 可以安装128KB、256KB或512KB的Cache。

6. 通过设置多条跳线可选择不同的CPU，CPU工作电压，CPU的主频；显示模式，Cache的容量等。

第五节 微型计算机系统的主要性能指标

衡量微型计算机系统的性能好坏的主要技术指标有以下五个方面：

1. 字长

字长指计算机内部一次可以处理的二进制数的位数。字长越长，计算机所能表示的数据精

度越高，在完成同样精度的运算时数据的处理速度越高。但字长越长，机器中的通用寄存器、存储器、ALU 的位数和数据总线的位数都要增加，硬件代价增大。因此应考虑精度/速度和成本兼顾的原则，来决定微型计算机的字长。PC/XT 微机的字长为 16 位，486 微机的字长为 32 位，586 微机的字长为 64 位。

2. 存储器容量

存储器容量是衡量计算机存储二进制信息量大小的一个重要指标。微型计算机中通常以字节为单位表示存储容量。1024B 为 1KB，1024KB 为 1MB，1024MB 为 1GB。8086/8088 微机内存为 1MB。486/586 微机大多有 8～64MB 内存和几 GB 的外存容量。

3. 运算速度

计算机的运算速度以每秒钟能执行的指令条数来表示。由于不同类型的指令执行时所需时间长度不同，因而有几种不同的速度的计算方法。

(1)MIPS(百万条指令/秒)方法

根据不同类型指令出现的频度，乘上不同的系数，求得统计平均值，得到平均运算速度，用 MIPS 作单位衡量。

(2)最短指令法

以执行时间最短的指令(如传送指令、加法指令)为标准来计算速度。

(3)直接计算法

给出 CPU 的主频和每条指令的执行所需要的时钟周期，可以直接计算出每条指令的执行所需的时间。

4. 扩展能力

主要指计算机系统配置各种外设的可能性和适应性。一台计算机允许配接多少种外设，对计算机的功能有重大影响。

5. 软件配置情况

软件是计算机系统不可少的重要组成部分。一台计算机软件是否配置齐全，是关系到计算机性能的重要标志。

软件分为系统软件和应用软件两大类。为了使用和管理计算机的各种软件称为系统软件。计算机配置的基本系统软件，通常应包括操作系统，各种高级语言处理程序，编译系统，各种服务程序，数据库管理系统等软件。这些软件可为用户使用计算机提供方便，扩大机器功能，提高工作效率。

一个计算机系统是硬件和软件相结合的统一整体。用户应当根据自己的需要和应用的场合来配置微机系统的硬件和软件的种类和数量。确定微机系统配置的基本原则是满足使用的要求，并兼顾近期的发展的扩展需要。

第六节　当前微机中应用的先进计算机技术

微型计算机发展至今，已成为性能价格比很高的超级微型机(Super Micro Computer)。当前微型计算机采用了许多先进的计算机技术，下面作一个简要的介绍。

一、流水线技术

流水线(Pipeline)技术是一种将每条指令分解为多步，并使不同指令的各步重叠操作，以

实现几条指令并行操作，加速程序的执行速度的一种技术。

采用流水线技术后，并没有加速单条指令的执行，每条指令的操作步骤也没有减少，只是多条指令同时执行，因而从总体上看加快了指令执行的速度。

80486 采用六级流水线结构，即同时有六条指令并行操作。若每条指令需要六个步骤，每个步骤需要一个时钟周期的时间，那末经过六个时钟后，每个时钟就有一条指令执行完毕。

流水线的实现是通过增加计算机硬件来实现的。通常一条指令的执行分为预取指令、译码、地址生成，取操作数、执行指令等多个步骤，每个步骤都需要有相应的硬件电路支持，才能实现流水线作业。

二、高速缓冲存储器技术

在 386 以后的微型机中，为了加快运算速度，都增设一级或二级的高速小容量存储器，称之为高速缓冲存储器(Cache)。高速缓冲存储器的存取速度比微机中的主存储器要快一个数量级，大体和 CPU 的处理速度相当。

由于程序中相关数据块一般都按顺序存放，并且大都存于相邻的存储单元，而程序常常重复使用同一代码和数据块，利用程序执行的这些重要特征，可采用 Cache 保存这些经常重复使用或当前将要使用的指令和数据。CPU 在对一条指令或操作数寻址时，首先到 Cache 中去查找，在一般正常情况下，CPU 对 Cache 的存取命中率可达 95%以上，从而大大提高了程序的执行速度。

Cache 技术和设计是一个复杂的课题，一般说 Cache 容量大，有较高的命中率则说明系统的性能较好。

Cache 和与它配合的高速缓冲控制器都由硬件实现，80486 中将 Cache 和高速缓冲控制器 82385 集成在 CPU 芯片中，因此对用户说是透明的，不需要用户自己去控制或操作。

三、虚拟存储器技术

虚拟存储技术是在内存储器和外存储器(软盘、硬盘或光盘)之间增加一定的硬件和软件支持，使内存和外存形成一个有机的整体。操作时，将程序预先放在外存储器中，由系统软件(操作系统)统一管理和调度，按某种置换算法将外存的内容依次调入内存中被 CPU 执行。这样，对使用者说，从 CPU 看到的是一个速度接近内存而容量却与外存相当的假想存储器，称为虚拟存储器，使编程人员在编写程序时可以不考虑内存容量的限制。在采用虚拟存储器的计算机系统中，存在着虚地址空间和实地址空间两个地址不同的空间。虚地址空间是程序可用的空间，而实地址空间是 CPU 可访问的内存空间。在 80486 中，实地址空间为 $2^{32}=4G$ 字节，而虚地址空间为 $2^{46}=64T$ 字节。

四、RISC 技术

精简指令集计算(Reduced Instruction Set Computing)技术简称 RISC 技术，其主导思想是精简 CPU 芯片中指令的数目，简化芯片的复杂程度，使指令的执行速度更快。传统的计算机都采用 CISC(Complex Instruction Set Computing)处理器，如我们现在常用的 Intel X86，其指令集中有许多指令非常复杂。用编译器对程序编译，结果证明，大多数复杂的指令很少被使用，编译器生成的总代码的 90%以上是只占 CISC 指令集中不足 10%的指令。设计更好的编译器，证明是困难的；而构筑一种简单的计算机，使它只有少数指令、大的寄存器阵列、对主存

的简单装入/存储访问，并且大多数指令执行只需要一个时钟周期，这就是 RISC 处理器组成的计算机。

RISC 处理器的主要特征表现在下列方面：

(1)采用统一的指令长度，以简化相应的逻辑电路。

(2)全 64 位实现，高流水线执行单元，很高的内部时钟速度(≥200MHz)。

(3)内置高性能浮点运算部件和大容量指令/数据 Cache。

(4)采用调入/存储体系结构，将内存中的数据预先调入内部寄存器以减少访问内存的指令数。

(5)支持多媒体和 DSP 的新指令

RISC 处理器的优点早就为人所知，但其执行中需要大容量的存储器和昂贵的 Cache，因此 RISC 技术的推广遇到了很大的阻力。目前 RISC 技术在低成本的消费者和商业领域仍未获得广泛认同，只在高性能的工作站中得到了应用。

思考题与习题

1. 简述微型计算机的发展概况。

2. 微型计算机系统的硬件由哪几部分组成？简述它们的主要功能。

3. 什么叫机器数？什么叫真值？试叙述带符号数和不带符号数的机器数主要有哪些表示方法？

4. 将下列数进行转换

(1)01011100B=？D　(2)0.10110011B=？D

(3)135D=？B　(4)99.4375D=？B

(5)10110010B=？O=？H　(6)308.DH=？B=？O

5. 简述计算机中数的编码方法。

6. PC 系列微机系统由哪几个基本部分组成？其中主机箱内应包括哪些部件？

7. PC 系列微机中有哪些常用的 I/O 适配器？

8. 衡量微机系统的主要性能指标有哪几个方面？

9. 什么叫流水线技术？采用流水线技术的优点是什么？

10. 什么叫高速缓冲存储器技术和虚拟存储器技术？微机中采用这两种存储器技术的根本目的是什么？

11. 简述 RISC 技术及 RISC 处理器的主要特征。

12. 用 ASCII 码表示

(1)A+B=C　(2)你的英文姓名

第二章　32位微处理器结构

在微处理器的发展历程中，Intel 公司的产品一直处于领先的地位。Intel 公司是世界上第一个推出微处理器(4 位微处理器 4004)的公司。目前，Intel 公司已经形成了自己的系列：80X86 系列，包括 8086/8088、80286、80386、80486、Pentium 等产品，这些微处理器在各种通用微机，专用微机和工作站中得到了广泛的应用。表 2.1 列出了 Intel 公司的微处理器芯片发展的情况。

由表 2.1 可看到 Intel80386/80486 是 Intel 公司的第四代产品，该产品是针对多任务操作系统设计的高性能 32 位微处理器。Intel 80386/80486 和 8086/80286 在目标代码级保持了兼容性，与 8086/80286 相比，在结构上和性能上有以下特点

(1)增加了片内寄存器的数量，寄存器由 16 位扩展到 32 位，具有 32 位数据处理能力。

(2)片内存储管理部件可实现段、负存储管理，比 8086/80286 提供更大的物理存储空间和虚拟存储空间。

(3)80386/80486 共有三种工作方式：实地址工作方式、保护虚地址方式和虚拟 8086 方式，比 80286 增加了虚拟 8086 方式。

(4)提供了 32 位外部总线接口，提高了数据传送速率。总线接口支持动态数据宽度控制，能自动地在 16 位和 32 位数据总线间进行切换，以适应不同位数的存储器和 I/O 设备。

(5)指令流水线增加到 6 级，即取指令、指令译码、地址生成、取操作数、执行指令和回送结果。

(6)可采用 16、20、25，33，66MHz 等多种时钟，运算速度大大加快。

80486 是 Intel 公司继 80386 之后推出的第二代 32 位高性能微处理器，80486 与 80386 相比作了许多改进，主要表现在以下几个方面。

(1)80486 采用单倍的时钟频率，即 80486CPU 的 CLK 端输入的外部时钟频率就是其内部处理器的工作时钟频率，因此可大大增加电路的稳定性。而 80386 则和 80286 一样，要求外部时钟频率必须是 CPU 内部工作时钟频率的两倍。

(2)内部包含了相当于增强型 80387 功能的浮点协处理器(FPU)。与 80386 系统中外置的 80387 芯片相比，其浮点处理速度提高 3～5 倍。

(3)对使用频度较高的基本指令，由原本的微代码控制改为硬件逻辑控制，并在指令执行单元采用了流水技术和 RISC 技术，便指令的执行速度加快，大部分基本指令可用一个时钟周期完成。

(4)和 80386 相比，内部数据总线的宽度 32 位改为 32 位、64 位和 128 位多种，分别用于不同单元之间的数据通路，对于加快数据传输速度、缩短指令执行时间有重要作用。

(5)对内部寄存器、高速缓冲器、总线接口部件等部件进行了功能的变动和增加。

上述的改进使 80486 的综合性能比 80386 提高了 3～4 倍。本章将以 80486CPU 为样板讲述 32 位微处理器的体系结构。

第一节　80486 CPU 的结构

一、80486 CPU 的内部结构

80486 CPU 的内部结构如图 2.1 所示。由总线接口部件、指令预取部件、指令译码器、指

表 2.1 Intel 微处理器发展年表及主要性能、特点

代序	型号	推出时间	工艺	数据位数	地址位数	主频 Hz	其他性能、特点
第一代	4004	1971 年	PMOS	4	12	740k	内含 1 200 个晶体管，基本指令执行时间 20 μs。
第二代	8008	1972 年	PMOS	4	12	800k	内含 2 000 个晶体管，基本指令执行时间 10 μs。
	8080	1976 年	PMOS	8	16	2M	内含 4 900 个晶体管，基本指令执行时间 2 μs。
	8080A	1976 年	NMOS	8	16	2～3M	与 8080 基本相同。
	8085A	1977 年	NMOS	8	16	3～6M	内含 9 000 个晶体管，基本指令执行时间 1 μs，它是将 8080A 微处理器、8224 时钟驱动器、8228 总线控制器三者合一而成。
第三代	8086	1978 年	NMOS	16	20	4.77～10M	16 位寄存器，内含 2.9 万个晶体管，<1MIPS。
	8088	1978 年	NMOS	8	20	4.77～10M	准 8086（即内部运算 16 位），性能与 8086 相同，外部交换 8 位。
	80186	1982 年	NMOS	16	20	8～16M	16 位寄存器，是 8086、两级 DMA、三个定时器、三级中断控制器的合成。
	80188	1982 年	NMOS	8	20	8～16M	是 8 位型 80186，类似于 8088 同 8086 的关系。
	80286	1982 年	CMOS	16	24	6～12.5M	16 位寄存器；内含 13.4 万个晶体管；保护模式操作；1～2MIPS；有虚存 1GB。
第四代	80386 DX	1985 年	CHMOS	32	32	16～33M	32 位寄存器；实存 4 GB，虚存 64 TB；内含 27.5 万个晶体管；6～12 MIPS。
	80386 SL	1988 年	CHMOS	16	32	16～20M	属低功耗型 80386DX，耗电少、主要面向便携机。
	80486 DX	1989 年	CHMOS	32	32	25～50M	32 位寄存器；实存 4 GB，虚存 64 TB；内含 120 万个晶体管，是将 80386DX、8KB Cache、80387 协处理器三者合一。
	80486 SX	1991 年	CHMOS	32	32	16～33M	是不带 80387 协处理器的 80486DX。
	80486 DX2	1992 年	CHMOS	32	32	50～66M	是双倍频时钟的 486DX，速度比之更高。
	80486 SL	1992 年	CHMOS	32	32	20～25M	属低功耗型 80486DX，更适合于便携机。类似于 386SL 同 386DX 的关系。
	80486 DX4	1994 年	CHMOS	32	32	75～100M	带有更大 Cache 的三倍频时钟的 486DX
第五代	Pentium (P5,586)	1993 年	BiCMOS	64	36	60～133M	32 位寄存器；内含 320 万个晶体管；超标量设计（2 条流水线）；分支指令预测；100～200MIPS。
	Pentium "P54C"	1994 年	BiCMOS	64	36	75～150M	3.3V，属低功耗的第二代 Pentium。
	Pentium "P55C"	1995 年	BiCMOS	64	36	90～150M	为多媒体计算机研制的第三代 Pentium。
	Pentium Pro (P6)	1995 年	BiCMOS	64	36	133～150M	32 位寄存器；内含 550 万个晶体管；内置式 L2Cache；乱序执行；超标量设计（3 条流水线）；<300MIPS。
	Pentium (P7)			64			超长指令集；1 000MIPS。

令执行部件、页管理部件、段管理部件、高速缓冲部件和浮点运算部件等 8 个逻辑部件组成。除了高速缓冲和浮点运算部件外，80486 和 80386 的内部结构基本相同。

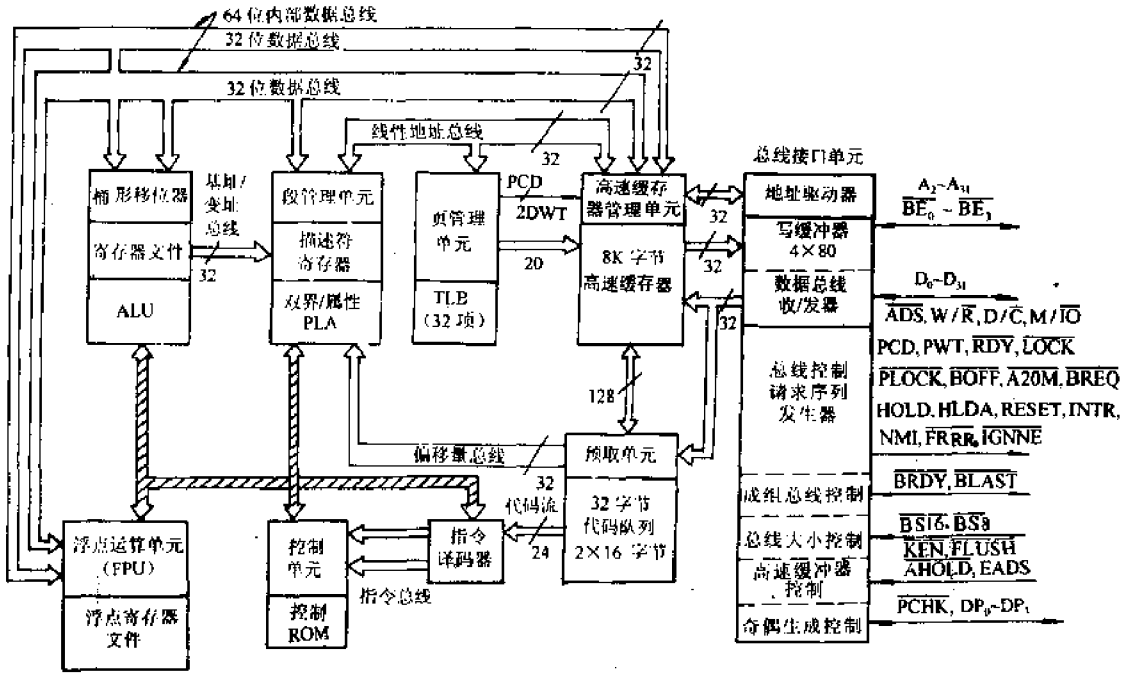


图 2.1　80486 的内部结构框图

1. 总线接口部件　主要用于管理访问外部存储器和 I/O 端口必须的地址、数据和控制总线，完成指令预取、读/写数据等总线操作。

2. 指令预取部件　包含有一个 32 字节的指令预取队列。当指令预取队列不满而总线正好空闲时，指令预取部件可通过总线接口部件从存储器中读取指令放入指令队列中。80486 的指令平均长度为 3.2 字节，所以指令预取单元平均可预取 10 条指令。

3. 指令译码部件　从指令预取队列中读取指令并进行译码。若发现是转移指令或调用指令，可通知总线接口部件去新的目标地址取指令，以刷新指令预取队列中的指令。

4. 指令执行部件　完成各种算术/逻辑运算，包括算术逻辑部件 ALU，8 个 32 位的通用寄存器，一个桶形移位寄存器和控制单元等。在控制单元中，大多数指令采用微程序控制结构执行。

5. 段管理部件　完成存储器分段管理，将指令指定的存储器的逻辑地址变换为 32 位线性地址。

6. 页管理部件　对存储器进行分页管理，将线性地址变换为存储器的 32 位物理地址。在存储器管理中，可不采用分页管理，此时线性地址即为物理地址。

7. 高速缓冲部件　在 80486CPU 中，增加片内高速缓冲器 Cache，由于片内 Cache 比片外 Cache 存取速度快，因此可加速指令和数据的访问。

8. 浮点运算部件　80486 增加了 FPU，相当于一个增强型协处理器 80387，专门用于浮点的运算，该部件可与 ALU 的整数运算并行进行。

80486 虽然是 32 位的 CPU，但其内部某些地方的数据总线不止 32 条。从图 2.1 可看到，在 Cache-ALU-FPU 之间采用了 64 位数据总线相连，使双精度数据(64 位)可一次传送。在 Cache 和预取指令部件之间采用了 128 位的数据总线，可一次预取 16 字节的指令。

由于有了上述各逻辑部件的支持，80486 可按 6 级流水线方式工作，图 2.2 列出了其工作方式。在理想情况下，指令执行中的 6 个步骤，即取指令、指令译码、地址生成、取操作数、执行指令和存储结果，每级需要一个时钟。一条指令在单独执行时，6 个步骤都不能少。但从整体上看，经过最初的 6 个时钟后，每个时钟都有一条指令执行完毕从流水线输出，因此采用 6 级流水线方式后，整体上看，每个时钟可完成一条指令的执行，提高了指令的执行速度。

指令步骤	时钟周期											
	1	2	3	4	5	6	7	8	9	10	11	
取指令	I_1	I_2	I_3	I_4	I_5	I_6						
指令译码		I_1	I_2	I_3	I_4	I_5	I_6					
地址生成			I_1	I_2	I_3	I_4	I_5	I_6				
取操作数				I_1	I_2	I_3	I_4	I_5	I_6			
执行指令					I_1	I_2	I_3	I_4	I_5	I_6		
存储结果						I_1	I_2	I_3	I_4	I_5	I_6	

图 2.2　80486 的流水线工作示意

二、内部寄存器

寄存器用于存储信息。由于内部寄存器在 CPU 中，其执行速度比 CPU 和存储器交换信息时要快，因此在微机 CPU 中，都设置了许多寄存器，以提高指令的执行速度。80486 的寄存器是从 8086/80286/80386 的基础上扩充而来的，除将原来的 16 位寄存器扩展为 32 位外，还增加了一些新的寄存器。

80486 的寄存器接功能可分为四类：基本寄存器、系统级寄存器、调试和测试寄存器、浮点寄存器。

1. 基本寄存器

基本寄存器包括通用寄存器、指令指针寄存器、标志寄存器和段寄存器，如图 2.3 所示

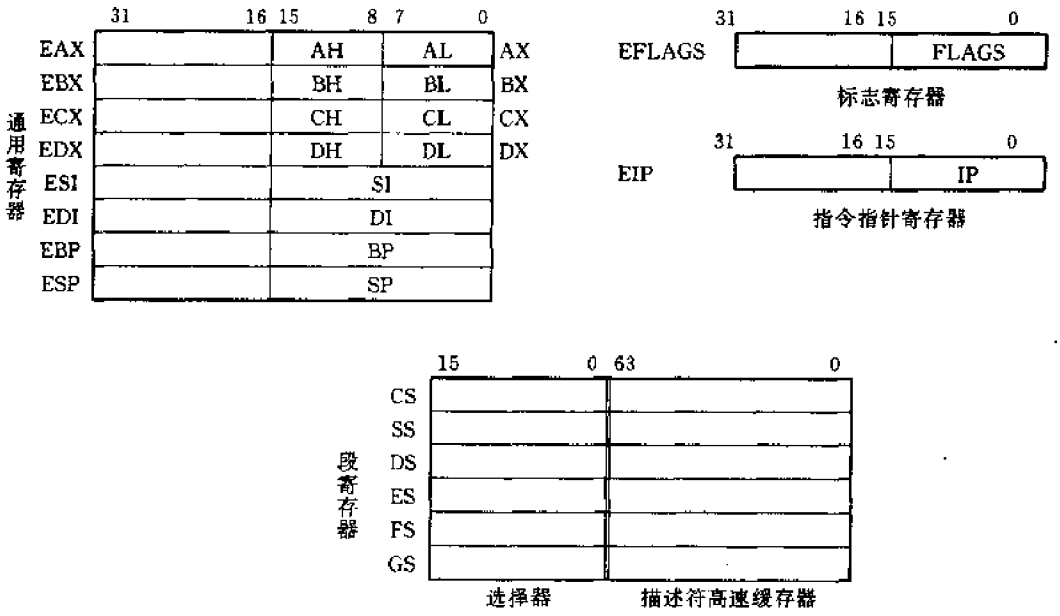


图 2.3　基本寄存器

(1)通用寄存器

80486 有 8 个 32 位通用寄存器，分别命名为 EAX，EBX，ECX，EDX，ESI，EDI，EBP，ESP。为了与 8086/8088 兼容，它们的低 16 位可以单独访问，并以 8086/8088 中的名称命名：AX，BX，CX，DX，SI，DI，BP，SP。其中 AX，BX，CX，DX 可分别分成两个 8 位的寄存器，其命名为 AH、AL，BH、BL，CH、CL，DH、DL，H 表示高八位(8～15 位)，L 表示低八位(0～7 位)。通用寄存器常用于存放操作数和中间结果，存放 8 位，16 位和 32 位的数据。

(2)指令指针寄存器(EIP)

EIP 用于存放指令在代码段中的偏移地址，在程序运行中，它总是指向下一条要预取的指令。EIP 和代码段的段基址寄存器(通常为 CS)共同确定下一条指令的物理地址。在指令执行时，CPU 自动修正 EIP 的内容，使 EIP 总是指向下一条指令的地址，以保证指令的连续执行。EIP 的低 16 位称为 IP，用于 16 位操作方式中。

(3)标志寄存器(EFLAGS)

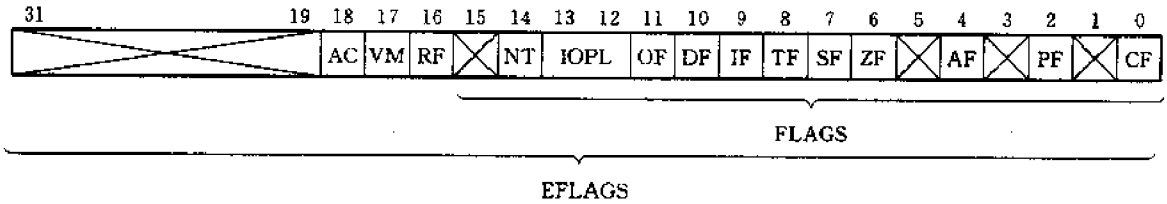
该寄存器为 32 位的寄存器，共包含三种状态。

①状态标志：反映算术/逻辑运算指令执行后的状态

②控制标志：反映串操作指令地址改变的方向

③系统标志：用于控制 I/O、中断、调试、任务转换和控制保护方式与虚拟 8086 方式之间的转换。

图 2.4 列出了 EFLAGS 各位的状况，由图可见，32 位标志寄存器仅用了其中的 15 位，共列出了 14 种标志。其中 CF、PF、AF、ZF、SF、TF、IF、DF、OF 是原 8086/8088 用的标志位；OPL、NT 属 80286 增加的标志位；VM、RF 属 80386 增加标志位；AC 为 80486 新增的标志位。下面对各标志位的意义作介绍。



注：×位表示未用，通常置 0

图 2.4 标志寄存器

• CF(进位标志)：CF＝1 表示运算结果的最高位产生进位(加法)或借位(减法)。CF 主要用于多字节数的运算和循环移位指令。

• PF(奇偶标志)：PF＝1 表示运算结果中有偶数个 1。PF 用于数据传送中检验错误。

• AF(辅助进位标志)：AF＝1 表示运算中低 4 位向高位产生进位(加法)或借位(减法)。AF 用于 BCD 码运算。

• ZF(零标志)：ZF＝1 表示运算结果为 0。

• SF(符号标志)：SF＝1 表示运算结果最高位为 1，对于补码表示的有符号数，SF＝1 表示运算结果为负数。

• TF(自陷标志)：TF＝1 表示 CPU 进入单步执行方式，每执行完一条指令后都产生一个中断，利用它可逐条的调试程序。

• IF(中断允许标志)：IF＝1 表示 CPU 允许外部可屏蔽中断的申请。

· DF(方向标志):DF=1 表示串运算时地址按减位方向进行。

· OF(溢出标志):OF=1 表示运算结果的数值超过了可表示的范围。

· IOPL(I/O 特权级标志):共占 2 位,表示 0～3 级 4 个 I/O 特权级,仅用于保护方式。只有当任务的现行特权级高于或等于 IOPL 表示的特权级时,执行 I/O 指令可保证不产生异常。

· NT(任务嵌套标志):NT=1 表示当前任务嵌套在另一个任务内,引起任务转换。该标志位用于控制被中断的链和被调用的任务。

· RF(恢复标志):该标志位与调试寄存器的代码断点结合使用,以保证不重复处理断点。当 RF=1 时,即使遇到断点或调试故障也不再产生异常中断。在成功执行每条指令后,RF 自动复位为 0。

· VM(虚拟 86 模式标志):VM=1 表示工作在虚拟 8086 方式。该位只能以两种方式设置:在保护方式下,由最高特权级(0 级)的代码段的 IRET 指令来设置;或由任务转换来设置。

· AC(对准检查标志):仅对 80486 有效。对准是指访问字操作时从偶地址开始,访问双字操作时从 4 的整数倍地址开始。当 AC=1,且 CR_0 的 AM 位也为 1,则进行字、双字或 4 字的对准检查。

EFLAGS 的低 16 位称 FLAGS 标志寄存器,用于 8086/80286 的 16 位操作。

(4)段寄存器

80486 有 6 个段寄存器:CS、SS、DS、ES、FS、GS。其中 CS 指明当前的代码段,SS 指明当前的堆栈段,DS、ES、FS 和 GS 指明当前的 4 个数据段。

80486 的段寄存器由 16 位段选择器和 64 位描述符高速缓冲器组成。在实地址方式和虚拟 8086 方式下,段选择器就是段寄存器,它保存逻辑段基址,其寻址方法和 8086 一样,即段基址左移 4 位加上偏移地址就是存储单元的物理地址,不必使用描述符高速缓冲器。

在保护虚地址方式下,每个段定义一个 16 位的选择符并装入段选择器中。由选择符在全局描述符表 GDT 或局部描述符表 LDT 中找到对应的描述符装入描述 符高速缓冲器中。有关保护虚地址方式下的寻址方法将在第六章存储器部分详细介绍。

2. 系统级寄存器

系统级寄存器包括 4 个控制寄存器和 4 个系统地址寄存器。这些寄存器一般由操作系统进行访问。

(1)控制寄存器

4 个控制寄存器 CR_0～CR_3 用于存放全局特征,如是否用 Cache 和 FPU,是否分段分页等。各控制寄存器都是 32 位,其格式如图 2.5 所示。

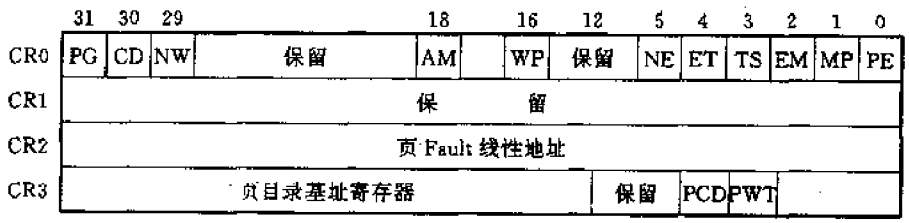


图 2.5 控制寄存器

①CR0:包含整个系统的控制标志。为了和 80286 兼容,CR_0 的低 16 位即为 80286 中的 MSW(机器状态字)。CR_0 共用 11 位,各位的作用如下:

· PE:保护模式允许位,当 PE=1,CPU 为保护模式。

• MP：协处理器控制位，MP＝1 表示系统有协处理器。

• EM：仿真协处理器控制位，EM＝1 表示用软件仿真协处理器。

• TS：任务转换控制位，每当进行任务转换时，由 CPU 自动将该位置位。

• ET：处理器扩展类型控制位，ET＝1 表示实际组装有 80387。

• NE：数值异常中断控制位，当 NE＝1 时，执行 FPU 指令发生故障。

• WP：写保护控制位，当 WP＝1 对由管理程序读取的专用页进行写保护。

• AM：对准屏蔽控制位，AM＝1 时，EFLAGS 中的 AC 位有效，否则无效。

• NW：非通写控制位，和 CD 位一起决定高速缓冲器是否有效。

• CD：Cache 禁止位，CD＝1，不允许用 Cache。

CD	NW	功　能
0	0	cache 有效
0	1	未定义
1	0	没有分配新 cache 行
1	1	cache 无效

• PG：页功能控制位，PG＝1 表示负功能有效

②CR_1：为 Intel 为其它产品保留，未用

③CR_2：保存页故障 32 位线性地址。只有 CR_0 中 PG＝1 时，CR_2 才有效。

④CR_3：CR_3 的 31～12 位为总目录基址寄存器，保存当前任务的总目录表在内存的基地址，PCD 位为是否对页目录进行高速缓冲，PCD＝0 时，进行高速缓冲。PWT 位为页级通写位，PWT＝1 表示片外 cache 对页目录进行通写，否则回写。

(2)系统地址寄存器

系统地址寄存器共 4 个，如图 2.6 所示，用于保存操作系统需要的保护信息和地址转换信息。系统地址寄存器只在保护方式下使用。

	32 位基地址	16 位界限
GDTR		
IDTR		

	16 位选择符	32 位基地址	16 位界限	16 位其他属性
LDTR				
TR				

图 2.6　系统地址寄存器

①全局描述符表寄存器 GDTR：48 位，保存全局描述符表的 32 位线性基地址和 16 位界限值，即用于保存全局描述符表 GDT 的起始地址和大小。

②中断描述符表寄存器 IDTR：48 位，保存中断描述符表基地址和 16 位界限值。80486 为每个中断定义一个中断描述符，所有的中断描述符都集中存放在一个中断描述表(IDT)中，需要由 IDTR 找到该中断描述符表在内存的位置。

③局部描述符表寄存器 LDTR：由 16 位选择符和 64 位描述符寄存器两部分构成。用于保存局部描述符表 LDT 的起始地址与大小。

④任务寄存器 TR：由 16 位选择符和 64 位描述寄存器构成；TR 提供任务状态段 TSS 在内存中的位置。

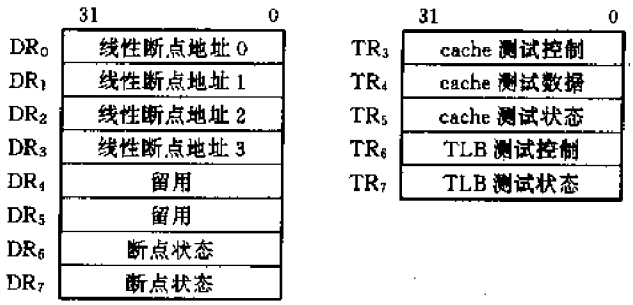
80486 CPU 的设计面向多任务系统，适用于实时或多用户操作系统。通常，每个任务占有

的存储空间称为局部空间，而多个任务共同占用的存储空间称为全局空间。在多任务系统中，每个任务有一个局部空间，因此局部空间可以有多个；而全局空间是对全部任务来说的，因此全局空间只有一个。一个任务占用的局部空间和全局空间所在内存中的位置和其它特性用局部描述符和全局描述符来记录，它们分别存放在局部描述符表 LDT 和全局描述符表 GDT 中。全局描述符表只有一个，因此全局描述符表寄存器 GDTR 和 GDT 是一一对应关系，不需要选择符来选择。而局部描述符表 LDT 可能有多个，对应的 LDTR 也就可能有多个，为利于对它们的寻址、LDTR 在结构上分成选择符和描述符寄存器两部分，选择符用于检索以找到指定的描述符，描述符寄存器中存放描述符的实质内容。对于任务寄存器 TR，由于任务状态段 TSS 也是每个任务对应一个，因此也需要 16 位选择符，以便对 TSS 进行检索。

(3)调试和测试寄存器

80486 提供了 8 个 32 位的可编程寄存器来支持调试功能，如图 2.7(a)所示。其中 DR_0～DR_3 各存放一个断点的线性地址；DR_4，DR_5 未用；DR_6 为断点状态寄存器，用于说明断点的性质和断点是否异常；DR_7 为断点控制寄存器，用于指明断点发生的条件和断点的类型。有了这些调试寄存器，80486 可进行数据断点的设置和 ROM 断点的调试。

80486 定义了 5 个测试寄存器 TR_3～TR_7，如图 2.7(b)所示。其中 TR_3～TR_5 用于 Cache 的测试；TR_6，TR_7 用于转换后援缓存器 TLB 的测试。测试寄存器是为了增强系统的可测性而增设的附加硬件，不是 80486 体系结构必需的标准部件。



(a)调试寄存器　　(b)测试寄存器

图 2.7　调试和测试寄存器

(4)浮点寄存器

80486 的 FPU 中包含有 13 个浮点寄存器，如图 2.8 所示。其中 R_0～R_7 8 个 80 位浮点数据寄存器用作固定寄存器组成堆栈；16 位的标记字寄存器用来标记每个数据寄存器的内容；16 位控制寄存器完成浮点运算中的精度控制、舍入控制和发生异常时的屏蔽功能；16 位状态寄存器反映浮点运算中 FPU 的全部状态；二个 48 位的指令和数据指针寄存器提供对程序异常处理的支持。

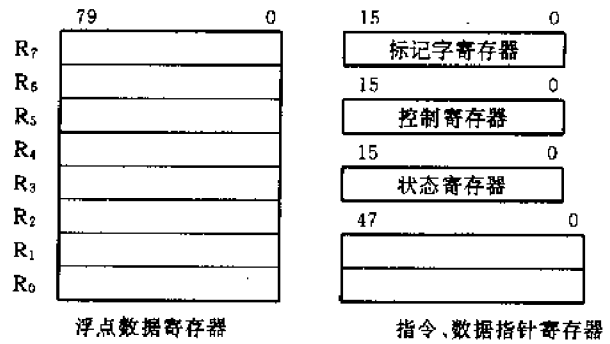


图 2.8　浮点寄存器

80486 进行浮点运算时，不再使用 CPU 中的 8 个通用寄存器，而使用上述专用的浮点寄存器，以保证浮点运算的正确和出错的处理。有了这些浮点寄存器和 FPU，80486 可以做到 ALU 和 FPU 的并行运算。

三、存储器

存储器用于存储程序和数据。存储器由若干个存储单元组成，存储器中含有存储单元的多少称为存储器的容量，存储器的容量大小是衡量计算机性能的一个重要指标。存储器存储信息的基本单位是一个二进制位，称为一个 bit；每个二进制位可存储一个二进制数：0 或 1。每 8 位(bit)组成一个字节，称为 byte；通常一

个字节构成一个存储单元，其 8 位的位编号如图 2.9(a)所示。8086/8088 CPU 组成的 IBMPC 机的字长为 16 位，由二个字节共 16 位组成。80486 微机的字长为 32 位，由四个字节组成如图 2.9(b)所示。

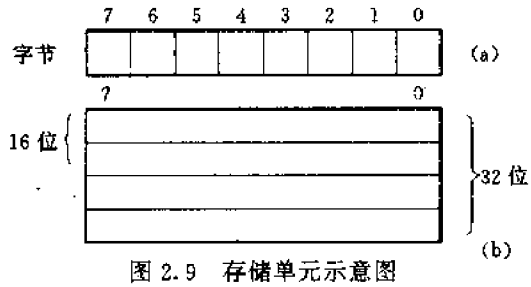


图 2.9 存储单元示意图

80486 的存储器工作方式有三种：实地址方式，保护虚地址方式和虚拟 8086 方式。

实地址方式是 80486 最基本的工作方式，其工作原理与 8086/8088 基本相同，主要区别是借助操作数长度前缀，80486 能处理 32 位数据。另外，可使用新增的两个数据段寄存器 FS 和 GS。

保护虚地址方式下，引入了虚拟存储器的概念。在保护虚地址方式下，CPU 可访问的物理存储空间为 $2^{32}=4GB$；程序可用的虚拟地址空间为 $2^{46}=64TB$。段的长度在启动页功能时是 $2^{32}=4GB$，不启动页功能时为 $2^{20}=1MB$。可支持多用户和单用户的多任务操作，并对各任务提供保护机制。

虚拟 8086 方式是一种既有保护功能又能执行 8086 代码的工作方式，可以说是保护方式的一种子方式。CPU 的工作原理与保护虚地址方式下相同，但程序指定的逻辑地址解释与 8086 相同，即可以和实地址方式下一样执行 8086 的应用程序。

80486 的这三种工作方式在一定条件下可以相互转换。

本节介绍 80486 的实地址工作方式，有关其它两种工作方式将在第六章存储器中再加以介绍。

80486 在上电和复位后自动建立实地址方式。80486 的实地址方式与 8086/8088 工作方式基本相同，只能在 1MB 范围内寻址，故不能管理和使用扩展存储器。

1. 存储器的组织

80486 在实地址方式下其存储器的组织与 8086/8088 基本相同，共能访问 1M 字节的存储空间，需要 20 条地址线，如图 2.10 所示。为了寻找和存取放在每个存储单元中的信息，每个存储单元都要给一个地址，地址也由二进制数来表示。1M(2^{20})字节存储器需 20 条地址线，共 20 位二进制数来表示存储单元的地址。为了书写方便，常用 16 进制数表示，共 5 位 16 进制数。图中，方框左边的 5 位 16 进制数表示一个地址，地址从 00000H 开始编号，到 FFFFFH 为止，这样 1M 字节的每个存储单元都有一个唯一的地址。

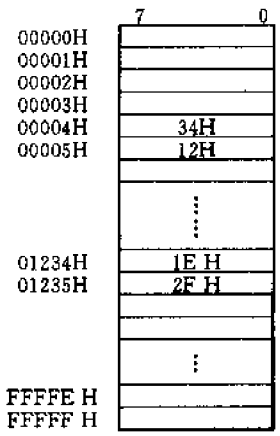


图 2.10 80486 实地址时的存储器

存储单元的地址和存储单元中存放的信息是不同的两个概念。例如：图 2.10 中地址为 00004H 的存储单元中存放的信息是 34H，地址为 00005H 的存储单元中存放的信息是 12H。存储器虽然由字节单元组成，但大部分的数据(实地址下)都是 16 位的，通常相邻的二个字节构成一个 16 位数据(又称一个字)，其中低 8 位存入低地址，高 8 位存入高地址。而 16 位数据的地址由低地址表示。例如，上图中，若 1234H 表示一个字，则这个字的地址应为 00004H，可表示为(00004H)=1234H。同一个地址既可看成是字节单元的地址，又可看成是字单元的地址，这要根据使用情况来确定。当 16 位数据的地址是偶数时，一次可存取 16 位，称为对准字；当 16 位数据的地址为奇数时，这时低字节在奇数地址中，高字节在偶数地址中，称为非对准

字；要存取非对准字，需要访问二次存储器，花费较多的时间，因此通常计算机中总是以对准字来存取 16 位数据，故在存放数据时应注意。

存储单元中存放的信息若还是一个地址，这种寻址方式称为间接寻址。例如，若上例中(00004H)＝1234H，若 1234H 仍是一个地址，则((00004H))＝(01234H)＝2F1EH。

存储器的重要特征是，从某个存储单元中取出内容后，该单元中原有内容仍保持不变，可多次读取。只有存入新的信息到这个存储单元中，它原来的内容才被新的内容代替。由于 80486 在实地址工作方式时与 8086/8088 基本相同，下面以 8086/8088 存储器为例讲述存储的结构。

2. 存储器的分段结构

8086/8088CPU 将 1MB 的存储空间分为若干个段，每个段的容量≤64KB，允许各段在整个存储空间中浮动，每个段都是独立寻址的逻辑单位。每个段的起始地址(又称段基址)有特殊规定，要求其地址为 16 的整数倍，即该地址的最低 4 位为 0。在程序设计中，需要设立几个段以及每个段的用途由用户自己确定，但在任何时刻，一个程序当前只能访问 4 个段中的内容，这 4 个段是：代码段、数据段、堆线段和附加段。这四个段的起始地址通常由 CS，DS，SS，ES 四个段寄存器来存放。

如图 2.11 所示。实际上，每个逻辑段在存储器中可以是邻接的，间隔的，也可以是重叠的(完全重叠或部分重叠)。

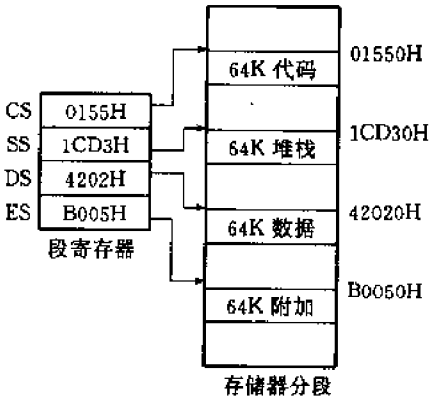


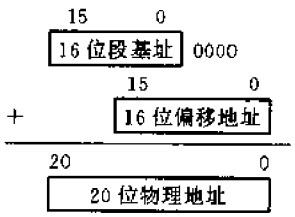
图 2.11　存储器分段

3. 逻辑地址与物理地址

逻辑地址不是存储单元的真正地址。在分段结构的存储器中，逻辑地址由段基址和偏移地址两部分组成，它们都是 16 位无符号二进制数。

在 8086CPU 中，段基址来源于段寄存器 CS，DS，SS 和 ES。偏移地址可以来自指针寄存器 SP，BP，变址寄存器 SI，DI 和指令指针寄存器 IP。段基址是一个段起始单元地址的 20 位中的高 16 位。偏移地址表示所寻找的存储单元与它所在段段基址之间的字节距离，当偏移地址为 0 时，就是这个段的起始单元的地址。在程序设计中，使用逻辑地址有利于程序的开发和存储器的动态管理。

物理地址是真正的存储单元的地址，CPU 与存储器之间的信息交换都使用物理地址。物理地址由逻辑地址计算得到，方法如下：



将段基址左移 4 位再加上偏移地址形成物理地址，可写为

段基址×16＋偏移地址＝物理地址

显然，每个存储单元只有一个物理地址，但此地址可由不同的段基址和不同的偏移地址组成。

例：设某指令的代码段(CS)＝0155H，(IP)＝10A4H

则该指令的物理地址＝(CS)×16＋IP＝01550H＋10A4H＝025F4H

四、堆栈(Stack)

堆栈是一个特定的存储区，主要用于数据的暂存和子程序调用或中断响应过程中保护断点和现场。堆栈采用后进先出的数据结构，即最后存入堆栈的数，最先从堆栈中取出。

8086 系统中的堆栈是用段定义语句在存储器中定义的一个堆栈段，堆栈段容量最大可达 64KB，它可以设定在 1MB 存储器允许的任意位置。堆栈段由堆栈寄存器 SS 给定其段的起始地址，由堆栈指针 SP 给定当前栈顶，当堆栈置空时，SP 指向堆栈底部，称为栈底。栈底是固定不变的，而栈顶是根据堆栈中存放的信息多少浮动的。

图 2.12 表示了一个堆栈的情况，若已知(SS)＝1000H，空栈时(SP)＝2000H 堆栈在存储器的分布情况为：堆栈的段基址：

$$1000H\times16=10000H$$

栈底　$1000H\times16+2000H=12000H$

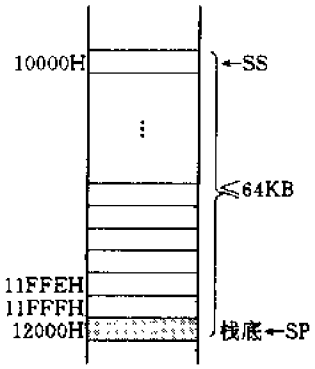


图 2.12　堆栈

对堆栈的操作主要有两个，进栈操作(PUSH)是把数据压入堆栈，进栈操作时，先将 SP 进行修改(SP)－2→SP，然而将信息压栈。弹出操作(POP)是把数据从堆栈顶部弹出，进行弹出操作时，先将数据出栈，然后修改堆栈指针(SP)＋2→SP。堆栈以字为单位进行操作，每次访问堆栈能压入/弹出一个字信息。

第二节　80486CPU 芯片的引脚信号

80486CPU 有 168 条外部引脚信号，如图 2.13 所示。下面对主要的引脚信号作简要的说明。

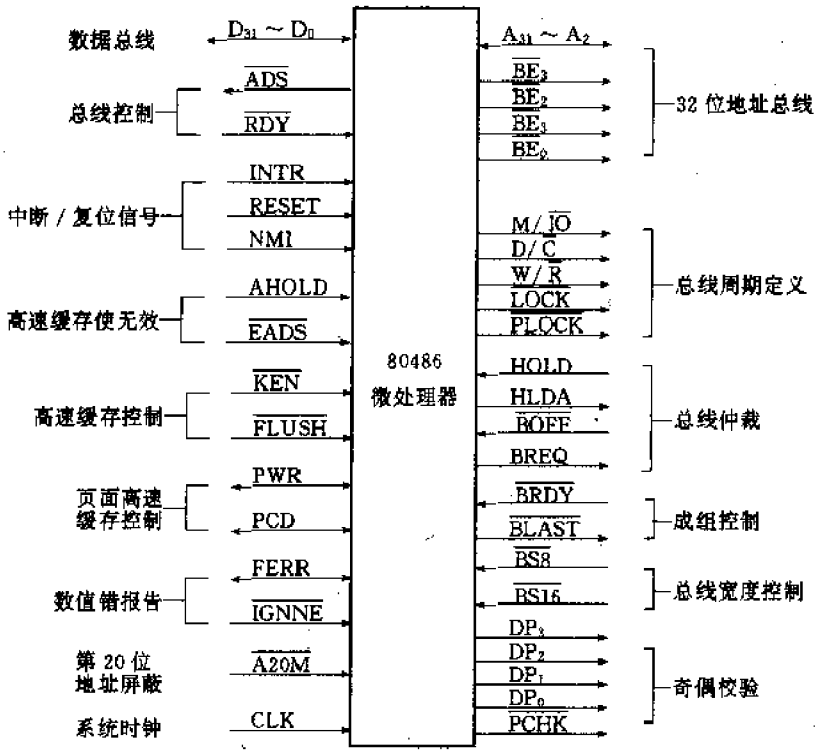


图 2.13　80486 外部引脚信号

1. 地址总线(A31～A2，$\overline{BE_3}\sim\overline{BE_0}$)

80486 由 A31～A2，$\overline{BE_3}\sim\overline{BE_0}$ 构成 32 位地址总线。A31～A2 指明 32 位地址(4 字节)，而

$\overline{BE_3}$～$\overline{BE_0}$指明访问 32 位中的哪些字节。80486 可直接寻址 4G 字节的物理存储空间和 64K 字节的 I/O 地址空间。

2. 数据总线(D31～D_0，总线宽度控制$\overline{BS_{16}}$、$\overline{BS_8}$)

(1)D_{31}～D_0 为 32 位(4 个字节)双向数据总线。其中 D_7～D_0 为最低有效字节。

(2)总线宽度控制$\overline{BS_{16}}$和$\overline{BS_8}$用来控制数据总线的宽度。由于 80486 允许传送 8 位和 16 位数据。当$\overline{BS_{16}}$有效时，选择 16 位数据总线，当$\overline{BS_8}$有效时，选择 8 位数据总线。

3. 总线控制、仲裁、和总线周期定义信号

(1)总线周期定义信号(M/$\overline{IO}$，D/$\overline{C}$，W/$\overline{R}$、$\overline{LOCK}$，$\overline{PLOCK}$)

总线周期定义信号用于定义主要的总线周期。

· M/$\overline{IO}$：定义内存或 I/O 周期

· D/$\overline{C}$：定义数据或控制周期

· W/$\overline{R}$：定义读还是写周期

· $\overline{LOCK}$：用于总线锁定，$\overline{LOCK}$＝0 表示当前的总线周期被锁定，此时 80486 将独占系统总线，不允许别的主控器访问系统总线。

· $\overline{PLOCK}$：伪锁定输出，当$\overline{PLOCK}$有效时允许 80486 访问 32 位以上的存储器操作数，如访问浮点长字(64 位)、段描述符(64 位)和高速缓存行(128 位)等。

(2)总线控制($\overline{ADS}$，$\overline{RDY}$)

· $\overline{ADS}$为地址状态输出信号，当$\overline{ADS}$有效时，标志一个总线周期开始，此时，地址信号和总线周期定义信号均为有效。

· $\overline{RDY}$为非突发"准备好"输入信号，$\overline{RDY}$有效表示当前的总线周期结束。用来响应读请求信号时，表示外部设备已经将准备好的数据送上数据总线，等待 CPU 读取。用来响应写请求信号时，表示外部设备已经收到 CPU 送来的数据。

(3)总线仲裁(BREQ，HOLD，HLDA，$\overline{BOFF}$)

总线仲裁用于多处理器结构时，决定总线由谁占有。

· BREQ　总线请求输出，每当总线周期在内部执行中时 80486 发出此信号。

· HOLD　总线保持请求输入，当此信号有效时，表示另一个总线主设备请示 80486 交出总线的控制权。

· HLDA　总线保持应答输出，它是对 HOLD 的应答。当 HLDA 有效时，表明 80486 交出总线控制权给另一个总线主设备。

· $\overline{BOFF}$　输入，强制 80486CPU 在下一个时钟期间释放对总线的控制，并使其引脚浮空。

(4)突发控制($\overline{BRDY}$，$\overline{BLAST}$)

· $\overline{BRDY}$　突发"准备好"输入，与$\overline{RDY}$作用相似，只是$\overline{BRDY}$　是突发周期时的响应信号，表明当前周期结束

· $\overline{BLAST}$　突发结束输出，用于终止高速缓冲的行填充或其它多数据周期的传送。

4. 有关高速缓冲的信号

(1)高速缓冲控制($\overline{KEN}$，$\overline{FLUSH}$)

· $\overline{KEN}$　高速缓冲允许输入，用来确定当前周期所传送的数据是否能用于高速缓冲。当$\overline{KEN}$信号有效且 80486 产生高速缓存周期时，该周期将成为高速缓存行填充周期。

· $\overline{FLUSH}$　高速缓冲清洗输入，当$\overline{FLUSH}$有效时强制 80486 清洗整个内部高速缓冲。

(2)高速缓冲使无效周期(AHOLD,$\overline{EADS}$)

这两个输入信号用于高速缓冲使无效周期中。

·AHOLD　当 AHOLD 有效时,允许另一个总线主控器控制 80486 的地址总线,并强制 80486 在下一个时钟周期浮空其地址总线。

·$\overline{EADS}$　$\overline{EADS}$有效时表明 80486 地址引脚上已放置了一个外部地址,并使 80486 去读取外部总线地址和 Cache 中登记的地址比较检验,如相等则使 Cache 中相应区域内容生效。

(3)页面高速缓冲控制(PWT,PCD)

·PWT　页通写输出,是以页为单位的写操作方式控制信号。

·PCD　页高速缓存禁止输出,PCD=1 禁止以页为单位的 Cache 操作。当允许调页时,PWT 和 PCD 分别对应于页表项中两位用户属性位:第 3 和第 4 位;当禁止调页时,它们对应于控制寄存器 CR_3 的位 3 和位 4,受它们的控制。

5.其它信号

(1)时钟信号(CLK)

80486 的 CLK 时钟信号就是 CPU 的主频,它为 CPU 提供基本的内部工作频率。

(2)奇偶校验(DP_3～DP_0,$\overline{PCHK}$)

·DP_3～DP_0　数据奇偶校验状态输出,分别对应数据的 4 个字节。

·$\overline{PCHK}$　奇偶校验状态输出,当$\overline{PCHK}$有效时表示有奇偶校验错。

(3)中断/复位(INTR,NMI,RESET)

·INTR　可屏蔽中断请求输入,INTR 有效表示有外部可屏蔽中断请求。

·NMI　非屏蔽中断请求输入,NMI 有效表示有非屏蔽中断请求。

·RESET　复位输入,强制 80486 回到初始状态执行程序。

(4)数据错报告($\overline{FERR}$,$\overline{IGNNE}$)

·$\overline{FERR}$　浮点出错输入,$\overline{FERR}$有效时说明 80486 中 PC 类型的浮点出错。

·$\overline{IGNNE}$　忽略数值错误输入,$\overline{IGNNE}$=0 时 80486 将忽略数值错误并继续执行非控制型浮点指令。当控制寄存器 CR_0 的 NE 位置 1 时,该信号不起作用。

(5)第 20 位地址屏蔽($\overline{A20M}$)

·$\overline{A20M}$　第 20 位地址屏蔽输入,$\overline{A20M}$有效时 80486 仿真在 8086 的 1M 字节地址。$\overline{A20M}$仅在实地址方式下被建立,在查找内部高速缓冲和驱动存储器周期之前,建立$\overline{A20M}$输入使 80486 屏蔽第 20 位物理地址。

第三节　微机系统中的总线结构

计算机总线是计算机模块间传递信息的通道。总线技术在计算机系统中占有十分重要的位置。一个计算机系统的研制和外围模块的开发,都必须服从一定的总线规范。按总线传送信息分类,总线可分为地址总线(AB),数据总线(DB)和控制总线(CB)三类。按总线在微机系统中所处的位置,可把总线分为以下四类:

1.片内总线,是 CPU 芯片内部传输信息用的总线。

2.芯片总线,又称片级总线,是指同一块集成电路板上 CPU 与各部件连接。

3.系统总线,也称板级总线。目前微机系统都采用模块结构,即一个微机系统由多块模板构成。一块模板往往就是一块集成电路板,如显示器卡,软、硬盘驱动板等。各电路板间用系统

总线连接，对系统总线的要求是规格化，可通用，服从某一总线标准。

4. 外部总线，用于微机系统与其它微机和电子仪器、设备的连接，这类总线并非微机所特有，可以借用电子工业的总线标准，如 EIA-RS232，IEEE-485 等。

系统总线在微机系统中占有重要的地位，本章将重点介绍系统总线。

一、总线的操作

微型计算机系统中各模块间的操作，如 MPU 和存储器间的信息交换，MPU 和 I/O 端口的信息交换，在多处理机系统中各处理机之间的信息交换都需要通过总线来进行。在同一时刻，总线上只能允许一对模块进行信息交换。当有多个模块都需要使用总线进行信息传输时，只能采用分时方式，即将总线时间分成段，每段时间完成一次信息的交换，通常称这段时间为一个总线周期。

为了在一个总线周期中完成信息交换，通常将一个总线操作周期分为 4 个阶段：

· 总线请求和仲裁阶段：当有多个模块提出总线请求时，必须由仲裁机构仲裁，确定将总线的使用权分配给哪一个申请的模块。仲裁通常由硬件构成，有串行仲裁和并行仲裁两种。由于篇幅有限，本书不作详细介绍，有兴趣的读者可查阅有关的资料。目前 80486 以上的微机已经将总线请求和总线仲裁逻辑集成在芯片内。

· 寻址阶段：取得使用总线权的模块需要经总线发出本次要访问的存储器或 I/O 端口的地址及有关命令，启动各模块工作。

· 传送数据阶段：主模块（指取得总线控制权的模块）与从模块之间进行数据的传送。

在寻址阶段和传送数据阶段，需要总线握手技术的支持。总线握手的作用是控制每个总线操作周期中数据传送的开始和结束，以实现主从模块间的协调和配合，确保寻址和数据传送的可靠性。总线握手的方法有 4 种：同步总线协定；异步总线协定；半同步总线协定；周期分裂式总线协定。有关这方面的内容请查阅有关资料。

· 结束阶段：主从模块将有关信息从总线上撤除，交出总线和使用权，以便其它模块的使用。

二、时钟周期、总线周期和指令周期

微处理器工作的过程就是执行程序的过程。程序是由若干条指令组成的，微处理机通过一条一条的执行指令，完成程序指定的特定功能。为了执行指令，微处理器必须有一个标准的时钟电路，在时钟脉冲的控制下，一个节拍一个节拍地进行操作，按规定的步骤一步步完成所要求的各个操作。

1. 时钟周期

时钟周期是微处理器执行指令的最小时间单位，通常又称为一个 T 状态。IBMPC/XT 中 8088 的时钟频率为 4.77MHz，故一个时钟周期 $T=\frac{1}{4.77\text{MHz}}$ 约为 210ns；80486 随型号不同其主频有 25MHz，33MHz，66MHz 等多种，其时钟周期为 40ns，30ns，15ns 等多种。

有两种产生时钟信号的方法，一种是由石英晶体加专用时钟电路产生时钟信号，并加到微处理器芯片的 CLK 引脚；另一种是直接将石英晶体加到微处理器的时钟引脚，由集成于微处理器内部的时钟电路处理后产生。

2. 总线周期

CPU 对存储器或 I/O 端口完成一次读/写操作需要的时间称为一个总线周期(总线周期也称机器周期)。一个总线周期由 2 个或 2 个以上时钟周期构成。IBM PC/XT 的基本总线周期由 4 个时钟周期 T_1～T_4 组成,对于低速外设,可在 T_3 和 T_4 周期之间插入一个或若干个等待周期 T_W。80486 的基本总线周期由两个时钟周期 T_1 和 T_2 组成,若外部硬件速度较慢,在 T_2 周期可重复一个或若干个 T_2 时钟周期。

3. 指令周期

CPU 执行一条指令所需的时间称为指令周期。指令周期由若干个总线周期组成,不同指令执行的时间不同;同一指令,寻址方式不同时,所需的时间也可不同。例如 8088 中,最快的传送指令需两个时钟周期,而加法指令,当寻址不同时需要的时钟周期可从几个至几十个不等。80486 的指令周期除因为指令和寻址方式不同而异外,还与高速缓冲是否命中等因素有关,少则一个时钟周期,多则二三百个时钟周期。由于 80486 的一些基本指令由微码控制改为硬件执行,所以其读/写指令最快可在单个总线周期甚至单个时钟周期内完成。

三、80486 的基本总线操作时序

虽然微处理器执行不同指令时的时间有很大的差别,但它们都有各自固定的时序。大多数指令都由存储器读/写、I/O 端口读/写或中断响应等基本的总线周期组成。微处理器执行指令的过程可由总线时序来反映,所谓总线时序,是指与完成总线操作有关的地址线、数据线、控制信号和时钟信号相互之间的定时关系,一般可用时序图来表示。

80486 支持多种总线操作,以满足高性能系统的需要。总线操作可以是单周期或多周期的;突发的或非突发的;高速缓冲的或不可高速缓冲的;传送的数据可以是 8 位、16 位或 32 位的。为了支持多处理器系统,还提供了高速缓存无效性周期的锁定周期,这就决定了总线操作的多样性和复杂性。本节只以两种基本的总线传送周期:非突发单周期和突发周期为例来说明 80486 的总线操作时序。

1. 非突发单周期总线传送

80486 支持的最快非突发单总线周期含有 2 个时钟周期,读、写都一样,所以称之为 2—2 周期。图 2.14 的左半部示出了基本的 2—2 周期的总线操作时序。从时序图中可看出:

①无论读还是写,80486 都是通过在第一个时钟(T_1)的上升沿建立地址状态信号$\overline{ADS}$而启动一个周期。$\overline{ADS}$输出指明,周期定义线和地址总线上的信号可供使用。

②外部系统在第二个时钟(T_2)中返回"非突发准备好"信号$\overline{RDY}$。$\overline{RDY}$有效(低电平)表明外部系统在响应读操作时已将数据置于数据总线上,或在响应写操作时已接收数据。80486 在 T_2 结束时采样$\overline{RDY}$,如采样的$\overline{RDY}$为低,则结束该周期。

③80486 在每个总线传送周期的 T_2 中发出一个"突发终止"信号$\overline{BLAST}$,以指明每个总线传送都在单周期后完成。

④每当读周期结束后的一个时钟周期内,80486 均输出一个$\overline{PCHK}$信号,以指明所读数据的奇偶校验状态。$\overline{PCHK}$信号可供外部系统使用。

当外部系统在 2 个时钟周期内完成不了读、写操作时,可通过在第二个时钟 T_2 结束时驱动$\overline{RDY}$为无效而将等待状态插入到基本的 2—2 周期中。图 2.14 右半部示出的是带有一个等待状态的简单的不可高速缓存的非突发单周期总线传送时序。因为读、写周期都包含 3 个时钟周期,所以称之为 3—3 总线周期。实际上,只要保持$\overline{RDY}$为无效,就可将任意数量的等待状态加到 80486 的总线周期中。可见,80486 可适应任何速度的外部系统工作需要。

要注意的是，在将$\overline{RDY}$驱动为无效的所有时钟边沿处，必须使“突发准备好”信号$\overline{BRDY}$也处于无效（高电平），以确保这些简单的非突发周期能正确工作。图 2.14 中未画出$\overline{BRDY}$时序。

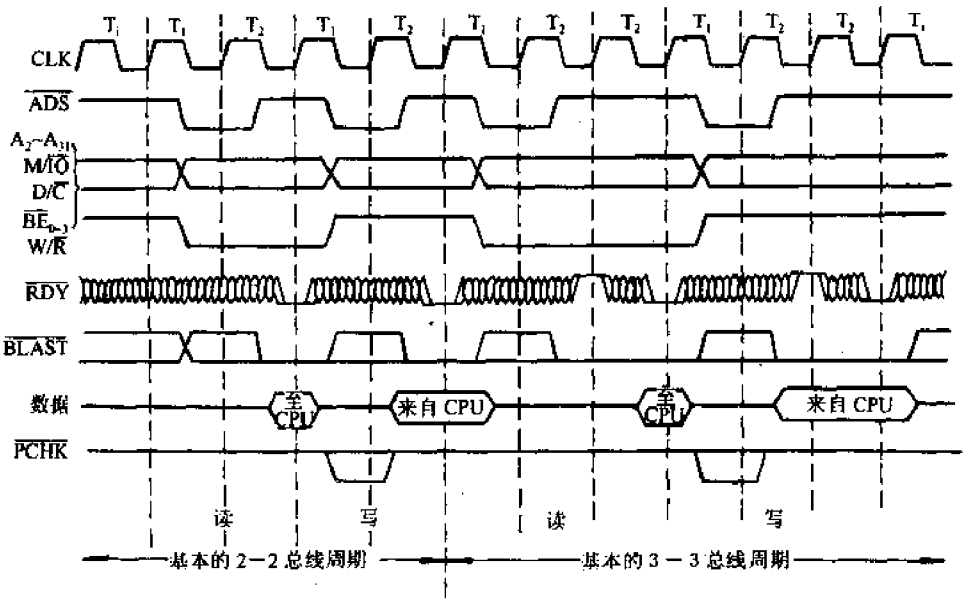


图 2.14　单周期总线传送时序

2. 突发周期总线传送

在 80486 系统中，许多传送操作如 64 位浮点读操作、128 位预取的内部请求、片内高速缓存行填充（每个 Cache 行为 16 字节）、段/表描述符的读操作、未对准数据的内部请求等，需要多个总线周期才能完成。当外部系统每个周期只能传送 8 位或 16 位数据时，执行 16 位/32 位或 32 位操作指令时也可能引起多周期总线传送。

突发周期总线传送是实现多周期传送最快的一种传送方法。在突发周期中，每个时钟都将新的数据项（字节，字或双字）选通到 80486 片内。最快的突发周期传送，对第一个数据项需 2 个时钟，以后则每个时钟一个数据项。

突发周期和非突发周期的启动方法一样，也是从 CPU 送出地址码并发出有效$\overline{ADS}$信号开始。80486 通过在该周期第 2 个时钟（T_2）中保持突发终止信号$\overline{BLAST}$为无效而表明它正进入突发周期，而外部系统则通过返回“突发准备好”信号$\overline{BRDY}$来作为响应突发周期的握手信号。

突发周期中数据项的地址将完全落在同一个 16 字节对界区内（对应于 80486 片上超高速缓存某一行）。16 字节对界区从单元 XXXXXXX0H 开始而结束于单元 XXXXXXXFH。在突发周期中，只能改变$\overline{BE_{0\sim3}}$、A_2 和 A_3。$A_4\sim A_{31}$、$M/\overline{IO}$、$D/\overline{C}$和 $W/\overline{R}$在整个突发周期中保持稳定。给出突发周期中第一个地址后，外部硬件能够很容易地预先计算出随后传送的地址。外部存储系统可以设计成能快速填充 80486 内部超高速缓存行的形式。

突发周期不限于超高速缓存行填充。80486 请求的任一多周期读出都可以转换成一个突发周期。80486 只突发完成一次传送所需要的字节数。例如，对于 64 位浮点的不可超高速缓存读只突发 8 个字节。

外部系统通过在某个传送的第一周期中返回$\overline{BRDY}$有效（而不是$\overline{RDY}$有效），将多周期请

求转换成突发周期。对于不可突发的周期，例如中断应答和停机，$\overline{BRDY}$的作用和$\overline{RDY}$相同，如果在同一时钟内返回$\overline{BRDY}$和$\overline{RDY}$，则忽略$\overline{BRDY}$。不能实现突发的存储区和外部设备必须用$\overline{RDY}$终结周期。

80486 在多周期传送中除最后周期外的所有周期都将$\overline{BLAST}$驱动成无效。$\overline{BLAST}$在第一个周期中被驱动成无效的作用是通知外部系统该传送可以占用附加周期。在传送的最后周期中$\overline{BLAST}$被驱动成有效，以指明当$\overline{BRDY}$或$\overline{RDY}$下一次有效时传送完毕。

80486 启动某一请求时知道需要传送多少个字节和该数据是否对界。外部系统必须在$\overline{RDY}$或$\overline{BRDY}$被返回前的一个时钟通过返回$\overline{KEN}$、$\overline{BS8}$和$\overline{BS16}$输入，告诉微处理器该数据是否可高速缓存和总线的宽度。80486 根据其内部信息和来自外部系统的输入确定完成某一传送将需要多少个周期。

由于 80486 在外部系统返回$\overline{KEN}$、$\overline{BS8}$和$\overline{BS16}$之前不可能确定完成某一传送需要多少个周期，所以$\overline{BLAST}$在总线周期的第一个时钟是无效的，应在周期的第二个和随后的时钟内，当外部系统返回$\overline{RDY}$或$\overline{BRDY}$时采样$\overline{BLAST}$。

图 2.15 给出了突发周期传送的总线操作时序。左半部为不可高速缓存的突发周期，外部系统通过在传送操作的第一个周期结束处送回有效的$\overline{BRDY}$（不是$\overline{RDY}$），将多周期请求转换为突发周期。右半部为可高速缓存的突发周期，外部系统通过在该周期第一个时钟（T_1）结束时返回有效的$\overline{KEN}$，使之成为一次高速缓存行填充（连续读入 16 个字节，即 4 个 32 位数据项）；同时通过在该次传送的第一个周期结束处驱动$\overline{BRDY}$为有效以通知 80486 CPU，它将以突发方式传送该缓存行数据。

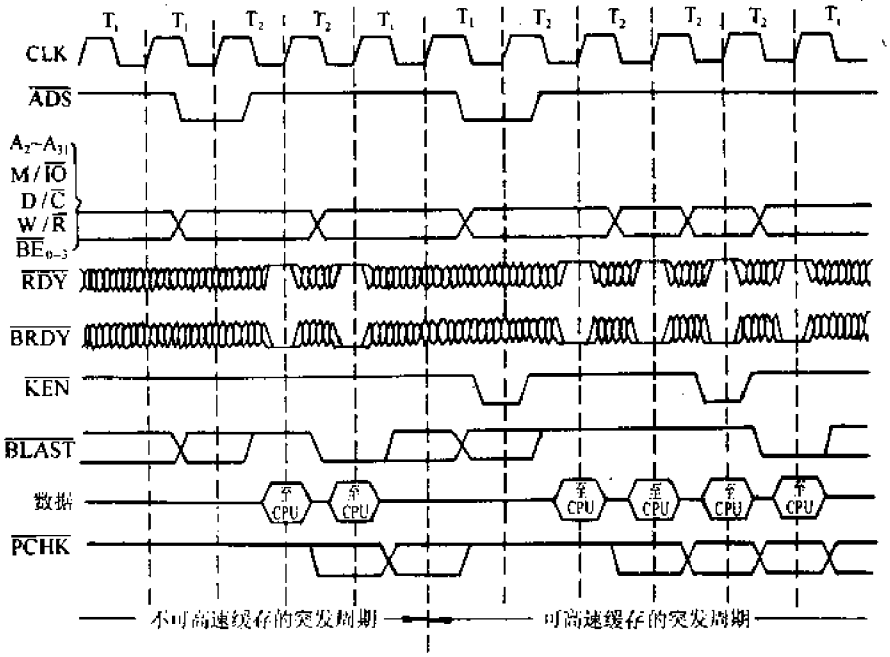


图 2.15 突发周期总线传送时序

无论是不可高速缓存还是可高速缓存的突发周期传送，$\overline{ADS}$都仅在传送的第一周期中被驱动为有效。且在$\overline{BRDY}$返回有效时，必须将$\overline{RDY}$驱动为无效。

四、80486 系列微机系统的标准总线

如前所述，采用总线结构是微型计算机系统体系结构的重要特点之一，总线是微机系统的

组成基础和重要资源。各微型机生产公司从一开始就十分重视总线的设计，并制订了自己的总线标准，以支持本公司的产品设计和开发。但是，由各厂商自己制订的总线标准专用性太强，适应面太窄，当系统设计人员需要选用不同厂家的模块和设备组建一个微机应用系统时，就感到很不方便。随着计算机技术和计算机应用的发展，在用户和厂商的共同推动下，国际电工委员会(IEC)和美国电气与电子工程师协会(IEEE)等组织经过对许多厂家所提总线标准的选优与修改，或组织专门人员制订，先后提出或推荐了一批总线作为国际标准总线。仅美国 IEEE 推荐的总线标准就有 20 多种。这就使计算机总线标准和总线结构逐步走上了通用性、规范化的轨道。不同的厂商可以按同样的总线标准和规范来生产各种不同功能的芯片、模块和整机，用户可以只根据功能需要去选用不同厂家生产的、基于同种总线标准的模块和设备，甚至可以方便地自行设计功能特殊的专用总线模块或设备，以组建合乎要求的应用系统。这样，可使各级别产品(包括芯片、模块、设备)的兼容性、互换性和整个系统的可维性、可扩性从根本上得到保证。

目前应用最广的 PC 系列微机的 I/O 总线有局部总线和系统总线两种。局部总线用于微处理器子系统内部，实现 CPU、协处理器、总线控制器、数据收发器、地址锁存器和时钟发生器等的连接。系统总线或者只是局部总线的缓冲形式，或者依据状态、控制线的编码和(或)信号的定时与装入不同而有别于局部总线。自从 IBM 公司推出第一台 PC 机以来，从 8 位机到 16 位机、32 位机、64 位机，为了适应数据宽度的增加和系统性能的提高，依次推出并为广大计算机界同行所承认、采用的总线标准主要有 XT 总线、AT 总线(即 ISA 总线)、MCA 总线、EISA 总线、VL 总线和 PCI 总线等。而经过多年的应用、比较、优选，到了 486、586 微机时代，应用最多的是 ISA 系统总线和 VL、PCI 两种局部总线。市场上流通的各种 486 系统或 486 主板的结构大都是建立在 ISA/VL 总线基础上的，而各种 586(Pentium)系统或 586 主板的结构则基本上是以 ISA/PCI 总线为基础。

各种标准总线之间尽管在设计细节上有很多不同，且各有特点，但从总体上看，无论哪种总线，在其规范(Specification)中都包含了信号系统、电气特性和机械物理规格等一系列规定，而其信号系统的规定中通常又不外乎信号分类、数据宽度、地址空间、传输速率、总线仲裁、总线握手、信号定时和资源共享分配等内容。本节不打算也不可能对各种常见总线标准作详细介绍，只对目前最流行的 ISA 总线、VL 总线和 PCI 总线作一简介。

(一)ISA 总线

ISA(Industrial Standard Architecture)总线是 IBM 公司 1984 年为推出 PC/AT 机而建立的系统总线标准，所以也叫 AT 总线。它是对 XT 总线的扩展，以适应 8/16 位数据总线要求。它在推出后得到广大计算机界同行的承认，以兼容这一标准为前提的微型计算机纷纷问世。目前人们所见到的 286、386、486 微机尽管工作频率各异，内部功能和系统性能有别，但大都采用了 ISA 标准总线。

1. ISA 总线的主要特点

(1)具有比 XT 总线更强的支持能力。它能支持：

- 64KB IO 地址空间(0000H～FFFFH)
- 16MB 存储器地址空间(000000H～FFFFFFH)
- 8 位或 16 位数据存取
- 15 级硬中断
- 7 级 DMA 通道

• 产生 I/O 等待状态

(2)是一种多主控(Multi Master)总线。即除主 CPU 外,DMA 控制器、DRAM 刷新控制器和带处理器的智能接口控制卡都可以成为 ISA 总线的主控设备(但它只支持一个智能接口控制卡)。这一特性是通过总线中的MASTER信号线来实现的。

(3)可支持 8 种类型的总线周期:

• 8 位和 16 位的存储器读周期

• 8 位和 16 位的 I/O 读周期

• 8 位和 16 位的存储器写周期

• 8 位和 16 位的 I/O 写周期

• 中断周期(包括中断请求周期和中断响应周期)

• DMA 周期

• 存储器刷新周期

• 总线仲裁周期

2. 总线信号

ISA 总线共包含 98 根信号线,它们是在原 XT 总线 62 线的基础上再扩充 36 线而成的。其扩充卡插头、插槽也是由两部分组成,一部分是原 XT 总线的 62 线插头、插槽(分 A、B 两面,每面 31 线),另一部分是新增的 36 线插头、插槽(分 C、D 两面,每面 18 线),新增的 36 线与原有的 62 线之间有一凹槽隔开。

ISA 总线信号及其在连接器上的排列情况如图 2.16 所示。各信号线的定义及功能则如表 2.2 和表 2.3 所示。从表中可见,前 62 根信号线按功能可分为时钟与定位数、数据总线、地址总线、控制总线、电源与地线五大类;后 36 根信号线除不提供时钟、定位功能外,其余四类都提供,是前面相应类功能的扩充。正因为这样,按 PC/XT 总线标准设计的插件板,才能照样原封

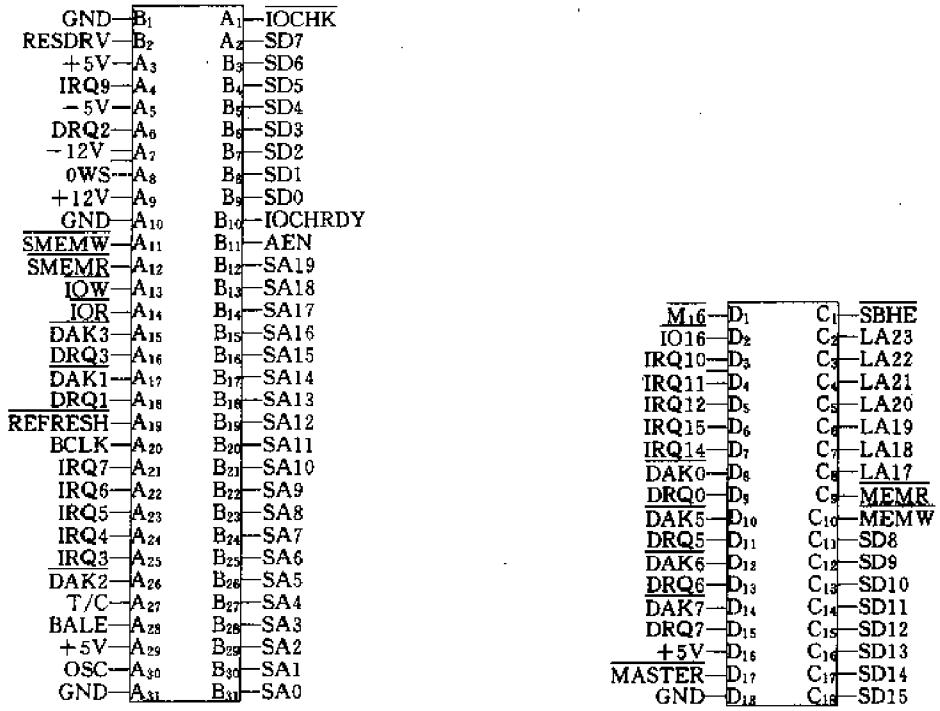


图 2.16 ISA 总线信号线排列

不动地插在ISA总线扩充槽的前62芯位置上工作。可见ISA总线系统具有向上与XT总线系统兼容的特性。

3. 存储器和IO地址空间

表 2.2 ISA 总线前 62 根信号线定义

类型	信号名称	I/O	有效电平	功能说明
时钟与定位	OSC	O		周期为70ns的振荡信号，占空比2：1
	CLK	O		周期为167ns的系统时钟，占空比2：1
	RESDRV	O	高	上电复位或初始化系统逻辑
	$\overline{OWS}$	I	低	零等待状态信号
数据总线	SD7～SD0	I/O	高	双向数据线0～7位，为CPU、存储器、I/O设备提供数据，SD0为最低有效位
地址总线	SA19～SA0	O	高	地址线19～0位，提供对存储器和IO设备寻址，A0为最低有效位
	BALE	O	高	地址锁存使能信号，允许锁存来自CPU的有效地址
	AEN	O	高	DMA允许信号，允许DMA控制器控制地址总线、数据总线及读/写命令线，进行DMA传输
	IRQ9,7～3	I	高	I/O设备的中断请求线。IRQ3优先级最高
	DRQ3～1	I	高	I/O设备的DMA请求线。DRQ1优先级最高。
	$\overline{DACK3～1}$	O	低	DMA应答信号线，分别对应DMA请求3～1级
控制总线	T/C	O	高	DMA通道计数结束信号，由DMA控制器送出
	$\overline{IOR}$	I/O	低	I/O读
	$\overline{IOW}$	I/O	低	I/O写
	$\overline{SMEMR}$	O	低	存储器读(小于1MB空间)
	$\overline{SMEMW}$	O	低	存储器写(小于1MB空间)
	$\overline{I/OCHCK}$	I	高	向CPU提供I/O设备或扩充存储器奇偶错
	$\overline{I/OCHRDY}$	I	高	I/O通道就绪，若是低速的存储器或I/O设备，则在检测到一个有效地址和一个读或写命令时，使该信号变低，总线周期用整数倍的时钟周期延长，但该信号低电平维持时间不得超过15个时钟周期(15×167ns=2.5μs)
	$\overline{REFRESH}$	I/O	低	该信号用来指示刷新周期
电源与地址	+5V			电源
	−5V			电源
	+12V			电源
	−12V			电源
	GND			地线

表 2.3 ISA 总线后 36 根信号线定义

类型	信号名称	I/O	有效电平	功能说明
数据总线	SD15～SD8	I/O	高	双向数据线15～8位，为CPU、存储器和I/O设备提供高8位地址
	$\overline{SBHE}$	I/O	低	数据高位允许信号
	$\overline{MEMCS16}$	I	低	存储器16位芯片选择信号
	$\overline{I/OCS16}$	I	低	I/O设备16位芯片选择信号
地址	LA23～LA17	I/O	高	存储器与I/O设备的最高7位地址
控制总线	IRQ15～10	I	高	中断请求信号，IRQ10为最高级，IRQ15为最低级
	DRQ7～5,0	I	高	DMA请求信号，DRQ0为最高级，DRQ7为最低级
	$\overline{DACK7～5,0}$	O	低	DMA应答信号，对应DRQ7～5和DRQ0
	$\overline{MASTER}$	I	低	I/O处理器发出的主控信号，使CPU总线处于高阻态
	$\overline{MEMR}$	I/O	低	对所有存储器读命令
	$\overline{MEMW}$	I/O	低	对所有存储器写命令
电源与地线	+5V			电源
	GND			地线

ISA 总线提供了两组地址信号线，即 SA19～SA0 和 LA23～LA17。使用这两组地址信号线可以对 16MB 的存储器空间和 64KB 的 I/O 地址空间进行访问。

16MB 的存储器地址空间编址为从 000000H 到 FFFFFFH，地址空间分配规则如图 2.17 所示。其中前 1MB(000000H～0FFFFFH)是基本存储器空间，它由三部分组成：系统板存储器，占最低 640KB(000000H～09FFFFH)；接下来为适配卡存储器，占 256KB(0A0000H～0DFFFFH)，用作显示器、硬盘等适配卡的数据缓冲区；最高 128KB(0E0000H～0FFFFFH)分配给 EPROM，其中包括一部分 BIOS 空间。16MB 存储器的最高 128KB(FE0000H～FFFFFFH)是前 1M 中 ROM BIOS 空间的复制。其他的存储器空间分配给总线 I/O 扩充卡。

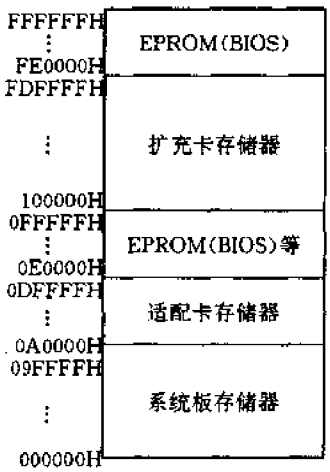


图 2.17　存储器地址空间分配

ISA 总线能对 64K 个 I/O 端口进行访问，但只用了 1K 个端口地址(0000H～03FFH)。其中头 256 字节(0000H～00FFH)分配给系统板上的 I/O 接口芯片，其余的 768 字节(0100H～03FFH)分配给扩充卡。

(二)VL 总线(VESA 总线)

VL 总线(VESA Local Bus)是 VESA(Video Electronics Standard Association)协会与 60 余家公司联合推出的一个通用的全开放局部总线标准，也叫 VESA 总线。它的推出为微机系统总线体系结构的革新奠定了基础。在此之前，PC 系列机一直是采用单一慢速的系统总线体系结构，尽管系统总线从 XT、AT(ISA)总线发展到 EISA 和 MCA 总线，在总线性能上逐步有所改善，但仍然远远跟不上 CPU 和软件的发展速度。在系统运行的大部分时间内，由于 I/O 总线上的数据传输速度低而使 CPU 处于等待状态，从而影响了系统的整体工作性能和效率。实践证明，限制 I/O 数据传输速度提高和系统整体性能增强的瓶颈不是外设，而是 I/O 总线，因为随着微电子技术的发展，厂家早已能生产出与 CPU 性能相适应的 I/O 芯片和 I/O 适配卡。

解决总线传输瓶颈问题的一个有效方法是将外设直接挂到 CPU 局部总线上并以 CPU 速度运行。这样做可极大地提高外设的运行速度，增强系统性能，而成本却只略有增加。这个优越的性能/价格比为局部总线产品创造了巨大的潜在市场。正是在这种市场需求的牵引下，VL 总线应运而生。

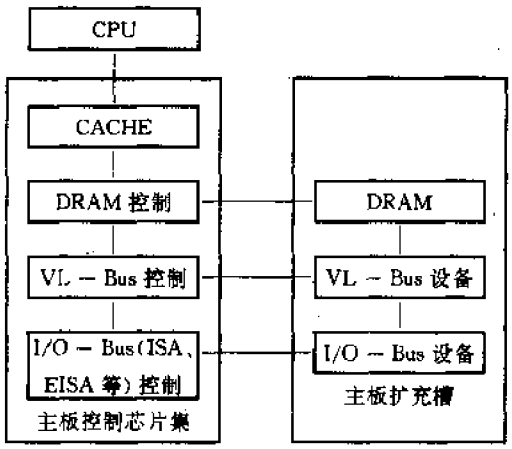


图 2.18　VL 总线接口方框图

VL 总线作为一种局部总线，不是一个单独的总线体系结构，它是对 ISA、EISA 等系统总线的补充，只能和系统总线共存于一个系统中，形成 ISA/VL 或 EISA/VL 等总线体系结构。图 2.18 描述了信息从主 CPU 通过 VL 总线到外设的逻辑流程。

1. VL 总线的主要特性

(1)支持的主 CPU 类型　VL 总线设计由单个主 CPU 系统优化而来，支持 386SX、386DX 和 486SX、486DX、486DX2 等主 CPU。多主 CPU 系统如果也使用 VL 总线，对 VL 总线控制器来讲应表现为单主 CPU 系统。

(2)VL 总线设备类型　VL 总线主要设计目标是支持高速视频控制器，其他外设如硬盘控制器、LAN 控制卡以及能从高速接口受益的外设，也可使用 VL 总线。

(3)总线传输速率　VL 总线与 CPU 同步工作，两者的时钟在频率上和相位上均相同，最高主频可达 66MHz(实际上受制于 VL 总线扩充槽性能，不能超过 40MHz)。数据总线宽度为 32 位，可扩展到 64 位。因此，VL 总线的最大数据传输率可达 132MB/s。(注意，当使用双倍时钟的 CPU 如 386 类 CPU 时，必须将 CPU 时钟分频才能驱动 VL 总线时钟。)

(4)未来兼容性　如果旧版 VL 总线视频卡插入到新版 VL 总线主机上，系统会照常工作；同样，新版 VL 总线扩充卡也能够照常工作在旧版 VL 总线主机上。

(5)对未来回写式高速缓存的支持　目前的总线协议允许未来支持先进的回写式高速缓存系统(Write back Cache)。未来 VL 总线标准规范的修改版将包含有回写式高速缓存与主 CPU 一同工作所需协议的全面描述。按目前版设计的 VL 总线扩充卡将来照样能够正常工作在新版的 VL 总线系统上。

(6)总线扩充槽和总线主控设备数目　VL 总线设计成能够支持最多 3 个 VL 总线扩充槽，每个槽只能包含 1 个 VL 总线设备。因此系统总共能支持最多 3 个 VL 总线设备，每个总线设备都可以是主控设备。根据负载情况，有些 VL 总线可设计成无需加三总线缓冲而直接连至 CPU 总线上；而另一些 VL 总线则需要加三总线缓冲以满足驱动 3 个扩充卡的负载要求。表 2.4 给出了不同主频下 VL 总线扩充槽和设备的推荐数目。

表 2.4　不同主频下 VL 总线扩充槽和设备数目的推荐值

主　频	不加缓冲	加　缓　冲
不高于 33MHz	2 个扩充槽/2 个设备	3 个扩充槽/3 个设备
40MHz	1 个扩充槽/2 个设备*	未推荐
不高于 50MHz	0 个扩充槽/2 个设备*	未推荐

* 有的 VL 总线设备可不经扩充槽而直接位于主板上。

(7)总线扩充槽类型及物理布局　VL 总线扩充槽是一种标准的 16 位微通道型扩充槽。它在主板上应位于与系统 I/O 总线扩充槽的中心线对齐的位置上，如图 2.19 所示。这种物理布局便于 VL 总线扩充卡充分利用所有系统 I/O 总线特性，同一块扩充卡上即可以有 VL 总线逻辑，又可以借用部分系统 I/O 总线接口逻辑。

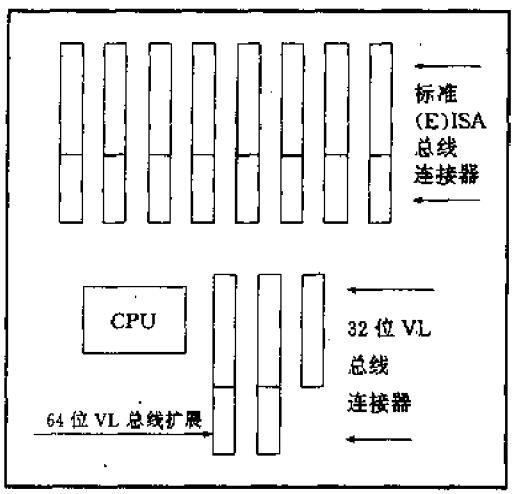


图 2.19　VL 总线物理布局图

2. VL 总线连接器的引脚信号

VL 总线连接器定义了 56×2＝112 根引脚信号，如表 2.5 所示。其中 A 面是扩充板的元件面，B 面是焊接面。各引脚信号的意义及使用要求从略，有需要者可参阅 VL 总线规范。

表 2.5 VL 总线连接器的引脚信号定义

引脚	信号名	引脚	信号名	引脚	信号名	引脚	信号名
A1	D1	B1	D0	A30	A16	B30	A17
A2	D3	B2	D2	A31	A14	B31	A15
A3	GND	B3	D4	A32	A12	B32	VCC
A4	D5	B4	D6	A33	A10	B33	A13
A5	D7	B5	D8	A34	A8	B34	A11
A6	D9	B6	GND	A35	GND	B35	A9
A7	D11	B7	D10	A36	A6	B36	A7
A8	D13	B8	D12	A37	A4	B37	A5
A9	D15	B9	VCC	A38	$\overline{\text{WBACK}}$	B38	GND
A10	GND	B10	D14	A39	$\overline{\text{BE0}}$	B39	A3
A11	D17	B11	D16	A40	VCC	B40	A2
A12	VCC	B12	D18	A41	$\overline{\text{BE1}}$	B41	N/C
A13	D19	B13	D20	A42	$\overline{\text{BE2}}$	B42	$\overline{\text{RESET}}$
A14	D21	B14	GND	A43	GND	B43	$\overline{\text{D/C}}$
A15	D23	B15	D22	A44	$\overline{\text{BE3}}$	B44	M/$\overline{\text{IO}}$
A16	D25	B16	D24	A45	$\overline{\text{ADS}}$	B45	W/$\overline{\text{R}}$
A17	GND	B17	D26	键位		键位	
A18	D27	B18	D28				
D19	D29	B19	D30	A48	$\overline{\text{LRDY}}$	B48	$\overline{\text{RDYRTN}}$
A20	D31	B20	VCC	A49	$\overline{\text{LEDV}}$	B49	GND
A21	A30	B21	A31	A50	$\overline{\text{LREQ}}$	B50	IRQ9
A22	A28	B22	GND	A51	GND	B51	$\overline{\text{BRDY}}$
A23	A26	B23	A29	A52	$\overline{\text{LGNT}}$	B52	$\overline{\text{BLAST}}$
A24	GND	B24	A27	A53	VCC	B53	ID0
A25	A24	B25	A25	A54	ID2	B54	ID1
A26	A22	B26	A23	A55	ID3	B55	GND
A27	VCC	B27	A21	A56	ID4	B56	VLCLK
A28	A20	B28	A19	A57	$\overline{\text{LKEN}}$	B57	VCC
A29	A18	B29	GND	A58	$\overline{\text{LEADS}}$	B58	$\overline{\text{LBS16}}$

(三)PCI 总线

PCI(Peripheral Component Interconnect)总线是继 VESA(VL)总线之后推出的一种高性能的 32 位局部总线(Local Bus)。

PCI 局部总线是 Intel 公司结构实验室于 1991 年底提出的。后来 Intel 公司又联合 IBM、DEC、Apple、Compaq、Motorola 等 100 多家 PC 工业界主要公司，于 1992 年 6 月成立了 PCI 集团(叫作 PCI SIG)，并组成专门小组，统筹、强化和推广 PCI 标准，使其成为开放的、非专利的局部总线标准，称之为 1.0 版标准。以后在应用实践的基础上又相继于 1993 年 4 月 30 日和 1995 年 6 月 1 日推出了 2.0 版和 2.1 版。

PCI 局部总线可同时支持多组外围设备，且不受制于微处理器，并能兼容现有的 ISA、EISA、微通道(MCA)等总线，与它们共存于 PC 系统中。

1. PCI 总线的主要特点

(1)性能优良

PCI 总线的时钟为 33MHz，与 CPU 时钟无关。它的总线宽度为 32 位，可扩充到 64 位，所以其带宽达 132MB/s～264MB/s。如果单从表面上看，VL 总线也能达到相似的速度，但实际上，PCI 总线可以比 VL 总线更快些，原因有两点：其一，总线支持无限读写突发方式。我们知

道，VL 总线实际上就是CPU 总线，在486CPU 的情况下，CPU 仅支持 16 字节的读突发方式，显然不及 PCI 总线。其二，PCI 总线支持并发工作，也就是 PCI 总线上的外围设备可与 CPU 并发工作。一般设计良好的 PCI 控制器（或称 PCI 桥路）有多级缓冲。例如 CPU 要访问 PCI 总线上的设备时，它可以把一批数据快速写入缓冲器中，当这些数据在不断写入 PCI 设备的过程中，CPU 可以去执行其他操作。这种并发工作提高了整体性能。此外，PCI 总线的潜伏期短且可以预测，一般为几十微秒，这样也提高了响应速度，并使扩充卡的设计更为方便。实验室测试结果表明，对于相同的系统（包括基于 i486 和 Pentium 处理器的系统），PCI 的性能是 EISA 的 3 倍以上，ISA 的 10 倍以上，比 VL 也要超出 10%～20%。

(2)灵活性、兼容性好

精密定义的 PCI 标准保证了它可与 ISA、EISA、MCA、VESA 总线兼容。由于 PCI 总线扩充卡的元件放置与一般 ISA 卡正好相反，这就可以使一个 PCI 总线扩充卡与一个 ISA 总线扩充卡共用一个位置（一个主板上可以有一个共享位置）。又由于 PCI 指标与 CPU 及时钟无关，使 PCI 总线扩充卡差不多成为一种通用卡，可插到任何一个有 PCI 总线的系统上去（不过实际上因卡上 BIOS 本身与 CPU 及操作系统等有关，不一定能做到如此通用。但至少，对同一类型 CPU 的系统，一般能够通用）。例如 PCI 总线扩充卡可以对所有 X86 体系结构的微机都通用，不论是 486 还是 Pentium CPU，也不管其时钟频率是多少。

此外，PCI 总线具有很大的灵活性，这是因为它在设计时就是针对多种外设，包括图形、磁盘控制器，网络、多媒体及其他扩展卡，而不像 VL 总线仅针对图形卡设计。

(3)自动配置，使用方便

近几年来，为了解决用户配置多种扩充卡所面临的多种难题，Intel、Microsoft 等公司提出了“即插即用”(Plug and Play)的概念，即要求做到任何扩充卡插入系统就能工作而不必设置开关或跳线。实际上也就是要求扩充卡具有自动配置的功能。PCI 总线的规范保证了自动配置的实现，因为按规范每个 PCI 总线扩充卡上都有 256 字节的空间用来存放自动配置信息，一旦它插入系统，系统 BIOS 将能根据读到的关于该扩充卡的信息，结合系统实际情况为扩充卡分配存储地址、端口地址、中断级和某些定量信息，从根本上免除了人工操作，给用户使用带来了极大方便。

(4)发展前途好

PCI 总线从一开始就是作为一种长期的总线标准加以制定的。例如它考虑到节能的绿色微机的要求，把支持 3.3V 的工作电压加入到规范中。为了便于从 5V 工作电压平稳地过渡到 3.3V，它容许在过渡时期设计一种通用扩充卡，既可插到 5V 工作的主板上，也可插到 3.3V 工作的主板上。另外，它可以容许从 32 位扩充到 64 位，而通用的扩充卡既可在 32 位的系统上工作，也可在 64 位的系统上工作。这些都是 PCI 的独到之处，它们决定了今天买到的 PCI 扩展卡可以在未来的任何 PCI 系统中运行。

(5)价格较低

PCI 总线不仅可提供高的性能，而且可降低系统成本，使用户受益。原因在于：

• 用于连接 PCI 总线的管脚数很少，对 PCI 外设来说，意味着其设计可以紧密集成；加之 PCI 扩展卡的外形尺寸较短，这样就可节省 PCB 板和元件的费用。

• PCI 被设计成一种无接合逻辑的总线，这意味着不再需要众多与开发 ISA、EISA 或 MCA 总线扩展卡相关的支持或缓冲芯片，从而降低了板级费用。

• 由于 PCI 可以在主板上与任何其他系统总线（如 ISA、EISA 或 MCA）相连接，制造商

们无须再为其扩展卡生产多种版本(即 1 种 ISA 卡,1 种 MCA 卡等)。因为一种卡即可适用于所有系统,所以可以实现大批量生产,降低生产成本。

· PCI 支持自动配置功能可节省系统集成的时间,间接降低系统研制、装配费用。

正因为 PCI 局部总线具有这么多突出的优点,所以一经推出便得到工业界的广泛支持和认可。正式标准公布不到一年,许多系统制造商就转而支持 PCI,使 PCI 接口的部件开发和应用达到可观的数量。目前各种 PCI 总线产品(包括总线标准基础产品和应用产品如图形控制器,IDE、SCSI I/O 控制器,网络适配器,多媒体声频/视频适配器及 PCI 数字信号处理器等)已广泛用于 PCI 总线个人计算机系统、PCI 总线网络服务器系统和 PCI 工作站系统中。计算机专家们普遍看好 PCI 总线,认为最终计算机市场上将是 PCI 总线的天下,PCI 总线将成为未来 PC 系统总线(而不单是局部总线)的新标准。

2. PCI 总线信号与总线连接器

PCI 总线的信号线包括两大类:必备的和可选的。PCI 接口要求的最少引脚数,对于只作为目标的从设备为 47 条,对于主设备为 49 条。只用这些信号线即可完成寻址、数据处理、接口控制、总线仲裁及其他系统功能。图 2.20 按功能分组表示了这些信号,左边是必备的,右边是可选的。图中的信号方向适用于主/从设备综合体的情况。当然,PCI 总线除了图 2.20 所示的这些信号线外,还有若干电源线、地线和保留线等,这些也是完成总线操作和方便未来系统/用户功能扩展所必要的。

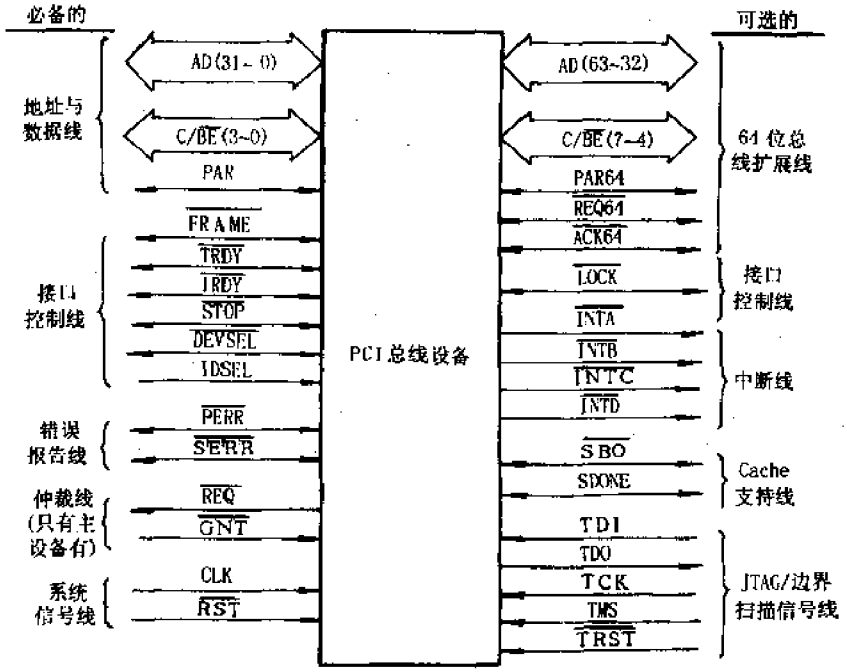


图 2.20 PCI 总线信号

PCI 总线规范定义了两种扩展板连接器,一种是 5V 信号环境,另一种是 3.3V 信号环境。同时定义了三种扩展板电气类型,即 5V 板、3.3V 板和 5V/3.3V 通用板。在连接器和板上都用了一套定位键(如图 2.21 所示),以防止把一块板插入到不适当的位置。每种 PCI 扩展板实际上又分为短型和长型两种,分别适用于 32 位和 64 位数据宽度。短型板连接器定义了 120 根引脚,长型板连接器定义了 184 根引脚。表 2.6 给出了 5V 和 3.3V 两种信号环境下 PCI 连接

器的引脚分配情况。各引脚信号的意义及使用要求请参阅“PCI 总线规范”，在此仅说明 3 点：

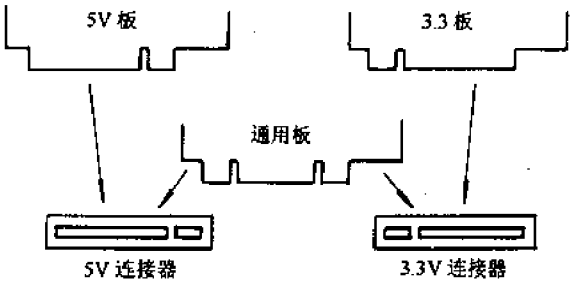


图 2.21　PCI 板与连接器

①引脚信号$\overline{\text{PRSNT1}}$(引脚 B9)和$\overline{\text{PRSNT2}}$(引脚 B11)是两个与连接器本身有关的“表示信号”。其用途有二：一是用来表明槽位上是否实际存在一块扩展板；二是表明该板对电源要求的有关信息。扩展板上这两个信号的设置定义如表 2.7 所示。

②对于不实现 JTAG/边界扫描的扩展板，必须将引脚 TDI(引脚 A4)和 TDO(引脚 B4)短接起来，以免扫描链断开。

表 2.6　PCI 连接器引脚分配

引脚号	5V 系统环境		3.3V 系统环境		备　注
	B 面	A 面	B 面	A 面	
1	−12V	$\overline{\text{TRST}}$	−12V	$\overline{\text{TRST}}$	32 位连接器开始
2	TCK	+12V	TCK	+12V	
3	GND	TMS	GND	TMS	
4	TDO	TDI	TDO	TDI	
5	+5V	+5V	+3.3V	+3.3V	
6	+5V	$\overline{\text{INTA}}$	+3.3V	$\overline{\text{INTA}}$	
7	$\overline{\text{INTB}}$	$\overline{\text{INTC}}$	$\overline{\text{INTB}}$	$\overline{\text{INTC}}$	
8	$\overline{\text{INTD}}$	+5V	$\overline{\text{INTD}}$	+3.3V	
9	$\overline{\text{PRSNT1}}$	保留	$\overline{\text{PRSNT1}}$	保留	
10	保留	+5V(I/O)	保留	+3.3V(I/O)	
11	$\overline{\text{PRSNT2}}$	保留	PRSNT2	保留	
12	GND	GND	键	位	3.3V 键位
13	GND	GND	键	位	3.3V 键位
14	保留	保留	保留	保留	
15	GND	$\overline{\text{RST}}$	GND	$\overline{\text{RST}}$	
16	CLK	+5V(I/O)	CLK	+3.3V(I/O)	
17	GND	$\overline{\text{GNT}}$	GND	$\overline{\text{GNT}}$	
18	$\overline{\text{REQ}}$	GND	$\overline{\text{REQ}}$	GND	
19	+5V(I/O)	保留	+3.3V(I/O)	保留	
20	AD[31]	AD[30]	AD[31]	AD[30]	
21	AD[29]	+5V	AD[29]	+3.3V	
22	GND	AD[28]	GND	AD[28]	
23	AD[27]	AD[26]	AD[27]	AD[26]	
24	AD[25]	GND	AD[25]	GND	
25	+5V	AD[24]	+3.3V	AD[24]	
26	C/$\overline{\text{BE}}$[3]	IDSEL	C $\overline{\text{BE}}$[3]	IDSEL	
27	AD[23]	+5V	AD[23]	+3.3V	

续上表

引脚号	5V 系统环境		3.3V 系统环境		备　注
	B 面	A 面	B 面	A 面	
28	GND	AD[22]	GND	AD[22]	
29	AD[21]	AD[20]	AD[21]	AD[20]	
30	AD[19]	GND	AD[19]	GND	
31	+5V	AD[18]	+3.3V	AD[18]	
32	AD[17]	AD[16]	AD[17]	AD[16]	
33	C/$\overline{BE}$[2]	+5V	C/$\overline{BE}$[2]	+3.3V	
34	GND	$\overline{FRAME}$	GND	$\overline{FRAME}$	
35	$\overline{IRDY}$	GND	$\overline{IRDY}$	GND	
36	+3.3V	$\overline{TRDY}$	+3.3V	$\overline{TRDY}$	
37	$\overline{DEVSEL}$	GND	$\overline{DEVSEL}$	GND	
38	GND	$\overline{STOP}$	GND	$\overline{STOP}$	
39	$\overline{LOCK}$	+3.3V	$\overline{LOCK}$	+3.3V	
40	$\overline{PERR}$	SDONE	$\overline{PERR}$	SDONE	
41	+3.3V	$\overline{SBO}$	+3.3V	$\overline{SBO}$	
42	$\overline{SERR}$	GND	$\overline{SERR}$	GND	
43	+3.3V	PAR	+3.3V	PAR	
44	C/$\overline{BE}$[1]	AD[15]	C/$\overline{BE}$[1]	AD[15]	
45	AD[14]	+3.3V	AD[14]	+3.3V	
46	GND	AD[13]	GND	AD[13]	
47	AD[12]	AD[11]	AD[12]	AD[11]	
48	AD[10]	GND	AD[10]	GND	
49	GND	AD[09]	GND	AD[09]	66MHz/GND
50	键	位	GND	GND	5V 键位
51	键	位	GND	GND	5V 键位
52	AD[08]	C/$\overline{BE}$[0]	AD[08]	C/$\overline{BE}$[0]	
53	AD[07]	+3.3V	AD[07]	+3.3V	
54	+3.3V	AD[06]	+3.3V	AD[06]	
55	AD[05]	AD[04]	AD[05]	AD[04]	
56	AD[03]	GND	AD[03]	GND	
57	GND	AD[02]	GND	AD[02]	
58	AD[01]	AD[00]	AD[01]	AD[00]	
59	+5V(I/O)	+5V(I/O)	+3V(I/O)	+3V(I/O)	
60	$\overline{ACK64}$	$\overline{REQ64}$	$\overline{ACK64}$	$\overline{REQ64}$	
61	+5V	+5V	+5V	+5V	
62	+5V	+5V	+5V	+5V	32 位连接器结束
	键	位	键	位	64 位空格
	键	位	键	位	64 位空格
63	保留	GND	保留	GND	64 位连接器开始
64	GND	C/$\overline{BE}$[7]	GND	C/$\overline{BE}$[7]	
65	C/$\overline{BE}$[6]	C/$\overline{BE}$[5]	C/$\overline{BE}$[6]	C/$\overline{BE}$[5]	
66	C/$\overline{BE}$[4]	+5V(I/O)	C/$\overline{BE}$[4]	+3.3V(I/O)	
67	GND	PAR64	GND	PAR64	
68	AD[63]	AD[62]	AD[63]	AD[62]	
69	AD[61]	GND	AD[61]	GND	
70	+5V(I/O)	AD[60]	+3.3V[I/O]	AD[60]	
71	AD[59]	AD[58]	AD[59]	AD[58]	
72	AD[57]	GND	AD[57]	GND	
73	GND	AD[56]	GND	AD[56]	

续上表

引脚号	5V 系统环境		3.3V 系统环境		备　注
	B 面	A 面	B 面	A 面	
74	AD[55]	AD[54]	AD[55]	AD[54]	
75	AD[53]	+5V(I/O)	AD[53]	+3.3V(I/O)	
76	GND	AD[52]	GND	AD[52]	
77	AD[51]	AD[50]	AD[51]	AD[50]	
78	AD[49]	GND	AD[49]	GND	
79	+5V(I/O)	AD[48]	+3.3V(I/O)	AD[48]	
80	AD[47]	AD[46]	AD[47]	AD[46]	
81	AD[45]	GND	AD[45]	GND	
82	GND	AD[44]	GND	AD[44]	
83	AD[43]	AD[42]	AD[43]	AD[42]	
84	AD[41]	+5V(I/O)	AD[41]	+3.3V(I/O)	
85	GND	AD[40]	GND	AD[40]	
86	AD[39]	AD[38]	AD[39]	AD[38]	
87	AD[37]	GND	AD[37]	GND	
88	+5V(I/O)	AD[36]	+3.3V(I/O)	AD[36]	
89	AD[35]	AD[34]	AD[35]	AD[34]	
90	AD[33]	GND	AD[33]	GND	
91	GND	AD[32]	GND	AD[32]	
92	保留	保留	保留	保留	
93	保留	GND	保留	GND	
94	GND	保留	GND	保留	64 位连接器结束

表 2.7　"表示信号"的定义

$\overline{\text{PRSNT1}}$	$\overline{\text{PRSNT2}}$	扩展板配置
开路	开路	不存在扩展板
地	开路	有扩展板，最大功耗为 25W
开路	地	有扩展板，最大功耗为 15W
地	地	有扩展板，最大功耗为 7.5W

③表中标着"+5V(I/O)"和"+3.3V(I/O)"的引脚是专用电源引脚，用于标识驱动通用板上的 PCI 信号环境电源线。在母板上，这些引脚分别连到+5V 或+3.3V 的主电源板上；通用板上 PCI 元件的 I/O 缓冲级则必须只从这些专用电源引脚供电，而不能从其他"+5V"或"+3.3V"电源引脚供电。

为了解决一般微机系统中主板上插槽数目有限的矛盾，PCI 总线标准还提供了与 ISA 总线共用的插槽，既可插 PCI 总线扩展卡，又可插 ISA 总线扩展卡，前提是 PCI 卡与一般 ISA 卡的元件面要正好相反。目前市场上销售的许多 Pentium 主板如 OCTEK(海洋)RHINO5、P/I－P55TP4、P/I－P55SP4 等和部分 486 主板如 M－423 等，就既提供了若干个独立的 PCI 总线插槽，又提供了 1～2 个 PCI/ISA 总线共用插槽。显然，这种 PCI/ISA 共用插槽(连接器)的引脚分配与机械规格都必须具有和 ISA 连接器的兼容性，在此不详述。

3. PCI 总线系统结构

PCI 总线系统的结构如图 2.22 所示。由图可见：

①驱动 PCI 总线所需的全部控制由 PCI 桥路实现。PCI 桥路实际上就是一个总线适配器，实现 CPU 总线与 PCI 总线之间的适配耦合。它提供了一个低延迟的访问通路，使 CPU 能直

接访问通过它映射于存储器空间或I/O空间的PCI设备。它在与CPU总线的接口中引入了FIFO缓冲器，使PCI总线上的设备可与CPU并行工作。另外，它还可使PCI总线和CPU总线的操作互相分离，以免相互影响。

②利用一种称为标准总线桥路的特殊模块，可将PCI局部总线转换为ISA、EISA、MCA等标准系统总线。例如Intel设计的SATUNG芯片组具有ISA标准总线桥路功能；而另一种为Pentium设计的MERCURY芯片组既可选择ISA标准总线桥路，也可选择EISA标准总线桥路。这样，在PCI总线系统中可继续使用原有的I/O设备，从而增强它的兼容性和选择范围。

③如果需要把许多设备接到PCI总线上，而总线驱动能力又不足时，可以采用多PCI总线。图2.22中共采用4级PCI桥路，形成了5组PCI总线和2组标准总线，这些总线都可并发工作，每组总线上都可接若干设备。

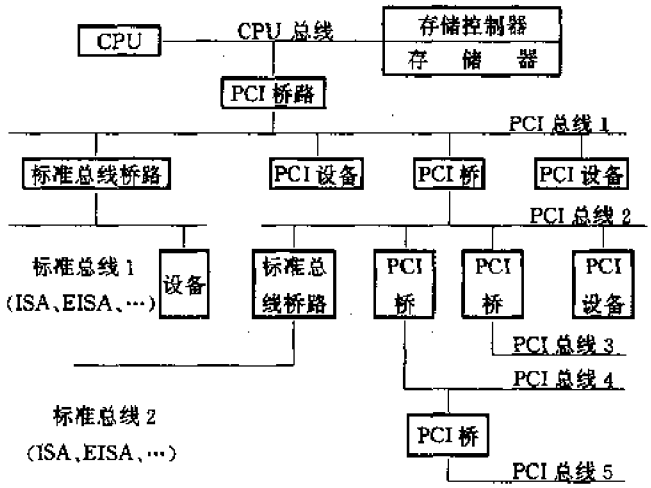


图2.22 PCI总线系统结构

思考题与习题

1. 与8086/8088相比，80386/80486在结构上和性能上有哪些主要改进？

2. 80486的综合性能比80386提高了3～4倍的主要原因是什么？

3. 简述80486 CPU的内部结构与各部件的功能。

4. 80486 CPU中有哪些基本寄存器，它们的主要功能是什么？

5. 标在寄存器EFLAGS中有哪些状态标志位？简述每个标志位的作用。

6. 存储器为什么要采用分段技术，80486在实地址方式下存储器如何分段？

7. 什么是逻辑地址？什么是物理地址？在实地址方式下如何求存储单元的物理地址？设一个16字的数据区，它的起始地为70A0H：DDF6H(段基址：偏移地址)，请写出这个数据区首字单元和末字单元的物理地址。

8. 堆栈的数据结构特点是什么？计算机中为什么要设置堆栈？

9. 80486有哪三种工作方式？简述每种工作方式的特点。

10. 什么叫总线？为什么微型计算机系统中普遍采用总线结构？

11. 试简述时钟周期、总线周期和指令周期的含义以及它们之间的关系。

12. 总线操作周期一般包含哪几个阶段？各个阶段的主要功能是什么？

13. 什么叫突发传送和非突发传送？80486的最快非突发单总线周期传送由几个时钟周期

组成？试根据时序图阐述总线操作的经过。

14. 什么是系统总线和局部总线？PC 系列微机中常用系统总线和局部总线有哪几种？它们各有什么特点？

第三章　80486的指令系统和寻址方式

计算机是通过执行指令来解决实际问题的。每种计算机都有一组由二进制代码编写的指令集提供给用户使用，这组指令集称为计算机的指令系统。指令集是在设计计算机的时候确定下来的，一旦设计完毕，指令集成为固有，一般不再改变。各种不同类型的计算机其指令系统是不同的。微型计算机的指令系统可包括几十条到上百条指令。

第一节　80486的数据类型

计算机执行指令过程中需要各种类型的数据，80486微机在其内部定点处理单元CPU和浮点处理单元FPU的支持下，可处理以下7种类型的数据。

1. 无符号二进制数

(1)字节：无符号8位数。

(2)字：2个相邻字节组成的无符号16位数。

(3)双字：2个相邻字，即4个相邻字节组成的无符号32位数。

这类数由CPU支持

2. 带符号二进制定点整数

此类数有正、负之分，均以补码表示，有8位数(字节)，16位数(字)，32位数(双字)、64位数(4字)四种。CPU只支持8位、16位和32位带符号整数。FPU支持16位、32位和64位整数。

3. 浮点数(实数)

80486微处理机中的浮点数由三个字段：符号位、有效数和阶码(即指数部分)组成，这类数由FPU支持，分单精度(32位)、双精度(64位)和扩展精度(80位)三种形式，如图3.1所示。单精度浮点数包括1位符号，8位阶码，24位有效数字(显示23位，外加1位隐含的整数"1.")。双精度浮点数包括1位符号，11位阶码，53位有效数字(显示52位，1位隐含的整数"1.")。扩展精度浮点数包含1位符号，15位阶码，64位有效数字(内含1位整数"1"，小数点"·"隐含)

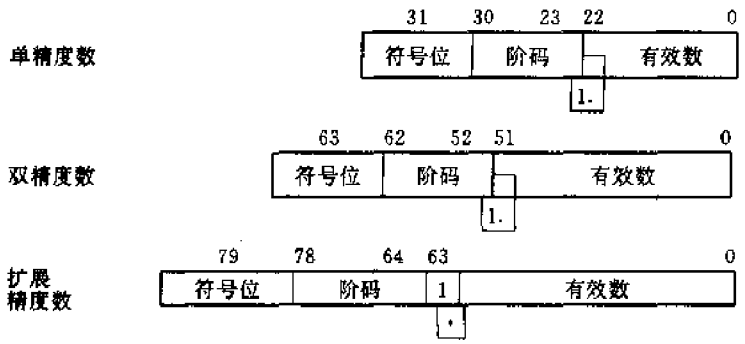


图3.1　浮点数格式

4. BCD 码

BCD 码有压缩 BCD 码和非压缩 BCD 码，压缩 BCD 码每字节包含二位十进制数，非压缩 BCD 码每字节包含一位十进制数。CPU 支持 8 位压缩和非压缩 BCD 码，FPU 只支持压缩 BCD 码，且最大长度为 80 位(10 个字节)，最多可处理 20 位 BCD 码。

5. 串数据

CPU 支持串数据，包括位串，字节串，字串和双字串。

(1)位串：一串连续的二进制数

(2)字节串：一串连续的字节

(3)字串：一串连续的字

(4)双字串：一串连续的双字

80486 微处理器中可处理的串数据最长可达 $2^{32}-1$ 字节。

6. ASC I 码数据

包括 AS I 码字符串和 ASC I 码数(0～F)两种

7. 指针数据类

32 位指针(近指针)：32 位偏移量，用于段内寻址。

48 位指针(远指针)：16 位选择符和 32 位偏移量，用于段间访问。

第二节　80486 的寻址方式

寻找指令中所需的操作数或操作数地址的方式称寻址方式。80486 微处理器各指令中所需操作数来自以下几个方面：

(1)操作数包含在指令中。在取指令的同时，操作数也随着取出。这种操作数称为立即数。

(2)操作数包含在 CPU 的某个内部寄存器中。由于该寄存器在 CPU 内部，因此取操作数也较简单。

(3)操作数在内存储器中。由于内存储器在 CPU 外，因此在寻找此种操作数时首先需要找到该操作数在内存中存放的地址。80486 系统中，任何内存单元的地址由段基址和偏移地址(又称偏移量)组成。段基址由段寄存器提供，而偏移地址则由以下几个基本部分组合而成：

- 基址寄存器
- 变址寄存器
- 比例因子
- 位移量

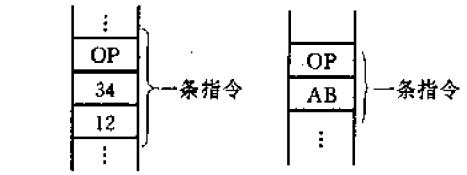
这四个部分称为偏移地址四元素。一般将这 4 种元素按某种计算方法组合形成的偏移地址称为有效地址 EA(Effective Address)。它们的组合方式和计算方法为：

$$EA=基址+(变址\times比例因子)+位移量$$

这 4 种元素可优化组合出 9 种有储器寻址方式，加上立即数寻址和寄存器寻址，80486 共有 11 种寻址方式。下面我们将详细讨论。

1. 立即数寻址(Immediate Addressing)

指令中所需操作数存放在指令中，放在操作码后。操作数可以是 8 位 16 或 32 位。见图 3.2。由于在指令执行过程中，立即数作为指令的一部分直接从指令预取单元中取出，不需要再访问存储器，因此这种寻址方式执行速度快。



(a)MOV AX,1234H　　(b)MOV AL,0ABH

图 3.2　立即数寻址方式

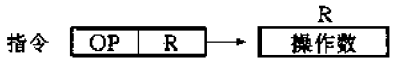


图 3.3　寄存器寻址

立即数寻址的不足之处是许多指令只允许源操作数使用立即数寻址方式，但目标操作数不允许出现立即数寻址方式。所以当某条指令中源操作数为立即数时，目标操作数必须是寄存器或存储器。

2. 寄存器寻址(Register Addressing)

指令中所需操作数在 CPU 某寄存器中(通用寄存器和段寄存器)，如图 3.3 所示。由于存取这类操作数时在 CPU 内部进行，所示执行速度快。其中 R 表示 8 位，16 位或 32 位寄存器

例如

MOV　EAX,EDX

MOV　DX,AX

INC　CL

3. 直接寻址(Direct Addressing)

操作数一般存放在存储器的数据段中，而操作数的有效地址 EA(16 位或 32 位整数)由指令给出。因此要取得操作数必须先求出操作数所在存储单元的物理地址，然后根据物理地址访问存储器才能得到此操作数。其过程如图 3.4 所示。如果操作数在数据段中，则物理地址＝(DS)×16＋EA。

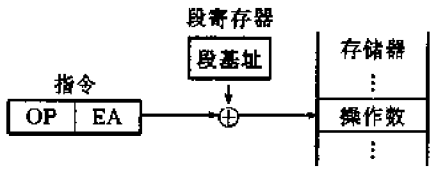


图 3.4　直接寻址方式

80486 微机允许数据存放在数据段以外的其它段中，此时应在指令中指定段跨越前缀，在计算物理地址时不再使用数据段寄存器 DS，而应使用指定的段寄存器。

例 1　MOV AX,DS:[2000H]

如果(DS)＝3000H，则执行情况如图 3.5 所示。物理地址＝3000H×16＋2000H＝32000H

通常"DS:"可省略，此例可写为 MOV AX,[2000H]

在 80486 汇编语言指令中，可以用符号地址来代替数值地址，如：

MOV AX,VALUE

此时 VALUE 为存放操作数的存储单元的符号地址。上述指令还可写为：MOV AX,[VALUE]

两者是等效的。

若设 VALUE 不在数据段中而在附加段中，则应指定段跨越前缀，如下：MOV AX,ES:VALUE

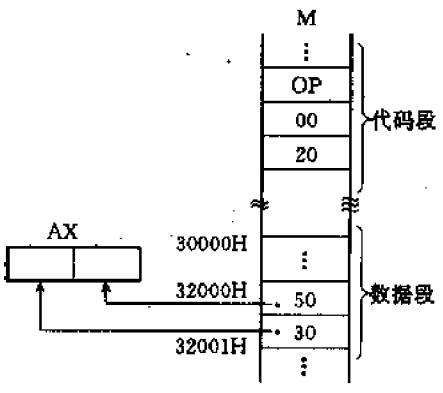


图 3.5　例/数执行情况

直接寻址方式适用于处理单个变量，例如要处理某个存放在存储器中的变量，可用直接寻址方式将变量送到一个寄存器中，然后再进一步处理。参看图 3.5。

4. 寄存器间接寻址方式(Register indirect Addressing)

操作数在存储器中,操作数的有效地址 EA 存放某个寄存器中,如图 3.6 所示。

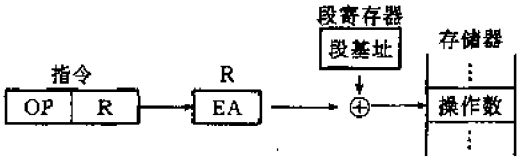


图 3.6 寄存器间接寻址方式

寄存器的使用在 16 位寻址和 32 位寻址时不一样。

(1)16 位寻址时,偏移地址放在 SI、DI、BX、BP 中。

如果指令中指定的寄存器是 BX,SI,DI,则操作数在数据段中,操作数的物理地址为:

$$\text{物理地址}=(DS)\times 16+\begin{cases}(BX)\\(SI)\\(DI)\end{cases}$$

如果指令中指定的寄存器是 BP,则操作数在堆栈段中,段基址在 SS 中,操作数的物理地址为:

$$\text{物理地址}=(SS)\times 16+(BP)$$

例 2 MOV AX,[BX]

若(DS)=2000H

(BX)=1000H

则物理地址=2000H×16+1000H

=21000H

执行情况如图 3.7 所示。

(2)32 位寻址时,8 个 32 位通用寄存器均可作寄存器间接寻址用。

例如 MOV EBX,[EAX]

MOV DX,[EBX]

除 ESP 和 EBP 默认段寄存器为 SS 外,其余 6 个通用寄存器均默认段寄存器为 DS。

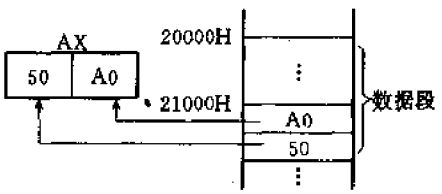


图 3.7 例 2 执行情况

寄存器间接寻址方式可用于表格处理,执行完一条指令后,只需修改寄存器内容就可取出表格中的下一项。

5. 基址寻址(Based Addressing)

操作数在存储单元中,操作数的有效地址 EA 由基址寄存器内容与指令中给出的位移量之和给出。

(1)16 位寻址情况下,BX 和 BP 作为基址寄存器。BX 以 DS 作为默认段寄存器,BP 以 SS 为默认段寄存器。位移量可为 8 位或 16 位

$$EA=\begin{cases}(BX)\\(BP)\end{cases}+\text{位移量(8 或 16 位)}$$

(2)32 位寻址情况下,8 个 32 位通用寄存器均可作基址寄存器,其中 ESP、EBP 以 SS 为默认段寄存器,其余均以 DS 为默认段寄存器。位移量为 8 位或 32 位

EA=8 个通用寄存器+位移量(8 位或 32 位)

16 位基址寻址方式如图 3.8 所示：

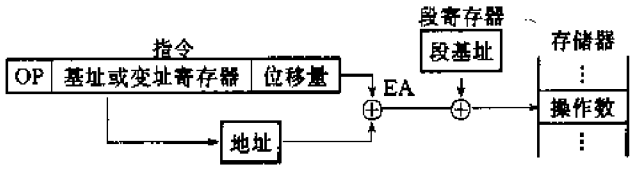


图 3.8　基址寻址和变址寻址

例 3　MOV DX,[EAX+1500H]

设(EAX)=2500 0000H,保护方式下求得 (DS)=40000000H

物理地址=(DS)+(EAX)+1500H

=65001500H

执行情况如图 3.9 所示

6. 变地寻址(Indexed Addressing)

变址寻址中,EA=(变址寄存器)+位移量。它的指令书写格式和寻址执行过程与基址寻址相同,区别仅是将基址寄存器改成变址寄存器

(1)16 位寻址时,仅 SI 和 DI 可作变址寄存器,默认 DS 为段基址寄存器。例如:

MOV AX,10H[SI]

MOV DX,COUNT[DI]

(2)32 位寻址时,除 ESP 外的其它 7 个寄存器均可作变址寄存器,EBP 默认 SS 作段基址寄存器,其余以 DS 作段基址寄存器。

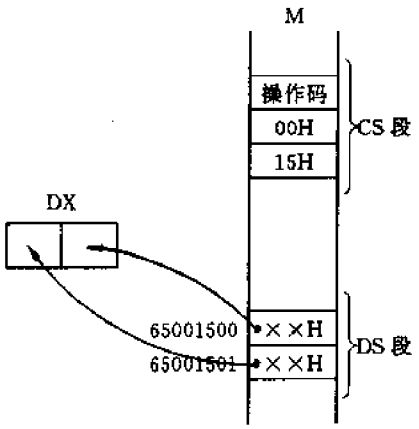


图 3.9　基址寻址

例如　MOV EAX,[EBP+50H]

MOV EDX,DATA[ECX]

基址寻址和变址寻址适用于对一维数组的数组元素进行检索操作。常用位移量表示数组的起始地址的偏移量,基址/变址寄存器表示数组元素的下标,可修改来获取数组元素的值。

7. 比例变址寻址(Indexed Proportion Addressing)

此种寻址方式中,EA=(变址寄存器)×比例因子+位移量

比例因子可取 1、2、4、8。这种寻地方式只适用于 32 位寻址方式。

例如:

MOV EAX,TABLE[ESI * 4]

上例中:TABLE 为位移量,4 是比例因子,ESI 乘以 4 的操作在 CPU 内部靠硬件来完成的。

8. 基址加变址寻址(Based Indexed Addressing)

这种寻址方式中,EA=(基址寄存器)+(变址寄存器)

例如　MOV CX,[BX+SI]

MOV EAX,[EDX][EBP]

同样,基址加变址寻址也有 16 位寻址和 32 位寻址两种情况,每种情况下基址寄存器和变址寄存器的使用规定和段寄存器的默认规定与前面所述相同,这里不再一一叙述。值得注意的是,当基址寄存器和变址寄存器默认的段寄存器不同时,我们如何来处理呢?

例如：MOV EAX,[EBP][ECX] 一般规定，此时由基址寄存器来决定默认的段寄存器为段基址寄存器，上例中，由于基址寄存器是 EBP，所以默认 SS 为段基址寄存器。

基址加变址寄存器主要用于二维数组的操作。

9. 基址加比例变址寻址

此种寻址方式中，EA＝(变址寄存器)＊比例因子＋(基址寄存器)

例如 MOV EAX,[EXC＊8][EDX]

此种方式只适用于 32 位寻址方式。其应用与基址加变址寻址相同。

10. 带位移的基址加变址寻址

这种寻址方式下，EA＝[变址寄存器]＋[基址寄存器]＋位移量

这种寻址方式也分 16 位寻址和 32 位寻址两种情况。变址、基址寄存器的使用约定和对段寄存器的约定与前面所述相同。格式举例：

MOV AX,[BX＋SI＋MASK]

ADD EDX,[ESI][EBP＋0FFF0000H]

11. 带位移的基址加比例变址寻址

其 EA＝(变地寄存器)×比例因子＋(基址寄存器)＋位移量

此种寻址方式只有 32 寻址一种情况。各种约定和默认情况同前所述。例如：

INC [EDI＊8][EDX＋40]

上面 11 种寻址方式实际上可分为二大类：(1)非存储器操作寻址方式，包括立即数寻址和寄存器寻址。这两种寻址方式不需要访问存储器，故执行速度较快。(2)访问存储器操作寻址方式，后九种都是这种方式。在进行访问存储器操作时，除要计算偏移地址 EA 外，还必须确定操作数所在的段，即确定有关的段寄存器。一般情况下，指令不特别指出段寄存器，是因为对于各种不同操作类型的存储器寻址，80486 约定了默认的段寄存器。有的指令允许段超越寻址，这时指令中应加上超越前缀。表 3.1 列出了不同的访问存储器操作类型对默认段寄存器，允许超越段寄存器和相应的偏移地址寄存器的约定情况。

从表中可看到，程序只能存放在代码段中，而且只能用 IP 作偏移地址寄存器。堆栈操作数只能在堆栈段中，而且只能用 SP 作偏移地址寄存器。串操作中目的操作数只能在附加数据段 ES 中，其它操作虽然也有默认段，但都是允许段超越的。

表 3.1 访问存储器时段和偏移地址寄存器约定

操作类型	默认段寄存器	允许超越的段寄存器	偏移地址寄存器
取指令代码	CS	无	(E)IP
堆栈操作	SS	无	(E)SP
通用数据访问	DS	CS、SS、ES、FS、GS	无特殊规定
以(E)BP、(E)SP 间接寻址的指令	SS	CS、DS、ES、FS、GS	无特殊规定
源串数据访问	DS	CS、SS、ES、FS、GS	(E)SI
目的串数据访问	ES	无	(E)DI

第三节 80486 的指令系统

80486 的指令系统是在 8086/8088、80286、80386 指令系统的基础上扩充而来的，在代码级具有向上兼容性。80486 增加了指令的种类，增强了一些指令的功能，并提供了 32 位寻址方式和 32 位操作方式。同时，由于 80486 中集成了 FPU 部件，所以其指令系统包含了浮点运算

指令。

80486 的指令的操作数可以是 8 位、16 位或 32 位，寻址宽度可以是 16 位或 32 位。当执行 8086/8088、80286 的 16 位指令代码时，操作数是 8 位或 16 位，寻址宽度为 16 位；而当执行 386、486 的 32 位指令代码时，其操作数是 8 位或 32 位，寻址宽度为 32 位。

一、80486 的指令格式

80486 采用变字节的指令格式，指令由 1～16 个字节组成，如图 3.10 所示。从图可看到，指令由 6 个字段组成，字段 1 为前缀字段，字段 2～6 为基本字段。下面将各个字段加以介绍。

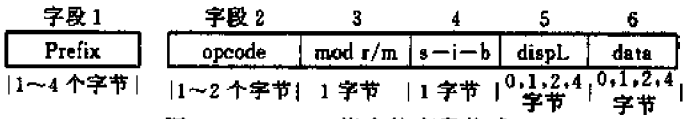


图 3.10　80486 指令的字段格式

1. 前缀字段

前缀字段用于修改指令操作的某些属性，为可选字段。常用的前缀有以下 5 类：

(1)段超越前缀：用于指明某个段寄存器取代指令中默认的段寄存器。

(2)操作数宽度前缀：用于改变当前操作数的默认宽度值。

(3)地址宽度前缀：用于改变当前地址的默认宽度值。

(4)重复前缀：用于重复串的操作。

(5)总线锁定前缀：用于产生 Lock 信号，防止其它设备中断 CPU 在总线上的操作。

每个前缀的编码为 1 个字节，在一条指令前可同时使用多个指令前缀。不同前缀的前后顺序是无关紧要的，但同类指令前缀用于同一指令前时，只有最后一个是有效的。当指令前缀过多而使指令长度超过 16 字节时，导致指令非法。

2. 操作码字段(OP Code)

此字段不可省略，它规定了指令的操作性质。

3. mod r/m 字段和 s－i－b 字段

此两个字段称为寻址方式说明符字段。mod r/m 为主寻址字节，而 s－i－b 为第二寻址字节。mod r/m 字节规定了操作数的寻址方式：包括操作数长度，采用的寻址方式，操作数的存放位置及 EA 的计算方法。s－i－b 为比例－变址－基址字节，它是否需要是由 mod r/m 字节规定的寻址方式来决定的。一般访问存储器的指令中都含有 mod r/m 字节。

4. 位移量字段(displ)

当寻址方式中需要位移量时，指令中将出现 displ 字段。其长度为 1，2，4 个字节，通常用带符号数表示。

5. 立即数字段(Data)

当寻址方式是立即数寻址，需用此字段。其长度为 1，2，4 个字节。立即数字段总是该指令的最后字段。

从上面可看到，除操作码字段为必选字段外，其它各字段都是可选字段，可根据不同的寻址方式选用。

二、80486 的指令系统

80486 的指令系统大致分为整数指令、浮点数指令和操作系统型指令三大类。本书只重点

介绍整数指令，要求读者对每条指令的助记符、操作数的正确书写格式和指令执行后对标志位的影响都能很好的掌握。掌握好指令是学习汇编语言程序设计的关键。对浮点数指令和操作系统型指令不作叙述，有需要的读者可查询有关资料。

80486 的整数指令按功能分为以下七种类型：

(1)数据传送指令

(2)算术运算指令

(3)逻辑运算指令

(4)串操作指令

(5)控制转移指令

(6)处理器控制指令

(7)高级和保护控制指令

由于 80486 的指令是在 8086/8088 指令系统的基础上发展而来。因此，本节先介绍 8086/8088 的指令系统，然后介绍 80486 的扩充指令。

(一)数据传送指令

数据传送指令将数据、地址或立即数(称源操作数)传送到寄存器或存储单元之中(称目标操作数)，完成寄存器与寄存器，寄存器与存储器、I/O 端口和累加器之间的字节、字或双字的传送。除 XCHG 等少量指令外，传送指令大多都不影响标志位寄存器的内容。8086/8088 数据传送指令可分成四种类型的指令，如表 3.2 所示。

表 3.2　数据传送指令

指令类型	指令格式	指令功能	备　注
通用数据传送	MOV 目标，源	将源字节/字/传送至目标	源：寄存器、存储器、立即数 目标：寄存器、存储器
	PUSH 源	将源字压入堆栈	源：寄存器、存储器
	POP 目标	从堆栈弹出字至目标	目标：寄存器(CS 除外)、存储器
	XCHG 目标，源	源、目标交换字节/字	源、目标：寄存器、存储器
目标地址传送	LEA 目标，源	装入有效地址	源：存储器，目标：通用寄存器
	LDS 目标，源	装入数据段指针到 DS	源：存储器，目标：通用寄存器
	LES 目标，源	装入附加段指针到 ES	源：存储器，目标：通用寄存器
标志位传送	LAHF	FR 低字节装入 AH	
	SAHF	AH 内容装入 FR 低字节	
	PUSHF	FR 内容压栈	
	POPF	弹出堆栈内容至 FR	
I/O 数据传送	IN 累加器，端口	从端口输入字节/字	累加器：AL、AX 端口：地址 0～255 或间址寄存器 DX
	OUT 端口，累加器	输出字节/字到端口	

1. 通用数据传送

(1)传送指令(MOV)

```
MOV  CL,   05H        ;字节传送，立即数送到 CL
MOV  [BX], 2FB4H      ;将 2FB4H 传送到 BX 为地址的两个存储单元，字传送
```

```
MOV  AL,   DL        ;寄存器传送,字节传送
MOV  DS,   AX        ;通用寄存器的段寄存器之间的字传送
MOV  [SI], BX        ;寄存器与存储单元间字传送
MOV  AX,   [2000H]   ;内存 2000H,2001H 内容传送到 AX,字传送
```

在应用传送指令传送数据时应当注意以下几点：

①立即数只能出现在源操作数位置

下面指令是错误的：MOV 15H,AL

②立即数不能直接传送到段寄存器

MOV DS,1000H 是错误的

可改为：MOV AX,1000H

MOV DS,AX

③源操作数和目标操作数不能同时为存储单元数据

MOV [BX],[1000H]是错误的，

可改为：MOV AX,[1000H]

MOV [BX],AX

④源操作数和目标操作数不能一个是字节，一个是字

例如 MOV AL,BX 是错误的

⑤两个段寄存器之间不能直接传送，段寄存器 CS 只能作源操作数。

例如 MOV DS,ES

MOV CS,AX 都是错误的

(2)压栈(PUSH)和出栈(POP)指令

```
PUSH  AX       ;将 AX 中 16 位数据压入堆栈
PUSH  [1000H]  ;将 1000H 和 1001H 中的 16 位数据压入堆栈
POP   AX       ;将堆栈中一个字内容弹出至 AX 中
POP   [1000H]  ;将堆栈中一个字内容弹出至 1000H 和 1001H 中
```

注意 目标操作数(POP 指令)和源操作数必须是 16 位寄存器或存储单元，并且低 8 位和堆栈低地址单元对应，高 8 位和堆栈高地址单元对应。

(3)交换指令 XCHG

例 XCHG AX,[BX+SI]

若指令执行前(AX)=6F30H,(BX)=0200H,(SI)=0046H

(DS)=2F00H,(2F246H)=4154H

源操作数的物理地址=(DS)×16+(BX)+(SI)

=2F000H+0200H+0046H

=2F246H

指令执行后：(AX)=4154H,(2F246H)=6F30H

注意：两个存储单元不能直接用交换指令

XCHG [1000H],[2000H] 是错误的

应改为

```
MOV AX,[2000H]
XCHG [1000H],AX
MOV [2000H],AX
```

2. 目标地址传送

(1)有效地址传送指令(LEA)

LEA 目标,源

该指令将源操作数的有效地址 EA 传数到目标操作数,源操作数必须是存储单元,目标操作数是 16 位通用寄存器

例 LEA BX,[SI+1005H]

若已知:(SI)=0054H,则 EA=(SI)+1005H=1059H

执行指令后,(BX)=1059H

(2)地址指针送 DS 和目标寄存器指令

LDS 目标,源

该指令将源操作数指定的四个存储单元中的内容送到目标寄存器和 DS 寄存器中。

例 LDS SI,[2130H]

指令执行后将 2130H,2131H 两个单元的内容作偏移地址送至 SI 中,将 2132,2133H 两个单元中的内容作段基址送至 DS 中。

(3)地址指针送 ES 和目标寄存器指令

LES 目标,源

此指令和 LDS 基本相同,不同处是段基址送至 ES 中。

此三条指令的用途是:LEA 指令用于表格处理中建立地址指针。LDS 和 LES 指令通常用于处理串操作。

3. 标志位传送指令

标志位传送指令用于标志寄存器的存取操作

(1)标志位送 AH 指令

指令格式:LAHF

指令功能:将标志位寄存器的低八位传送给 AH 寄存器。传送后 SF、ZF、AF、PF 和 CF 标志位送至 AH 寄存器的第 7、6、4、2、0 位。指令对标志寄存器各位内容无影响。

(2)AH 送标志位指令

指令格式:SAHE

指令功能:将 AH 内容送入标志寄存器的低八位。

(3)标志进栈指令

指令格式:PUSHF

指令功能:将 16 位标志寄存器 FR 的内容压栈保存。

(4)标志出栈指令

指令格式:POPF

指令功能:将堆栈顶部的一个字的内容弹出送入标志寄存器 FR 中。

标志寄存器只有少数标志位(CF,DF,IF)有专门的指令进行置"1"或清"0"操作,其余各位都没有直接指令对它们进行设置或修改。当需要对这些标志位进行操作时,可用 LAHF 和 SAHF 实现。当标志位在操作中需要保护时,可用 PUSHF 和 POPF 来实现。

4. I/O 数据传送指令

I/O 数据传送指令用于 I/O 端口与 CPU 之间的通信。传送中 CPU 只能用累加器 AL 和 AX 接收或发送信息。外部设备最多可有 65536 个 I/O 端口。端口地址从 0000H~FFFFH。其

中前 256 端口(0000～00FFH)可以用指令直接指定。当端口号大于 256 时,需将端口号装入 DX 寄存器中,再用 I/O 数据传送指令传送。

例 (1)IN AX,28H

(2)OUT 15H,AL

(3)MOV DX,03FCH

IN AX,DX

5.80486 的传送指令中增加的指令和功能

(1)MOV,XCHG,PUSH,POP,LEA 指令中寄存器,存储器和立即数允许用 32 位的双字进行传送

(2)IN,OUT 指令中可用 EAX 累加器进行交换数据

例如 OUT DX,EAX

IN EAX,28H

(3)全部通用寄存器内容压栈和弹出指令 PUSHA/PUSH AD 和 POPA/POPAD

指令格式:	功能
PUSHA	将 16 位通用寄存器内容压栈,顺序为 AX,CX,BX,DX,SP,BP,SI,DI
PUSHAD	将 32 位通用寄存器内容压栈,顺序为 EAX、ECX、EBX、EDX、ESP、EBP、ESI、EDI
POPA	弹出堆栈中保存的 16 位通用寄存器内容,顺序和 PUSHA 相反
POPAD	弹出堆栈中保存的 32 位通用寄存器内容,顺序和 PUSHAD 相反

(4)字节交换指令 BSWAP

指令格式		功能
BSWAP	寄存器(32 位)	将指定的 32 位寄存器中的双字以字节为单位进行互换。位 31～24 与 7～0,位 23～16 与 15～8 互换。

(5)标志位操作指令

80486 增加了	PUSHFD	将 32 位 EFLAGS 内容压栈,操作后,SP 减 4。
	POPFD	将栈顶 32 位数据弹出到 EFLAGS 中

(6)LDS,LES,LSS 指令中,目标操作数可以是 32 位通用寄存器,源操作数可以是 32 位或 48 位指针。另外增加了 LFS 和 LGS 指令。

(7)扩展传送指令 MOVSX 和 MOVZX

格式: MOVSX / MOVZX 目标,源;　源:8 位或 16 位寄存器或存储器　目标:16 位或 32 位寄存器

功能:①MOVSX 为带符号扩展传递指令。其作用是将寄存器或存储器中的 8 位或 16 位符号数带符号扩展成 16 位或 32 位带符号数,然后传递给 16 位或 32 位寄存器。

②MOVZX 为零扩展传送指令。其作用是将寄存器或存储器的 8 位或 16 位无符号数高位加 0 扩展成 16 位或 32 位,然后传送给 16 位或 32 位寄存器。

例如 MOVSX ECX,AL;将 AL 内容带符号扩展为 32 位送入 ECX

MOVZX EAX,CX;将 CX 中 16 位无符号的高位加 0 扩展为 32 位后送入 EAX

(二)算术运算指令

8086/8088 指令系统中有加、减、乘、除等算术运算指令,可进行字或字节的运算。参加运算数可以是二进制数和十进制数,这些数可以是无符号数,也可以是带符号数。算术运算指令

影响标志寄存器的状态标态位。表 3.3 给出了 8086/8088 指令系统中算术运算类指令。

表 3.3 算术运算指令

指令类型	指令格式	指令功能	备 注
加 法	ADD 目标,源	源和目标相加(字/字节)结果送目标	源:通用寄存器、存储器、立即数 目标:通用寄存器、存储器
	ADC 目标,源	源、目标和 CF 相加(字/字节)结果送目标	
	INC 目标	目标加 1 后送目标	
减 法	SUB 目标,源	目标减源(字/字节)结果送目标	源:通用寄存器、存储器、立即数 目标:通用寄存器、存储器
	SBB 目标,源	目标减源和 CF(字/字节)结果送目标	
	DEC 目标	目标减 1 后送目标	
	NEG 目标	目标操作数取补送目标	
	CMP 目标,源	目标减源,结果不送目标,只置标志位	
乘 法	MUL 源	不带符号乘法,被乘数隐含在 AL/AX 中	源:通用寄存器、存储器,字节乘法时,结果存入 AX 中,字乘法时,结果存入 DX:AX 中
	IMUL 源	带符号数乘法,被乘数隐含在 AL/AX 中	
除 法	DIV 源	不带符号除法,被除数隐含在 AX 或 DX:AX 中	源:通用寄存器,存储器,字节除法,商 AL 中,余数 AH 中,字除法,商 AX 中,余数 DX 中
	IDIV 源	带符数数除法,被除数隐含在 AX 或 DX:AX 中	
	CBW	字节转换成字,扩展后变 16 位被除数	
	CWD	字转换成双字,扩展后变 32 位被除数	
十进制调整	DAA	加法的十进制调整(压缩 BCD 码)	
	AAA	加法的十进制调整(非压缩 BCD 码)	
	DAS	减法的十进制调整(压缩 BCD 码)	
	AAS	减法的十进制调整(非压缩 BCD 码)	
	AAM	乘法的非压缩 BCD 码调整	
	AAD	除法的非压缩 BCD 码调整	

1. 加法指令

(1)不带进位位的加法指令

```
ADD AL,50H              ;AL 和 50H 相加,结果存入 AL
ADD AX,1000H            ;AX 和 1000H 相加,结果存入 AX
ADD BX,DX               ;BX 和 DX 相加,结果存入 BX
ADD [BX+SI],AX          ;BX+SI 和 BX+SI+1 所指的两个存储单元和 AX 相加,结果
                         存入 BX+SI,BX+SI+1 中
ADD AL,[DI+1000H]       ;DI+1000H 所指的存储单元和 AL 相加,结果存入 AL 中
```

(2)带进位位的加法指令

```
ADC AX,BX               ;AX 和 BX 中内容和 CF 值相加,结果存入 AX 中
ADC DX,[SI]             ;DX 和 SI 和 SI+1 所指存储单元及 CF 内容 相加,结果存
                         入 DX
ADC AL,5                ;AL 和 5 及 CF 相加,结果存入 AL 中
```

(3)加 1 指令

```
INC AL                          ;将 AL 内容加 1 存入 AL
INC DX                          ;将 DX 内容加 1 存入 DX
INC BYTE PTR[BX+DI+500]         ;将 BX+DI+500 所指存储单元的内容加 1,存入 BX
                                 +DI+500 单元中
```

2. 减法指令

(1)不带借位的减法指令

```
SUB BX,CX                    ;BX 减去 CX 中内容,结果送 BX
SUB AL,20                    ;AL 减去 20,结果送 AL
SUB WORD PTR [DI],1000H      ;将 DI,DI+1 两单元中的 16 位数减去 1000H 后,结果
                              送 DI 和 DI+1
```

(2)带借位的减法指令

```
SBB AX,2000H          ;AX 减 2000H 和 CF,结果送 AX
SBB [BP+2],CL         ;将堆栈段 BP+2 所指的有储单元内容减去 CL 和 CF,结果送
                       BP+2 单元
```

(3)减 1 指令

```
DEC AX                  ;AX 内容减 1 结果送 AX
DEC CL                  ;CL 内容减 1 结果送 CL
DEC BYTE PTR[SI+2]      ;SI+2 所指存储单元中内容减 1 后结果存入 SI+2 中
```

(4)取补指令

```
NEG AL          ;将 AL 中的数取补后送回 AL
NEG DX          ;将 DX 中的数取补后送回 DX
```

(5)比较指令

```
CMP AX,0              ;AX 内容与 0 比较
CMP AL,50H            AL 内容与 50H 比较
CMP BX,DX             ;BX 与 DX 比较
CMP AX,[SI+100H]      ;AX 与 SI+100H 和 SI+101H 为地址的存储单元比较
```

比较指令与减法指令的不同点是,比较指令虽然也是由目标操作数减去源操作数,但结果不存入目标操作数,只是影响标志位。比较指令执行后,源和目标操作数都不变,适用于两个数比较大小。

3. 乘法指令

(1)不带符号乘法　用于 2 个不带符号数相乘

```
MUL    BL      ;BL * AL   结果存入 AX 中(AH 放高位)
MUL    SI      ;SI * AX   结果存入 DX,AX 中(DH 放高位)
```

(2)带符号乘法　用于 2 个带符号数相乘

```
IMUL CL                    ;CL * AL,结果存入 AX
IMUL WORD PTR[DI+10]       ;DI+10,DI+11 所指的单元中的 16 位数乘以 AX,结果
                            存入 DX,AX 中
```

4. 除法指令

(1)不带符号除法　两个不带符号相除

```
DIV DL          ;AX÷DL,商在 AL 中,余数在 AH 中
DIV SI          ;DX 和 AX 中 32 位数除以 SI,商在 AX 中,余数在 DX 中
```

(2)带符号除法

```
IDIV BX              ;DX 和 AX 中 32 位数除以 BX,商在 AX,余数在 DX 中
IDIV BYTE PTR[SI]    ;AX 除以 SI 所指单元中的 8 位数,商在 AL 中,余数在 AH 中
```

5. BCD 码运算调整指令

在8086指令系统中，采用“8421”码进行十进制数的存储和运算。表示十进制数可用压缩的BCD码或非压缩的BCD码两种格式。压缩的BCD码用4位二进制数表示一个十进制数位，每个字节表示两位十进制数。非压缩的BCD码用8位二进制数表示一个十进制数位，8位的低4位是以8421码表示的十进制数，高4位则没有意义。例如ASCII码就是一种非压缩BCD码。

十进制调整指令共六条：

DAA　（加法的十进制调整指令）

AAA　（加法的ASCII调整指令）

DAS　（减法的十进制调整指令）

AAS　（减法的ASCII调整指令）

AAM　（乘法的ASCII调整指令）

AAD　（除法的ASCII调整指令）

1. DAA(Decimal adjust for addition)

格式：DAA

功能：将AL寄存器中的加法运算结果调整为二位压缩BCD码。此指令前必须执行ADD或ADC指令，结果放在AL中。调整的方法为：(1)若AF=1或AL寄存器低4位是十六进制的A～F，则AL寄存器内容加06H。(2)若CF=1或AL寄存器高4位是十六进制数的A～F，则AL寄存器内容加60H。

DAA指令除对OF标志无影响外，其它各标志位都按结果置位。

例　ADD AL,BL

DAA

(1)如果指令执行前(AL)=28,(BL)=68

```
   2 8                  0 0 1 0 1 0 0 0
+) 6 8               +) 0 1 1 0 1 0 0 0
   9 6                  1 0 0 1 0 0 0 0
```

执行ADD指令后(AL)=90 CF=0,AF=1,结果错误。

执行DAA指令后，由于AF=1,AL寄存器内容加06H进行调整，(AL)←(AL)+06H

结果(AL)=96　　正确

(2)如果指令执行前(AL)=66,(BL)=92

```
   6 6                  0 1 1 0 0 1 1 0
+)   9 2              +) 1 0 0 1 0 0 1 0
   1 5 8                1 1 1 1 1 0 0 0
```

执行ADD指令后

(AL)=F8H,CF=0,AF=0

执行DAA指令后，由于AL高4位出现非法码F，应当加60H进行调整

(AL)←(AL)+60

```
     1 1 1 1 1 0 0 0
+)   0 1 1 0 0 0 0 0
   1 0 1 0 1 1 0 0 0
```

结果(AL)=158　　正确

2. AAA(ASCII Adjust for Addition)

格式:AAA

功能:将 AL 中的加法运算结果调整为非压缩格式的 BCD 码。如果向高位有进位(AF=1),则进到 AH 寄存器中。

这条指令不能单独使用,应当跟在 ADD 或 ADC 指令之后,加法指令必须是两个非压缩的 BCD 码相加。AAA 指令的调整过程与 DAA 指令大体相同,但不需要对高 4 位进行调整,而只对低四位进行调整。调整后清除 AL 寄存器的高 4 位,若 AF=1,则将 AF 的值送 CF 位。

例　ADD AL,BL

　　AAA

若指令执行前　　(AL)=35H,(BL)=39H,它们分别是 ASCII 的'5'和'9'。

第一条指令执行后,低四位相加　　0101 + 1001 = 1110　　AF=0

第二条指令执行后对低四位调整　　1110 + 0110 = 1 0100　　AF=1

清除 AL 高 4 位

得到结果(AL)=04H

　　(AH)=01H

　　AF=1,CF=1

AAA 指令只影响 AF 和 CF 位,其它标志位无定义。

3. DAS(Decimal Adjust for Subtraction)

格式:DAS

功能:将 AL 寄存器中的减法运算结果调整为二位压缩 BCD 码。DAS 不能单独使用,总是跟在 SUB 和 SBB 指令之后。

调整方法为:(1)当 AF=1(表示低位向高位有借位)或 AL 低四位出现非法码 A～F,则需调整,将 AL 寄存器的内容减去 06H,并将 AF 位置"1"。(2)当 CF=1(表示高位向高字节有借位)或 AL 高四位出现非法码 A～F,将 AL 寄存器的内容减去 60H,并将 CF 位置"1"。

DAS 指令对 OF 标志无定义,但影响其它各标志位。

例　SUB AL,AH

　　DAS

若指令执行前(AL)=86,(AH)=07

执行 SUB 指令后(AL)=7FH,AF=1　　86 − 07 = 79　　10000110 − 00000111 = 01111111

执行 DAS 指令后,因 AF=1,(AL)=(AL)−6=79

AF=1,CF=0

4. AAS(ASCII Adjust for Subtraction)

格式:AAS

功能:将 AL 寄存器的减法运算结果调整为非压缩的 BCD 码。此指令总是跟在 SUB 或 SBB 指令后,减法指令必须把两个非压缩的 BCD 码相减,并将结果存放在 AL 寄存器中。

AAS 指令的调整过程与 DAS 指令相似，但 AAS 只对低四位进行调整。当 AL 寄存器的低四位出现非法数据 A～F，或 AF 位为 1，则把 AL 寄存器内容减去 06H，AH 寄存器内容减 1，将 AF 位置 1，调整后要清除 AL 寄存器的高 4 位，并将 AF 位的值送 CF 位。

AAS 指令不影响 AF 和 CF 标志位以外的其它标志位。

5. AAM(ASCII Adjust for Multiplication)

格式：AAM

功能：将 AL 寄存器的乘法运算结果调整为非压缩的 BCD 码。此指令总是跟在 MUL 指令后，MUL 指令执行两个非压缩的 BCD 码相乘，并将结果存放在 AL 寄存器中。在乘法前应将两个操作数高 4 位置 0，可以用 AND 指令实现。

AAM 的调整方法是：把 AL 寄存器的内容除以 0AH，商放在 AH 寄存器中，余数保存在 AL 寄存器中。本指令根据 AL 寄存器的内容设置条件码 SF，ZF 和 PF；OF，CF 和 AF 位无定义。

例
```
AND AL,0FH
AND BL,0FH
MUL AL,BL
AAM
```

设 (AL)＝'9'

(BL)＝'5'

在计算中这样来实现

(1)两数相乘

```
      00001001 (AL)
×     00000101
        1001
      1001
     ---------
     00101101 (AL)
```

(2)调整

```
        00000100   →(AH)
1010 ) 00101101
       1010
       ------
          101     →(AL)
```

结果(AX)＝45 正确

AX

0100	0101
4	5

6. AAD(ASCII Adjust for Division)

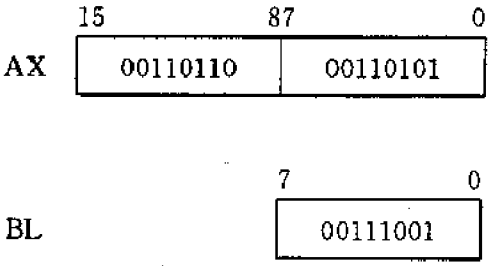
格式：AAD

功能：将被除数调整为二进制数，并存放在 AL 寄存器中。AAD 指令和加法、减法乘法调整指令不同，它的调整是在除法运算之前进行。即先对非压缩 BCD 码的被除数进行调整操作，将它调整为二进制数，然后进行二进制除法运算，得到十进制的商和余数。具体调整过程为：若设累加器 AX 中存放二位非压缩 BCD 码的被除数，将 AH 中的高位十进制数乘以 10 与 AL

中的低位十进制数相加，结果存放在 AL 寄存器中。然后再进行除法得到十进制的商和余数。

本指令根据 AL 寄存器的结果设置 SF、ZF 和 PF 位，OF，CF 和 AF 无定义。

例 设累加器 AX 中有二位被除数 65(ASCII 码)，在 BL 寄存器中有一位除数为 9(ASCII 码)，如下所示：



现要求将两数相除，商保留在 AL，余数保留在 AH 中，则指令如下：

```
AND BL,0FH          ;将 BL 高位置 0，变为非压缩 BCD 码
AND AX,0F0FH        ;将 AX 中低 8 位中的高 4 位，高 8 位中的高 4 位置 0。
AAD
DIV  BL
```

其调整运算如下：

```
    00000110  (AH)
  ×     1010
    ---------
      01100
     0110
    ---------
    00111100
  +  00000101
    ---------
    01000001  (AL)
```

```
              0111  →AL
          ---------
1001 ) 01000001
         1001
         ----
          1110
          1001
          ----
           1011
           1001
           ----
           0010  →AH
```

结果商(AL)=7　　余数(AH)=2 正确

• 80486 算术运算指令

(1)ADD、ADC、SUB、SBB、INC、DEC、CMP、MUL、IMUL、DIV、IDIV 指令中寄存器、存储单元和立即数可以用 32 位进行运算。

(2)符号位扩展指令　CWDE、CDQ

CWDE　将 AX 中 16 位有符号数带符号扩展成 32 位存放在 EAX 中，高 16 位与原符号位值相同。

CDQ　将 EAX 中的 32 位有符号数带符号扩展成 64 位数，扩展后的高 32 位放在 EDX 中，且都与原符号位值相同。

(3)交换加法指令　XADD

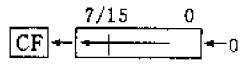
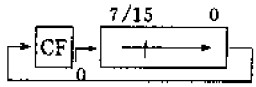
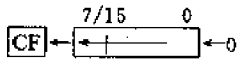
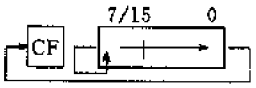
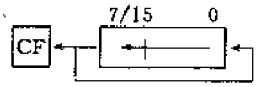
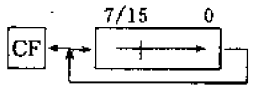
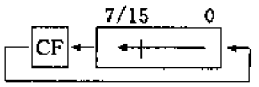
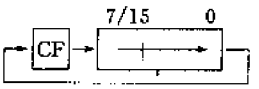
指令格式　　功能

XADD　目标，源　　将源与目标中的 8、16、32 位寄存器或存储单元值相加，结果存于目标中，并将目标的原值交换到源中。

(三)逻辑运算

逻辑运算指令分为逻辑运算和移位两大类，如表 3.4 所示。

表 3.4　逻辑运算和移位指令

指令类型	指令格式	指令功能	备　注
逻辑运算	NOT　目标	目标(字/字节)取非后送目标	源：通用寄存器、存储器、立即数 目标：通用寄存器、存储器
	AND 目标，源	源和目标相与后送目标	
	OR 目标，源	源和目标相或后送目标	
	XOR 目标，源	源和目标异或后送目标	
	TEST 目标，源	源和目标相与，结果不送目标	
移位和循环移位	SHL 目标，计数值	逻辑左移 	目标：通用寄存器和存储器 计数值：1 或 CL(>1 时) 决定移位次数 当大于 1 时，将数送入 CL
	SHR 目标，计数值	逻辑右移 	
	SAL 目标，计数值	算术左移 	
	SAR 目标，计数值	算术右移 	
	ROL 目标，计数值	循环左移 	
	ROR 目标，计数值	循环右移 	
	ROL 目标，计数值	带进位循环左移 	
	RCR 目标，计数值	带进位循环右移 	

1. 逻辑运算指令

(1)NOT 指令

NOT 指令对标志位无影响

例　MOV AL，0FH

　　NOT AL

执行结果(AL)＝0FOH

(2)AND 指令

AND 指令可用来屏蔽操作数的某些位，操作时将源操作数设置成立即数，需要屏蔽的位设为 0，AND 指令对 SF、PF、ZF 有影响，AF 任意，CF＝OF＝0

例　屏蔽 AX 寄存器的高八位

　　设(AX)＝2450H

　　AND　　AX，00FFH；结果 AX＝0050H

(3)OR 指令

OR 指令可用于对某位置 1 或合并字段，并对 SF，PF，ZF 有影响，AF 任意 CF＝OF＝0

例　设(AL)＝40H　(BL)＝02H

OR AL,BL

执行后(AL)=42H

```
        01000000  (AL)
OR)     00000010  (BL)
        01000010→AL
```

(4)XOR 指令

XOR 指令可用于将操作数的某些位求反、需求反的位置 1，XOR 指令对 SF、PF、ZF 有影响，AF 任意，CF=OF=0

例 设(AL)=11H

XOR AL,0FH

执行后(AL)=1EH，即对低 4 位操作数求反

```
        00010001  (AL)
XOR)    00001111
        00011110→AL
```

(5)TEST 指令

TEST 指令与 AND 指令一样将两个操作数按位相与，区别在于 TEST 指令不回送结果到目标操作数，仅仅影响标志位

例 测试寄存器 AL 中的操作数的最高位是否为 0

```
TEST AL,80H
```

指令执行后，判断 SF 是否为 0，即可说明 AL 的最高位的状态

2. 移位和循环移位指令

(1)移位指令

移位指令对 OF,SF,ZF,PF,CF 有影响，AF 任意

例

```
SAL DX,1        ;将 DX 中的值左移 1 位，最低位补 0
SAL AX,CL       ;将 AX 的值左移若干位，左移的次数由 CL 决定
SHR AL,CL       ;将 AL 的值右移若干位，右移的次数由 CL 决定
```

算术移位指令在执行时，将操作数看作带符号数来进行移位；逻辑移位指令在执行时，将操作数看成无符号数来进行移位。

SHL 和 SAL 两条指令的功能完全相同，每移一次，最低位补 0，最高位进入 CF，对带符号数来说移位后，CF 的值若和最高位不同，则 OF=1 表示溢出。

SAR 和 SHR 两条指令的功能有区别。SAR 指令在执行时最高位保持不变，表示符号位不变，而 SHR 指令在执行时，最高位补 0。

通常左移 1 位相当于乘 2，右移 1 位，相当于除 2，下例是将 AL 中的数乘以 10。

```
SAL AL,1        ;AL 左移 1 位，结果为 2xAL
MOV BL,AL       ;BL=2xAL
MOV CL,2
SAL AL,CL       ;AL 左移 2 位，结果为 8xAL
ADD AL,BL       ;AL 结果为 10xAL
```

(2)循环移位

循环移位指令只对标志位 OF 和 CF 有影响

循环移位指令在执行一次左移时，移位结果使最高位(符号位)发生变化，则 OF=1，若循环移多位时，OF 无效。而 CF 标志总是保持移出的最后一位的状态。

例 1

```
ROL  BL,1                ;BL 中内容不带进位位循环左移 1 位
RCL  WORD PTR[DI],CL     ;DI 和 DI+1 所指单元中的 16 位数带进位位循环左移若干次，次数由 CL 决定
```

例 2 将 DX,AX 中的双字左移 1 位

SAL AX,1

RCL DX,1

·80486 逻辑运算指令

(1)AND、OR、XOR、TEST、NOT、SAL、SHL、SAR、SHR、ROL、ROR、RCL、RCR 中寄存器和存储单元可以用 32 位数进行操作。80486 移位次数不允许大于 31,并可直接用立即操作数。

(2)双精度左移/右移指令　SHLD/SHRD

指令格式

SHLD/SHRD 目标 1,目标 2,移位位数(或 CL)

指令功能:将目标操作数 1 和 2 的两个 16 位或 32 位数连接成双精度值(32 位或 64 位),按移位位数向左或右移位。移位时将目标操作数 2 的内容移入目标操作数 1,而目标操作数 2 不变。

例:SHLD　TABLE〔EBX〕,EAX,8　　;将 TABLE〔EBX〕指向的双字单元的内容左移 8 位,空位由 EAX 的内容左移 8 位填入,但 EAX 中的内容不变。

(四)串操作指令

串操作指令就是用一条指令实现对一串字符或数据的操作。"串操作指令对向量或数组运算提供支持,可有效地简化程序。表 3.5 给出了 8086/8088 指令系统中串运算类指令

串操作指令中,串比较指令(CMPSB/CMPSW)和串扫描指令(SCASB/SCASW)对标志位 OF,SF,ZF,AF,PF,CF 有影响。其它指令对标志位均无影响。

表 3.5　串运算类指令

指令类型	指令格式	指令功能	备　　注
串传送	MOVSB MOVSW	从源串中取一个字节/字送目标串	1. 源串由 DS 提供段基址,SI 提供偏移地址。目标串由 ES 提供段基址,DI 提供偏移地址。 2. 待处理的串长度存放在 CX 寄存器中,每执行一次指令,自动对 SI 和 DI 进行修改。 3. 修改的方向由方向标志 DF 决定,当 DF=0,串操作由低地址向高地址进行,SI 和 DI 内容+1(字节)或+2(字)修正。当 DF=1 时,则串操作由高地址向低地址进行,SI 和 DI 内容−1(字节)或−2(字)修正。
串比较	CMPSB CMPSW	目标串中一个字节/字减源串中一个字节/字,不回送结果只置标志位	
串扫描	SCASB SCASW	目标串中一个字节/字减 AX/AL 中内容,不传送结果,只是标志位	
读串	LODSB LODSW	将源串中一个字节/字取到 AX/AL 中	
写串	STOSB STOSW	将 AX/AL 内容写入目标串中	
重复前缀	REP	无条件重复执行,直到计数器 CX=0 为止	用于 MOVSB/MOVSW / STOSB/STOSW 前进行重复操作
	REPE/REPZ REPNE/REPNZ	ZF=1 且 CX≠0 重复执行 ZF=0 且 CX≠0 重复执行	用于 CMPSB/CMPSW / SCASB/SCASW 前进行重复操作

例:REP MOVSB

设执行指令前(DS)=1000H,(ES)=2000H,(SI)=0020H,(DI)=0100H,(CX)=0030H 执行后将数据段 10020H 开始的 30H 个字节传送到附加段 20100H 为起始地址的存储单

元中。

·80486 串操作指令

(1)80486 在 8086/8088 的串操作指令中增加了双字的字符串操作:MOVSD、LODSD、STOSD、CMPSD、SCASD 用于双字串的传送、装入、存、比较和扫描。

(2)串输入和输出指令

①INSB/INSW/INSD　　用于从 DX 寄存器指定的端口输入一个字节、字或双字到 ES:〔(E)DI〕所指定的存储单元,同时自动完成目标指针的修改。

②OUTSB/OUTSW/OUTSD　　将 DS:〔(E)SI〕指定的存储单元的字节、字或双字输出到 DX 寄存器指定的端口,同时自动修改源指针。

(五)程序转移指令

在 8086/8088 微机中,指令执行的顺序由 CS 和 IP 决定。CS 存放当前代码段的段基址,IP 存放指令的偏移地址。在正常顺序执行时,要执行的指令已经由总线接口部件 BIU 按 CS 和 IP 的内容从存储单元中取出存入指令队列中,CPU 执行一条指令后,按指令队列中的指令顺序,执行下一条指令。但实际上经常会遇到改变程序执行流程的情况,程序转移指令就是用改变 CS 和 IP 的内容,使程序执行的顺序发生变化。当程序需要发生转移时,存放在指令队列中的指令将被清除,BIU 根据改变后的 CS 和 IP 重新确定指令的地址,并按此地址将指令从存储单元中取出,重新送入指令队列,并送到 CPU 去执行。

8086/8088 指令系统中有以下四类程序转移指令:

·无条件转移指令

·条件转移指令

·循环控制指令

·中断指令

如表 3.6 所示

表 3.6　转移指令

指令类型		指令格式	指令功能	测试条件
无条件转移		JMP　目标标号	无条件转移到目标标号去执行	无条件
		CALL　过程名	过程调用,转移到子程序去执行	
		RET　弹出值	过程返回,从子程序返回到主程序	
条件转移	单标志位条件转移	JC　目标标号	进位位为 1 时转移到目标标号执行	CF=1
		JNC　目标标号	进位位为 0 时转移到目标标号执行	CF=0
		JE/JZ　目标标号	等于/结果为 0 转移到目标标号执行	ZF=1
		JNE/JNZ　目标标号	不等于/结果不为 0 转移到目标标号执行	ZF=0
		JO　目标标号	溢出 转移到目标标号执行	OF=1
		JNO　目标标号	不溢出 转移到目标标号执行	OF=0
		JNP/JNO　目标标号	奇偶位为 0/为奇 转移到目标标号执行	PF=0
		JP/JO 目标标号	奇偶位为 1/为偶 转移到目标标号执行	PF=1
		JNS　目标标号	符号位为 0 转移到目标标号执行	SF=0
		JS　目标标号	符号位为 1 转移到目标标号执行	SF=1
	无符号数	JA/JNBE　目标标号	高于/不低也不等于 转移到目标标号执行	CF=0 且 ZF=0
		JAE/JNB　目标标号	高于或等于/不低于 转移到目标标号执行	CF=0 且 ZF=1
		JB/JNAE　目标标号	低于/不高也不等于 转移到目标标号执行	CF=1 且 ZF=0
		JBE/JNA　目标标号	低于或等于/不高于 转移到目标标号执行	CF=1 且 ZF=1

续上表

指令类型		指令格式	指令功能	测试条件
条件转移	带符号数	JG/JNLE 目标标号	大于/不小也不等于 转移到目标标号执行	ZF=0 且 OF⊕SF=0
		JGE/JNL 目标标号	大于或等于/不小于	ZF=1 或 OF⊕SF=0
		JL/JNGE 目标标号	小于/不大也不等于	SF⊕OF=1 且 ZF=0
		JLE/JGE 目标标号	小于或等于/不大于	SF⊕OF=1 或 ZF=1
循环控制		LOOP 目标标号	CX−1→CX,CX≠0 转目标标号循环	
		LOOPE/LOOPZ 目标标号	ZF=1 且 CX≠0 循环	
		LOOPNE/LOOPNZ 目标标号	ZF=0 且 CX≠0 循环	
		JCXZ 目标标号	CX=0 时转移到目标标号执行	
中断		INT 中断类型	中断	
		INTO	溢出时中断	
		IRET	中断返回	

有关程序转移指令作以下说明：

1. 无条件转移指令分为段内转移和段间转移两种类型。段内转移是指程序转移的目标地址在当前代码段中，因此程序在转移时不需要改变 CS 的内容而只改变 IP 的值。段间转移是指程序转移到其它的代码段中，因此程序在转移时需要同时改变 CS 和 IP 的值。段内转移执行速度比段间转移快，在程序设计中，大量使用段内转移。

在段内和段间转移中还分为直接转移和间接转移。直接转移是指指令中直接给出目标地址的 IP 和 CS 的值，而间接转移是采用寄存器或存储器寻址方式得到转移的目标地址。

2. 子程序调用指令(CALL)实现主程序向子程序的转移，执行时是将主程序断点处的 CS 和 IP 中的值压入堆栈保存，然后将子程序的入口地址(即子程序的首地址)送入 CS 和 IP 中，转移到子程序去执行。

子程序返回指令(RET)执行时将原来压栈的主程序断点的值从堆栈中弹出送回 CS 和 IP，使其返回主程序断点处继续执行主程序。

应用时 CALL 指令和 RET 指令必须成对使用，CALL 指令放置在调用程序的主程序中，而 RET 指令则放置在子程序中。

3. 条件转移指令是根据 CPU 中标志寄存器中各标志位的状态来决定程序执行的流程。可以形成测试的条件有 CF，PF，ZF，SF，OF 5 个标志位。在使用条件转移指令前，应当先应用算术运算、逻辑运算或比较、测试指令等指令设置标志位的状态，然后根据不同条件的要求，选择相应的条件转移指令。

4. 有关中断和中断指令将在后面章节中详细介绍，这里不再讲述。

(六)处理机控制指令

处理机控制指令完成简单的控制功能，如表 3.7 所示。

表 3.7 处理机控制指令

指令类型	指令格式	指令功能
标志位操作	CLC	清除进位位 CF←0
	STC	进位位置位 CF←1
	CMC	进位位求反 CF←$\overline{CF}$
	CLD	清除方向标志 DF←0

续上表

指令类型	指令格式	指 令 功 能
标志位操作	STD	方向标志量1 DF←1
	CLI	清除中断标志 IF←0
	STI	中断标志置1 IF←1
其它	WAIT	使CPU进入等待状态,直到出现外部中断为止
	ESC	交权,使CPU将控制权交给FPU
	LOCK	封锁总线,使指令的执行受到保护
	HLT	暂停执行程序
	NOP	空操作,NOP执行后,CPU可继续执行其它指令

由上表可看到,8086/8088指令系统中,允许用指令操作的标志位有CF,DF和IF三位。其中DF用于串运算中决定地址变化的方向,当DF=0,串运算为地址增量方向,DF=1,串运算为地址减量方向执行。IF用于可屏蔽中断,当IF=0表示禁止可屏蔽中断(关中断),IF=1时表示允许可屏蔽中断(开中断)

·80486位操作指令

8086/8088指令系统中只允许对CF,DF,IF三位进行位操作,而80486增加了位操作的指令,包括位测试指令和位扫描指令两类,从而增加指令的功能。下面加以详细介绍。

(1)位测试指令 BT/BTS/BTR/BTC

指令格式:BT/BTS/BTR/BTC 目标,源

其中目标为寄存器或存储单元,源为寄存器或立即数

指令功能:对目标操作数指定的16位或32位位串数据的指定位(源操作数作位串下标)进行测试并送入CF,然后按指令要求置1、置0或求反。

BT:只完成测试并送CF

BTS:完成BT功能后,再将测试位置1

BTR:完成BT功能后,再将测试位置0

BTC:完成BT功能后,再将测试位变反

当目标操作数是16位位串,源操作数的取值范围为0~15

当目标操作数是32位位串,源操作数的取值范围为0~31。

(2)位扫描指令 BSF/BSR

指令格式:BSF/BSR 目标,源

其中目标为寄存器,源为寄存器或存储单元

功能:BSF为向前扫描指令,其功能是对源操作数指定的16位或32位位串从低位向高位扫描,并将扫描到的第一个"1"位存入目标操作数指定的寄存器中。若位串中所有位均为0,则将ZF标志位置1。BSR的功能与BSF相同,但扫描时从高位向低位进行。

(七)高级语言指令

80486提供了三类高级语言指令,为实现C和PASCAL这样的结构化语言提供支持。

(1)数组边界检查指令BOUND

指令格式:BOUND 目标,源 ;目标:寄存器,源:存储单元

指令功能:验证目标寄存器中的16位或32位操作数是否在源操作数所指定的两个界限内。若有效地址值小于下界或大于上界,则产生异常5。

例如:ARRAY　DD　LOWER
　　DD　UPPER
　　⋮
　　BOUND EDI,ARRAY

执行后若(EDI)<LOWER 或(EDI)+4>UPPER,则产生异常 5。

(2)对过程设置堆栈指令 ENTER

在许多高级语言中,每个子程序或函数都有自己的局部变量,为保存这些局部变量,在执行时,应为其建入相应的堆栈。ENTER 指令为过程参数建立一个堆栈区。

指令格式:ENTER　立即数 1(16 位),立即数 2(8 位)

功能:建立堆栈,立即数 1 指出堆栈字节数,立即数 2 指出过程的嵌套层数:0～31。

(3)过程退出指令　LEAVE

指令格式:LEAVE

功能:撤消前面 ENTER 指令的动作。当退出子程序时,应撤消对堆栈框架的设定。

在 C 语言中,不允许函数的嵌套调用,故 C 语言的编译程序中只采用 ENTER m,o 的语句形式。PASCAL 语言则允许子程序和函数的多层嵌套调用,故可采用指令 ENTER m,n。

有关操作系统型指令和浮点运算指令本书不再叙述,需要时可查阅有关手册。

思考题与习题

1. 80486 有哪几种数据类型?

2. 什么叫寻址方式? 80486 有多少种寻址方式? 各种寻址方式的特点是什么?

3. 简述立即寻址和直接寻址,寄存器寻址和寄存器间接寻址的区别。

4. 80486 的指令一般由哪几个字段组成? 各字段的主要作用是什么?

5. 80486 的指令系统分为几种类型?

6. 分别指出下列指令中源操作数和目标操作数的寻址方式。若是存储器寻址,用表达式表示 EA=?

(1)AND AX,00FFH　　(2)ADD　BX,[00FFH]

(3)MOV AX,[BX+10H]　　(4)ADD AX,[ESI * 8]

(5)SUB　[BP][SI],AX　　(6)MOV AX,[BX+DI+20H]

(7)CMP　[SI],AX　　(8)OR AX,DX

(9)MOV EAX,[ESI][EDI * 2]　(10)PUSH　DS

7. 80486 工作在实地址模式下,已知(DS)=1000H,(SS)=2000H,(SI)=007FH,(BX)=0040H,(BP)=0016H,变量 TABLE 的偏移地址为 0100H。指出下列指令中源操作数的寻址方式,求它的有效地址(EA)和物理地址(PA)。

(1)MOV AX,[1234H]　　(2)MOV AX,TABLE

(3)MOV AX,[BX+100H]　　(4)MOV AX,TABLE[BP][SI]

8. 指出下列指令的错误,并加以改正。

(1)MOV DS,100　　(2)MOV 1020H,DX

(3)SUB[1000H],[SI]　　(4)PUSH AL

(5)IN AL,[80H]　　(6)MOV DS,ES

(7)JMP BX　　(8)SHR DX,4

(9)OUT 380H,AX　　(10)ADD AL,BX

(11)POP CS　　(12)MOV CL,3300H

9. 已知(DS)=091DH,(SS)=1E4AH,(AX)=1234H,(BX)=0024H,(CX)=5678H,(BP)=0024H,(SI)=0012H,(DI)=0032H,〔09226H〕=00F6H,〔09228H〕=1E40H,〔1E4F6H〕=091DH,试求下列各指令单独执行后的结果。

```
(1)MOV CL,20H〔BX〕〔SI〕        ;(CL)=?
(2)MOV〔BP〕〔DI〕,CX             ;〔1E4F6H〕=?
(3)LEA BX,20H〔BX〕〔SI〕         ;(BX)=?
   MOV AX,2〔BX〕                 ;(AX)=?
(4)LDS SI,〔BX〕〔DI〕
   MOV 〔SI〕,BX                  ;〔SI〕=?
(5)XCHG CX,32H〔BX〕
   XCHG 20〔BX〕〔SI〕,AX          ;(AX)=? 〔09226H〕=?
```

10. 已知(AL)=0C4H,DATA 单元中内容为 5AH,写出下列每条指令单独执行后的结果。

(1)AND AL,DATA　　(2)OR AL,DATA

(3)XOR AL,DATA　　(4)NOT DATA

(5)AND AL,OFH　　(6)OR AL,1H

(7)XOR AL,OFFH　　(8)TEST AL,80H

11. 设(CS)=0405H,(IP)=3D8FH,(SP)=0F17CH,当执行 CALL 2000:009AH(段基址:偏移地址)后,指出(IP)、(CS)、(SP)、〔SP〕、〔SP+1〕,〔SP+2〕、〔SP+3〕的内容。

12. 已知(AC)=8EH,(BL)=72H,执行以下指令后标志位 OF,SF、ZF、AF、PF 和 CF 的值是什么?

(1)ADD AL,BL　　(2)AND BL,AL

(3)CMP AL,BL　　(4)SHL AL,1

13. 设程序在数据段中定义的数据如下:

```
NAMES DB 'GOOD MORNING!'
      DW 2050H
      DB 'PRINTER'
      DB  48
      DB 'MOUS. EXE'
      DW  3080H
```

请指出下列指令是否正确,若正确,A 累加器中的结果是多少?

```
(1)MOV  EBX,OFFSET NAMES
   MOV EAX,〔EBX+13〕
(2)MOV EAX,NAMES
(3)MOV BX,12
   MOV SI,6
   MOV AX,NAMES〔BX〕〔SI〕
```

(4)MOV EBX,16 * 2
MOV ESI,4
MOV EAX,OFFSET NAMES[EBX][ESI]
INC [AX]
(5)MOV BX,12
MOV SI,6
LEA DI,NAMES[BX][SI]
MOV AL,[DI]

14. 试比较无条件转移指令、条件转移指令,子程序调用指令的异同。

15. 试用 CMP 指令和条件转移指令实现以下判断

(1)AX 和 CX 中的内容为无符号数

①(AX)≥(CX)则转至 BIGER 标号执行

②(AX)<(CX)则转至 LESS 标号执行

(2)BX 和 DX 中的内容为带符号数

①(BX)>(DX)则转至 BIGER 标号执行

②(BX)≤(DX)则转至 LESS 标号执行

第四章　汇编语言

汇编语言是面向机器的语言，它将机器语言符号化，用助记符代替机器语言指令的二进制代码，因此汇编语言源程序与其经过汇编后产生的机器语言程序之间具有一一对应的关系。用汇编语言编程时，允许编程人员直接使用存储器、寄存器、I/O 端口，直接对位、字节、字、寄存器、存储单元和 I/O 端口进行处理；同时也能直接使用 CPU 的指令系统和各种寻址方式，因此汇编语言源程序比高级语言编写的源程序生成的目标代码精炼，占内存少，执行速度快。由于汇编语言的上述优点，所以它主要用于系统软件、实时控制软件、I/O 接口驱动程序的设计中。

支持 PC 系列微机的汇编程序有 ASM，MASM，OPTASM 等多种，现在广泛使用的 MASM 是美国 Microsoft 公司开发的宏汇编程序，它不仅包含了 ASM 的功能，还增加了宏指令，结构，记录等高级宏汇编语言功能。

第一节　MASM 汇编语句结构

MASM 有两种基本语句，它们是指令语句和指示语句(又称伪指令语句)

一、指令语句(Instruction statements)

指令语句又称可执行语句，它要求 CPU 执行某种操作。在第三章中介绍的各种指令都是指令语言，指令语句经汇编后产生机器目标代码。

指令语句的格式为：

〔标号:〕　指令助记符　〔操作数〕　〔;注释〕

一条指令语句可分为 4 个字段，其中方括号内的字段为可选项。

1. 标号字段(Label field)

这是一个任选项，以":"作为结束符。标号是该指令的符号地址，它代表该指令的第一个字节地址。一个程序段或子程序的入口处通常用一个标号来表示。当程序需要转移或调用时，可直接引用此标号，这给编程带来了方便。标号又称标识符，MASM 的标识符组成规则如下：

(1)标识符有效长度 1～31 个字符。

(2)标识符由字母(大写或小写的英文字母)，数字(0～9)或某些特殊字符如：@ $ _ . : ? /等。

(3)第一个字符必须是字母，不允许用数字开头，也不允许用特殊符号单独作为标识符。

(4)标识符不能使用系统专用字，如 AX(寄存器名)，MOV(助记符)等。

2. 指令助记符字段(Mnemonic field)

助记符是指令的关键部分，不可省略，例如：MOV，ADD，SHL 等。助记符表示要 CPU 完成什么具体操作，必要时可在助记符前加前缀实现某些附加操作(如重复前缀 REP)

3. 操作数字段(Operand field)

操作数是指令运算时需要的数据，由于有些指令不需要数据，因此，这部分可省缺。当指令

要求有两个操作数时，必须用逗号将两个操作数分开。操作数可以用表达式来表示。

4. 注释字段(Comment field)

注释字段是可选项，如果语句中带注释字段则必须用分号开始，用于对程序或指令加注释，提高程序的可读性，汇编程序不对它作任何处理。

二、指示语句(pseude instruction statements)

指示语句又称伪指令，它不要求 CPU 执行某种操作，汇编时也不产生对应的机器代码，而仅仅给汇编程序提供汇编信息，如源程序起止信息，段内存的安排等。

伪指令的格式为：

[名字] 伪指令定义符 [操作数] [;注释]

其中方括号内的字段为可选项。

指示语句的格式与指令语句类似，不同之处有以下几点：

1. 名字是给伪指令取的名称，相当于指令语句的标号，但在名字后面不允许带冒号(:)，名字可以省缺。

2. 伪指令定义符是由 MASM 规定的符号，又称为汇编命令，不可省缺。如 DB，DW，PROC 等。

3. 操作数个数随不同伪指令而相差悬殊，有的伪指令不允许有操作数，有的伪指令允许带许多个操作数，这时必须用逗号将各个操作数分开。

第二节 MASM 汇编语言数据、表达式和运算符

数据是指令中操作数的基本组成部分，数据的形式对语句格式有很大的影响。汇编程序能识别的数据项有常数，变量和标号。

MASM 汇编语言还有两种特殊的数据结构形式：结构和记录。

一、常 数

常数是没有任何属性的纯数值。在汇编时常数的值已确定，并且在程序运行过程中，常数的值不会改变。

常数分为两种类型：数值型常数和字符串型常数。

(一)数值型常数

1. 二进制数：以字母 B 结尾，如 01011101B。

2. 八进制数：以字母 Q 或 O 结尾，如 723Q，211O。

3. 十进制数：以字母 D(或省略)结尾，如 1995D，3508。

4. 十六进制数：以字母 H 结尾，例如：3A40H，8BH。

(二)字符串型常数

用单引号括起来的字符串。如 'ABCD'，'$123456' 等。每个字符在机内以 ASCII 码存放，例如 'A'。在内存中为 41H。

二、变 量

变量用来定义存放在存储器单元中的数据。

当存储单元中的数据在程序运行中随时可以修改时，这个存储单元中的数据可以用变量来定义。为了便于对这个变量的访问，要给这个变量取一个名字称为变量名。变量名是存放数据存储单元的符号地址。

定义变量可用数据定义伪指令(后面将介绍)

例如
```
VAR_DATA    SEGMENT
            DATA1    DB    12H
            DATA2    DB    20H
            DATA3    DW    5678H
VAR_DATA    ENDS
```

经过定义的变量，有三重属性：

1. 段属性(SEG)

表示变量存放在哪个逻辑段中(通常指变量所在段的段基址)。如上例中：变量 DATA1、DATA2、DATA3 放在逻辑段 VAR_DATA 中。

当指令中要对这些变量进行存取操作时，需将段基址送至段寄存器中。

2. 偏移地址属性(OFFSET)

表示变量在所在段的偏移地址，以字节数表示。例如上面例子中 DATA1 的偏移地址为 0。DATA2 的偏移地址为 1。DATA3 的偏移地址为 2。

段基址和偏移地址组成变量的逻辑地址。

3. 类型属性(TYPE)

表示变量占用存储单元的字节数。这一属性是由数据定义伪指令来规定的。变量可分别定义为 8 位，16 位，32 位，48 位，64 位和 80 位数据。

三、标　号

标号是指令语句所在地址的符号表示。它表明该指令在内存中的位置，常做为转移指令的操作数，确定程序转移的目标地址。

例如
```
NEXT：……
      ⋮
LOOP NEXT
```

与变量类似，每个标号也有三重属性，如下所述：

1. 段属性(SEG)　该标号所在段的段基址。

2. 偏移地址属性(OFFSET)　该标号所在段的偏移地址。

3. 距离属性(Distance)

当标号做为转移类指令的操作数时，可在段内或段间转移，这时它们的距离属性不同。

NEAR：只允许在本段内转移。

FAR：允许在段间转移。

设置标号距离可用隐含方式或伪指令 LABEL 定义其距离属性两种方式。

(1)隐含方式

标号后面跟“：”隐含其距离属性为 NEAR，只能在本段中转移。

(2)用 LABEL 伪指令定义距离属性。LABEL 伪指令格式为：

```
名称    LABEL    类型
```

有关 LABEL 伪指令后面将详细介绍。

四、表达式

表达式是操作数的常见形式，表达式由常数，变量，标号及连接它们的运算符组成。表达式的运算不由 CPU 完成，而是在程序汇编过程中进行计算确定并将表达式的结果做为操作数参加指令所规定的操作。

MASM 允许使用的表达式分两类：

1. 数字表达式

数字表达式的结果是数字

例如　MOV DX,(6 * A－B)/2

指令的源操作数(6 * A－B)/2 是一个表达式。若设变量 A 的值为 1，变量 B 的值为 2，则此表达式的值为(6 * 1－2)/2＝2，是一个数字结果，此表达式是数字表达式。

2. 地址表达式

地址表达式的结果是一个存储单元的地址，当这个地址中存放的是数据时，称为变量；当这个地址中存放的是指令时，则称为标号。

当在指令的操作数部分用到地址表达式时，应当注意其物理意义。例如两个地址相乘或相除是无意义的，两个不同段的地址相加减也是无意义的。经常使用的是地址±(加减号)数字量。如 SUM＋1 是指向 SUM 字节单元的下一个单元的地址。

例如　MOV AX,ES:[BX＋SI＋1000H]

BX＋SI＋1000H 为地址表达式，结果是一个存储单元的地址。

五、运 算 符

MASM 的运算符分为六大类，它们是：

(1)算术运算符　(2)逻辑运算符　(3)关系运算符

(4)分析运算符　(5)合成运算符　(6)其它运算符

MASM 汇编程序支持的运算符如表 4.1 所示。

表 4.1　MASM 支持的运算符

运　算　符			运算结果	实　例
类　型	符　号	名　称		
算术运算符	＋	加法	和	2＋6＝8
	－	减法	差	8－2＝6
	*	乘法	乘积	8 * 5＝40
	/	除法	商	22/5＝4
	MOD	模除	余数	12MOD3＝0;32MOD6＝2
	SHL	左移	左移后二进制数	0010B SHL2＝1000B
	SHR	右移	右移后二进制数	1100B SHR 1＝0110B
逻辑运算符	NOT	非运算	逻辑非结果	NOT 1010B＝0101B
	AND	与运算	逻辑与结果	1011B AND1100B＝1000B
	OR	或运算	逻辑或结果	1011B OR1100B＝1111B
	XOR	异或运算	逻辑异或结果	1010B XOR1100B＝0110B

续上表

运算符			运算结果	实例
类型	符号	名称		
关系运算符	EQ	相等	结果为真输出全"1" 结果为假输出全"0"	6EQ11B=全'0'
	NE	不等		6NE11B=全'1'
	LT	小于		5LT8=全'1'
	LE	不大于		7LE101B=全'0'
	GT	大于		6GT100B=全'1'
	GE	不小于		6GE111B=全'0'
分析运算符	SEG	返回段基址	段基址	SEG N1=N1 所在段段基址
	OFFSET	返回偏移地址	偏移地址	OFFSET N1=N1 的偏移地址
	LENGTH	返回变量单元数	单元数	LEHGTH N2=N2 单元数
	TYPE	返回元素字节数	字节数	TYPE N2=N2 中元素字节数
	SIZE	返回变量总字节数	总字节数	SIZE N2=N2 总字节数
合成运算符	PTR	修改类型属性	修改后类型	BYTE PTR[BX]
	THIS	指定类型/距离属性	指定后类型	ALPHA EQU THIS BYTE
	段寄存器名	段前缀	修改段	ES:[BX];DS:BLOCK
	HIGH	分离高字节	高字节	HIGH 2345H=23H
	LOW	分离低字节	低字节	LOW 2345H=45H
	SHORT	短转移说明		JMP SHORT LABEL
其它运算符	( )	圆括号	改变运算符优先级	(8-4)*4=16
	[ ]	方括号	下标或间接寻址	MOV AX,[BX]
	.	点运算符	连接结构与变量	TAB.T1
	< >	尖括号	修改变量	<,8,5>
	MASK	返回字段屏蔽码	字段屏蔽码	MASK C<
	WIDTH	返回记录宽度	记录/字段位数	WIDTH W

要注意的是，有些运算符尽管与指令的名字相同，但它们与指令有着本质区别：指令是在程序运行时执行的语句，而运算符是在汇编时由汇编程序完成其功能的。

当各种运算符同时出现于一表达式中时，具有不同的优先级。运算符的优先级规定如表4.2所示。优先级相同的运算符操作顺序为先左后右。

表4.2 运算符的优先级表

优先级		符号
高↑低	1	LENGTH,SIZE,WIDTH,MASK,( ),[ ],< >
	2	PTR,OFFSET,SEG,TYPE,THIS
	3	HIGH,LOW
	4	+,-(单目)
	5	*,/,MOD,SHL,SHR
	6	+,-(双目)
	7	EQ,NE,LT,LE,GT,GE
	8	NOT
	9	AND
	10	OR,XOR
	11	SHORT

第三节　伪指令(指示语句)

指示语句又称伪指令,它没有对应的机器指令,不由 CPU 执行,而是在汇编程序汇编时识别,用于说明 CPU 的类型、段结构、源程序起止信息和段内存的安排等。

MASM 有丰富的伪指令,必须搞清这些伪指令的功能及用法,才能编写出高质量的汇编语言源程序。

MASM 的伪指令可分为变量定义语句、符号定义语句、段定义语句、段分配语句、过程定义语句、群定义语句、结构定义语句、记录定义语句等。

本节重点讨论前五种伪指令语句,这些语句在程序中比较重要,使用频繁。

一、变量定义语句

变量定义语句用来定义字节、字或双字变量,变量定义语句有两种不同的格式。

1. 格式 1

[变量名]　DB/DW/DD/DF/DQ/DT　表达式

功能:为变量分配 1 个字节/字/双字/长字/四字/十字节的存储单元,并给它们赋值

格式 1 又可分为以下几种具体用法:

(1)用数值表达式定义变量

```
例如  K1    DB    5AH          ;将 5AH 赋给字节变量 K1
      K2    DW    2354H        ;将 2354H 赋给字变量 K2
      K3    DD    2A00 4455H   ;将 2A00 4455H 赋给双字变量 K3
```

K_1,K_2,K_3 在存储器中的存放格式如图 4.1 所示。

(2)用字符串定义变量

```
ONE   DB    'HOW ARE YOU';将 "HOW ARE YOU"字符串赋给变量 ONE。
TWO   DW    'OK'         ;将"OK"字符串赋给变量 TWO。
```

变量 ONE,TWO 在存储单元中存放格式如图 4.2 所示。

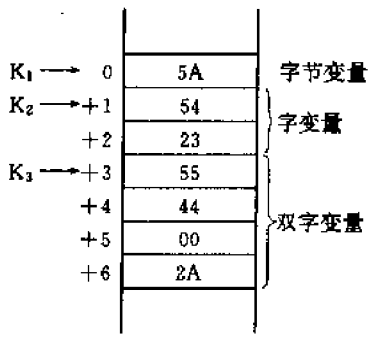


图 4.1　数值变量存储格式

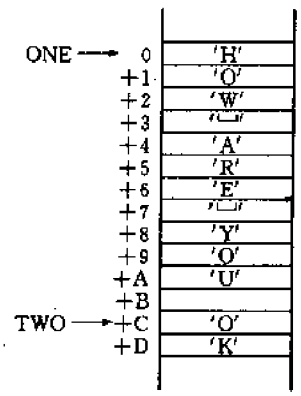


图 4.2　字符串变量存储格式

需要的注意的是,用 DW 语句定义的字符串只允许包含两个字符,如果字符少于或多于两个时,必须用 DB 语句来定义。

(3)存放结果时,用问号(?)定义不确定值的变量

```
ABOV    DB?      ;定义变量 ABOV 为不确定值的字节变量
BLOW    DW?      ;定义变量 BLOW 为不确定值的字变量
CLER    DD?      ;定义变量 CLER 为不确定值的双字变量
```

(4)用数组表达式定义变量

```
例如  A1    DB    00H,11H,22H,33H
      A2    DW    3456H,1025,25 * 25,1000/3,10A0H
```

定义后 A1 中包含 4 个元素,A1 为 4 个元素的字节数组变量。A2 中包含 5 个元素,A2 为 5 个元素的字数组变量。这种变量又称为多重变量。

2. 格式 2

格式 2 用于定义重复变量,其格式如下所示

```
          DB
          DW
          DD
变量名    DF      n DUP(表达式)
          DQ
          DT
```

与格式 1 不同处格式 2 增加了 n DUP 用于表示重复数,表达式需用圆括号括起。例如:

```
T1    DB    10    DUP(0)                ;定义 T1 为 10 个"0"组成的字节变量
T2    DW     2    DUP(?)                ;定义 T2 为 2 个不确定数值的字变量
T3    DB     4    DUP(1,2 DUP(50H))     ;T3 为(1,50H,50H)重复 4 次共 12 个字节的字节变量
```

利用格式 2 可以方便的定义重复数组变量。

二、符号定义语句

编制源程序时,经常把常数、变量、标号、表达式用一特定符号表示,这时需要用符号定义语句。符号定义语句不占用内存,有两种基本格式。

1. 等值语句(EQU)

格式:符号　EQU　表达式

功能:将表达式的值赋给 EQU 左边的符号。表达式可以是常数、变量、标号、数值或地址表达式。例如:

(1)常数或数值表达式

```
COUNT    EQU    5
NUM      EQU    13+6-2
```

(2)变量

```
CR       EQU    CX
CBD      EQU    DAA
L        EQU    SEG    N1
```

(3)标号或地址表达式

```
WO       EQU      WORD PTR DA_BYTE
```

```
ADR1        EQU        DS:[BP+14]
```

等值语句在汇编时不产生任何目标代码，也不占用存储单元。但在同一个程序中，不能对已定义的符号重新定义。

2. 等号语句

格式为：符号＝表达式

等号语句(＝)与 EQU 语句有同样的功能，区别在于等号语句定义的符号允许重新定义，使用更加方便灵活。

下列等号语句是有效的：

```
COUNT=5
NUM=14H
  ⋮
NUM=NUM+10H
COUNT=COUNT+100
  ⋮
```

三、段定义伪指令

在汇编程序设计中，需要将存储器分成若干段，80486CPU 访问存储器时，凭借段寄存器对各段进行访问。通常用段定义语句(SEGMENT/ENDS)来定义各种类型的逻辑段。

在 MASM5.0 以上的汇编语言版本中，有完整定义伪指令和简化段定义两种。

(一)完整段定义伪指令

1. 段定义语句

格式：

```
段名   SEGMENT  [定位类型][,组合类型][,字长选择][,类别]
          ⋮
       (段体)
          ⋮
 段名     ENDS
```

任何一个逻辑段从 SEGMENT 语句开始，以 ENDS 语句结束。伪指令名 SEGMENT 和 ENDS 是本语句的关键字，不可以省缺，并且总是成对出现。语句中段名是必选项；定位类型、组合类型、字长选择，分类名为可选项。

(1)段名(SEG Name)

用户自己选定，不能省略，一个段开始与结尾用的段名应一致。

(2)定位类型(ALIGN TYPE)

定位类型表示对段的起始边界的要求，可有以下 5 种选择：

①PAGE(页)：表示本段从一页的边界开始，一页为 256 个字节，所以 PAGE 定义的边界其地址能整除 256，这样段的首地址的最后八位二进制数一定为'0'(即以 00H 结尾)。

②PARA(节)：表示本段从一个节的边界开始，一节为 16 个字节，所以段的起始地址最后四位二进制数一定为'0'(即以 0H 结尾)。PARA 通常隐含，即未定义定位类型时，其定位类型为 PARA。

③DWORD(双字)：表示本段从 4 的倍数地址开始。

④WORD(字)：表示本段从偶数地址开始，该地址的最低位应为 0。

⑤BYTE(字节):表示本段可从任何地址开始定位。

(3)组合类型(COMBINE TYPE)

组合类型用来对各个逻辑段之间的连接方式提出要求,可供选择的参数有以下 6 种:

①NONE:表示该段与其它同名段不进行连接,独立存于存储器中,如果语句中省略组合类型,则 MASM 把它作为 NONE 处理。

②PUBLIC:该段可与其它模块中的同名段在满足定位类型前提下,由低地址到高地址连接起来,组合成一个较大的逻辑段。

③COMMON:该段与其它模块中的同名段采用复盖方式在存储器中定位,即它们具有相同的段首地址,共享同一个存储区,共享存储器的长度由同名段中最大的段确定。段的内容为所连接的最后一个模块中的内容及没有覆盖到的前面 COMMON 段的部分内容。

④MEMORY:定义该段与其它模块中的同名段具有相同的首地址,采用覆盖方式在存储器中进行连接,与 COMMON 不同处是带有 MEMORY 参数的逻辑段覆盖在其它同名段的最高地址。

⑤STACK:表示该段为堆栈段,其它与 PUBLIC 同样处理。对于多个模块只需设置一个堆栈段实体,它的容量应当与各模块中设置的多个堆栈段中的最大的相同。对于堆栈段,STACK 参数不可省缺,否则需要在程序中用指令设置 SS 和 SP 寄存器的值。

⑥AT 表达式

表示本段可定义在表达式所指示的节边界上。例如:"AT 0931H",表示该段从首地址 09310H 开始。一般情况下各个逻辑段在存储器中的分配由系统自动完成,但在某些情况下,如果要求某个逻辑段必须分配在某个节的边界上,则可用本参数来实现。如果要求某个逻辑段从绝对地址 20508H 开始,可采用如下语句格式:

```
MCODE SEGMENT AT 2050H
        ORG 0008
START:MOV AX,O
        ⋮
MCODE ENDS
```

(4)字长选择

用于定义段中使用的偏移地址和寄存器的字长。只用于设置·386 和·486 语句的段。有两种字长选择:

USE16:段字长为 16 位,最大段长为 64KB

USE32:段字长为 32 位,最大段长为 4GB

若字长选择省略,确认为 USE32

(5)分类名(CLASSNAME)

分类名必须用单引号括起来,并且不超过 40 个字符,用于控制段的存放次序。MASM 将所有分类名相同的逻辑段存放在连续的存储空间中。

在段定义语句中,定位类型、组合类型、字长选择、分类名为可选项,可根据要求选其中一项或几项,各参数间用空格分隔,未选的项可省缺,但它们之间的顺序不能交换。

2. 段分配语句(ASSUME)

段分配语句用来完成段的分配,说明当前哪些逻辑段被分别定义为代码段、数据段、堆栈段和附加段。代码段用来存放被执行的程序,数据段用来存放程序执行中需要的数据和运算结

果，当用户程序中使用的数据量很大时或使用了串操作指令，可设置附加段来增加数据段的容量，堆栈段用来设置堆栈。

格式：ASSUME 段寄存器：段名[，段寄存器：段名，…]

功能：说明源程序中定义的段由哪个段寄存器去寻址。段寄存器可以是 CS，SS，DS，ES，FS 或 GS。

格式中，ASSUME 是伪指令名，是语言中的关键字，不可省缺。段寄存器名后面必须有冒号，如果分配的段名不止一个，则应用逗号分开，段名是指用 SEGMENT/ENDS 伪指令语句中定义过的段名。ASSUME 语句设置在代码段内，放在段定义语句之后。

下面是一段源程序：

```
DATA SEGMENT
        W1 DB 00H,11H,22H,33H,'$'
        W2 DW 4142H,0A0BH
DATA ENDS
STACK1 SEGMENT STACK
        DW 100 DUP(0)
STACK1 ENDS
CODE SEGMENT
        ASSUME CS:CODE,DS:DATA,SS:STACK1
START:
  ⋮
CODE ENDS
```

程序中用 ASSUME 语句指出了各段的分配情况。代码段寄存器 CS 将存放 CODE 段的段基址，数据段段寄存器 DS 将存放 DATA 段的段基值，而堆栈段段寄存器 SS 将存放 STACK1 为名的堆栈段的段基值，附加段没有设置。

在用 ASSUME 语句来完成段的分配时，要注意以下几点：

(1)在一个代码段中，如果没有另外的 ASSUME 语句重新设置，原有的 ASSUME 语句的设置一直有效。

(2)每条 ASSUME 语句可设置 1～6 个段寄存器。

(3)可以用 NOTHING 将以前的设置删除，例如：

ASSUME ES:NOTHING

删除对 ES 与某段的关联设置。

ASSUME NOTHING

删除全部六个段寄存器的设置。

3. 段寄存器的装入

ASSUME 语句只建立当前段和段寄存器之间的联系，但 ASSUME 语句并不能将各段的段基值装入各个段寄存器，段基值的装入是用程序的办法，六个段寄存器的装入也不相同。

1. DS、ES、FS 和 GS 的装入

DS、ES、FS 和 GS 不能用立即数寻址方式直接装入，段基值需先送入通用寄存器，再传送给段寄存器。

例如
```
DATA1 SEGMENT
    BYTE1 DB 12H
DATA1 ENDS
DATA2 SEGMENT
    BYTE2 DB 10H DUP(?)
DATA2 ENDS
CODE  SEGMENT
      ASSUME CS:CODE,DS:DATA1,ES:DATA2
START:MOV AX,DATA1
      MOV DS,AX          ;装入 DS
      MOV AX,DATA2
      MOV ES,AX          ;装入 ES
        ⋮
COPE ENDS
```

2. SS 的装入

SS 的装入有两种方法

(1)在段定义伪指令(SEGMENT)的组合类型项中选择“STACK”参数，且在 ASSUME 语句中，将堆栈段分配给段寄存器 SS。

例如
```
STACK1 SEGMENT PARA STACK
        DB 40 DUP(?)
STACK1  ENDS
        ⋮
CODE    SEGMENT
        ASSUME CS:CODE,SS:STACK1
        ⋮
```

这样，当程序目标代码装入存储器后，SS 中已自动装入了 STACK 段的段基值。

(2)如果在段定义伪指令中，组合类型中未选用“STACK”参数项，则可用程序实现对 SS 和 SP 的装入。

例如
```
STACK2  SEGMENT
        DW 40 DUP(?)
TOP     LEBEL WORD
STACK2  ENDS
        ⋮
CODE    SEGMENT
        ⋮
        MOV AX,STACK2
        MOV SS,AX
        MOV SP,OFFSET TOP          ;装入 SS 和 SP
        ⋮
```

4. CS 的装入

CS 和 IP 共同决定程序中当前要执行的指令的地址。CS 中存放的是当前要执行指令的段基址，IP 中存放该指令的偏移地址。任何一个源程序都以 END 伪指令来结束，END 伪指令的格式是

END 起始地址

起始地址可以是一个标号或表达式，因此为了使 CS 的装入正确应当在程序执行的第一条语句设置一个标号，与 END 语句呼应。当程序装入内存时，此标号作为起始地址，它的段基址和偏移地址将自动装入 CS 和 IP 中。

例如：

```
    ⋮
CODE SEGMENT
        ASSUME CS:CODE,…
START:……
              ⋮
CODE    ENDS
        END START
```

标号 START 的段基址和偏移地址将自动装入 CS 和 IP

(二)简化段定义伪指令

简化段有利于实现汇编语言程序模块与 Microsoft 高级语言程序模块的连接，它可以由操作系统自动安排段序，自动保证名字定义的一致性。但是命令文件(.COM)的编程不可使用简化段定义。

1 段次序语句(DOSSEG)

格式：DOSSEG

功能：各段在内存的顺序按 DOS 段次序约定排列。

说明：各段在内存的次序决定于很多因素，多数程序对段次序无明确要求，可由操作系统安排。本语句用于主模块前面，其它模块不必使用。

2 内存模式语句(.MODEL)

格式：.MODEL 模式类型[,高级语言]

功能：指定数据和代码允许使用的长度。

说明：[高级语言]是可选项，可使用 C、BASIC、FORTRAN、PASCAL、SYSCALL、STDCALL 等关键字来指定与哪种高级程序设计语言接口，还可用关键字 OS__OS2 或 OS__DOS 告诉 MASM 使用的是哪种操作系统。程序中凡数据或代码的长度不大于 64KB 时为近程，否则为远程。近程的数据通常定义在一个段中，对应一个物理段，只要程序一开始设置其段值于 DS 中，以后数据的访问只改变偏移值，不改变段值。

本语句一般放在用户程序中其它简化段定义语句前。可选内存模式有 5 类，如表 4.3 所示。

当独立的汇编语言源程序不与高级语言程序连接时，多数情况下只用小模式即可，而且小模式的效率也最高。

3 段语句

简化段定义的段语句如表 4.4 所示。

表 4.3 内存模式类型

内存模式	说　　明
SMALL	小模式。数据、代码各放入一个物理段中，均为近程。
MEDIUM	中模式。数据为近程，代码允许为远程。
COMPACT	压缩模式。代码为近程，数据允许为远程，但任一个数据段所占内存不可超过 64KB。
LARGE	大模式。数据与代码均允许为远程。但任一个数据段不可超过 64KB。
HUGE	巨型模式。数据与代码均允许为远程，且数据语句所占内存也可大于 64KB。

表 4.4 简化段语句

段语句名	格　式	功　能
代码段语句	.CODE[名字]	定义一个代码段。如有多个代码段，要用名字区别。
堆栈段语句	.STACK[长度]	定义一个堆栈段，并形成 SS 及 SP 初值。(SP)＝长度，如省略长度，则(SP)＝1024。
初始化近程数据段语句	.DATA	定义一个近程数据段。当用于与高级语言程序连接时，其数据空间要赋初值。
非初始化近程数据段语句	.DATA?	定义一个近程数据段。当用于与高级语言程序连接时，其数据空间只能用"?"定义，表示不赋初值。
常数段语句	.CONST	定义一个常数段。该段是近程的，用于与高级语言程序连接。段中数据不能改变。
初始化远程数据段语句	.FARDATA[名字]	定义一个远程数据段，且其数据语句的数值应赋初值。用于与高级语言程序连接。
非初始化远程数据段语句	.FARDATA?[名字]	定义一个远程数段，但其数据空间不赋初值，只能用"?"定义数值。用于与高级语言程序连接。

不同的内存模式允许的段定义语句有所不同。表 4.5 给出了标准内存模式允许的段及其隐含的段名、段参数和组名。

对于简化段定义，有两点需要说明：

a. 凡是与高级语言程序连接的数据，必须把常数与变量分开，变量中又要把赋初值的与不赋初值的分开，并分别定义在. CONST、. DATA/. FAR－DATA 和. DATA? /. FAR－DATA? 中。远程数据段只能在压缩模式、大模式和巨型模式中使用，其它数据段和代码段可在任何模式下使用。

表 4.5 标准内存模式允许的段及其隐含的内容

MODEL	段定义符	段隐含内容				
		段　名	定位类型	组合类型	'类别'	组　名
Small	.CODE	_TEXE	WORD	PUBLIC	'CODE'	
	.DATA	_DATA	WORD	PUBLIC	'DATA'	DGROUP
	.CONST	CONST	WORD	PUBLIC	'CONST'	DGROUP
	.DATA?	_BSS	WORD	PUBLIC	'BSS'	DGROUP
	.STACK	STACK	PAPA	STACK	'STACK'	DGROUP
Medium	.CODE	name _TEXT	WORD	PUBLIC	'CODE'	
	.DATA	_DATA	WORD	PUBLIC	'DATA'	DGROUP
	.CONST	CONST	WORD	PUBLIC	'CONST'	DGROUP
	.DATA?	_BSS	WORD	PUBLIC	'BSS'	DGROUP
	.STACK	STACK	PARA	STACK	'STACK'	DGROUP
Compact	.CODE	_TEXT	WORD	PUBLIC	'CODE'	
	.FARDATA	FAR _DATA	PARA	独立段	'FAR _DATA'	
	.FARDATA?	FAR _BSS	PARA	独立段	'FAR _BSS'	

续上表

MODEL	段定义符	段隐含内容				
		段 名	定位类型	组合类型	'类别'	组 名
	.DATA	_DATA	WORD	PUBLIC	'DATA'	DGROUP
	.CONST	CONST	WORD	PUBLIC	'CONST'	DGROUP
	.DATA?	_BSS	WORD	PUBLIC	'BSS'	DGROUP
	.STACK	STACK	PARA	STACK	'STACK'	DGROUP
Large	.CODE	name _TEXT	WORD	PUBLIC	'CODE'	
	.FARDATA	FAR _DATA	PARA	独立段	'FAR _DATA'	
	.FARDATA?	FAR _BSS	PARA	独立段	'FAR _BSS'	
	.DATA	_DATA	WORD	PUBLIC	'DATA'	DGROUP
	.CONST	CONST	WORD	PUBLIC	'CONST'	DGROUP
	.DATA?	_BSS	WORD	PUBLIC	'BSS'	DGROUP
	.STACK	STACK	PARA	STACK	'STACK'	DGROUP

b. 独立的汇编语言源程序(即不与高级语言连接的源程序)只用前述的 DOSSEG、.MODEL、.CODE、.STACK 和 .DATA 5 种简化语句,并且不区分常数与变量以及赋初值与不赋初值。在 .DATA 语句定义的段中,所有数据语句均可使用。其一般格式如下:

```
DOSSEG
.MODEL SMALL
.STACK[长度]
.DATA
…
数据语句
  …
.CODE
启动标号:MOV AX,DGROUP;或 MOV AX,@DATA
    MOV DS,AX
    …
执行性语句
  …
END 启动标号
```

这种简化段的源程序结构中只有一个堆栈段、一个数据段和一个代码段。代码段长度可达 64KB,数据段与堆栈段为一个组,其总长度可达 64KB,组名为 DGROUP。组名 DGROUP 与数据段名@DATA 都代表组对应物理段的段界地址,装入内存时,系统给 CS 和 IP 赋初值,使其指向代码段。同时系统还给 SS 和 SP 赋初值,使(SS)=DGROUP,(SP)=数据段长度+堆栈段长度,从而使堆栈段为对应的物理段。这样处理是使堆栈元素也能用 DS 寄存器访问,以便同高级语言程序连接。在代码段开始运行处(启动标号处),用户应设置 DS 指向组的段界地址。

四、过程定义伪指令(PROC/ENDP)

在程序设计中,常把具有一定功能的程序段设计成一个子程序,又称为过程。MASM 宏汇编程序用过程定义语句来构造过程,定义后的过程可供其它程序调用,通常把调用过程的程序

称为主程序。过程调用作为一种常用的程序结构，显示出很强的生命力，使用过程可有效地缩短程序的总长度，节省存储空间，提高程序的可读性，加快程序的设计周期。

过程定义伪指令的格式如下：

```
过程名  PROC (NEAR/FAR)
        ⋮
        RET
        ⋮
过程名  ENDP
```

其中过程名不可省缺，过程的开始(PROC)和结束(ENDP)应使用同一个过程名。过程名也是调用指令(CALL)的目标操作数。过程名像标号一样具有三重属性：段属性指该过程所在段的段基址，偏移地址属性是指该过程第一个字节与段首地址之间的字节距离，距离属性可选择 NEAR 和 FAR。如果是 NEAR 过程，则只允许实现段内调用，如果是 FAR 过程，则允许实现段间调用。在定义过程中如没有选择距离属性，则隐含为 NEAR。

过程内部至少设置一条返回指令 RET 作为过程的出口，它可以在过程的任何位置。如果一个过程有多个出口，它可能有多个返回指令，一个过程执行的最后一条指令必定是返回语句。

五、其它伪指令

1. LABEL 伪指令

LABEL 伪指令用于给已定义的变量或标号取一个别名，并可重新定义它的类型或距离属性。

LABEL 伪指令的格式如下：



LABEL 语句通常和下一个语句所定义的变量和标号联用，给下一个语句所定义的变量和标号取别名，别名由 LABEL 左边的名称决定，而其类型或距离属性则由 LABEL 右边的参数来给定。

LABEL 语句使用可分为二种情况：

(1)与变量连用

这种情况 LABEL 语句用来给下一条语句中的变量取一个别名，并可修改变量的类型属性，LABEL 右边的参数可以是 BYTE，WORD 和 DWORD。

例如

```
TIMB LABEL BYTE
TIMW DW 4142H,4344H
       ⋮
```

第一条语句为 LABEL 语句，它给第二条语句定义的字变量 TIMW 取一个别名 TIMB，并将其类型修改为字节，这样在后续程序中当要以字节类型访问该变量时可使用 TIMB 变量名。

例如：

```
MOV AX,TIMW[0]
MOV BL TIMB[0]
```

这两条语句都是有效的，但如果没有使用前面的 LABEL 语句，汇编程序将报告第二个语

句产生操作数不匹配的错误。前面定义堆栈段时，常用的格式中也用到了 LABEL 语句，

例如：

```
ASTACK SEGMENT
        DW 20 DUP(?)
TOP LABEL WORD
ASTACK ENDS
```

这里定义了一个由 20 个字组成的堆栈，其栈底的名字取为 TOP，它的类型定义为字。

(2)与标号连用

与标号连用时，LABEL 语句给标号取一个别名，并可改变其距离属性，这时 LABEL 左边仍是别名的名称，右边则 NEAR 或 FAR。

例如 SUBF LABEL FAR

```
SUBN:MOV AX,[BX+SI]
      ⋮
```

第二条语句用冒号隐含定义了 SUBN 为 NEAR 距离属性，第一条 LABEL 语句中将 SUBN 取一个别名 SUBF，SUBF 的距离属性修改为 FAR。SUBF 和 SUBN 有相同的逻辑地址，但它们的类型属性不同，可作为程序转移或调用时不同情况下的入口。当段内转移或调用时可使用标号名称 SUBN，当段间转移或调用时则使用标号名称 SUBF，但实际上是指向同一条指令。

2. 定位伪指令(ORG)和当前位置计数器($)

在汇编程序时，为了指示程序中指令或数据在相应段中的偏移地址，可使用定位伪指令和当前位置计数器。

(1)定位伪指令

定位伪指令的格式是：ORG　　表达式

其中 ORG 是本语句操作符，不可省缺，表达式给出偏移地址值，即 ORG 语句后的指令或数据从表达式给出的值作起始的偏移地址。表达式运算的结果必须是正整数，并且以 65536 为模。

在大多数情况下，不需要用 ORG 语句设置位置指针，段定义语句是段的起点，它的偏移地址为 0，以后每分配一个字节，位置指针将自动加 1，所以每条指令都有确定的偏移地址。只有程序要求改变这个位置指针时，才需要安排 ORG 语句。

例如：

```
SDATA SEGMENT
        ORG 10H
D1      DB 00H,5AH,80H,24H
        ORG 30H
D2      DW 4142H,6162H,OAODH
SDATA ENDS
```

如果不设置 ORG 语句，字节变量 D_1 的第 0 个元素 00H 的偏移地址为 0，字变量 D_2 的第 0 个元素 4142H 的偏移地址应为 4，由于 ORG 的设置，D_1 的第 0 个元素 00H 的偏移地址为 10H，而 D_2 的第 0 个元素 4142H 的偏移地址为 30H。通常，ORG 语句可以出现在程序中任何位置上。

(2)当前位置计数器

在汇编程序时,用 $ 表示当前位置计数器的现行值。

例如:

```
DATA SEGMENT
        ST     DB 'ABCDEFGHI'
        COUNT EQU $-ST
DATA ENDS
```

其中 $ 表示当前的偏移地址值,ST 表示上一条语句中变量 ST 的偏移地址值,$-ST 表示两个偏移地址之差,正好是以变量 ST 为首址的连续字节数,即 ST 为首址的字符个数。

3. END 伪指令

END 伪指令作为汇编语言源程序的结束语句,一般放在源程序的最后一行。一个程序模块只允许有一个 END 语句。END 语句的格式为:END␣标号名称

其中标号为该程序中第一条可执行语句的标号。

4. 模块定义伪指令

一个可执行的汇编源程序可由多个模块组成,每个模块是一个独立的汇编单位。在操作系统中,汇编源程序是一个 *.ASM 源文件。汇编源程序的模块与汇编源文件是一一对应的。

(1)模块开始语句(NAME)

格式:NAME[模块名]

功能:表示源程序开始并指出模块名。

说明:本语句一般可省略。省略时,模块名取源程序中 TITLE 语句之页标题的前 6 个字符。若没有 TITLE 语句,则取该模块的源程序文件名为模块名。

(2)模块结束语句[END]

格式:END[标号/过程名]

功能:模块结束语句表示源程序到此结束,并可指出程序的启动地址。

说明:一个源程序必须有且只能有一个 END 语句来指出源程序文件的结束。方括号中的标号或过程名是可选项,只有主模块才具有该项,其作用是指出该程序的第一条可执行指令的位置(为此,系统将给 CS、IP 赋初值)。

5. 模块连接伪指令

模块连接伪指令又叫外部引用伪指令。这类伪指令用于解决多模块连接问题,实现多模块之间数据和过程的共享。这里介绍 4 种这类伪指令语句。

(1)全局符说明语句(PUBLIC)

格式:PUBLIC 符号名 1[,符号名 2,…]

功能:将本文件中定义的符号名说明为全局符号,允许程序中其它模块使用。

说明:本语句中的符号名可以是标号、变量名、过程名或由 EQU(或=)伪指令定义的名字。这些符号名必须是在当前源程序中定义的,其它模块不能再用它们去定义别的内容。未被说明的符号名不能被其它模块引用。

需要注意的是,PUBLIC 伪指令与 SEGMENT 伪指令中的 PUBLIC 组合类型是两个不同的概念。

(2)外部符说明语句(EXTRN)

格式:EXTRN 符号名 1:类型[,符号名 2:类型,…]

功能：本语句中指定的符号名是在其它模块中用 PUBLIC 伪指令语句定义过的。类型可以是 NEAR、FAR、BYTE、WORD、DWORD、FWORD、QWORD、TBYTE 或 ABS（由 EQU 伪指令定义的常数符），具体类型必须与其它模块中定义的相同符号名的类型相一致。

说明：当前模块要引用其它模块中定义的符号名时，必须用 EXTRN 伪指令说明。如果当前模块中没有用 EXTRN 伪指令说明，或者在被引用模块中没有用 PUBLIC 伪指令说明，均不能引用，否则 LINK 程序将会产生一个错误信息。可见，EXTRN 伪指令和 PUBLIC 伪指令是互相对应的。

(3)包含语句(INCLUDE)

格式：INCLUDE 文件名

功能：将指定文件的内容完整地插入到本语句出现的位置。

说明：使用本伪指令语句可避免重复书写多个模块都要使用的相同程序块。当 MASM 汇编某一源程序文件时，若遇到 INCLUDE 伪指令，就按文件说明打开磁盘上存在的相应文件，并将它插入到当前文件的该 INCLUDE 伪指令处，然后汇编插入进来的文件中的语句。当该文件中的所有语句处理完后，MASM 继续处理当前文件中 INCLUDE 语句后面的语句。

汇编语言程序设计时，常用 INCLUDE 伪指令语句将一个标准的宏定义插入到程序中。

(4)公用符号说明语句(COMM)

格式：COMM [NEAR/FAR]符号名：尺寸[：元素数]，……

功能：将语句中的符号名说明为公用符号。公用符号既是全局的又是外部非初始化的。

说明：语句中的符号名可以是近程(NEAR)或远程(FAR)数据段的符号，NEAR/FAR 缺省时，在完整段和简化段的中、小模式下为 NEAR，其它模式下为 FAR；尺寸可以是 BYTE、WORD、DWORD、QWORD 及 TBYTE；元素数为符号的个数，缺省值为 1。一条语句中可说明多个公用符号，多个符号间用逗号隔开。本伪指令语句经常用在 INCLUDE 文件中。如果一个变量用于多个模块中，可以在 INCLUDE 文件中将它说明为公用变量，然后在每一个模块中都用 INCLUDE 指令嵌入这个 INCLUDE 文件。

6. 方式选择伪指令

80486 汇编语言是在 8086/8088 的基础上逐步发展起来的，它们具有向上兼容的特点，因此汇编程序提供了方式选择伪指令来区分当前的源程序是针对哪一种 CPU 的。

方式选择伪指令的功能和格式如表 4.6 所示。

表 4.6　方式选择伪指令

伪指令格式	功　能
.8086	这是默认方式，它告诉汇编程序只接受 8086/8088 指令
.286/.286C	告诉汇编程序只接受 8086/8088 及 80286 非保护方式(即实地址方式)下的指令
.286P	允许汇编程序接受 8086/8088 及 80286 的所有指令(包括保护方式和非保护方式下的指令)。该伪指令一段只有系统程序员使用
.386/.386C	允许汇编 8086/8088 及非保护方式下的 80286/80386 指令
.386P	允许汇编 8086/8088 及 80286/80386 的所有指令，该伪指令一段只有系统程序员使用
.486/486C	允许汇编 8086/8088 及 80286/80386/80486 非保护方式下的指令。MASM6.0 可用
.486P	允许汇编 8086/8088 及 80286/80386/80486 的所有指令。MASM6.0 可用

7. 宏定义伪指令

宏下义语句格式与过程定义语句格式相似。其格式与功能如下：

格式：宏名 MACRO [形式参数表]

⋮ ;宏体

ENDM ;宏定义结束

功能：定义一个宏

说明：(1)宏名必须是唯一的，它代表着所定义的宏体的内容，可通过该名字来调用宏。

(2)形式参数表是可选项，是由逗号分隔的一个或多个形式参数组成。若选用了形式参数，当调用宏时，需要用对应的实际参数去取代，以实现向宏中传递信息。

(3)宏体可以是汇编语言所允许的任何指令或伪指令语句。宏体中还可以定义或调用另一个宏，称为宏嵌套。

例：
```
ADDUP  MACRO  A1,A2,A3
       MOV  AX,A1          ┐
       ADD  AX,A2          ;├宏定义
       ADD  AX,A3          ┘
       ENDM
         ⋮
       ADDUP  BX,24,DX     ;宏调用，实际参数 BX,24,DX
         ⋮                   分别取代形参 A1,A2,A3
```

宏可用来简化源程序。宏操作的特点是可以直接传递和接收参数，因此编程较容易。引入宏操作不会在执行目标代码时增加时间；但宏调用并不能缩短目标代码的长度，在这方面不如过程的调用。宏调用适用于执行速度比内存容量更为重要，且调用次数不太频繁的地方，否则选用过程调用更合适。

有关结构定义伪指令、记录定义伪指令、联合定义伪指令、条件汇编伪指令语句本书不再加以介绍，读者可查阅有关的手册和资料。

思考题与习题

1. 什么是宏汇编语言，宏汇编语言与高级语言相比有什么优缺点？
2. 解释下列名词：指令、程序、寻址方式、指令语句、指示语句。
3. MASM 宏汇编程序的运算符分为几类？简述各类运算符的特点及用途。
4. 什么是变量？什么是变量的三重属性？
5. 变量定义语句的格式如何？有什么用途？
6. 什么是标号？什么是标号的三重属性？
7. 汇编程序设计中为什么要用段定义语句和段分配语句，它们的格式如何？
8. 试阐述六个段寄存器段基值的装入有何不同？
9. 某数据段中有

```
COUNT1 EQU 16H
COUNT2 DW 16H
```

下面两条指令有何异同：

```
        MOV AX,COUNT1
        MOV BX,COUNT2
```

10. 试用伪指令编写一数据段与下面程序等效：

```
        MOV AX,0913H
        MOV DS,AX
        MOV BX,20H
        MOV [BX],'AB'
```

11. 下面各题中有语法错误，试分别用两种办法修改，使其正确。

```
(1)  M1    DW 5060H
           ⋮
           MOV BL,M1
(2)  M2    EQU 10H
           ⋮
           MOV M2,AL
(3)        M3 DW 'ABCD'
(4)        M4 DB 1234
(5)        DATA1 SEGMENT
                 DA1 DW 1234H
           DATA1 ENDS
           DATA2 SEGMENT
                 DA2 DW 5678H
           DATA2 ENDS
           CODE SEGMENT
                ASSUME CS:CODE,DS:DATA1
                ⋮
                MOV BX,DA2
                ⋮
           CODE ENDS
```

12. 按下面要求写出程序的框架

(1)数据段的位置从 0E000H 开始，数据段中定义一个有 100 个字节的数组。

(2)堆栈段组名为 STACK，留 100 个字的空间。

(3)代码段中指定段寄存器，主程序从 1000H 开始，给有关段寄存器赋值。

(4)程序结束。

13. 简述过程定义伪指令的格式。

14. 80486 汇编语言中方式选择伪指令的作用是什么？

第五章　汇编语言程序设计

第一节　程序设计概述

与高级语言程序设计类似，在用汇编语言进行程序设计时，同样应按以下步骤进行。

1. 分析问题，建立数学模型

2. 确定算法

3. 编制程序流程图

初学程序设计者往往不习惯编制程序流程图，实际上在编写程序之前先构思程序流程图不仅能加速程序的编制，而且对程序在逻辑上的正确性也比较容易查找和修改，尤其对比较复杂的问题更是如此。流程图由以下几种主要框图符号组成，见图 5.1。

(1)起始、结束框

表示程序开始或结束。起始框内通常用程序名，标号或“开始”字符来表示，它仅有一个出口。结束框内通常用“结束”，“返回”等字符表示，它仅有一个入口。

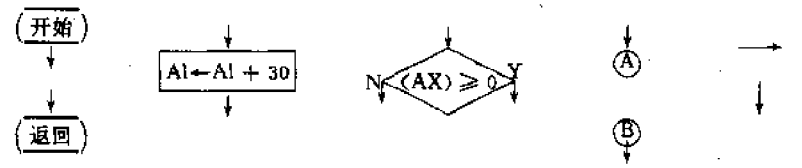


图 5.1　程序流程图的框图符号

(2)处理框

用于说明程序段(或指令)所完成的具体功能。这种框图通常是一个入口，一个出口。

(3)判断框

表示程序的分支流向判断，框内记入判断条件。这种框图通常是一个入口，两个或两个以上的出口。

(4)连接框

当程序比较复杂时，经常需要几张纸来绘制流程图，每页之间的连接可用连接框。框内有相同字母或数字则表示它们有连线关系，连接框仅有一个出口或一个入口。

(5)流向线

表示程序的流向，决定程序执行的顺序关系。

用汇编语言进行程序设计时，与高级语言进行程序设计的一个最大的不同点是，汇编语言编写程序要直接调用寄存器和存储单元。因此在构思流程图前应当考虑好程序所需占用的寄存器和存储单元数。合理利用寄存器，充分利用存储单元，是汇编语言能编写出高质量程序的关键。

4. 编制程序

有了程序流程图，即可根据流程图，逐条编写程序，在编写程序时要注意程序的基本格式，分清指令语句和伪指令的不同用途，正确使用各种寻址方式和指令系统中的各种指令。

一个源程序的基本格式如下：

```
DATA    SEGMENT
          ⋮                    ;存放数据项的数据段
DATA    ENDS
EXTRA   SEGMENT
          ⋮                    ;存放数据项的附加段
EXTRA   ENDS
STACK1  SEGMENT PARA STACK
          ⋮                    ;堆栈段
STACK1  ENDS
CODE    SEGMENT
        ASSUME CS:CODE,DS:DATA,SS:STACK1,ES:EXTRA
START:  MOV AX,DATA
        MOV DS,AX              ;DS 装入段基地
        MOV AX,EXTRA
        MOV ES,AX              ;ES 装入段基地
          ⋮      (核心程序段)
        MOV AH,4CH             ;系统功能调用
        INT 21H                返回操作系统
CODE    ENDS
        END START
```

有关源程序的基本结构作如下说明

(1)一个汇编语言源程序一般具有代码段、数据段、附加段和堆栈段 4 个段。8086/8088/80286 允许同时使用 4 个段，80386/80486 除以上 4 个段外，还增加了 FS 和 GS 两个附加数据段。实际程序中，只有代码段是必需的，其它段都为可选。在 8086/8088 和 80486 实地址方式下，每个段的大小为≤64K 字节；在保护方式下，80486 每个段最大长度允许 4G 字节。

(2)ASSUME 伪指令只说明各段寄存器和逻辑段的关系，并没有为段寄存器赋值，因此在源程序中，除代码段 CS(有时还有堆栈段 SS)外，其它定义的段寄存器由用户在代码段起始处用指令进行段基址的装入。

(3)每个源程序在代码段中都必须含有返回 DOS 操作系统的指令语句，以保证程序执行结束后能自动返回 DOS 状态。终止当前程序，使其正确返回 DOS 状态有以下几种：

①采用 DOS 功能与程序中的 4CH 功能调用

这种方法在代码段结束前加以下调用语句：

```
MOV AH,4CH        ;功能号 4CH→AH
INT 21H           ;中断调用
```

②采用标准序法，适用于定义为 FAR 的过程中

这种方法在过程定义语句 PROC 后加入以下语句：

```
过程名  PROC    FAR
```

```
        PUSH    DS
        MOV     AX,0;  } 标准序
        PUSH    AX
          ⋮
        RET
过程名   ENDP
```

③利用 DOS 的 0 号功能调用

```
调用方式：  MOV     AH,0
            INT     21H
```

此方式仅用于.COM 格式的可执行文件。

④利用 20 号软中断调用

调用方式：INT 20H

此方式仅用于.COM 格式的可执行文件。

5. 调试程序

程序编写结束后应当输入计算机中进行调试，上机调试大致有以下四步：

(1)使用编辑程序，将编写好的程序送入计算机，在盘中建立一个扩展名为.ASM 的文件。

(2)使用宏汇编程序，将扩展名为.ASM 的源程序汇编成目标程序，在盘上获得扩展名为.OBJ 文件。

(3)使用连接程序(LINK)将目标程序连接装配成可执行文件，其扩展名为.EXE，并存于盘上。

(4)程序可运行，如果运行中仍有问题，可使用调试程序(DEBUG)进行调试，直到问题全部解决，运行正确为止。

只有在计算机上通过运行的程序，才能认为是正确的程序。

第二节　顺序结构程序

顺序结构程序是一种最基本的程序结构形式。这种结构的程序中，指令从开始到结尾一直是顺序执行的，没有任何分支和判断框。

例 1　编制程序，求下列公式中的 Z 值，并将结果存放到 RESULT 单元中

$$Z=\frac{(X+Y)*8-X}{2}$$

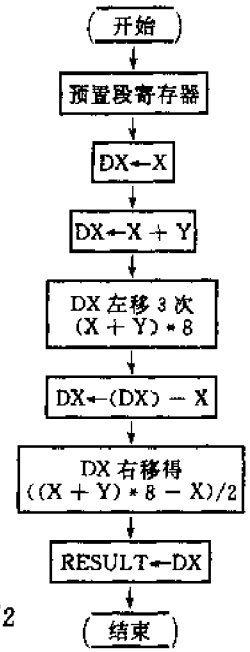
设 X，Y 的值分别存放在 VARX 和 VARY 单元中，运算结果不超过 16 位。

解：(1)存储单元 RESULT 用来存放结果，VARX 和 VARY 放 X、Y 的值。寄存器 DX 放中间运算结果，左移 1 位相当于乘 2，右移 1 位相当于除 2，所以可以用左移 3 位得到乘以 8 的结果。

(2)程序流程图和程序如下

```
DATA    SEGMENT
        VARX    DW 0006H
        VARY    DW 0007H
        RESULT  DW?
DATA    ENDS
```

```
STACK1    SEGMENT RARA STACK
          DW 20H DUP(0)
STACK1    ENDS
COSEG     SEGMENT
          ASSUME  CS:COSEG,DS:DATA
          ASSUME  SS:STACK1
START:    MOV  AX,DATA
          MOV  DS,AX
          MOV  DX,VARX      ;DX←X
          ADD  DX,VARY      ;DX←X+Y
          MOV  CL,3
          SAL  DX,CL        ;DX←(X+Y)*8
          SUB  DX,VARX      ;DX←(X+Y)*8-X
          SAR  DX,1         ;DX←((X+Y)*8-X)/2
          MOV  RESULT,DX
          MOV  AH,4CH       ;系统功能调用
          INT  21H          返回操作系统
COSEG  ENDS
       END  START
```

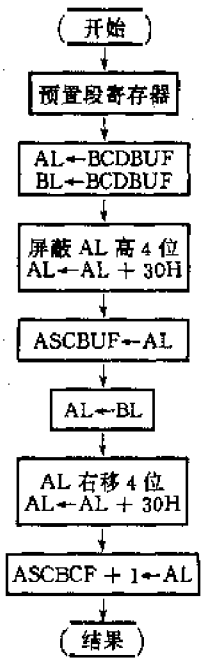


例 2 在 BCDBUF 单元中，存放二位用压缩型 BCD 码表示的十进制数，将它们转换为 ASCII 码并存放在 ASCBUF 和 ASCBUF+1 两个单元中。设高位 BCD 码转换后放在地址较高的单元中。

解：(1)存储单元 BCDBUF 中存放二位 BCD 码表示的十进制数，每 4 位二进制码表示一位十进制数。存储单元 ASCBUF 和 ASCBUF+1 用来存放转换的结果。用寄存器 AL 和 BL 存放中间结果。

(2)程序流程图和程序

```
          DOSSEG
          .MODEL  SMALL
          .STACK  200H
          .DATA
BCDBUF    DB  XXH
ABCBUF    DB  ?
          DB  ?
          .CODE
START:MOV  AX,DATA
      MOV  DS,AX
      MOV  AL,BCDBUF
      MOV  BL,AL
      AND  AL,OFH
      OR   AL,30H
```

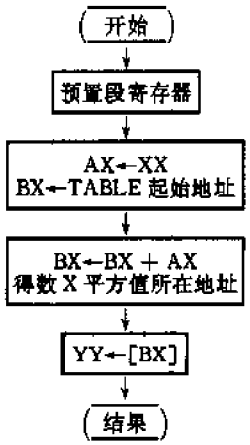


```
        MOV   ASCBUF,AL
        MOV   AL,BL
        MOV   CL,4
        SHR   AL,CL
        OR    AL,30H
        MOV   ASCBUF+1,AL
        MOV   AH,4CH
        INC   21H
        END   START
```

例 3 在内存 XX 单元中存放数 x(0≤x≤9 的整数),查表求 x 的平方值,并将结果存入 YY 单元中。

解:指定用查表的方式需建一个表 TABLE,将 0～9 的整数的平方值按顺序列出;表的起始地址与数 x 的和,正好是 x 平方值所在存储单元的地址。所需存储单元:表 TABLE 共 10 个单元存放 0～9 的平方值,XX 存放数 x,YY 存放结果。寄存器 AX 存放 XX 单元中的数 x,BX 存放起始地址与 x 的和,即 x 平方值所在的存储单元地址。

程序流程图如下:



程序编写如下:

```
DATA SEGMENT
TABLE     DB      0,1,4,9,16,25,36,49,64,81
XX        DB      X
YY        DB      ?
DATA      ENDS
STACK1    SEGMENT PARA STACK
          DW 20H DUP(0)
STACK1    ENDS
COSEG     SEGMENT
          ASSUME CS:COSEG,DS:DATA,SS:STACK1
START:    MOV AX,DATA
          MOV DS,AX
          MOV AH,0
          MOV AL,XX
          MOV BX,OFFSET TABLE
          ADD BX,AX
          MOV AL,[BX]
          MOV YY,AL
          MOV AH,4CH
          INT 21H
COSEG     ENDS
          END START
```

第三节　分支程序

计算机可根据不同条件进行逻辑判断，从而选择不同的程序流向。程序的流向是由 CS 和 IP 决定的，当程序的转移仅在同一段内进行时，只需修改偏移地址 IP 的值；如果程序的转移是在不同的段之间进行，则段基址 CS 和偏移地址 IP 均需要修改。

转移指令分为无条件转移指令和条件转移指令。在设计分支程序前，请复习第四章有关转移指令的内容，尤其是条件转移指令。能否正确使用这些转移指令是能否编写好分支程序的关键。

在进行分支程序设计时，首先要根据处理的问题用比较，测试的方式，或者用算术运算，逻辑运算使标志寄存器产生相应的标志位。例如比较两个单元的地址的高低，两个数的大小，测试某个数据是正还是负，测试数据的某位是"0"还是"1"等。将处理的结果反映在标志寄存器的 CF，ZF，SF，DF 和 OF 位上，然后根据转移条件选择转移指令。通常一条条件转移指令只能产生两路分支，因此要产生 n 路分支需 n-1 条条件转移指令。

例 1　符号函数 $y=\begin{cases}1 & x>0\\0 & x=0\\-1 & x<0\end{cases}$　其中 x 为 8 位有符号数

解：设 x 的值放在 XX 单元中，y 的值放在 YY 单元中。寄存器选用 AL 存放中间结果。

程序流程图见图 5.2

程序如下：

```
DATA        SEGMENT
  XX        DB    X
  YY        DB    ?
DATA        ENDS
CODE        SEGMENT
            ASSUME CS:CODE,DS:DATA
START:MOV   AX,DATA
      MOV   DS,AX
      MOV   AL,XX
      CMP   AL,0      ;x-0 建标志位
      JGE   BIGR      ;x≥0 转移
      MOV   AL,-1     ;x<0
      JMP   JUS2
BIGR: JG    JUS1      ;x>0 转移
      MOV   AL,0      ;x=0
      JMP   JUS2
JUS1: MOV   AL,1      ;x>0
JUS2: MOV   YY,AL
      MOV   AH,4CH
```

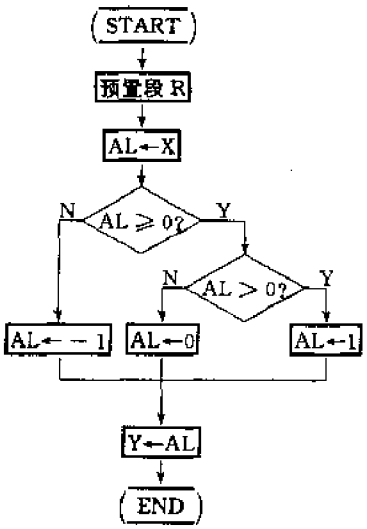


图 5.2　程序流程图

```
        INT   21H
CODE    ENDS
        END   START
```

例 2 已知 X,Y,Z 为三个无符号 16 位数,编程序求三者中最大值,并将其送入 MAX 单元(设 X,Y,Z 三个数不等)

解:程序流程图见图 5.3,存储单元 XX,YY,ZZ,MAX。用寄存器 AX 存放中间结果。

程序如下

```
DATA    SEGMENT
    XX      DW  X
    YY      DW  Y
    ZZ      DW  Z
    MAX     DW  ?
DATA    ENDS
CODE    SEGMENT
        ASSUME CS:CODE,DS:DATA
BEING:  MOV AX,DATA
        MOV DS,AX
        MOV AX,XX
        CMP AX,YY
        JA XGY
        MOV AX,YY
XGY:    CMP AX,ZZ
        JA GMAX
        MOV AX,ZZ
GMAX:   MOV MAX,AX
        MOV AX,4COOH
        INT 21H
CODE    ENDS
        END BEING
```

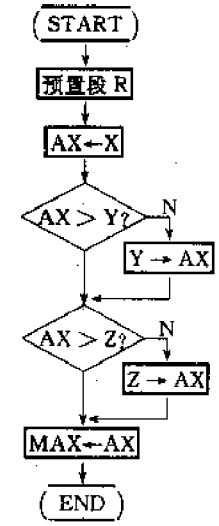


图 5.3 程序流程图

例 3 数据块搬移程序

程序要求把内存中一数据块(源数据块)传送到内存的另一存储区(目标数据块)。为了程序的通用性,应考虑以下三种情况:

(1)两块分离 (2)两块重叠,源数据块首址<目标数据块首址

(3)两块重叠,源数据块首址>目标数据块首址 如图 5.4 所示。

根据图 5.4 三种情况分析,可得出以下规律:

当源数据块首址>目标数据块首址:从数据块首地址开始传送。

当源数据块首址<目标数据块首址:从数据块末地址开始传送。程序流程图如图 5.5 所示。

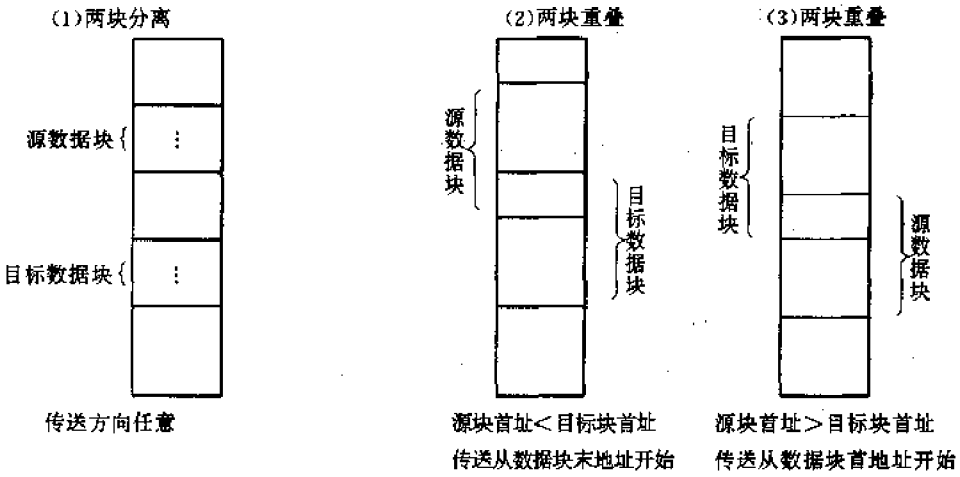


图 5.4　数据块搬移的三种情况

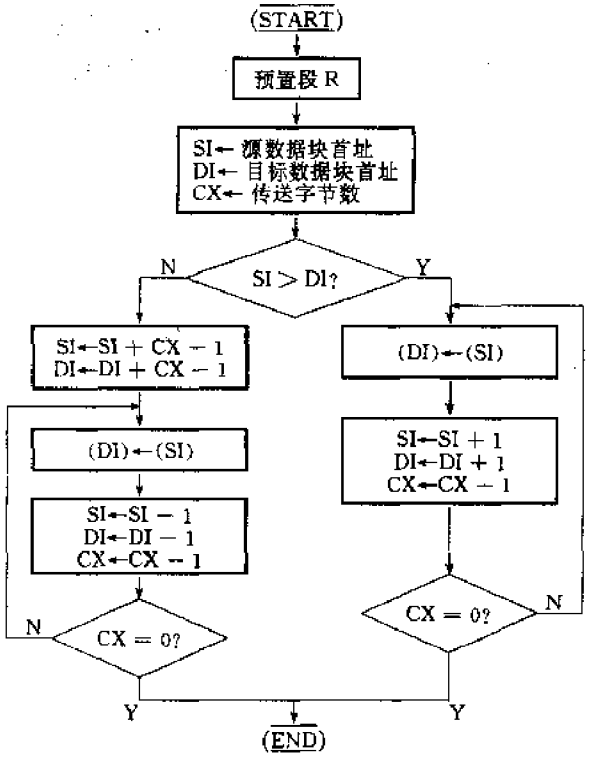


图 5.5　数据块搬移流程图

按流程图编写程序如下：

```
DATA   SEGMENT
   DA   DB 200 DUP('1,2,3,4,5')
   DA1   EQU   DA+10H
   DA2   EQU   DA+50H
   COUNT   EQU   50
DATA   ENDS
STACKI   SEGMENT   PARA   STACK
```

```
        DW   20H  DUP(0)
STACK1  ENDS
COSEG   SEGMENT
        ASSUME  CS:COSEG,DS:DATA,SS:STACK1
MOVE:MOV AX,DATA
     MOV DS,AX
     MOV CX,COUNT
     MOV SI,OFFSET DA1          ;源串首偏址→SI
     MOV DI,OFFSET DA2          ;目标串首偏址→DI
     CMP SI,DI
     JA  OK2
     ADD SI,CX
     ADD DI,CX
OK1: MOV AL,-1[SI]              ;SI<DI,从数据块
     MOV [DI-1],AL               末地址开始传送
     DEC SI
     DEC DI
     DEC CX
     JNE OK1
     JMP OK3
OK2: MOV AL,[SI]                ;SI>DI,从数据块首地址开始传送
     MOV [DI],AL
     INC SI
     INC DI
     DEC CX
     JNE OK2
OK3: MOV AH,4CH
     INC 21H
COSEG   ENDS
     END MOVE
```

本例源程序的数据段中，定义了长度为200×5个字节的字符串，源数据块和目标数据块可在其间选取。本例中源数据块从DA+10H开始，将50个字节传送到从DA+50H开始的目标数据块中。改变源数据块和目标数据块的首地址可以形成三种数据块搬移的情况，因此，此程序具有通用性。由于偏移地址是无符号数，故条件转移指令用JA，请加以注意。

第四节　循环程序

在程序中重复执行的操作可用循环程序来实现。486指令系统中有专门的循环控制指令来简化循环程序的设计，循环控制指令共4条，见第三章表3～6。

一个循环程序通常由以下四部分组成：

1. 初始化部分

为循环操作作准备工作，建立循环的初始值，如初始化地址指针，计数器及给变量赋初值等。

2. 循环体

循环体为循环的工作部分，用于完成各种具体操作。循环体可以是一个顺序结构，分支结构或又一个循环程序。若循环体内又包含有循环程序，则称为多重循环。

3. 修改部分

为执行循环而修改某些参数，如地址指针，计数器或某些变量。

4. 控制部分

判断循环是否结束，通常判断循环是否结束主要有两种方法

(1)用计数器控制循环，这种方式一般用于循环次数已知的情况。

(2)用条件控制循环，用于循环次数未知，根据条件决定是否结束。下面举例来说明循环程序的组成。

例1 计算 $Y=\sum_{i=1}^{20}a_i$

设 $a_1,a_2,\cdots,a_{20}$ 是一组无符号十六位二进制数，并设其和不大于2字节。

解 定义数组名 TABL 存放 $a_1\sim a_{20}$；和存放于单元 YY 中。中间结果存于寄存器 AX 中。BX 寄存器为地址指针，CX 寄存器作计数器。

程序如下：

```
DATA   SEGMENT
TABL   DW a1,a2,…a10
       DW a11,a12,…,a20
YY     DW?
DATA   ENDS
CODE   SEGMENT
       ASSUME  CS:CODE,DS:DATA
START:MOV  AX,DATA
       MOV  DS,AX
       MOV  AX,0                    ┐
       MOV  BX,OFFSET TABL          ├初始化
       MOV CX,20                    ;┘
LOP:   ADD AX,[BX]                  ;循环体
       INC BX                       ┐
       INC BX                       ├修改部分
       DEC CX                       ;┘
       JNZ LOP                      ;控制部分
       MOV YY,AX
       MOV AH,4CH
       INT 21H
CODE   ENDS
```

END START

从程序中看到，如果不用循环结构，程序中需要有20条加法指令，用循环程序后明显缩短了程序的长度。

一、循环的基本结构

在循环程序中初始化、循环体、修改和控制这四部分是密切相关的，但这四部分的顺序是可以改变的。根据循环控制部分的位置不同，循环的基本结构可分为先执行后判断和先判断后执行两种。

1. 先执行，后判断结构

这种循环结构的流程图见图5.6，其特点是循环控制部分在循环体之后，因此不管条件如何，循环体至少被执行一次。

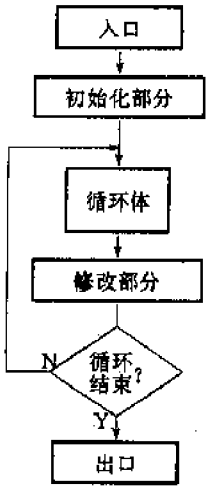


图5.6 先执行后判断

例2 编程序统计数据块中正数的个数，并将其送RS单元中。

解 数据块定义为D1，RS单元存放正数的个数，数据块长度用COUNT表示。BX作地址指针，CX作计数器，DX记录正数的个数，程序流程图见图5.7。程序如下：

```
DATA    SEGMENT
D1      DB,-1,-3,5,7,-9,…,-6
COUNT   EQU  $-D1
RS      DW  ?
DATA    ENDS
CODE    SEGMEND
        ASSUME  CS:CODE,DS:DATA
START:MOV  AX,DATA
      MOV  DS,AX
      MOV  BX,OFFSET D1
      MOV  CX,COUNT
      MOV  DX,0
LOP1: MOV  AL,[BX]
      CMP  AL,0
      JLE  JUSI
      INC  DX
JUSI: INC  BX
      DEC  CX
      JNZ  LOP1
      MOV  RS,DX
      MOV  AH,4CH
      INT  21H
CODE  ENDS
      END  START
```

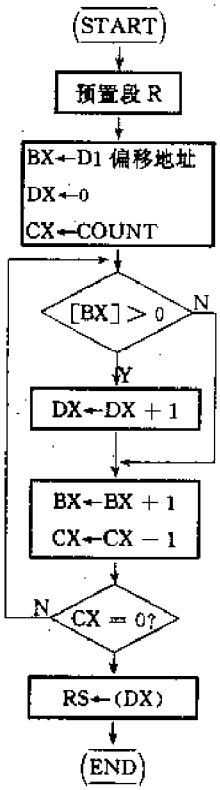


图5.7

2. 先判断，后执行结构

此种结构的流程图见图5.8，其特点是循环控制部分在循环体之前，在初始化后先判断循环的结束条件，因此此种结构有可能循环体一次都不执行。

例3 AX 寄存器中有一个16位的二进制数，编程序统计其中值为1的位的个数，并将结果存于 CX 中。

解 由于 AX 有可能为0，若 AX＝0则不必再往下作统计工作，因此可采用先判断，后执行结构

程序流程图如图5.9，程序段如下：

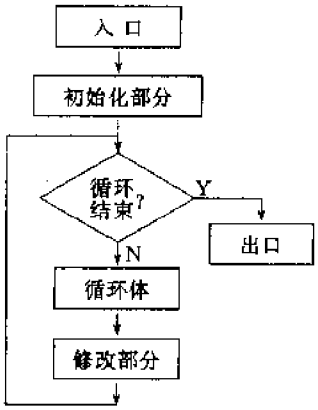


图5.8

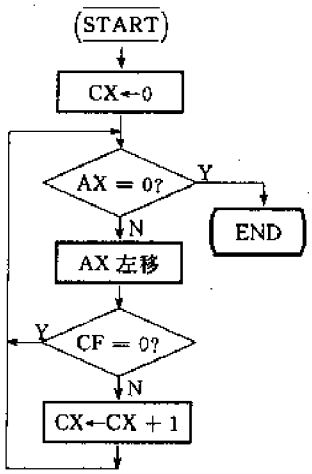


图5.9

```
    ⋮
MOV   CX,0
LOP:AND AX,AX      ;建标志位
    JZ  STOP       ;AX=0停机
    SAL  AX,1
    JNC  NOD
    INC  CX
NOD:JMP  LOP
STOP:HLT
    ⋮
```

二、循环控制方式

如何控制循环是循环程序设计中的重要环节，常用的循环控制方式有两种，对计数次数已知的循环常采用计数循环，而当计数次数无法事先确定时，要根据问题中的某些条件来决定循环是否结束，但此条件必须可以检测。

1. 计数控制

用于计数次数已知的情况，可采用正计数或倒计数法。

例4 编程序计算 $C=\sum_{i=1}^{4}a_ib_i=a_1b_1+a_2b_2+a_3b_3+a_4b_4$

设 a_i、b_i 为8位无符号数，运算结果不超过16位二进制数。

解 BUFA 中存放 a_1,a_2,a_3,a_4，BUFB 中存放 b_1,b_2,b_3,b_4，存储单元 C 中放结果。SI 放 BUFA 偏移地址，DI 放 BUFB 的偏移地址，CX 作计数器。

程序流程图见图5.10

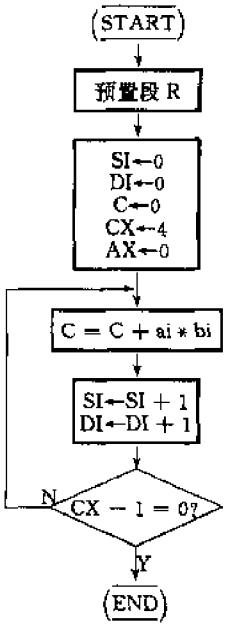


图5.10

程序如下：

```
DATA    SEGMENT
BUFA    DB  X,X,X,X
COUNT   EQU  4
BUFB    DB  X,X,X,X
C       DW  ?
DATA    ENDS
CODE    SEGMENT
        ASSUME  CS:CODE,DS:DATA
BEGIN:  MOV  AX,DATA
        MOV  DS,AX
        MOV  SI,0
        MOV  DI,0
        MOV  CX,COUNT
        MOV  C,0
        MOV  AX,0
LL:     MOV  AL,BUFA[SI]
        MUL  BUFB[DI]
        ADC  C,AX
        INC  SI
        INC  DI
        LOOP  LL
        MOV  AH,4CH
        INT  21H
CODE    ENDS
        END BEGIN
```

2. 条件控制

当计数次数无法确定，但循环次数与某些条件有关，并且条件可检测时，可采用条件控制，例如检测，比较或判断等形式。

例5 在一串字符串中查找某一关键字'#'，若查到，记下存放该关键字单元的偏移地址，若找不到用－1作标志。

解 设串变量名 STRING，关键字单元 CHAR，关键字单元的偏移地址放 FLAG 单元。寄存器 SI 为字符串的地址指针，CX 为字符串长度，AL 用于放关键字'#'。

程序流程图与程序如下：

```
DATA    SEGMENT
STRING  DB 'THIS IS A  #  OF…'
COUNT   EQU  $ -STRING
```

```
CHAR    EQU  '#'
FLAG    DW   ?
DATA    ENDS
CODE    SEGMENT
        ASSUME CS:CODE,DS:DATA
START:  MOV  AX,DATA
        MOV  DS,AX
        LEA  SI,STRING
        MOV  AL,CHAR
        MOV  CX,COUNT
LP:     CMP  AL,[SI]
        JZ   FOUND
        INC  SI
        LOOP LP
        MOV  FLAG,-1
        JMP  RETDOS
FOUND:  MOV  FLAG,SI
RETDOS: MOV  AH,4CH
        INT  21H
CODE    ENDS
        END  START
```

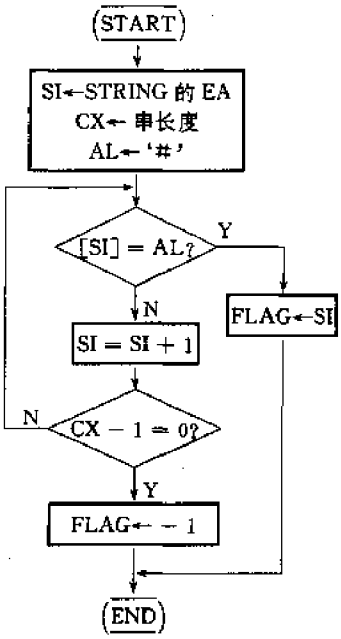


图5.11

三、多重循环

循环体中又有循环称多重循环。在多重循环程序设计时，应分别考虑每重循环的控制条件，注意每次通过外层循环进入内层循环时，初始条件必须重新设置。经常，内层循环要受外层循环的控制。

例6 在首地址为A的存储单元中有N个数的数组，编程序将此N个数按从大到小的次序排列。

表5.1

序号	数	比较次数		
1	8	8	16	84
2	5	16	84	32
3	16	84	32	16
4	84	32	8	8
5	32	5	5	5

本题排序采用汽泡排序法，从第一个数开始依次对相邻的两个数进行比较，如果次序对则不做任何操作，如果次序不对则将此两个数交换位置。表5－1表示这种算法的例子。第一次先用8和5比较由于8大于5次序正确不做任何操作，再用5和16比较，次序不对应交换位置，然后再用5和84比较，次序不对交换位置，最后用5和32比较，也需交换位置，第一次5个数经4次比较后，最小的数已经到了最后。对于N个数来说，第一次应比较N－1次。第二次比较时由于最小数已经到了最后不需要再参与比较，所以只需考虑N－1个数比较，共比较N－2次。依次继续下去，最多比较N－1次就可以完成排序，程序流程图见图5.12。程序如下

```
DATA  SEGMENT
A  DW  n  DUP(?)
DATA  ENDS
```

```
CODE   SEGMENT
       ASSUME  CS:CODE,DS:DATA
START:MOV   AX,DATA
       MOV   DS,AX
       MOV   CX,N
       DEC   CX
LOOP1:MOV   DI,CX
       MOV   BX,0
LOOP2:MOV AX,A[BX]
       CMP   AX,A[BX+2]
       JG   CONT
       XCHG   AX,A[BX+2]
       MOV   A[BX],AX
CONT:ADD   BX,2
      DEC   DI
      JNZ   LOOP2
      LOOP   LOOP1
      MOV   AH,4CH
      INT   21H
CODE  ENDS
      END   START
```

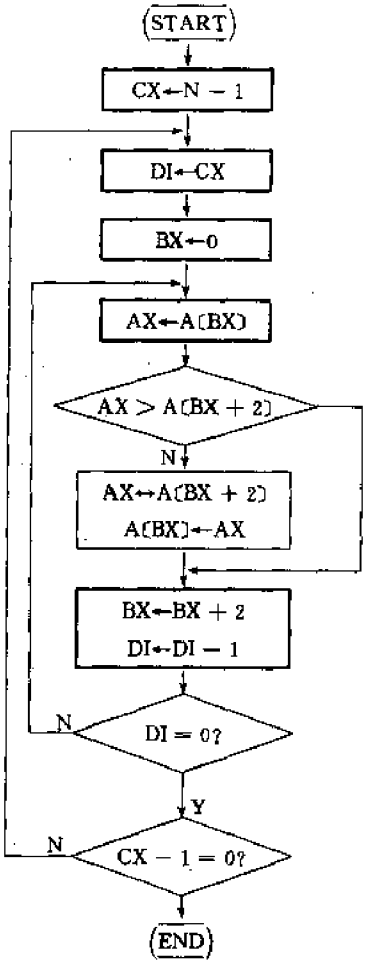


图5.12

第五节　子　程　序

程序设计过程中常常把多次引用的相同程序段编成一个独立的程序段，当需要执行这个程序段时，可以用调用指令调用它。具有这种独立功能的程序段称为过程或子程序。调用子程序的程序通常称为主程序，或调用程序，主程序向子程序的转移叫子程序调用，简称转子。

(一)子程序的设计方法

1. 适合编成子程序的程序有以下两大类

(1)程序需要反复使用，这类程序编写成子程序可避免重复编写程序，并节省大量存储空间。

(2)程序具有通用性，这类程序大家都要用到，如键盘管理程序，磁盘读写程序，标准函数程序等。编成子程序后便于用户共享。

2. 编写子程序文件

为了使用户使用方便，子程序应当以文件形式编写。子程序文件由子程序说明和子程序本身两部分构成。

(1)子程序说明部份

子程序说明部份应提供足够的信息，使不同的用户看了此部分后就知道该子程序的功能。子程序说明部分要求语言简洁、确切，一般由以下几部分组成：

(1)子程序的名称

(2)子程序功能描述
(3)使用的寄存器和存储单元
(4)子程序的入口,出口参数
(5)本子程序是否又调用其它子程序
下面是一个子程序说明部分的例子:

```
;子程序  DTOB
;将两位十进制数(BCD 码)转换成二进制数
;寄存器 BX 传递参数
;入口参数:AL 寄存器存放十进制数
;出口参数:CL 寄存器存放转换后的二进制数
;执行时间0.06ms
```

2.子程序程序部分
子程序用过程定义伪指令(PROC/ENDP)定义,子程序的编写格式如下:

```
子程序名  PROC  (NEAR/FAR)
            ⋮
          RET
            ⋮
子程序名  ENDP
```

子程序从 PROC 语句开始,以 ENDP 语句结束。程序中至少应当包含一条 RET 语句用以返回主程序。在定义子程序时应当注意其距离属性,当子程序和调用程序在同一代码段中时用 NEAR 属性,当子程序及其调用程序不在同一个代码段中时应当定义为 FAR 属性。
下面是一个编写子程序文件的例子

```
;子程序       SU
;将内存名为 BUF 的缓冲区清0
;寄存器 BX 作缓冲区的地址指针
;入口及出口参数  BUF 为名的20H 个存储单元
DATA  SEGMENT
BUF   DB  20H DUP(?)
DATA  ENDS
CODE  SEGMENT
      ASSUME  CS:CODE,DS:DATA
;清0子程序
SU    PROC  FAR
      PUSH  DS        ;
      MOV   AX,0      ; }返回操作系统
      PUSH  AX        ;
      MOV   AX,DATA
      MOV   DS,AX
      MOV   BX,OFFSET  BUF
LP:   MOV   [BX],0
```

```
        INC   BX
        CMP   BX,OFFSET   BUF+20H
        JNZ   LP
        RET
SU      ENDP
CODE    ENDS
        END   SU
```

当由DOS系统进入子程序时，子程序应当定义为FAR属性，为执行子程序后返回操作系统，在子程序的前几条指令中设置返回信息。

(二)子程序使用中的问题

1. 子程序的调用和返回

主程序调用子程序是通过CALL指令来实现，子程序执行后，通过RET指令返回主程序调用指令CALL的下一条指令继续执行主程序。一个子程序可以由主程序在不同时刻多次调用。如果在子程序中又调用了其它的子程序，称为子程序的嵌套。特别是当子程序又能调用子程序本身时，这种调用称为递归。

有关CALL指令和RET指令在第三章指令系统中已经详细介绍，这里不再重复。

2. 调用子程序时寄存器及所用存储单元内容的保护。

如果子程序中要用到某些寄存器或存储单元时，为了不破坏原有的信息，要将寄存器或存储单元的原有内容压栈保护或存入子程序不用的寄存器或存储单元中。

保护可以放在主程序中，也可以放在子程序中，但放在子程序中较好。例如：

```
SUBP    PROC   NEAR
        PUSH   AX
        PUSH   BX
        PUSH   CX
          ⋮
        POP    CX
        POP    BX
        POP    AX
        RET
SUBP    ENDP
```

用于中断服务的子程序则一定要把保护指令安排在子程序中，这是因为中断是随机出现的，因此无法在主程序中安排保护指令。

3. 子程序调用时参数的传递方法

调用程序在调用子程序时需要传送一些参数给子程序，这些参数是子程序运算中所需要的原始数据，子程序运行后要将处理结果返回调用程序。原始数据和处理结果的传递可以是数据，也可以是地址，统称为参数传递。

参数传递必须事先约定，子程序根据约定从寄存器或存储单元取原始数据，又称入口参数，进行处理后将处理结果，又称出口参数，送到约定的寄存器或存储单元返回到调用程序。参数传递一般有下面三种方法：(1)用寄存器传递(2)用堆栈传送(3)用存储单元传送。下面分别举例说明。

(1)用寄存器传递参数

寄存器传递参数适用于参数较少的情况，其特点是参数传递速度快。下面是一个用寄存器传递参数的子程序调用的例子：

```
MDATA    SEGMENT
     M1  DB  06H,75H,?
     M2  DB  14H,27H,?
     M3  DB  34H,12H,?
MDATA    ENDS
MSTACK   SEGMENT
         DW  10  DUP(?)
         TOP  LABEL  WORD
MSTACK   ENDS
MCODE    SEGMENT
         ASSUME  CS:MCODE,DS:MDATA,SS:MSTACK
START:   MOV  AX,MDATA
         MOV  DS,AX
         MOV  AX,MSTACK
         MOV  SS,AX
         MOV  SP,TOP
         MOV  BX,OFFSET  M1
         MOV  AL,[BX]
         MOV  AH,[BX+1]
         CALL  MPROC
         MOV  [BX+2],AL
         MOV  BX,OFFSET  M2
         MOV  AL,[BX]
         MOV  AH,[BX+1]
         CALL  MPROC
         MOV  [BX+2],AL
         MOV  BX,OFFSET  M3
         MOV  AL,[BX]
         MOV  AH,[BX+1]
         CALL  MPROC
         MOV  [BX+2],AL
         HLT
MPROC    PROC  NEAR
         PUSH  BX
         ADD  AL,AH
         DAA
         POP  BX
```

```
        RET
MPROC   ENDP
MCODE   ENDS
        END  START
```

上例中子程序 MPROC 为 NEAR 属性，它包含在代码段 MCODE 内，允许在本代码段中用 CALL 语句调用。子程序 MPROC 完成将 AL 和 AH 寄存器中两个压缩型十进制数相加，结果保留在 AL 寄存器中。在用 CALL 语句调用 MPROC 子程序前，入口参数送 AL 和 AH 中，子程序从 AL 和 AH 中得到传送来的原始数据处理后将出口参数送入 AL 返回主程序。

(2)用存储单元传送参数

当相互传送的参数较多时，可以用存储单元来传送。将上例稍加修改，就成为通过存储单元传递参数的实例。为简洁，相同部分省略。

```
MDATA   SEGMENT
        ⋮
YCODE   SEGMENT
        ASSUME  CS:YCODE,DS:MDATA,SS:MSTACK
START:  MOV  AX,MDATA
        MOV  DS,AX
        MOV  AX,MSTACK
        MOV  SS,AX
        MOV  SP,TOP
        MOV  BX,OFFSET  M1
        CALL  YPROC
        MOV  BX,OFFSET  M2
        CALL  YPROC
        MOV  BX,OFFSET  M3
        CALL  YPROC
        HLT
YPROC   PROC  NEAR
        MOV  AL,[BX]
        ADD  AL,[BX+1]
        DAA
        MOV  [BX+2],AL
        RET
YPROC   ENDP
YCODE   ENDS
        END  START
```

调用子程序前将操作数的偏移地址送寄存器 BX，子程序中利用 BX 寄存器间接寻址得到参加运算的操作数，运算后的结果送入存储单元返回调用程序。

(3)用堆栈传送参数

用堆栈传送子程序调用中的参数适用于相互传送参数多的情况，特别是存在嵌套或递归

调用时，采用堆栈方式不容易出错。下面的例子是将两个给定的二进制数转换为二进制数的ASCII码。

```
DATA    SEGMENT
        BIN1    DS  35H
        BIN2    DW  2B48H
        ASCBUF  DB  20H  DUP(?)
DATA    ENDS
STACK1  SEGMENT  PARA  STACK
        DW  20H  DUP(?)
STACK1  ENDS
COSE G  SEGMENT
        ASSUME  CS:COSEG,DS:DATA,SS:STACK1
BEING:MOV  AX,DATA
      MOV  DS,AX
      MOV  AX,0
      MOV  AH,BIN1
      PUSH  AX
      LEA  DI,ASCBUF
      PUSH  DI
      MOV  AX,8H
      PUSH  AX
      CALL  BINASC
      MOV  AX,BIN2
      PUSH  AX
      ADD  DI,10H
      PUSH  DI
      MOV  AX,10H
      PUSH  AX
      CALL  BINASC
      MOV  AH,4CH
      INT  21H
   ;转换子程序  BINASC
BINASC  PROC
      PUSH  DI
      PUSH  CX
      PUSH  DX
      MOV  BP,SP
      MOV  DX,[BP+12]
      MOV  DI,[BP+10]
      MOV  CX,[BP+8]
```

```
LOP:  ROL   DX,1
      MOV   AL,DL
      AND   AL,1
      ADD   AL,30H
      MOV   [DI],AL
      INC   DI
      LOOP  LOP
      POP   DX
      POP   CX
      POP   DI
      RET   6
BINASC  ENDP
COSEG   ENDS
        END   BEING
```

主程序两次调用子程序，每次调用前将被转换数，存放 ASCII 码首地址和转换位数压栈。子程序中用 BP 作基址寻址，将堆栈作为存储器用 MOV 指令将被转换数送入 DX，将 ASCII 码首地址送入 DI，将转换位数送入 CX。MOV 指令与 POP 指令不同，MOV 指令不修改 SP 的栈顶内容，且用 MOV 指令传送后，堆栈中相应单元的内容依然存在。图5.13给出了上例中堆栈的变化情况，返回指令 RET 6执行后，除了断点偏移量弹出外同时还要舍去6个字节，即将无用的参数全部清除，以便下一次压栈时堆栈中没有无用的参数所占据。

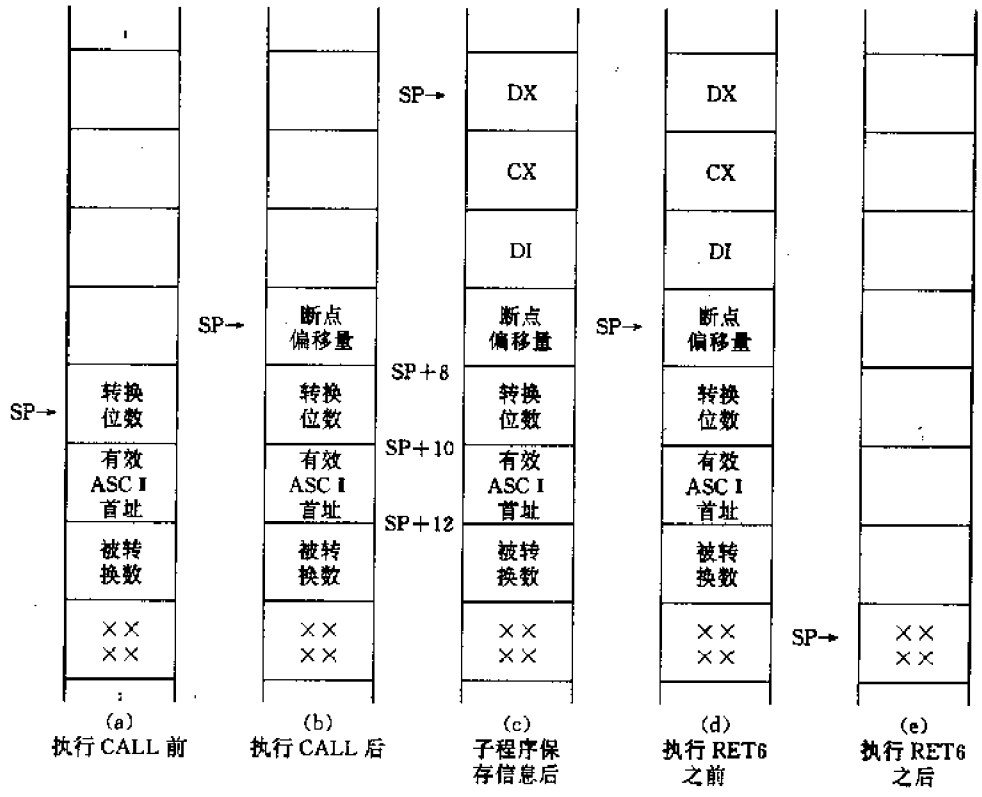


图 5.13

(三)DOS 功能子程序

为程序员编写汇编语言源程序时提供方便,MS－DOS 系统中设置了几十个内部子程序,可供用户调用。调用时需要使用软中断指令:

INT n

其中 n 为软中断调用的类型号,有 n＝20H,21H,23～24H,2FH,33H 和67H 等多种。其中 DOS 功能子程序的调用使用中断指令"INT 21H",它共有80多个功能子程序,可以完成磁盘读写控制,文件管理,存储管理,基本输入/输出管理等功能。

系统功能子程序调用成为汇编语言编写的重要工具,程序员不必了解所使用设备的物理特性、接口方式及内存分配等,不必编写繁锁的控制程序。调用功能子程序时需要用以下方式:

(1)传送入口参数到指定寄存器中

(2)调用的功能号送入 AH 寄存器中

(3)INT 21H

若子程序无入口参数,则只需要安排后2个语句,调用结束后,系统将出口参数送到指定寄存器或从屏幕显示出来。下面选择部分常用的系统功能调用作简要说明:

1. 带显示的键盘输入单字符(1号功能)

格式:MOV AH,01H

INT 21H

功能:按下任何一个键,将其字符的 ASCII 码送入 AL 中,并在屏幕上显示该字符。

如果按下的是 Ctrl-Break 键,则中止程序执行。1号功能调用,无入口参数,出口参数在 AL 中。

2. 不带显示的键盘输入单字符(8号功能)

格式: MOV AH,08H

INT 21H

功能:同1号功能,但字符不在屏幕上显示。

3. 键盘输入字符串(0AH 号功能)

格式: MOV DX,缓冲区偏移量

MOV AH,0AH

INT 21H

功能:将字符串写入内存缓冲区中

在使用0AH 号功能调用时,应当注意以下几点:

(1)应当先在内存中建立一个缓冲区,缓冲区的第一个字节给定该缓冲区能存放的字节数,第二个字节留给系统填写实际键入的字符个数,从第三个字节开始存放键入的字符串,最后键入回车键(↵)表示字符串结束。

(2)设置的缓冲器容量应当多于希望键入的字符数,若字符数超过缓冲器容量,则后面输入的字符被略去。

(3)调用时,用 DS:DX 指向缓冲区的段基值:偏移量。

下面是0AH 号功能调用的一个例子:

⋮

```
BUF   DB   30H
      DB   ?              ;定义缓冲区
      DB   30H  DUP(?)
  ⋮
MOV   DX,OFFSET  BUF
MOV   AH,0AH              ;0AH 号系统功能调用
INT   21H
```

4. 单字符显示(2号功能)

格式：
```
MOV   DL,'字符'
MOV   AH,02H
INT   21H
```

功能：将置入 DL 寄存器中的字符在屏幕上显示输出。

5. 单字符打印(5号功能)

格式：
```
MOV   DL,'字符'
MOV   AH,05H
INT   21H
```

功能：将置入 DL 寄存器中的字符送打印机接口，打印输出。

6. 字符串输出(9号功能)

格式：
```
MOV   DX,字符串偏移量
MOV   AH,09H
INT   21H
```

功能：在屏幕上显示字符串

在使用9号功能调用时应当注意以下问题：

(1)要显示的字符串必须先放在内存一数据区中，字符串以'$'符号作为结束标志。

(2)应当将字符串首地址的段基值和偏移量分别存入 DS 和 DX 中

例如：
```
CHAR  DB"I  AM  STUDENT",OAH,ODH,"$"
        ⋮
      MOV   DX,OFFSET  CHAR
      MOV   AH,09H
      INT   21H
```

上例中第一行用于将显示的字符串放入数据区中，其中 OAH 为回车，ODH 为换行。

7. 直接输入、输出单字符(6号功能)

格式：
```
MOV   DL,输入输出标志
MOV   AH,06H
INT   21H
```

功能：执行键盘输入操作或屏幕显示输出操作。执行这两种操作的选择由 DL 寄存器中的内容决定。

(1)当(DL)＝0FFH 时，执行键盘输入操作

(2)当(DL)≠0FFH 时，表示将 DL 中内容送屏幕显示输出。

8. 返回操作系统(4CH 号功能)

格式：　MOV　AH,4CH
　　INT　21H

功能：结束当前程序的执行，返回操作系统，这时屏幕上将显示操作系统提示符(C>)。

9. 设置系统日期(2BH 功能)

格式：　MOV　CX,年号
　　MOV　DH,月号
　　MOV　DL,日期
　　MOV　AH,2BH
　　INT　21H

功能：设置有效的年、月、日。当 AL＝0时，设置成功，当 AL＝0FFH 时设置失败。

10. 设置系统时间(2DH 功能)

格式：　MOV　CH,小时
　　MOV　CL,分
　　MOV　DH,秒
　　MOV　AH,2DH
　　INT　21H

功能：设置有效的时间。同样设置是否成功看 AL 寄存器的值。当 AL＝0时，设置成功，否则 AL＝0FFH。

对于其它的 DOS 功能调用，请查阅有关资料。

(四)子程序举例

例1　回车/换行标准显示输出子程序

```
CRLF  PROC  FAR
      PUSH  DX
      PUSH  AX
      MOV   DL,0DH     ;回车的 ASCII 码为0DH
      MOV   AH,02H
      INT   21H
      MOV   DL,0AH     ;换行的 ASCII 码为0AH
      MOV   AH,02H
      INT   21H
      POP   AX
      POP   DX
      RET
CRLF  ENDP
```

例2　用变址寻址实现多个16位数的加法

```
DATA  SEGMENT
  TABLE  DW  1234H,5678H,…,EAFBH     ;10个数
  LSBA   DW?
  HSBA   DW?
DATA  ENDS
```

```
STACK1   SEGMENT  PARA  STACK
         DB  64  DUP(0)
STACK1   ENDS
CODE     SEGMENT
MP       PROC  FAR
         ASSUME  CS:CODE,DS:DATA,SS:STACK1
         PUSH  DS
         SUB  AX,AX          ;返回操作系统
         PUSH  AX
         MOV  AX,DATA
         MOV  DS,AX
         LEA  BX,TABLE
         MOV  SI,00H
         MOV  AX,TABLE[SI]
         MOV  CX,09H
AGIN:    ADD  SI,02H
         ADD  AX,TABLE[SI]
         ADC  DX,00H
         LOOP  AGIN
         MOV  LSBA,AX
         MOV  HSBA,DX
         RET
MP       ENDP
CODE     ENDS
         END
```

例3 菜单选择程序。编写标准输入/输出子程序及信息输出标准程序，并编制主程序实现菜单选择。

子程序如下：

```
DATAS  SEGMENT
MENU   DB  ODH,OAH,'MESSAGE  DEMONSTRATION  PROGRAM',ODH,
       OAH
       DB  ODH,OAH,'Press 1 or 2 for Messages or CTRL/C to Stop!'
MESS1  DB  ODH,OAH,'Message Nomber One',ODH,OAH
MESS2  DB  ODH,OAH,'Message Number Two',ODH,OAH
SESS3  DB  ODH,OAH,'You hit an invalid key',ODH,OAH
DATAS  ENDS
STACKS  SEGMENT  STACK
        DW 20 DUP(?)
STACKS  ENDS
CODES  SEGMENT
```

```
        ASSUME  CS:CODES,DS:DATAS,SS:STACKS;
标准输入子程序
STDIN   PROC    FAR
        MOV     AH,01H
        INT     21H
        RET
STDIN   ENDP
;标准输出子程序
STDOUT      PROC    FAR
            PUSH    DX
            MOV     DL,AL
            MOV     AH,2H
            INT     21H
            POP     DX
            RET
STDOUT      ENDP
;信息输出标准子程序
STDME       RPOC    FAR
            PUSH    SI
            PUSH    AX
STDME1:     MOV     AL,[SI]
            INC     SI
            CMP     AL,0
            JZ      STDMES
            CALL    STDOUT
            JMP     STDME1
STDMES:     POP     AX
            POP     SI
            RET
STDME   ENDP
;菜单选择主程序
MAIN        PROC    FAR
            MOV     AX,DATAS
            MOV     DS,AX
MAINO:      LEA     SI,MENU
            CALL    STDME
            CALL    STDIN
MAIN1:      CMP     AL,'1'
            JNZ     MAIN2
            LEA     SI,MESS1
```

```
        CALL   STDME
        JMP    MAIN4
MAIN2:  CMP    AL,'2'
        JNZ    MAIN3
        LEA    SI,MESS2
        CALL   STDME
        JMP    MAIN4
MAIN3:  LEA    SI,MESS3
        CALL   STDME
MAIN4:  JMP    MAINO
MAIN    ENDP
CODES   ENDS
        END    MAIN
```

第六节　算术运算、代码转换和串操作

(一)算术运算

计算机中执行的是二进制数的运算，但是在实际生活中，人们都习惯用十进制数，希望能直接进行十进制数的输入和输出。为了实现十进制数的运算，可采用以下两种方法：

1. 数制转换

首先将输入的十进制数转换为二进制数，计算机进行二进制数的运算，运算结束后，再将结果转换为十进制数输出。

2. 直接用十进制数运算

给计算机提供能直接进行十进制数运算的指令。众所周知，计算机内部数据表达都是以二进制数形式，如果要表示一位十进制数，通常要用四位二进制数，所以叫作二一十进制数，即BCD码。计算机内部要直接进行十进制数运算，就需要有能进行BCD码运算的运算指令。目前计算机内部实现BCD码运算的方法有以下两种：

(1)指令系统中专门设置直接进行BCD码的加、减、乘、除运算指令

(2)先用二进制数的加、减、乘、除指令进行运算，然后对运算结果进行BCD码的校正，这种方法只需增加校正指令即可实现十进制数的运算，80486CPU就是采用此种方法。有关BCD码校正指令已在第三章作了介绍。

例1　多位十进制数(组合型BCD码)取补程序。采用主程序子程序结构，调用子程序时要求用堆栈来传递数据，取补后的十进制仍存放在原处。

对多位十进制数取补采用的方法为求反加1。例如设四位十进制数为0803，其补码求法如下：

```
求反        F F F F
        -)  0 8 0 3
        ------------
            F 7 F C
校正        9 1 9 6
加1     +)        1
        ------------
            9 1 9 7
```

程序编制如下：

```
DATA    SEGMENT
NUM     DB  03H,08H
DATA    ENDS
STACK1  SEGMENT  PARA  STACK
        DW  20H  DUP(0)
STACK1  ENDS
COSEG   SEGMENT
        ASSUME  CS:COSEG,DS:DATA,SS:STACK1
START:  MOV  AX,DATA
        MOV  DS,AX
        LEA  AX,NUM
        PUSH  AX                    ;数首址压入堆栈
        MOV  AX,4
        PUSH  AX                    ;位数压入堆栈
        CALL  SUB1
        MOV  AH,4CH
        INT  21H
;十进制数取补子程序
SUB1    PROC
        MOV  BP,SP
        MOV  SI,[BP+4]              ;取首址
        MOV  CX,[BP+2]              ;取位数
        SHR  CX,1                   ;一次处理二位十进制数
        XOR  AL,AL
        CLC                         ;清进位位
SUBNEXT: MOV  AL,FFH
        SBB  AL,[SI]                ;求反
        DAS                         ;减法校正
        MOV  [SI],AL                ;送回
        INC  SI
        DEC  CX
        JNZ  SUBNEXT
        MOV  SI,[BP+4]              ;以下为加1,将求反
        MOV  CX,[BP+2]              ;校正后的数取出
        SHR  CX,1
        STC                         ;CF=1
ADDNEXT: MOV  AL,[SI]
        ADC  AL,O                   ;加1
        DAA                         ;加法校正
```

```
        MOV  [SI],AL              ;送回
        INC  SI
        DEC  CX
        JNZ  ADDNEXT
        RET  4
SUB1    ENDP
COSEG   ENDS
        END  START
```

(二)代码转换

代码转换是计算机应用中经常遇到的问题。输入设备(如键盘)提供的是字符和数字的ASCII码,而计算机内部是以二进制数进行运算的,因此必须将ASCII码转换为二进制数。当数据处理结束后,又需要进行相反的转换,即将二进制数转换成ASCII码形式,以便在CRT显示器上显示。实现代码转换可以用硬件或软件。在实际应用中,软件方法较为常用,其不足之处是转换时程序执行时间较长。下面举几个代码转换的例子。

例2 编程序将十六位二进制数转换成四位十六进制数的ASCII码

十六进制数的ASCII码与二进制数之间的对照关系如表6-2所示。

表6-2 ASCII码和二进制数对照

十六进制数字符	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
ASCII码	30H	31H	32H	33H	34H	35H	36H	37H	38H	39H	41H	42H	43H	44H	45H	46H
二进制数	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111

从表6-2可看到数字0～9,它的二进制数与ASCII码相差30H,而A～F的二进制数与ASCII码相差37H,所以在转换时应当先对四位二进制数进行判断,如果是0000～1001,只需加上30H,若是1010～1111,则加上37H。

程序编制如下:

```
DATA    SEGMENT
BIN1    DW   1001110011010111B
HEX1    DB   4  DUP(0)
DATA    ENDS
STACK1  SEGMENT  PARA  STACK
        DW   20H  DUP(0)
STACK1  ENDS
COSEG   SEGMENT
        ASSUME  CS:COSEG,DS:DATA,SS:STACK1
BINHEX: MOV  AX,DATA
        MOV  DS,AX
        MOV  CH,4                 ;16进制数个数
        LEA  DI,HEX1              ;
        MOV  BX,BIN1              ;取待转换的二进制数
CONV1:  MOV  CL,4
```

```
        ROL   BX,CL              ;循环左移4位,将最高4位
        MOV   AL,BL              移至最低位
        AND   AL,OFH
        CMP   AL,09H             ;是0～9吗?
        JLE   ASCI
        ADD   AL,07H             ;是 A～F
ASCI:   ADD   AL,30H
        MOV   [DI],AL            ;存结果
        INC   DI
        DEC   CH                 ;计数
        JNZ   CONV1
        MOV   AH,4CH
        INT   21H
COSEG   ENDS
        END   BINHEX
```

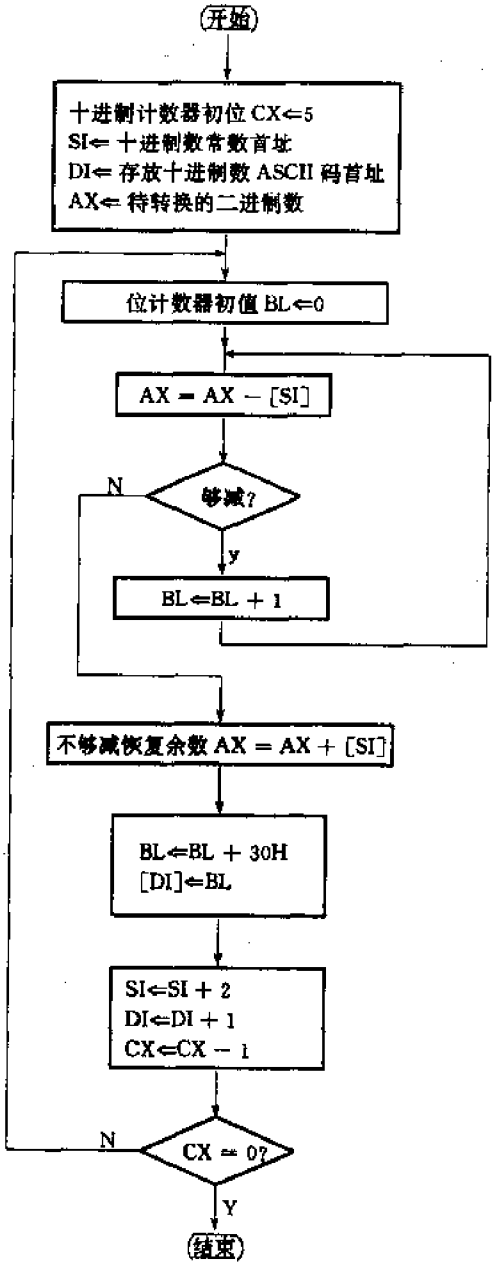
例3 将16位二进制数转换为十进制数的 ASCII 码

16位二进制数可表示的最大十进制数为$2^{16}-1=65535$,即5位十进制数,把二进制数转换为十进制数可用减法,二进制减去10000的次数,就是十进制数的万位,余数减去1000的次数,为十进制数的千位,依次类推,最后余下的是十进制数的个数。

十进制数10000,1000,100,10和01以常数形式存放于数据段中,用一个计数器记录二进制数减去上述5个常数中某一个的次数,计数器记录的值即为十进制数某位的数值,再加上30H,便转换为一位十进制数的 ASCII 码。注意用减法先得到的是十进制数的最高位。程序流程图如下:

源程序编写如下:

```
DATA    SEGMENT
BIN     DW  0110110110101001B
CONST   DW  10000,1000,100,10,1
DEC5    DB  5  DUP(0)
DATA    ENDS
STACK1  SEGMENT  PARA  STACK
        DW  20H  DUP(0)
STACK1  ENDS
COSEG   SEGMENT
        ASSUME  CS:COSEG,DS:DATA,SS:STACK1
BINDEC: MOV  AX,DATA
        MOV  DS,AX
        MOV  CX,5        ;转换后十进制数的位数
        LEA  SI,CONST    ;转换常数首址
        LEA  DI,DEC5     ;存十进制数 ASCII 码首址
        MOV  AX,BIN
```




```
CONV3:  MOV  BL,0
LOP1:   SUB  AX,[SI]
        JC   NEXT
        INC  BL
        JMP  LOP1
NEXT:   ADD  AX,[SI]     ;不够减,恢复余数
        OR   BL,30H      ;变十进制数 ASCII 码
        MOV[DI],BL       ;存结果
        INC  SI
        INC  SI
        INC  DI
```

```
        LOOP   CONV3
        MOV    AH,4CH
        INT    21H
COSEG   ENDS
        END    BINDEC
```

(三)串操作

串操作常用于文本编辑、数据处理等方面，串操作指令已在第三章中作了详细介绍，主要指令为取串指令(LODS)，存串指令(STOS)，串传送指令(MOVS)，串比较指令(CMPS)和串搜索指令(SCAS)。要特别注意的是，串操作指令中的源操作数地址由 DS:SI 提供，目标操作数地址由 ES:DI 提供。下面举例说明串操作的应用。

例4 编程序比较 BUF 和 STRING 两个字符串，将不相同字符前相同的字符个数送 RESU 单元中，设字符个数≤255。

每个字符占一个字节，故用指令 CMPSB，并选用重复前缀 REPE。

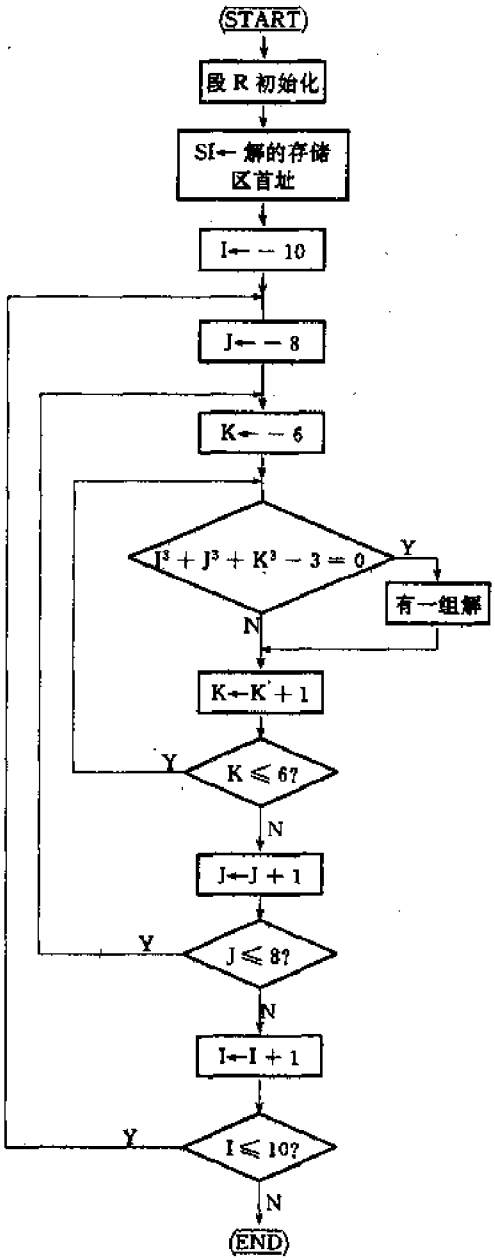
```
DATA   SEGMENT
BUF    DB'ABCDEFGHIJKLMNOP'
CUNT   EQU   $-BUF
STRING DB'ABCDEFGHIJKOMNOP'
RESU   DB?
DATA   ENDS
CODE   SEGMENT
       ASSUME CS:CODE,DS:DATA,ES:DATA
COMPS:MOV AX,DATA
       MOV DS,AX
       MOV ES,AX
       MOV SI,OFFSET BUF          ;取源串首址
       MOV DI,OFFSET STRING       ;取目标串首址
       MOV CX,CUNT
       MOV BX,CUNT
       CLD
       REPE  CMPSB                ;两个字符串比较
       JZ ENDO                    ;两个字符串若相同则转移
       SUB SI,OFFSET BUF          ;中间有不相同字符取前面
       MOV BX,SI                  ;相同字符的个数
       DEC BX                     ;去掉一个不相同的字符
ENDO   MOV RESU,BL
       MOV AH,4CH
       INT 21H
CODE   ENDS
       END COMPS
```

第七节　实用程序设计举例

例1　求 $I^3+J^3+K^3=3$ 的全部整数解，其中 $-10\leqslant I\leqslant 10$，$-8\leqslant J\leqslant 8$，$-6\leqslant K\leqslant 6$。

解：此题采用筛选法来实现，即在给定的 I,J,K 的范围内，对每一组 I,J,K 的组合进行测试看它们是否满足给定的方程。完成本例需要用三重循环。

程序流程图和程序如下：




```
DATA   SEGMENT
 ANS   DB 60 DUP(0)          ;存放结果
 I3    DW ?                  ;两个中间工作单元
```

```
 J3     DW ?
DATA   ENDS
CODE   SEGMENT
     ASSUME CS:CODE,DS:DATA
START:MOV    AX,DATA
      MOV    DS,AX
      MOV    CH,-10          ;I 初值──→CH
      LEN    SI,ANS
L1:   MOV    AL,CH
      IMUL   CH              ;
      IMUL   CH              ;  I³──→I3
      MOV    I3,AX
      MOV    CL,-8           ;J 初值──→CL
L2:   MOV    AL,CL
      IMUL   CL              ;
      IMUL   CL              ;  J³──→J3
      MOV    J3,AX
      MOV    BH,-6           ;K 初值──→BH
L3:   MOV    AL,BH
      IMUL   BH              ;
      IMUL   BH              ;  K³──→AX
      ADD    AX,I3
      ADD    AX,J3
      SUB    AX,3
      JNZ    L4              ;不是解跳转 L4
      MOV    [SI],CH
      MOV    [SI+1],CL       ;是解,存放结果
      MOV    [SI+2],BH
      ADD    SI,3            ;修改 SI
L4:   INC    BH              ;K+1──→K
      CMP    BH,6
      JLE    L3              ;K≤6转 L3
      INC    CL              ;J+1──→J
      CMP    CL,8
      JLE    L2              ;J≤8转 L2
      INC    CH              ;I+1──→I
      CMP    CH,10
      JLE    L1              ;I≤10转 L1
      MOV    AH,4CH
      INT    21H
```

```
CODE    ENDS
        END     START
```

例2 表插入，删除程序。设内存数据区 DS:2000H 地址开始有150个 ASCII 码字符组成的表，以 $ 符结束。(1)从表首插入五个"!"符，删除表中的第31至第50元素，显示修改后的内容。(2)由键盘输入一个字符，删除表中的这个元素，显示表内容。

解：表插入元素时，先将指定位置以后的全部元素向后移动，留出空位后将元素插入。

删除表中元素时，只要将后面元素向前移动，覆盖要删除的元素。

程序如下：

```
DATA SEGMENT
   ORG 2000H
A0 DB 0DH,0AH
A1 DB 10 DUP('QWERTYUIO')
   DB 50 DUP ('0')
   DB '$'
DATA ENDS
STACK1 SEGHENT PARA STACK
        DW 100 DUP(?)
STACK1 ENDS
CODE SEGMENT
ASSUME CS:CODE,DS:DATA
ASSUME SS:STACK1,ES:DATA
STA  PROC   FAR
    PUSH    DS
    XOR     AX,AX
    PUSH    AX
    MOV     AX,DATA
    MOV     DS,AX
    MOV     ES,AX
    CALL    IT
    STD
    MOV     CX,100      ;
    LEA     SI,A1
    MOV     DI,SI
    ADD     SI,99
    ADD     DI,104
    REP     MOVSB
    MOV     CX,5
    MOV     AL,'!'
    REP     STOSB
```

显示字符表子程序 IT，在表中插入5个!字符

```
CLD
LEA     SI,A1
ADD     SI,50
ADD     DI,30       ;删除第31～50元素显示插入、删除后的表元素
MOV     CX,100
REP     MOVSB
CALL    IT
MOV     AH,1
INT     21H         ;键盘输入
    LEA   SI,A1
T1:CMP    BYTE PTR[SI],'$'
    JZ    T3
    CMP   AL,[SI]   ;查找要删除的元素，遇到$符结束
    JZ    T2
    INC   SI
    JMP   T1
T2:CALL   EX4
    JMP   T1        ;调用数据块移动子程序EX4，调显示表元素子程序IT
T3:CALL   IT
    RET
STA ENDP
IT PROC
    LEA DX,A0
    MOV AH,9        ;显示表元素子程序IT
    INT 21H
    RET
IT ENDP
EX4 PROC
     PUSH SI
     MOV DI,SI
     INC SI
R: MOVSB
     CMP   BYTE PTR[DI-1],'$'
     JZ    R1       ;数据块移动子程序EX4
     JMP   R
R1:POP     SI
     RET
EX4 ENDP
CODE ENDS
      END STA
```

例3　编写如下矩阵相乘的程序：

$$\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} \cdot \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ b_4 \end{bmatrix} = \begin{bmatrix} c_1 \\ c_2 \\ c_3 \\ c_4 \end{bmatrix}$$

计算公式为：

$$c_i = \sum_{j=1}^{4} a_{ij} \cdot b_j \qquad (i=1,2,3,4)$$

展开来写就是：

$$c_1 = a_{11}b_1 + a_{12}b_2 + a_{13}b_3 + a_{14}b_4$$
$$c_2 = a_{21}b_1 + a_{22}b_2 + a_{23}b_3 + a_{24}b_4$$
$$c_3 = a_{31}b_1 + a_{32}b_2 + a_{33}b_3 + a_{34}b_4$$
$$c_4 = a_{41}b_1 + a_{42}b_2 + a_{43}b_3 + a_{44}b_4$$

我们首先考虑编出计算一个 c_i(先固定 $i=1$)的程序。为了便于循环，把矩阵 A 的元素按行依次相邻存放，即 $a_{11},a_{12},a_{13},a_{14},a_{21},a_{22},a_{23},a_{24},a_{31},\cdots$，把向量 B 和 C 的元素也依次相邻存放，在数据段里如下定义数据项：

```
A  DB   a11,a12,a13,a14
   DB   a21,a22,a23,a24
   DB   a31,a32,a33,a34
   DB   a41,a42,a43,a44
B  DB   b1,b2,b3,b4
C  DW 4 DUP(?)
```

则它们的存储分配如下：

A[0]:a_{11}	A[4]:a_{21}	A[8]:a_{31}	A[12]:a_{41}
A[1]:a_{12}	A[5]:a_{22}	A[9]:a_{32}	A[13]:a_{42}
A[2]:a_{13}	A[6]:a_{23}	A[10]:a_{33}	A[14]:a_{43}
A[3]:a_{14}	A[7]:a_{24}	A[11]:a_{34}	A[15]:a_{44}
B[0]:b_1	C[0]:c_1		
B[1]:b_2	C[2]:c_2		
B[2]:b_3	C[4]:c_3		
B[3]:b_4	C[6]:c_4		

其中，小写字母 $a_{11},a_{12},\cdots,a_{44},b_1,\cdots,b_4$ 等均代表具体的无符号数。

这样，计算 c_1 的框图如图5.14中的内层循环所示。

紧接着应考虑 c_2 的计算，从框图上看，计算 c_2 和计算 c_1 有两点不同：一是数据 A 从 a_{21} 开始而不是 a_{11}，也就是从 A[4]开始，SI 从4开始而不是从0开始；二是结果存放在 C[2]而不是 C[0]。其中，第一点不同，由于在计算 c_1 的过程中 SI 每次加1，到计算完 c_1 时，SI 的值恰好是4，这一要求自然满足；第二点不同，可以利用基址寄存器 BX，开始给 BX 送初值0，每循环一次加2，利用 C[BX]间接寻址方式访问 C_i 的方法解决。c_3、c_4 的计算也是一样的。这里出现了二重循环，计算一个 c_i 的循环为内循环，计算所有 c_i 的循环为外循环。内、外循环都是用 CX 作为计数器，因此需要将控制计算4个 c_i 的 CX 的计数值暂时保存起来。本程序通过堆栈保存和恢复。

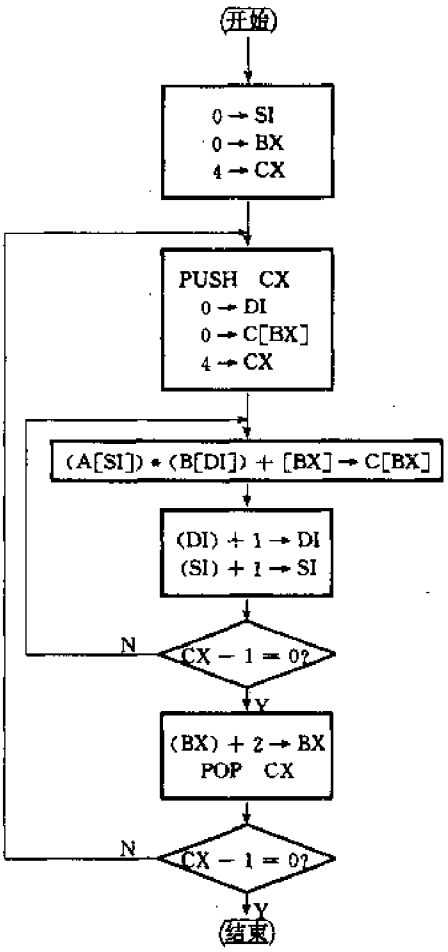


图5.14 矩阵计算程序框图

根据框图很容易编写出程序如下(对矩阵任意假定一组数据)：

```
DATA    SEGMENT
A       DB    1,0,2,3
        DB    0,1,1,0
        DB    3,0,1,0
        DB    4,2,0,1
B       DB    0,1,1,0
C       DW    4DUP(?)
DATA    ENDS
STACK   SEGMENT STACK'STACK'
STAPN   DW    100DUP(?)
TOP     EQU   LENGTH STAPN
STACK   ENDS
CODE    SEGMENT
        ASSUME CS:CODE,DS:DATA,SS:STACK
START:
        MOV AX,DATA
```

```
        MOV DS,AX
        MOV AX,STACK
        MOV SS,AX
        MOV SP,TOP
        MOV SI,0                    ;以 SI 作为 a_ij 值指针,并赋初值0
        MOV BX,0                    ;以 BX 作为 c_i 值指针,并赋初值0
        MOV CX,4                    ;以 CX 作为外循环计数器,并置初值4
LOOP1:
        PUSH CX                     ;将 CX 压栈保存
        MOV DI,0                    ;以 DI 作为 b_j 值指针,并赋初值0
        MOV WORD PTR C[BX],0;给 C[BX]清0
        MOV CX,4                    ;CX 也为内循环计数器
LOOP2:                              ;进入内循环
        MOV AH,0
        MOV AL,A[SI]
        MUL B[DI]                   ;a_ij * b_j ——→AX
        ADD C[BX],AX                ;a_ij * b_j + c_i-1 ——→c_i
        INC SI                      ;SI 指针加1
        INC DI                      ;DI 指针加1
        LOOP LOOP2                  ;内循环结束
        ADD BX,2                    ;BX 指针加2
        POP CX                      ;弹出 CX
        LOOP LOOP1                  ;外循环结束
        MOV AH,4CH                  ;返回 DOS
        INT 21H
CODE    ENDS
        END START
```

例4 编程序实现 BCD 码的加法运算。(1)从键盘输入两个6位的十进制数。(2)将键盘输入的 ASCII 码 转换成压缩 BCD 码。(3)两个 BCD 码相加。(4)将压缩的 BCD 码转换成 ASCII 码并显示结果。程序编写如下：

```
    ·MODEL  SMALL
DISPLAY  MACRO TEXT             ;定义宏用于显示字符串
         MOV DX,OFFSET TEXT
         MOV AH,9
         INT 21H
         ENDM
        ·CODE
  START:
          MOV     AX,@DATA
          MOV     DS,AX
```

```
        MOV     ES,AX
        DISPLAY MSG1                ;宏调用,显示 MSG1开始的字符串
        CALL    READ_NUM            ;输入被加数(子程序调用)
        MOV     DI,OFFSET ASC1
        CALL    STOR_NUM            ;保存被加数
        DISPLAY MSG2                ;宏调用,显示 MSG2开始的字符
        CALL    READ_NUM            ;输入加数
        MOV     DI,OFFSET ASC2
        CALL    STOR_NUM            ;保存加数
        MOV     SI,OFFSET ASC1
        MOV     DI,OFFSET BCD1
        CALL    ASCTOBCD            ;将被加数转换成 BCD 码
        MOV     SI,OFFSET ASC2
        MOV     DI,OFFSET BCD2
        CALL    ASCTOBCD            ;将加数转换成 BCD 码
        CALL    BCDADD              ;两个 BCD 码数相加
        CALL    BCDTOASC            ;将结果转换成 ASCII 码
        DISPLAY MSG3                ;宏调用,显示结果
        MOV     AX,4C00H            ;返回 DOS
        INT     21H
;Input number
READ_NUM    PROC                    ;键盘输入子程序
        MOV     DX,OFFSET BUFSIZE
        MOV     AH,0AH
        INT     21H
        RET
READ_NUM ENDP
;store number
STOR_NUM    PROC                    ;保存数据子程序
        MOV     SI,OFFSET NUMFLD
        MOV     CH,0
        MOV     CL,NUMLEN
        REP   MOVSB                 ;重复将[SI]中的字符串存放于[DI]
        RET
STOR_NUM    ENDP
;Convert ASCII to BCD
ASCTOBCD    PROC                    ;ASCII 码转换为压缩 BCD 码子程序
        ADD     SI,4                ;源指针指向 ASCII 码数据的次低位
        ADD     DI,2                ;目标指针指向 BCD 码数据的最低位
        MOV     CL,4
```

```
                MOV     DX,3                ;6位压缩 BCD 码数只占3个字节
ATB1:           MOV     AH,[SI]             ;次低位 ASCII 码数——→AH
                MOV     AL,[SI+1]           ;最低位 ASCII 码数——→AL
                SHL     AL,CL               ;AL 中内容左移4位
                SHL     AX,CL               ;转换成压缩 BCD 码在 AH 中
                MOV     [DI],AH             ;保存 BCD 码
                SUB     SI,2
                DEC     DI
                DEC     DX
                JNZ     ATB1
                RET
    ASCTOBCD    ENDP
    ;Convert BCD to ASCII
BCDTOASC        PROC                        ;"压缩 BCD 码——→ASCII 码"子程序
                MOV     DX,4                ;BCD 码字节数为4,作为初值赋给 DX
                MOV     SI,OFFSET BCD3+3
                MOV     DI,OFFSET ASC3+7
BTA1:           MOV     AL,[SI]             ;取 BCD 码
                AND     AL,0FH
                OR      AL,30H              ;将低位转换成 ASCII 码
                MOV     [DI],AL             ;保存低位
                DEC     DI
                MOV     AL,[SI]             ;取同一字节 BCD 码
                AND     AL,0F0H
                MOV     CL,4
                SHR     AL,CL               ;将高位转换成 ASCII 码
                OR      AL,30H
                MOV     [DI],AL             ;保存高位
                DEC     DI
                DEC     SI                  ;指针指向 BCD 码的前一字节
                DEC     DX
                JNZ     TAB1                ;DX≠0,未转换完,转 TAB1
                RET                         ;DX=0,转换完4字节,返回
BCDTOASC ENDP
;BCD add
BCDADD          PROC                            ;两个6位数的 BCD 码相加子程序
                MOV     SI,OFFSET BCD1+2
                MOV     DI,OFFSET BCD2+2
                MOV     BX,OFFSET BCD3+3
                MOV     CX,3
```

```
                MOV     AH,0
                CLC
BADD1:          MOV     AL,[SI]
                ADC     AL,[DI]
                DAA
                MOV     [BX],AL
                DEC     SI
                DEC     DI
                DEC     BX
                LOOP    BADD1
                JNC     EXIT                    ;6位数相加后,CF=0,退出
                MOV     DL,1                    ;CF=1,将1送BCD3+0单元
                MOV     [BX],DL
EXIT:           RET
BCDADD          ENDP
                .DATA
BUFSIZE  DB     7
NUMLEN   DB     ?
NUMFLD   DB     7DUP(' ')
MSG1     DB     13,10,'Please enter first number: $'
MSG2     DB     13,10,'Please enter second number: $'
MSG3     DB     13,10,'The result:'
ASC1     DB     6 DUP('0')                      ;为被加数 ASCII 码分配6个字节
PLUS     DB     '+'
ASC2     DB     6DUP('0')                       ;为加数 ASCII 码分配6个字节
EQUAL    DB     '='
ASC3     DB     8DUP('0')13,10,' $'             ;为和数 ASCII 码预留8个字节
BCD1     DB     3DUP(0)                         ;为被加数 BCD 码预留3个字节
BCD2     DB     3DUP(0)                         ;为加数 BCD 码预留3个字节
BCD3     DB     4DUP(0)                         ;为和数 BCD 码预留4个字节
         MOV    [BX+SI],AX
         MOV    WORD PTR[BP-2],1                ;局部参数置"1"
INCS:    ADD    SI,2
         LOOP   COMP
         TEST   WORD PTR[BP-2],1                ;测试局部参数是否为"0"
         JZ     FINISH                          ;是"0",转结束
         SHR    SI,1                            ;地址除以2为下次元素个数
         MOV    CX,SI
         JMP    SORTA
FINISH:  POP    SI
```

```
        MOV  SP,BP
        POP  BP
        RET
CPROTC  ENDP
        END
```

例5 读入一个文本文件，将大写的字符转换成小写字符并将结果显示在屏幕上。首先，使用 DOS 的功能调用打开文件，如果打开不成功，则出错返回；否则使用顺序读方式逐字节地读文件内容，转换成小写字符，显示在屏幕上，直到文件结束。

```
DATA    SEGMENT
FILENM  DB   'TEST',00                    ;文本的文件名
BUF     DB   2DUP(0)
MSG1    DB   'ERROR!',0DH,0AH,'$'
DATA    ENDS
CODE    SEGMENT
        ASSUME   CS:CODE,DS:DATA
START:
        MOV      AX,DATA
        MOV      DS,AX
        MOV      DX,OFFSET FILENM
        MOV      AX,3D00H
        INT      21H                     ;打开文件
        JB       ERR                     ;打开不成功,转出错处理
        MOV      BX,AX                   ;文件标识送入 BX
READ_LOOP:
        MOV      AH,3FH
        MOV      CX,1
        MOV      DX,OFFSET BUF
        INT      21H                     ;读入文件一个字符
        CMP      AX,1                    ;读入成功?
        JNZ      DONE                    ;文件结束,返回 DOS
        MOV      AL,BUF                  ;取出文件内容
        CALL     LOWER                   ;调子程序,将字符转换成小写
        MOV      AH,02
        MOV      DL,AL
        INT      21H                     ;显示该字符
        JMP      READ_LOOP               ;继续读文件
DONE:
        MOV   AH,3EH
        INT   21H                        ;关闭文件
        MOV   AH,4CH
```

```
        INT     21H                         ;返回 DOS
ERR:
        MOV     DX,OFFSET MSG1
        MOV     AH,9
        INT     21H                         ;显示错误信息
        MOV     AH,4CH
        INT     21H                         ;返回 DOS
LOWER
        PROC    NEAR
        CMP     AL,'A'
        JB      LRET
        CMP     AL,'Z'
        JA      LRET
        ADD     AL,32                       ;将大写字符转换成小写字符
LRET:   RET
LOWER   ENDP
CODE    ENDS
        END     START
```

例6 在 IBM PC/AT 微机的主板上有一片由电池供电的 CMOS 存储器，它保存微机系统的硬件信息。CMOS 中有多个数据区域，保存多种不同的信息，每个区域都有一个指定的标识号。若想访问某个数据区域，只需将它的标识号送到70H 端口，然后从71端口读取相应的信息。下面的程序用来读取当前的日期(世纪、年、月、日)。

```
DISPLAY  MACRO TEXT                         ;定义宏，用于显示当前日期
         MOV       DX,OFFSET TEXT           ;将字符串缓冲区首址送 DX
         MOV       AH,9
         INT       21H                      ;显示字符串
         ENDM
         .DATA
DATE     DB        13,10,Century:           ;13和10分别为回车、换行符 ASCII 码
CENTURY  DB        ?,?
         DB        13,10,'Year:'
YEAR     DB        ?,?
         DB        13,10,'Month:'
MONTH    DB        ?,?
         DB        13,10,'Day:'
DAY      DB        ?,?
         DB        '$'                      ;字符串结束符
         .CODE
START:   MOV       AX,@DATA
         MOV       DS,AX
```

```
GET_CENTURY:                          ;读"世纪"
        MOV     AL,32H                ;32H 为世纪标识号
        OUT     70H,AL                ;输出标识号
        IN      AL,71H                ;取世纪值
        CALL    BCDTOASC              ;将世纪值转换成 ASCII 码
        MOV     CENTURY,AH
        MOV     CENTURY+1,AL          ;保存结果
GET_YEAR:                             ;取"年"
        MOV     AL,09H
        OUT     70H,AL
        IN      AL,71H
        CALL    BCDTOASC
        MOV     YEAR,AH
        MOV     YEAR+1,AL
GET_MONTH:                            ;取"月"
        MOV     AL,08H
        OUT     70H,AL
        IN      AL,71H
        CALL    BCDTOASC
        MOV     MONTH,AH
        MOV     MONTH+1,AL
GET_DAY:                              ;取"日"
        MOV     AL,07H
        OUT     70H,AL
        IN      AL,71H
        CALL    BCDTOASC
        MOV     DAY,AH
        MOV     DAY+1,AL
        DISPLAY DATE                  ;宏调用,从 DATA 地址开始显示,
                                        直至遇到"$"
        MOV     AX,4C00H              ;退出本程序,返回 DOS
        INT     21H
Convert BCD to ASCII                  ;BCD 码转换为 ASCII 码子程序
    BCDTOASC PROC
        MOV     AH,AL
        MOV     CL,4                  ;将高位 BCD 码转换为 ASCII 码,
        SHR     AH,CL                 ;存于 AH
        OR      AH,30H
        AND     AL,0FH                ;将低位 BCD 码转换为 ASCII 码,
                                        存于 AL
```

```
            OR          AL,30H
            RET
BCDTOASC    ENDP
.STACK
            END  START
```

思考题与习题

1. 编写程序段计算(各变量均为字节变量)

(1)$Z=((X-Y)/10+W)\times 4$

(2)$Z=X^2+34XY-X/Y$

2. 两个 ASCII 码数串定义如下：

```
STR1  DB '352678'
STR2  DB '123400'
```

编写程序段，计算 STR1+STR2，要求结果仍然是 ASCII 码数串

3. 若程序中数据定义如下：

```
FIRST   DB  30  DUP(?)
SECOND  DB  30  DUP(?)
THIRD   DB  30  DUP(?)
```

编写程序将 FIRST 和 SECOND 中的30个字节数分别相减，结果存放到 THIRD 中。

(1)假定数据为无符号数，若有借位则保持结果为0。

(2)假定数据为带符号数，若溢出则保存结果为-1。

4. 将 MBUF 为起始单元的5个数按相反次序传送到 NBUF 单元开始的存储单之中。

5. 试编写程序，完成下列数字表达式的功能。

$$Y=\begin{cases}1 & (x\geqslant 1)\\ 0 & (-1<x<1)\\ -1 & (x\leqslant -1)\end{cases}$$

6. 将内存中某数据块中的正数和负数分开，并分别将这些正数和负数送同一数据段的两个缓冲区中，并在屏幕上显示正数和负数的个数。

7. 试分析下列两个程序段执行的功能

(1)
```
CLD
LEA   DI,[0406H]
MOV   CX,0080H
XOR   AX,AX
REP   STOW
```

(2)
```
MOV   CX,10
LEA   SI,FIRST
LEA   DI,SECOND
REP   MOVSB
```

8. 从内存 BLOCK 开始，存放着256个字节的带符号数，编写程序从这些数中找出绝对值最大的数，将其存入 MAX 中。

9. 将数据段中自 ARER1开始的100个字数据搬移到附加段中以 ARER2开始的区域中。

(1)用 MOV 指令(2)用重复串传送指令

10. 逐条注释下列两个程序段的每条指令，并说明它们完成的功能。

(1)
```
        LEA  BX,ARRAY
        LEA  DI,RESULT
        MOV  CL,4
AGAIN:  MOV  AL,[BX]
        TEST AL,80H
        JZ   NEXT
        NEG  AL
        MOV  [DI],AL
        INC  BX
        INC  DI
        DEC  CL
        JNZ  AGAIN
```

(2)
```
        MOV  AL,0
        MOV  SI,-1
        MOV  CX,100
LOP:    INC  SI
        MOV AL,A1[SI]     ;A1、A2和 SUM
        ADD AL,A2[SI]     为数据区中的
        MOV SUM[SI],AL    缓冲区
        LOOPNZ  LOP
        JZ   NEXT
ZERO:   RET
NEXT:   INC  CX
        JMP  ZERO
```

11. 从 FIRST 开始存放着100个字符的字符串，结束符为'$'。编写程序，统计该字符串中字母'B'的个数。

12. 三个数据区 A1，A2，A3中分别存放30个学生三门功课的成绩(百分制)，编写子程序计算每个学生三门功能的平均分，并存放在数据区 A4中。

13. 一个十进制数 ABC＝12345678，用压缩 BCD 码格式存储如下：

ABC DB 78H,56H,34H,12H　　编写程序：

(1)将 ABC 转换成二进制数　(2)将 ABC 转换成 ASCII 码

14. 编写程序实现

(1)从键盘输入4位十六进制数的 ASC I 码，并将其转换成4位十六进制数存入 DX 寄存器中。

(2)将 EAX 的内容以十六进制格式显示在计算机屏幕上。

15. 编程序将内存中10个无符号数据字排序(从小到大)后存放在原来的区域内。

16. 编程序打印输出自然数1000以内的素数。

17. 编程序求正数 X 和 Y 的最大公因数和最小公倍数。

18. 设某年级200个学生数学课的成绩以压缩 BCD 码存入变量 BUF 中，编程序统计(1)高于80分，低于60分，60～80分之间的学生人数，仍以压缩 BCD 码分别存入 GOOD、PASS、BAD 变量中。(2)计算全年级平均成绩，也以压缩 BCD 形式存入变量 AVRG 中(舍去小数点后的数)。

19. 编程序求级数 $1^2+2^2+3^2+\cdots$ 的前 n 项和刚大于1000的项数 n。

20. 编写程序求 N!

第六章　存储器

第一节　概　　述

存储器是计算机的重要组成部分，用于存放程序和数据。计算机在运行中大部分的总线周期都是对存储器的读/写访问，因此存储器系统性能的好坏将在很大程度上影响计算机的性能。

目前大多数微机系统都采用分级结构来组成整个存储器系统。通常存储器可分为4级：内部寄存器组、高速缓冲器、内存储器和外存储器。

1. 内部寄存器组

内部寄存器组是最高一级的存储器。在微型计算机中，内部寄存器组与CPU共同集成在一片芯片上。寄存器可以存放待使用的数据和运算的中间结果。由于微处理器对寄存器的读/写速度快，一般在一个时钟周期内即可完成，充分利用并恰当安排寄存器，可以提高系统的功能。但是，由于寄存器是制作在微处理器内部的，受芯片面积和集成度的限制，寄存器的数量是有限的。

2. 高速缓冲存储器(Cache)

Cache为第二级存储器。Cache由高速存储器芯片组成，其速度可与微处理器的速度相匹配，容量可达几百K字节。Cache中装载当前用得最多的程序和数据，使微处理机能快速的对其访问，以提高微机的运算速度。

3. 内存储器

内存储器的特点是容量较大，而其速度较慢。由于降低了对存储器芯片速度的要求，就可能以低价格实现大容量。内存的容量可从几兆字节～几十兆字节，用来存放程序和数据。内存储器常分为RAM(随机读/写存储器)和ROM(只读存储器)。只读存储器用以解决微机初始化操作和启动操作系统等工作。ROM的存取速度比较慢，达300～400ns，但这种存储器只在机器启动时运行，因此对系统性能并无大的影响。

4. 外存储器

磁盘、软盘、硬盘、光盘都称为外存储器。它们的特点是容量大，但其存取速度比内存要慢得多。由于外存储器平均存储费用低，所以大量用作后备存储器，存储大量的数据和程序。在高档微机中，硬盘、光盘等外存广泛用作虚拟存储器的硬件支持。

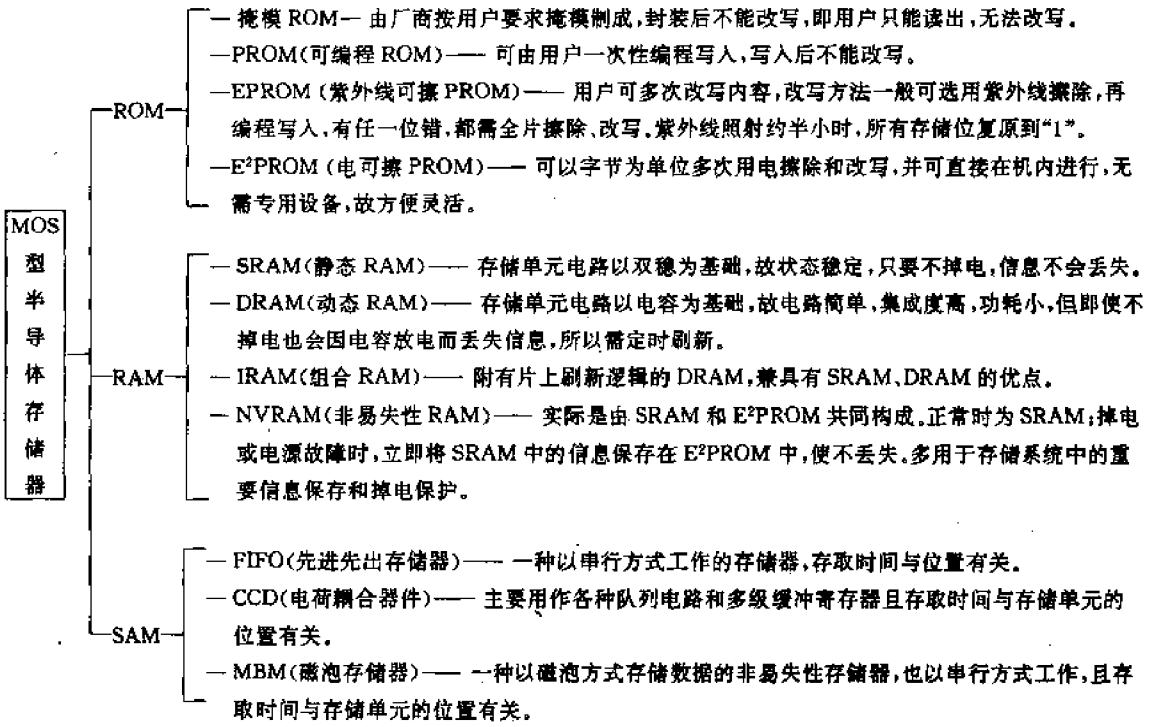
上述4级存储器可以根据微机系统的性能要求来配置。

第二节　存储器芯片及其选择

微型计算机的内存采用半导体存储器。半导体存储器按照制造工艺不同，可分为双极型和MOS型两大类。微型计算机中的内存和Cache一般使用MOS型存储芯片，MOS型半导体存储器从功能和应用角度可分为ROM、RAM和SAM(顺序读写存储器)，表6.1列出了MOS

型半导体存储器的分类情况及简单原理和功能。

表 6.1 存储芯片分类



1. 只读存储器 ROM(Read-Only Memory)

ROM 是一种只读存储器，存储器中内容一经写入，在工作过程中就只能读出不能重写，即使掉电，写入的内容也不会丢出。ROM 主要用于存放各种系统软件（如 ROMBIOS、监控程序等）。目前常用的 ROM 芯片有掩模 ROM、PROM、EPROM、E²PMOM 和闪电存储器等。

(1)掩膜 ROM

掩膜 ROM，是指厂家在生产过程中根据用户提供的程序对芯片图形（又称掩膜）进行光刻写入信息。一旦掩膜 ROM 制成后，用户不能再修改。这种 ROM 成本低，适用于大批量生产。

(2)PROM

PROM 中存放的程序是由用户根据自己需要，使用专门的 PROM 编程仪自己写入的。在写入前，存储陈列的晶体管、布线和熔断丝是完整无损的。编程时，用户可将某些存储单元中的熔断丝熔断，将信息写入存储器。PROM 成本也较低，适用于成批生产的微机产品中。

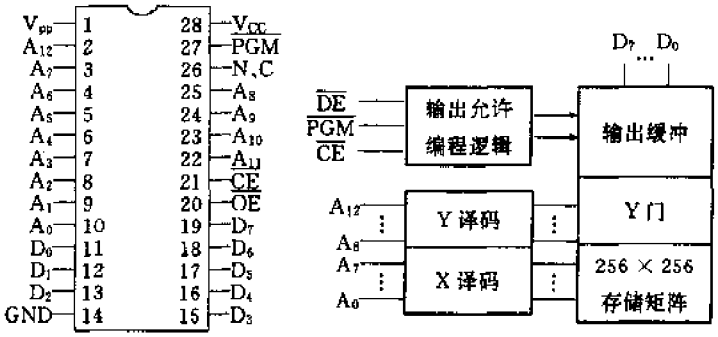
(3)EPROM

EPROM 是一种可擦只读存储器。这种存储器利用编程器将信息写入，信息可长久保留。当其中内容需要变更时，可用擦抹器（紫外线灯照射）将信息擦除，再根据需要利用 EPROM 编程器将新的信息写入，EPROM 可反复使用。

EPROM 芯片有多种型号，如 2716(2K×8)，2732(4K×8)，2764(8K×8)，27128(16K×8)，27256(32K×8)等。用户可以根据需要选用不同的芯片。下面以 2764A 为例，对 EPROM 的性能和工作方式作一介绍。

Intel 2764A 的芯片引脚和功能框图见图 6.1。

由图中看出，Intel2764A 有 13 条地址线 A_0～A_{12}。8 条数据线 D_0～D_7。一个片选端$\overline{CE}$，输出允许$\overline{OE}$和编程控制端$\overline{PGM}$。此外还有电压输入端 V_{CC}和 V_{PP}。Intel 2764A 有七种工作方式，下面介绍几种工作方式：



注：N.C 为未接

图 6.1　2764A 芯片和功能框图

①读方式

这是 2764A 通用的方式，在此方式中应将 V_{CC}和 V_{PP}接＋5V，$\overline{PGM}$接高电平。当从芯片中读信息时，先通过 A_0～A_{12}由 CPU 送地址信号到芯片，然后 CPU 发控制信号使$\overline{CE}$和$\overline{OE}$有效，经过一段时延，指定的芯片中的某单元的内容即送至数据线上并送到 CPU。

②备用方式

当$\overline{CE}$为高电平时，2764A 处于备用方式，这时芯片信息不能读出。

③编程方式

要对 Intel 2764A 写入程序，V_{PP}需接 12.5V，V_{CC}仍接 5V，$\overline{CE}=0$，$\overline{OE}=1$，并由数据线输入要存储的数据。每写一个地址单元，都必须在$\overline{PGM}$引脚给一个低电平脉冲，其宽度为 45mA，这时信息写入芯片中。

④编程禁止

在编程过程中，若使$\overline{CE}=1$，则编程被禁止。

⑤编程校验

在编程过程中，可对写入的信息进行校验。在一个字节编程完成后，使$\overline{PGM}=1$，$\overline{CE}=\overline{OE}=0$，则同一单元的内容即可由数据线输出，即可校验结果是否正确。

目前由于工业控制计算机的发展，已发展了 27C010(128K×8)，27C020(256K×8)，27C040(512K×8)等大容量的 EPROM 芯片，需要时可查有关手册选用。

(4)E^2PROM

E^2PROM 是一种可用电擦除和编程的只读存储器。其特点是可以字节为单位多次用电擦除或改写，并且无需专用设备，可直接在机内进行。由于 E^2PROM 具有 RAM 和 ROM 的双重特点，因此得到了日益广泛的应用。一般 E^2PROM 可有读方式、写方式、维持方式和擦除方式等工作方式。

2. 随机存取存储器 RAM(Random Access Memomy)

RAM 能够随时从任一单元读出或写入内容，其优点是读/写方便，使用灵活。但 RAM 中的信息不能长期保存，一旦停电时，所存信息会丢失，因此 RAM 主要用作信息的暂存。RAM

主要用于以下几个方面：

• 存放当前正在执行的程序和数据，中间运算结果和 I/O 数据等。

• 作堆栈(Stack)保护中断和子程序调用时 CPU 的现场信息。

• 作 I/O 数据缓冲存储器，如显示输出、打印输出、键盘输入缓冲存储器。

• 构成 CMOS 电路，用于保存系统当前设置的各种参数。在这种情况下需要有后备电源及掉电保护电路的支持。RAM 可分为静态 RAM，动态 RAM 和非易失性 RAM 等几类。

(1)静态 RAM

静态 RAM(Static RAM)简称 SRAM 其优点是状态稳定，不需要刷新，速度快。但 SRAM 集成度低、功耗大，成本较高，故只用于高速缓冲器和不需要大存储容量的微型计算机，如单片机、单板机中。

SRAM 分为单端口和双端口两种。通常的 SRAM 为单端口存储器，即只有一个地址输入端和数据输入/输出端。在 DMA 方式中需要外加切换控制电路，使系统的工作效率降低。双端口的 SRAM 设有两组独立的地址、数据和控制信号、常用于高速数字信号处理系统中。

常用的 SRAM 芯片有 Intel 6116，6264，62256，628128 等。下面以 Intel 6116 为例，描述 SRAM 的工作过程。

Intel 6116 芯片的容量为 2K×8 位，有 2048 个存储单元，共用 11 根地址线，7 根用于行地址译码，4 根用于列地址译码输入，每条列线控制 8 位形成 128×128 个存储陈列。其芯片引脚和功能框图如图 6.2 所示。

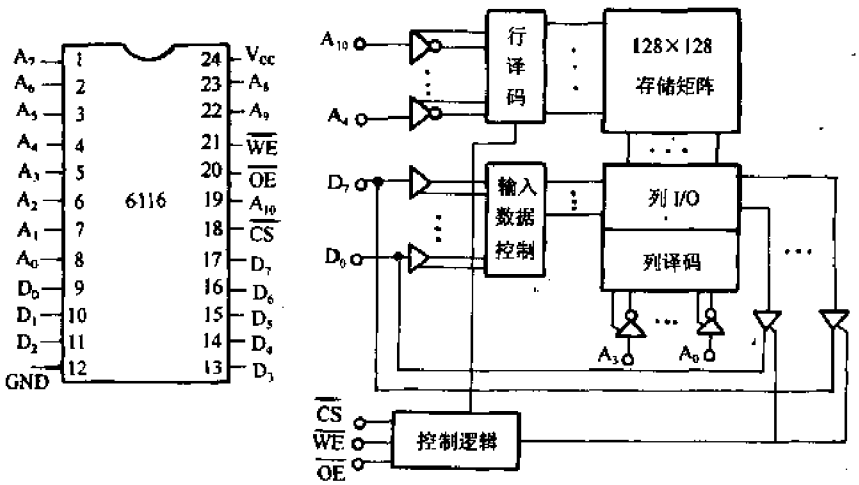


图 6.2　6116 引脚和功能图

Intel 6116 芯片的工作过程如下：

• 读出时，地址线 A_0～A_{10}送来的地址信号经行、列地址译码器译码后选中某个存储单元(8 位)，当$\overline{CS}=0$，$\overline{OE}=0$，$\overline{WE}=1$ 时构成读出逻辑，打开三态门，数据经 I/O 电路送到 D_7～D_0 后输出到 CPU。

• 写入时，地址选择与读出相同，而$\overline{CS}=\overline{WE}=0$，$\overline{OE}=1$，打开左边的三态门，数据经 D_7～D_0 和数据输入控制电路送到 I/O 电路，写入存储单元中。

• 当片送无效时，即$\overline{CS}=1$，输入/输出三态门呈高阻，从而存储芯片与系统总线被隔离。

(2)动态 RAM

动态 RAM(Dynamic RAM)简称 DRAM,其优点是集成度高、功耗小、价格低,适用于大存储量的计算机,常用的内存条由 DRAM 组成。但 DRAM 是电容充电作为存储基础,由于电容电荷会因为漏电而逐渐丢失,因此 DRAM 必须定时刷新,对电容进行充电。

常见的 DRAM 有 64K×1 位、64K×4 位、256K×1 位,256K×4 位,1M×1 位,1M×4 位等产品。下面以 Intel 2164A 为例说明其工作过程。

DRAM 芯片 2164A 的容量为 64K×1 位,8 片 2164A 构成 64KB 的存储器,每片 Intel 2164K 其芯片引脚和内部结构框图如图 6.3 所示。

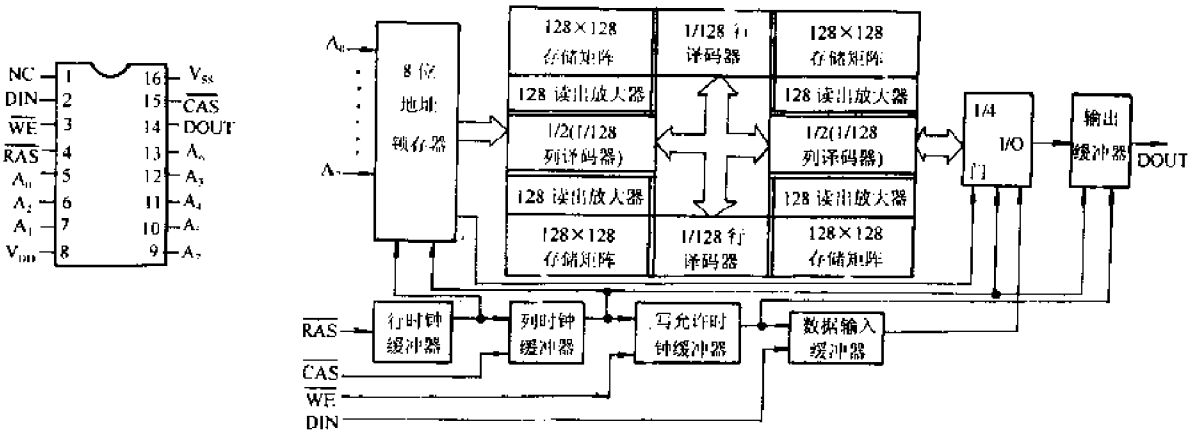


图 6.3 Intel 2164A 芯片和内部结构示意图

要寻址 64KB,通常需要 16 条地址线。为减少地址线引脚数目,采用了行地址线和列地址线分时工作,这样只需 A_0~A_7 8 条地址线。分时工作的方法是利用地址锁有器和多路开关,先由行地址选通信号$\overline{RAS}$,将 8 位地址信号送至行地址锁有器锁存,随后出现的列地址选通信号$\overline{CAS}$把后送来的 8 位地址送至列地址锁存器锁存。从内部结构框图可见,64K 存储体由 4 个 128×128 的存储矩阵组成,每个 128×128 的存储矩阵,由 7 条行地址线和 7 条列地址线进行选择。锁存在行地址锁存器中的 7 位行地址 RA_6~RA_0 同时加到 4 个存储器矩阵上,在每个存储矩阵中选中一行;锁存在列地址锁存器中的 7 位列地址 CA_6~CA_0 选中 4 个存储矩阵中的一列,选中 4 行 4 列交点的 4 个存储单元再经过 4 送 1 的 I/O 门控电路(由 RAS 和 CAS)控制选中其中的一个单元,可对该单元进行读写。2164A 数据的读出和写入是分开的,由$\overline{WE}$信号控制,当$\overline{WE}$为高电平时,读出,当$\overline{WE}$为低电平时,写入。芯片进行刷新的时候,只加上行选通信号$\overline{RAS}$,不加列选通信号$\overline{CAS}$,可以把地址加到行译码器上,使指定的 4 行存储单元只被刷新,而不读/写数,一般 2ms 可全部刷新一次。实现 DRAM 定时刷新的方法和电路有多种,可以由 CPU 通过一定控制逻辑实现,也可以用 DMA 控制器实现,还可以用专用 DRAM 控制器实现,这里不再叙述。

(3)单列直插式 DRAM 存储条

在 386 以上的微机系统中,现在都使用 SIMM(Single In Line Memory Module)封装形式塑封在一起的厚膜集成电路 DRAM 存储条来构成 32 位或 64 位数据总线的内存。目前市场上内存条按容量分为 256KB,512KB,1MB,4MB,8MB,16MB 等多种。按内存条上所装存储器的

位数分为 9 位和 8 位两种，9 位的内存条带有奇偶校验位，对硬件的适应性好。按电路板引脚数又可为 30 线和 72 线两种通用标准。在 486 和 586 系统中，72 线内存条是主流产品。

一般主板的存储器安装座分为几个体，每个体中有 2～4 个存储器安装座，可安装 2～4 个存储条。对 16 位数据线的 CPU，使用 30 线内存条时，由于每条可以提供 8 位有效数据，所以系统主板存储条安装数应为偶数，即 2 条、4 条等。对 32 位数据线的 CPU，如 486DX/DX2/DX4，最好安装 72 线内存条插座，由于 72 线的内存条可提供 32 位有效数据，所以只需安装一条存储条即可工作。对于 Pentium 类 CPU，其数据线为 64 位，必须安装 2 条或 4 条 72 线 SIMM 存储器。最近国际上又推出了 100 线的 SIMM 存储条接口标准(64 位有效数据位)，只需安装一个内存条就可支持 64 位数据的一次存取。

第三节　内存储器的构成原理

内存储器的构成与设计，一般包括以下三项工作：存储器结构的确定，存储器芯片的选择，存储器接口的设计。其中存储器接口的设计实际上就是要解决存储器同 CPU 三大总线的正确连接与时序匹配问题。而与地址总线的连接，本质上就是在存储器地址分配的基础上实现地址译码，以保证 CPU 能对存储器中的所有单元正确寻址。它又包括两方面：一是高位地址译码，用以选存储芯片；二是低位地址线连接，用以通过片内地址译码器译码选单元。

一、存储器结构的确定

存储器结构的确定，主要指采用单存储体结构还是多存储体结构。在微机系统中，为能支持各种数据宽度操作，存储器一般都按字节编址、以字节为单位构成，就是说它的数据宽度为 8 位。对于 CPU 的外部数据总线为 8 位的微机(如 8088 系统等)，其存储器只需用单体结构；而对于 CPU 的外部数据总线为 16 位的微机系统(如 8086/80186/80286 系统等)，一般需用两个 8 位存储体。比如对 80286 微机，是将 $2^{24}=16M$ 字节物理地址空间的存储器分为偶数地址的存储体和奇数地址的存储体，如图 6.4 所示。偶数地址存储体的数据线与数据总线 D_0～D_7 连接，而奇数地址存储体的数据线与数据总线 D_8～D_{15} 连接。地址总线中 A_1～A_{23} 同时连到两个存储体，A_0 作为偶数存储体的片选信号。奇数存储体则利用控制信号 $\overline{BHE}$ 作片选信号。用这

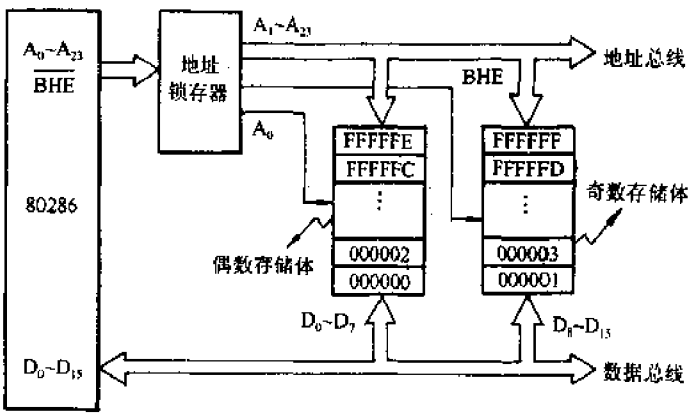


图 6.4　80286 存储器结构

种连接方法，$\overline{A_0}=0$，$\overline{BHE}=1$ 时，只是偶数地址的存储体工作；$\overline{A_0}=1$，$\overline{BHE}=0$ 时，只是奇数地址的存储体工作；当 $\overline{A_0}=0$，$\overline{BHE}=0$ 时，偶数地址的存储体和奇数地址的存储体则同时工作。在进行 16 位数据传送时，如果低 8 位字节在偶地址存储体中，高 8 位字节在奇地址存储体中（此称为字对准），一个总线周期即可完成数据传送；反之，如果低 8 位字节在奇地址存储体中，高 8 位字节在偶地址存储体中（此称为字未对准），则需两个总线周期才可完成一次传送。

对于以 80386、80486 等 32 位 CPU 为核心的微机系统，一般使用 4 个由字节组在的存储体，以支持 8 位字节操作、16 位字操作和 32 位双字操作。例如图 6.5 给出的是 80386/80486 系统的存储器结构，它将整个存储器分成 4 个存储体，分别由 $\overline{BE_0}$～$\overline{BE_3}$ 来选通，这样可以构成 32 位数据。当 $\overline{BE_0}$～$\overline{BE_3}$ 同时有效且双字对准时，在一个总线周期里就可完成 32 位数据的存储器读写操作。

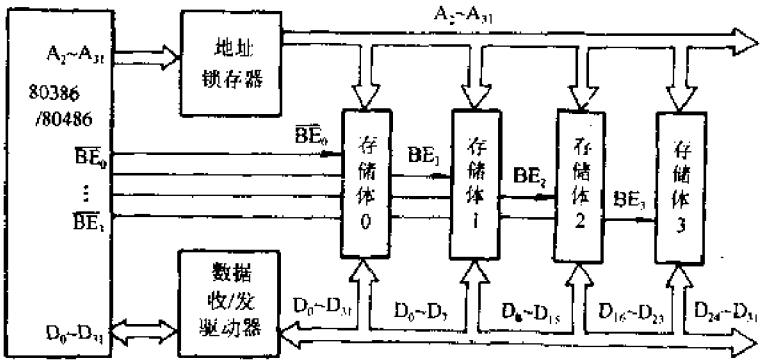


图 6.5　386/486 存储器结构

可见，8 位字长的单体存储器是构成微型计算机内存系统的基础。下面集中介绍单体存储器的构成原理。而单体存储器的构成，在选定了存储器芯片的情况下，剩下的就是存储器接口的设计了。

二、存储器片选控制方法

一般说来，一个微机系统的内存储器不可能仅由一个存储器芯片组成，而是由几片甚至几十片组成。因此，内存储器的构成原理实质上就是用多个存储器芯片构成存储器系统，并使之与 CPU 总线正确接口的原理。为了简化存储器地址译码电路设计，首先应尽量选择存储容量相同的芯片，然后将芯片地址线与低位地址总线一一相连，而将剩下的高位地址线通过译码产生片选控制信号。地址总线的高、低位划分因芯片的容量不同而异，如 4K×n 位芯片的低位地址总线为 A_0～A_{11} 共 12 位，1M×n 位芯片的低位地址总线则为 A_0～A_{19} 共 20 位，等等。其余部分均为高位地址总线。

根据对高位地址总线的译码方案不同，片选控制方法通常有线选法、局部译码法和全译码法三种。

1. 线选法

线选法除将低位地址线直接接片内地址外，将余下的高位地址线，分别作为各个存储器芯片的片选控制信号，如图 6.6(a)所示。要注意的是，这些片选地址线每次寻址时只能有一位有效（图中为低电平），不允许同时有多位有效，否则不能保证每次只选中一个芯片。

2. 局部译码法

局部译码法是对高位地址总线中的一部分(而不是全部)进行译码,以产生各存储器芯片的片选控制信号,如图 6.6(b)所示。当采用线选法地址线不够用,而又不需要全部存储空间的寻址能力时,可采用这种方法。

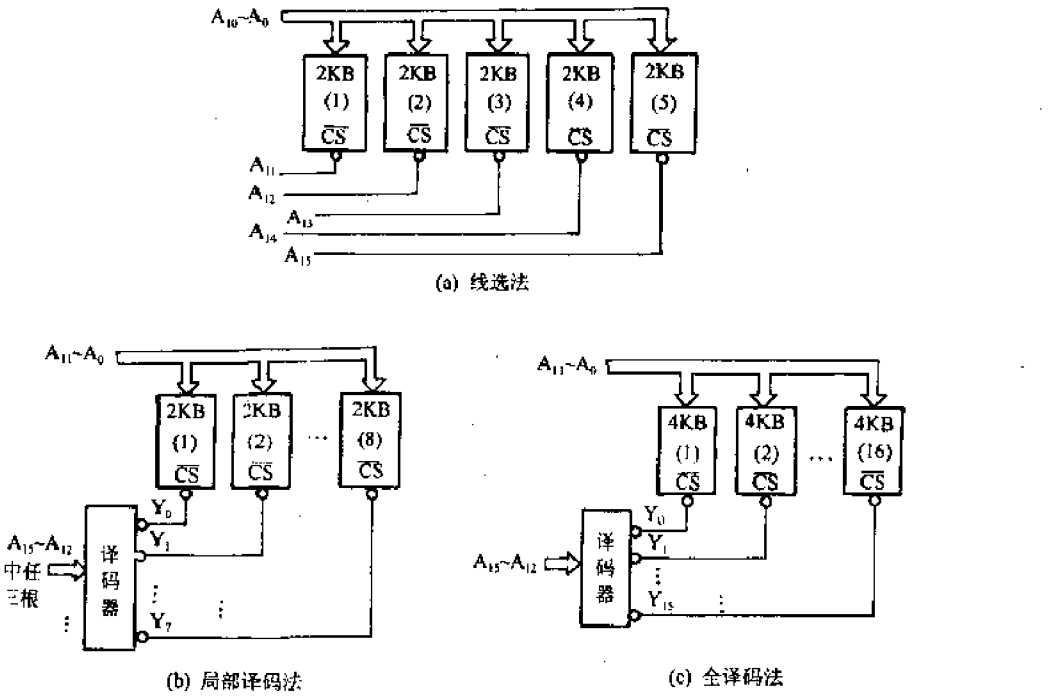


图 6.6 三种常用片选控制方法(假设地址总线为 16 根)

线选法和局部译码法的优点是电路简单(尤其线选法,无需片选译码电路),但都存在地址重叠和(或)地址不连续问题,使寻址空间利用率降低。所以一般多采用全译码法。

3. 全译码法

这种方法除了将低位地址总线直接连至各芯片的地址线外,将余下的高位地址总线全部译码,译码输出作为各芯片的片选信号,如图 6.6(c)所示。

显然,这种选址方法可以提供对全部存储空间的寻址能力。即使不需要全部存储空间,也可采用全译码法,多余的译码输出让它空着,便于需要时扩充。

三、存储器地址分配与设置

在微型计算机中,实际的存储器装机容量往往比允许的存储空间小。比如 PC/XT 中,CPU 是 8088,有 20 条地址线,可寻址的存储空间为 1MB,但系统板上实际配置的存储器只有 64KB ROM 和 256KB RAM。因此在实际存储器系统设计时,需要在地址分配的基础上进行地址设置。

在设置存储器地址时,通常可按下列步骤进行:

① 根据系统实际装机存储容量,确定存储器在整个存储空间中的位置。

② 选择合适的存储芯片,画出地址分配图或列出地址分配表。

③ 根据地址分配图或表及选用的译码器件,画出相应的地址位图,以此确定"片选"和片

内单元选择的地址线，进而画出片选译码电路。

④ 画出存储器与地址总线的接口连线图。

[**举例**] 为某 8 位微机(地址总线为 16 位)设计一个 12KB 容量的存储器，要求 EPROM 区为 8KB，从 0000H 开始，采用 2716 芯片；RAM 区为 4KB，从 2000H 开始，采用 2114 芯片。

本例对存储器的容量及其在存储空间的位置提出了明确要求，存储芯片也已给定，因此设计工作从上述第②步开始进行。

① 根据要求列出存储器地址分配表如表 6.2 所示。当然，也可画出如图 6.7 所示的存储单元地址分配图。

表 6.2 示例地址分配表

容量分配	芯片型号	地址范围	容量分配	芯片型号	地址范围
2KB	2716	0000～07FFH	1KB	2114	2000～23FFH
2KB	2716	0800～0FFFH	1KB	2114	2400～27FFH
2KB	2716	1000～17FFH	1KB	2114	2800～2BFFH
2KB	2716	1800～1FFFH	1KB	2114	2C00～2FFFH

② 画出地址位图，进而画出片选译码电路。由于采用的EPROM芯片 2716 和 RAM 芯片 2114 的存储容量不同，前者为 2KB，后者为 1KB，这就导致了这一步有两种方案：

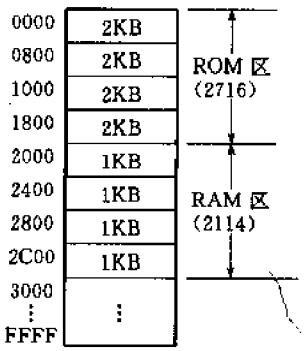


图 6.7 地址分配图

方案一——ROM、RAM 分别译码方式。

若选用 74LS139 中的两个 2—4 线译码器对 ROM 和 RAM 分别进行选片译码，则根据地址分配图/表可画出 ROM、RAM 两个地址位图，分别如图 6.8(a)和(b)所示。

比较图 6.8(a)、(b)两个地址位图可以看出，由于 ROM 区和 RAM 区的"片"地址不同，体现在地址位图上用于芯片内部译码和外部译码的地址线数目不同；同时，由于 ROM 和 RAM 在地址空间的位置不同，作为外部译码的译码输入线和译码允许地址线的逻辑值也不同，换言之，在用于外部译码的地址线中选择哪几条做译码输入、哪几条作译码允许以及它们的状态如何，是由地址分配图中的地址位置决定的。根据地址位图，即可画出存储器片选信号的外部译码电路，如图 6.9 所示。注意，图中译码允许条件中除了地址条件外还加了$\overline{\text{MERQ}}$条件，目的是区分存储器操作和 I/O 操作。

方案二——二次译码方式。

所谓二次译码方式，即先按"片"地址为 2KB 进行译码，得到一些 2KB 为一片的片选信号；再利用其中的某一条或某几条输出与一条地址线进行二次译码，得到片地址为 1KB 的片选信号。这种方法可推广到多种不同容量的存储芯片一起使用的场合，这时可通过多层译码来相继获得容量从大到小的不同芯片的片选信号。

对于本例，由于整个存储器 ROM 和 RAM 共为 12KB，按 2KB 为一片共需 6 根"片选"信号线，故可选 74LS138 作为译码器。这样，根据前面的地址分配表或分配图可画出如图 6.10 所示的地址位图。图中第一次译码为 2KB ROM 芯片提供片选信号，第二次译码为 1KB RAM 芯片提供片选信号。

外译码(选片)					内译码(选单元)										
译码允许			译码输入												
A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_9	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0
0	0	0	0	0	ROM(1)										0000~07FF
0	0	0	0	1	ROM(2)				(全 0 到全 1)						0800~0FFF
0	0	0	1	0	ROM(3)										1000~17FF
0	0	0	1	1	ROM(4)										1800~1FFF

(a) ROM 地址位图

外译码(选片)					内译码(选单元)										
译码允许				译码输入											
A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_9	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0
0	0	1	0	0	0	RAM(1)									2000~23FF
0	0	1	0	0	1	RAM(2)				(全 0 到全 1)					2400~27FF
0	0	1	0	1	0	RAM(3)									2800~2BFF
0	0	1	0	1	1	RAM(4)									2C00~2FFF

(b) RAM 地址位图

图 6.8　方案一的地址位图

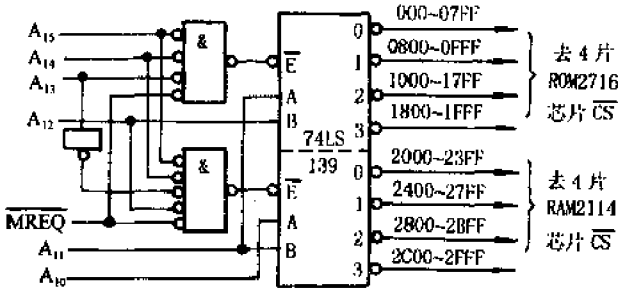


图 6.9　方案一的片选译码电路

根据图 6.10 的地址位图，可画出二次译码方式所对应的外部译码电路如图 6.11 所示。

③ 画出存储器与地址总线接口连线图。对应于上面两种方案的存储器接口连线图分别如图 6.12(a)和(b)所示。

TP801 Z－80 单板机上的存储器系统就是按照上面方案二设计的。如果在上面的设计中，没有限定存储芯片的型号，则最好选用存储容量相同的 EPROM 和 SRAM 芯片，比如选都是 2K×8 位的 EPROM2716 和 SRAM2128，便可明显简化片选译码电路的设计和连线。这样，由于所有芯片片内地址一样，只要用一个“3－8 线译码器”74LS138 即可完成全部片选的译码任务，如图 6.13 所示。而且，由于 2128 和 2716 的外形尺寸相同，引脚排列相容，因此实际组装也很方便。

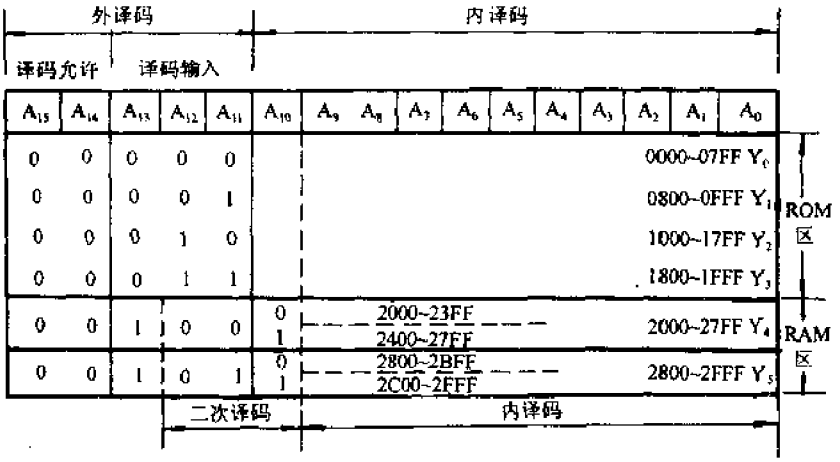


图 6.10　方案二的地址位图

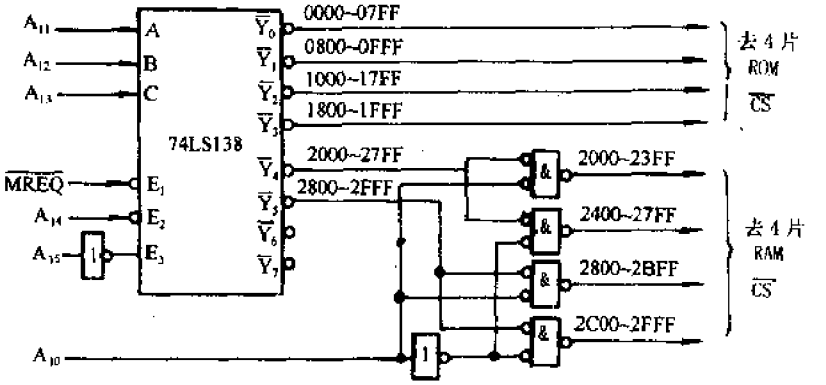


图 6.11　方案二的片选译码电路

四、读写控制线与数据总线的连接

1. 读写控制线的连接与时序匹配

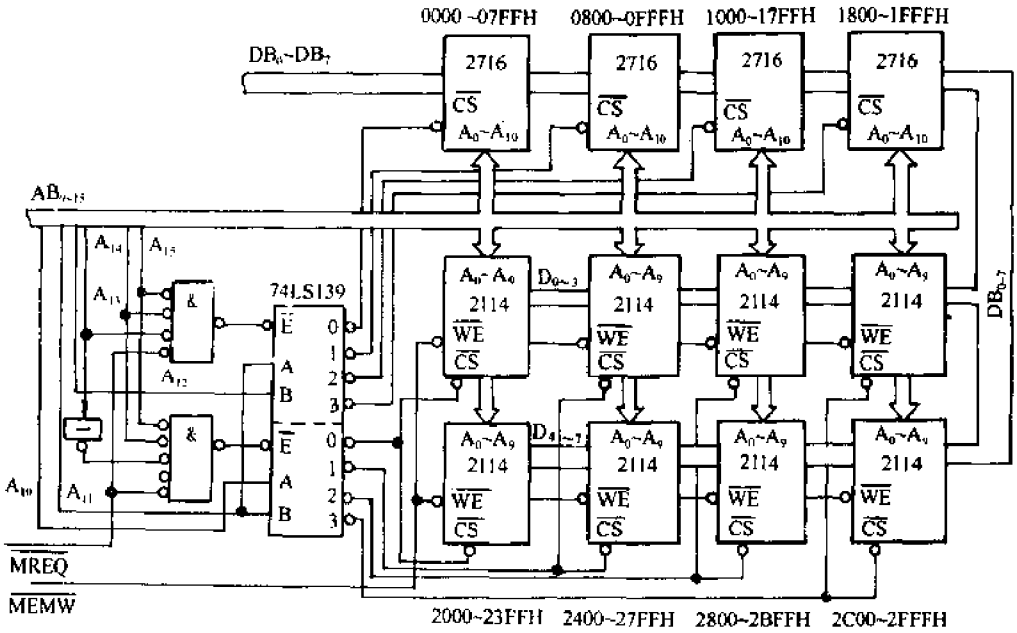
存储芯片读/写操作所要求的控制方面的条件，除如前所述的片选控制外，还有读/写控制，用于决定操作的类型。

对于工作速度与 CPU 大体相当的存储芯片，读写控制线的连接非常简单，只需将存储芯片的读/写控制端直接连到 CPU 总线或系统总线的相应功能端（如$\overline{\text{MEMR}}$和$\overline{\text{MEMW}}$信号端）即可。当然，$\overline{\text{MEMR}}$和$\overline{\text{MEMW}}$信号往往并不是由 CPU 直接提供，而是通过对 MPU 输出的有关控制信号（例如 Z80 的$\overline{\text{MREQ}}$和$\overline{\text{RD}}$、$\overline{\text{WR}}$，80486 的 M/$\overline{\text{IO}}$和 W/$\overline{\text{R}}$ 等）译码而得到。

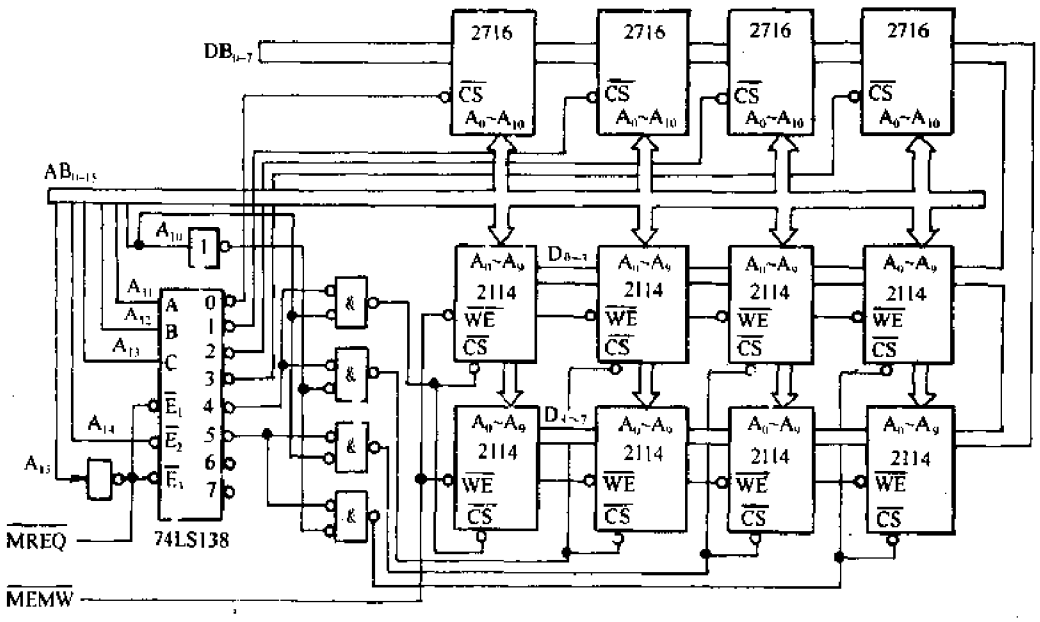
如果存储芯片的工作速度比较慢，以至于不能在 CPU 的读写周期内完成读数、写数操作，那么 CPU 就需要在正常的读写周期之外再插入一个或几个等待周期，以实现读写时序的匹配与操作的同步。为此，作为存储器接口，就必须能向 CPU 提供相应的等待信号。

2. 数据线的连接

在微机中，无论字长是多少，一般每个存储体（8 位机为单存储体，16 位机为双体，32 位机



(a) 方案一



(b) 方案二

图 6.12　存储器接口连线图

为 4 体)都是以一个字节为基本单位来划分存储单元的,即每 8 位为一个存储单元,对应一个存储地址。但由于存储芯片的内部结构不同,有的芯片一个地址对应 8 个存储位,有 8 条数据引线,如 2716、2128;而有的芯片一个地址对应 4 位,数据引线只有 4 条,如 2114;还有的芯片只有一个存储位,只有一根数据输入、输出线,如 2118。当用这些存储字长不是 8 位的芯片构成内存时,必须用多片合在一起,并行构成具有 8 位字长的存储单元。比如 2114,需同时用两片;而 2118,则需同时用 8 片。而在用多片构成存储单元时,它们的地址线、控制线完全是并联在一起的,数据线则分别接在数据总线的不同线上,如前面方案一、方案二的连线图所示。

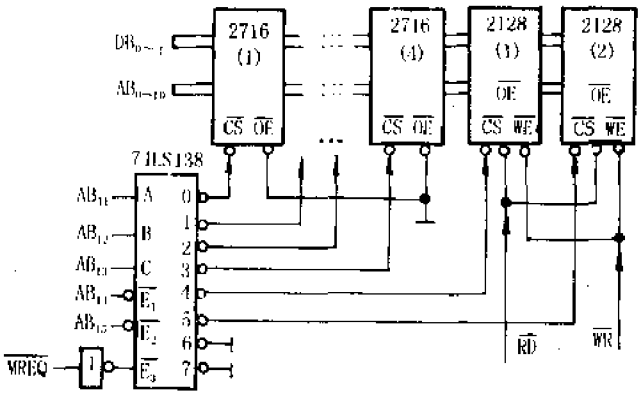


图 6.13　用 2716 和 2128 构成的存储器

第四节　80486 的存储器结构与存储管理

80486 的存储器系统具有典型的"内部寄存器、Cache、内存、外存"层次结构,并且通过分段分页来支持虚拟存储器和保护技术。

一、内存配置及结构

80486 系统的内存寻址范围为 4 千兆字节(4GB),但实际装机的内存容量一般为 4～64MB,如果用户需要,可自行扩充,不过应有一定的硬件软件支持。

当刚推出 DOS 操作系统和 IBM PC 机时,设计者认为配置 640KB 内存就绰绰有余了。后来,需要支持的软件越来越大、越来越多,640KB 内存明显不够,于是又不断增配了扩展存储器。但为了保持向上兼容性,有关内存的配置和结构已形成了一定的规则。目前各种 386、486、586 系统板都遵守 PC/AT 系统的技术标准,DOS、Windows 等操作系统也都是按照适应这种内存配置标准的需要而设计的。

有关内存的配置和结构标准如表 6.3 所示。

对表 6.3 中出现的几个有关内存配置的名称说明如下:

· 0～640KB 是最早推出 DOS 和 IBM PC 时配置的,称之为主存储器。

· 640KB～1024KB 区域的 384KB 称为内存保留区,专门留给各种接口卡使用,如显示 RAM、I/O 扩展 ROM、BIOS 等系统板 ROM 等。实际上,内存保留区有不少空闲部分。这些空闲部分,DOS 5.0 以上版本的操作系统可支持应用程序利用起来,这时就称之为上位存储器 UMBs(Upper Memory Blocks)。

表 6.3 80486 的内存配置

配置名称	地址范围(H)	配 置 说 明
主存储器 (640KB)	0~07FFFF	系统板上 512KB 系统存储器
	080000~09FFFF	系统板上 128KB I/O 通道基本 RAM
内 存 保留区 (384KB)	0A0000~0BFFFF	128KB 视频显示 RAM
	0C0000~0DFFFF	128KB I/O 扩展 ROM
	0E0000~0EFFFF	系统板上保留的 64KB,作为 FE0000~FEFFFF 的副本
	0F0000~0FFFFF	64KB 系统板 ROM(BIOS 等),其副本在 FF0000~FFFFFF
扩展 (扩充) 存储器 XMS (EMS)	100000~10FFFF	这 64KB 叫高位内存区 HMA
	110000~F5FFFF	14.32MB I/O 通道扩充用存储器,不用于系统板
	F60000~FDFFFF	512KB I/O 通道扩充用存储器,不用于系统板
	FE0000~FEFFFF	系统板上保留的 64KB,其副本在 0E0000~0EFFFF
	FF0000~FFFFFF	系统板上 64KB ROM(BIOS 等),以 0F0000~0FFFFF 为副本
	01000000~FFFFFFFF	

• 1024KB 以上的存储器统称为扩展存储器(Extended Memory),也叫 XMS。其中 1024KB~1088KB 的 64KB 称为高位内存区 HMA(High Memory Area)。

• 扩展存储器 XMS 实际上也就是扩充存储器 EMS(Expanded Memory)。但按定义,EMS 是利用 EMM(Expanded Memory Manipulation)管理程序进行管理的 XMS,对它访问时,必须把 EMS 中的内容拷贝到"内存保留区"中的空闲地方即 UMBs 才行。为此,EMM 支持在内存保留区空闲部分建立一个 64KB 的页框(每页 16KB,4 页为一个页框),专门供拷贝 EMS 中内容用。

上述内存配置和结构中,HMA、UMBs 或 EMS 都是在 DOS 5.0 以上版本的操作系统支持下,通过驱动程序向 XMS"要"来的,因此,要使用它们,需利用 DOS 提供的存储器管理软件。

二、80486 与 32 位、16 位、8 位内存的接口

在 80486 系统的 32 位物理内存中,每个 4 字节的数都从一个为 4 的倍数的字节地址处开始。A_2~A_{31}用来选择一个 4 字节的数,$\overline{BE_0}$~$\overline{BE_3}$ 用来选择 4 字节中的各个字节。对于涉及 32 位操作数的所有总线周期,$\overline{BS16}$和$\overline{BS8}$均无效(高电平)。

对于 16 位和 8 位的内存,需要在 486CPU 总线与内存之间加字节交换逻辑和字节选择逻辑,以便将数据送到相应的数据线上。在混合使用几种宽度内存的系统中,还需要有额外的地址译码逻辑,用来发$\overline{BS16}$或$\overline{BS8}$信号。

80486 与 32 位、16 位、8 位内存的接口如图 6.14 所示。寻址 16 位内存时,必须对字节允许信号$\overline{BE_0}$~$\overline{BE_3}$译码(通过字节选择逻辑),以产生 A_1、$\overline{BHE}$和$\overline{BLE}$(A_0)。寻址 8 位内存时,则必须译码产生 A_1 和 A_0($\overline{BLE}$)。由于$\overline{BLE}$和 A_0 完全相同,所以在 16 位和 8 位系统中可使用同样的字节选择逻辑。

可按表 6.4 所示的真值表对$\overline{BE_0}$~$\overline{BE_3}$进行译码,产生 A_1、$\overline{BHE}$和$\overline{BLE}$。表中 0 表示低电平,1 表示高电平,X 表示无关。为产生这三个信号所需的字节选择逻辑如图 6.15 所示。当 16 位内存的 D_0~D_7 被选时,发出$\overline{BLE}$(A_0),D_8~D_{15}被选时发出$\overline{BHE}$。对所有偶数字,A_1 为 0;对

所有奇数字，A_1 为 1。

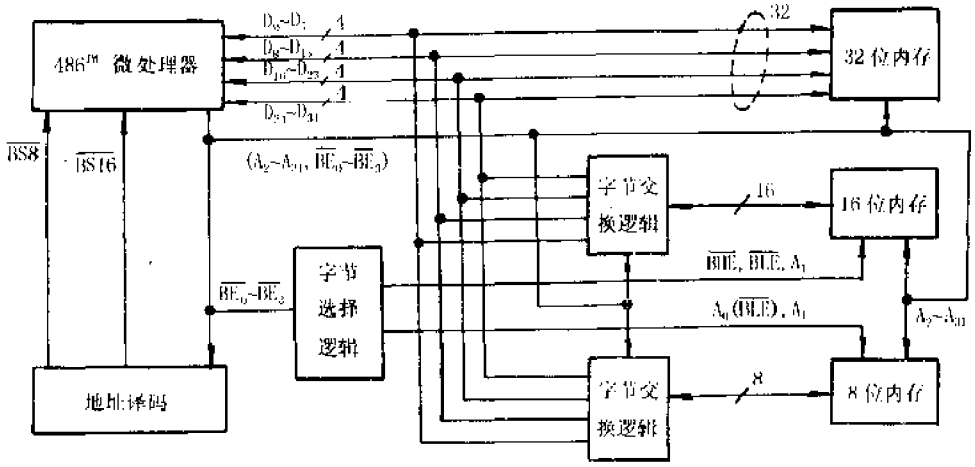


图 6.14　80486 与 32/16/8 位内存的接口

表 6.4　字节选择逻辑真值表

486 字节允许信号				16/8 位内存字节选择信号		
$\overline{BE_3}$	$\overline{BE_2}$	$\overline{BE_1}$	$\overline{BE_0}$	A_1	$\overline{BHE}$	$\overline{BLE}(A_0)$
0	0	0	0	0	0	0
0	0	0	1	0	0	1
0	0	1	0	×	×	×[注]
0	0	1	1	1	0	0
0	1	0	0	×	×	×
0	1	0	1	×	×	×
0	1	1	0	×	×	×
0	1	1	1	1	0	1
1	0	0	0	0	0	0
1	0	0	1	0	0	1
1	0	1	0	×	×	×
1	0	1	1	1	1	0
1	1	0	0	0	0	0
1	1	0	1	0	0	1
1	1	1	0	0	1	0
1	1	1	1	×	×	×

注：凡是有效的 2 个或 3 个字节允允信号（"0"）被 1 个或 2 个无效的字节允许信号（"1"）隔开的那些$\overline{BE_0}$~$\overline{BE_3}$组合，均不可能出现，被视为无关项。

三、存储器工作方式及管理

80486 的存储器工作方式实质上也就是第二章所述的三种工作方式，即实地址方式、保护虚地址方式和虚拟 8086 方式。

实地址方式是 80286~80486 最基本的工作方式，与 8088/8086 工作方式基本相同，只能在 1MB 范围内寻址，故不能管理和使用扩展存储器。上电或复位后自动建立起实地址方式。这时

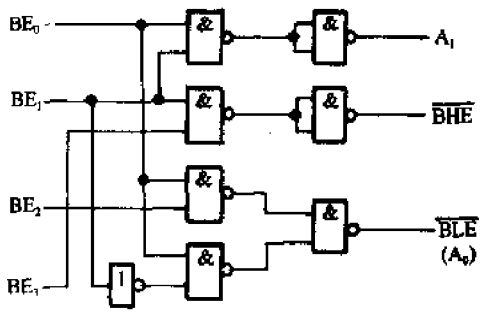


图 6.15　16/8 位内存字节选择逻辑

启动地址为 FFFF0H，在此地址安排一条跳转指令，进入上电自检和自举程序。另外，保留 00000～003FFH 的中断向量区。可以认为该方式只使用低 20 位地址线，与 8088/8086 工作情况是一致的。DEBUG 调试程序只能在实地址方式下使用。80386/80486 的指令除了少数保护方式指令外，其余均可在实地址方式下运行，且通过指令前缀字节，可以改变后面跟着的那条指令的某些特性，允许以 32 位操作数和 32 位寻址方式编写和执行 32 位运算的程序，不过地址偏移值仍不能超过 64K。

虚拟 8086 方式实质上是保护虚地址方式下的一种子方式，它们都是建立在虚拟存储和保护两大机制的基础上的工作方式，并且支持多用户、多任务操作。在保护虚地址方式下，80486 的所有指令和功能都可以使用，所编程序均在保护环境中执行。

本节主要讨论保护虚地址方式的虚拟存储和保护两大机制，以及虚拟 8086 方式与实地址方式的异同点。

1. 虚拟存储管理机制

80486/80386 的虚拟存储空间最大可达 64TB(10^{12}字节)。对于虚拟存储器的管理，和 80286 一样，采用的是分段分页机制，但最大分段长度从 80286 的 64KB 扩展到了 80486/80386 的 4GB。

分段分页机制的基本思想是：首先使用分段机制，将虚拟地址空间分成一个个大小不等的逻辑段。将虚拟地址用间接指向段基址的段选择符和段内偏移量两部分表示，并将虚拟地址转换为一个中间地址空间的地址，这一中间地址空间称为线性地址空间，其地址称为线性地址。线性地址空间是一个不分段的连续的地址空间。然后再使用分页机制，将线性地址空间分成若干固定大小的页，将线性地址用页基址和页内偏移量表示，并将线性地址转换为物理地址。图 6.16 给出了 80486 存储器分段分页管理机制的原理示意图。

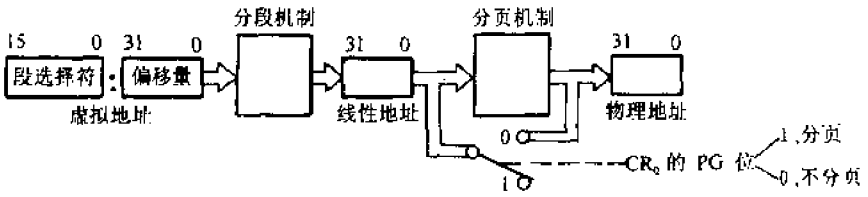


图 6.16　80486 存储器分段分页机制示意图

虚拟地址空间是二维的，它所包含的段数最大可到 16K 个，每个段最大可到 4GB，从而构成 64KGB 容量的庞大虚拟地址空间。线性地址空间和物理地址空间都是一维的，其容量为 2^{32}=4GB。事实上，分页机制可通过设置 80386/80486 中控制寄存器 CR_0 的 PG 位为 1 或 0 来确定是否启用，当被禁止使用时，线性地址就是物理地址。

分段是虚拟存储器管理机制的基础。虚拟地址中的段部分指的是 CS、DS、SS、ES、GS、FS 六个 16 位段寄存器，由段选择符标识，段选择符在程序执行时装载到相应的段寄存器中；偏移量的计算方法因指令的寻址方式不同而异，由于 80486 的寻址方式很多，所以为偏移量的形成提供了灵活的机制。

每个段均由三个参数定义：

①段的基地址，即线性空间中段的开始地址。它是线性地址空间对应于段内偏移量为 0 的虚拟地址。

②段的界限(Limit),即指段内可以使用的最大偏移量,它指明该段的长度。

③段属性,如可读出或写入段的特权级等。

以上三个参数都存储在段的描述符中。

分页是存储器管理机制的第二部分。当控制寄存器CR_0中的PG位等于1时,启用分页机制。它把线性地址空间中的任何一页映射到物理空间的一页。

无论分段、分页,都是使用驻留在内存中的转换表来完成相应的地址变换。如图6.17所示。

虚拟地址一线性地址变换时要访问描述符,而描述符又存于描述符表中。显然,有多少个段就有多少个段描述符,因此最多有16K个段描述符;每个段描述符中含有32位段基址,以选择4G个内存地址中之任一个作为基地址;为了区分16K个段描述符,段选择符应使用14位。32位段基址加上虚拟地址中的32位段内偏移量,即得32位线性地址。

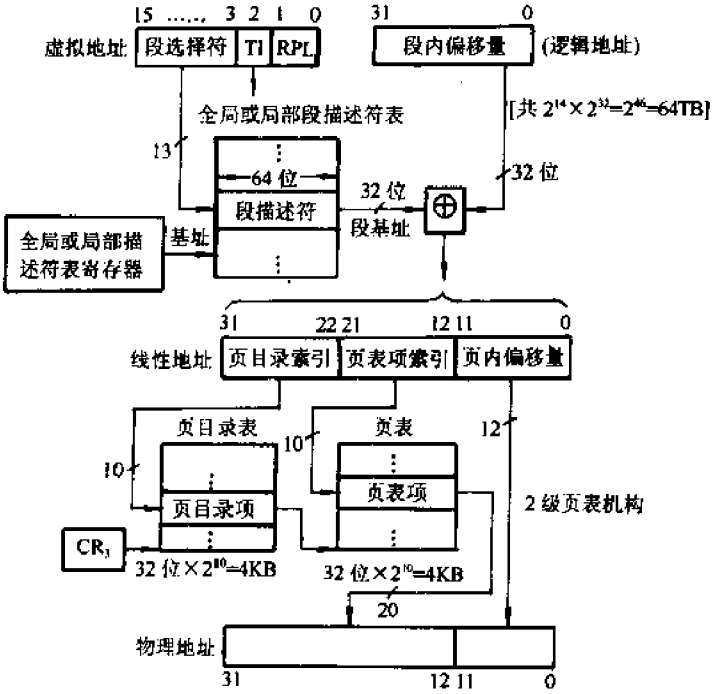


图6.17 80486的虚拟地址向物理地址的转换

分页转换函数由称为页表的存储器常驻表来描述。为了节省页表所占的内存空间,80486中采用两级页表机构:第一级用2^{10}个表项构成页目录表,每项4个字节,共占4KB内存;第二级再用2^{10}个表项构成页表,每项也是4字节,共占4KB内存。这样,两级表组合起来只占8KB内存,即可描述2^{20}个表项。(如果用一级页表机构来描述这2^{20}个表项,则页表所占内存将达4MB(4字节$\times20^{20}$)。)

为了与两级页表机构相适应,将32位线性地址分成页目录索引(10位)、页表项索引(10位)和页内偏移量(12位)三个字段。通过依次访问页目录表和页表,即可找到与线性地址相对应的物理地址所在的页号(即物理地址的31～12位),再根据页内偏移量,即可知道物理地址。

采用两级页表机构固然具有节省内存的优点,但也将带来多访问一级页表、使地址变换速度降低的缺陷。为此,80486的页部件中特设置了一个转换后援缓冲器TLB,将当前最可能访

问的 32 个页表项存于其中，构成一个内存页表的局部副本。TLB 的结构及地址变换原理如图 6.18 所示。TLB 由两个字段组成：页表数据字段中存有页表项；标记字段里存有与其页表项对应的线性地址的位 31～12。TLB 中共可保存 32 个页表项。

页部件将线性地址变换为物理地址时，先用 TLB 的标记字段来搜索线性地址位 31～12，若搜索到，说明 TLB 命中，则使用 TLB 中对应的页表数据字段作为页物理地址位 31～12；若未搜索到（TLB 未命中），才经内存中的二级页表机构去寻找物理地址页号（页表项），同时将它复制到 TLB 中。如复制时 TLB 已满，则按某种算法进行置换。

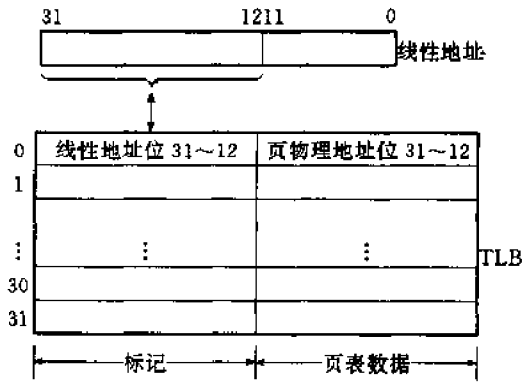


图 6.18　TLB 结构及原理示意图

2. 保护机制

存储器管理机制的保护功能表现在两方面：

①每个任务有不同的虚地址空间，使不同任务间互相隔离，受到保护。当 486CPU 同时运行多个任务程序时，分段部件、分页部件都可起到保护程序免受其他程序干扰的作用。尤其分段机制对程序保护起的作用更大。CPU 对段进行访问时，严格受段定义的大小及其访问特权级等信息的控制，任何越限和越权的访问都将被拒绝，并引起异常处理中断。

②同一任务内的不同程序段受到保护。由于对不同程序段定义了四种特权级，一般 OS 内核特权级最高（0 级），OS 其余部分次之（1 级），其他系统软件为 2 级，应用程序为最低级（3 级）；另外，每个应用程序有自己单独的段表、页表和局部描述符表，而为许多任务共享的 OS、其他系统软件和工具软件则有其独立的段表、页表和全局描述符表。这样一来，同一任务中特权级高的程序不可能被应用程序所破坏。

3. 虚拟 8086 方式与实地址方式的比较

（1）共性　两者都是为了与 8086/8088/80186/80188 兼容而设置的 16 位操作方式；它们的段基址都为段寄存器值乘以 16，最大段容量均为 64KB；最大寻址空间均为 1MB。

（2）区别

两者的区别主要表现在两方面：

①实地址方式是整个 CPU 的工作方式，是实实在在的 8086 方式，此时的 80486 相当于一个快速的 8086，只能运行 8086 程序的任务。而虚拟 8086 方式是一种模拟（仿真）8086 方式，是在虚拟存储器、保护和多任务操作等概念支持下一个任务的工作方式，所以，在一个多任务操作系统（如 UNIX、Windows 等）环境中，既可在虚拟 8086 这种仿真方式下创建和运行 8086 程序的任务，又可在保护虚地址方式下创建和运行 80286 程序的任务和 80386/80486 程序的任务。

②在实地址方式下，各个段的特权级都为 0（最高级）；而在虚拟 8086 方式下各个段的特权级则都为 3 级（最低级）。这样，在虚拟 8086 方式的，就限制了一些特权指令的执行。

思考题与习题

1. 微机系统中通常有哪几级存储器？它们在性能上有什么特点？

2. 存储芯片如何分类？各类芯片的功能有何不同。

3. 简述 Intel 2764A 的工作方式。

4. 试述 Intel 2164A 芯片的内部结构特点。

5. 常用存储器片选的控制方法有几种?? 它们各有什么优缺点?

6. 下列 RAM 各需要多少个地址端?

(1)512×4 位 (2)1K×8 位 (3)4K×1 位 (4)16K×1 位

7. 下列 ROM 各需要多少个地址端? 多少个数据端?

(1)16×4 位 (2)32×8 位 (3)256×4 位 (4)512×8 位

8. 某微机系中有 16 条地址线,画出下列情况下存储器的地址译码和连接图。

(1)采用 8K×1 位存储芯片,组成 64K 字节存储器

(2)采用 4K×1 位存储芯片,组成 32K 字节存储器。

9. 试为某 8 位微机系统设计一个具有 8KB ROM 和 40KB RAM 的存储器。ROM 采用 EPPOM 芯片 2732 组成,从 0000H 地址开始;RAM 用 SRAM 芯片 6264 组成,从 4000H 地址开始。

10. 试简述 80486 的内存配置。

11. 试比较 80486 的实地址、保护虚地址和虚拟 8086 方式有什么区别和联系。

12. 简述 80486 虚拟存储管理机制,并说明它的虚拟地址—物理地址变换原理。

第七章　中断控制

第一节　中断概述

一、中断与中断功能

中断是现代微型计算机系统中广泛采用的一种资源共享技术。

所谓中断是指 CPU 在正常执行程序的过程中，由于某个外部或内部事件的作用，强迫 CPU 停止当前正在执行的程序，转去为该事件服务（称为中断服务），待服务结束后，又能自动返回到被中断的程序中继续执行。

中断控制可以实现如下功能：

1. 实现 CPU 与外部设备的速度匹配与并行工作。

由于 CPU 的运行速度很快，而多数外部设备的工作速度较慢，在 CPU 和外设传送数据时，CPU 需花费大量的时间等待与外设交换数据，致使 CPU 的利用率很低。为了使 CPU 与外设之间的工作同步，可通过中断控制方式来实现。

在 CPU 执行程序过程中，如需要和某个外设传送数据时，先执行一条启动该设备工作的命令，然后继续执行现行程序，待外设为传送数据作好准备后，向 CPU 发出中断请求信号，请求 CPU 暂时停止正在执行的程序，转去执行中断服务程序，完成数据的输入或输出操作；当中断服务程序执行完毕，又自动返回到原来的程序中继续运行。由此可见，中断方式不仅可以实现 CPU 和外设之间的并行工作，而且大大提高了 CPU 的利用率。

2. 实现实时信息监测和控制

微型计算机的重要应用领域之一是用来进行实时信息的采集处理和控制。所谓实时是指使计算机能够对现场采集到的信息及时作出分析和处理，以便对被控制的对象立即作出响应，使被控制的对象保持在最佳工作状态。

3. 实现故障检测和自动处理

在计算机运行过程中，常常会发生一些事先无法预料到的故障。如：电源和硬件故障，数据存储和运算错误等。这些故障可随机地发生。有了中断系统，计算机就能对这些故障自动发现并及时进行自动处理。

在现代计算机系统中，随着计算机软、硬件技术的发展，中断技术也得到了不断地发展和完善，中断系统功能的强弱已成为评价计算机系统整体性能的一项重要指标。

二、中断控制

为了实现中断控制，中断系统应具有如下功能：

1. 设置中断源，确定它们提出中断请求的方式。

把能够提出中断请求的设备和事件称为中断源。不同的计算机系统，中断源的设置有所不同，通常按照 CPU 与中断源的位置关系可分为内部中断和外部中断。

内部中断是CPU在处理某些特殊事件而引起或通过自己内部逻辑去调用的中断。通常有以下几种：

(1)电源故障；

(2)存储错误；

(3)运算溢出；

(4)单步执行程序和设置断点；

(5)中断指令。

外部中断是由于外部设备要求数据输入或输出操作时请求CPU为之服务的一种中断。主要有以下几种：

(1)数据输入/输出；

(2)实时处理；

(3)故障处理；

(4)人机联系；

(5)总线请求。

当中断源需要CPU服务时，是以中断申请的方式进行的。外部中断请求的方式通常有电平触发和边沿(跳变信号)触发两种。

2.确定中断优先级和中断的嵌套方式

在一个系统中一般设置有多个中断源，当它们同时向CPU发出中断请求时，CPU应先为哪一个中断源服务；当CPU正在执行某一个中断服务程序时，是否允许被另外的中断请求打断。前者属于CPU为中断源服务的先后次序问题，后者属于中断嵌套问题。

在中断系统中，应根据中断源执行任务的紧迫程序不同事先规定好各中断源的中断优先权级别。当有多个中断源向CPU发出中断申请时，系统能够自动地对它们进行排队判优，优先权级别较高者优先得到服务。并且允许高优先级中断请求可以打断低优先级中断服务，使CPU对于急需处理的事件立即作出响应。

3.中断现场的保护与恢复

CPU执行中断服务的过程可以用图7.1来描述。为了使执行流程由中断服务程序执行之后能正确地返回到被中断的现行程序中继续运行，要求系统在执行中断服务程序之前，必须把被中断程序的断点，即返回之后将要执行的第一条指令的地址和一些中间结果(称为现场)送入堆栈保护起来；当中断服务程序执行完毕，又能自动地从堆栈中取出断点，并通过执行数据出栈指令将中间结果重新送回原来的存储单元中(称为现场恢复)，至此CPU的一次中断过程结束。

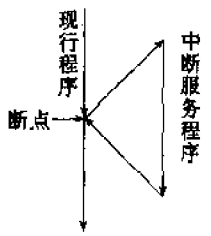


图7.1

三、中断处理过程

中断处理通常由中断申请、中断响应、中断处理和中断返回四个过程来完成。中断处理流程如图7.2所示。

(1)中断申请

当外设需要中断服务时，由硬件产生一个中断请求信号INT发送给CPU，并且要求INT信号保持到CPU作出响应为止。

(2)中断响应

CPU 在每条指令执行结束后去采样和查询有无中断请求，若查询到有中断请求，并且在允许响应中断的情况下，系统自动进入中断响应周期，由硬件完成关中断、保存断点、取中断服务程序的入口地址等一系列操作，而后转向中断服务程序执行中断处理。

(3)中断处理

中断处理就是执行中断服务程序中规定的操作。在进入中断处理之前通常要做如下两件事：

· 保护现场。执行压栈指令 PUSH 将有关寄存器的内容入栈保护。

· 开中断。CPU 响应中断后，中断自动关闭，为了能够实现中断嵌套，需在适当的位置安排一条开中断指令，使系统处于开放中断状态，随时可对高级中断作出响应和处理。

(4)中断返回

中断返回是由执行中断服务程序中的中断返回指令 IRET 来完成的。执行该条指令时，自动从堆栈中弹出断点送入指令计数器 PC，CPU 又转向被中断的程序继续执行。

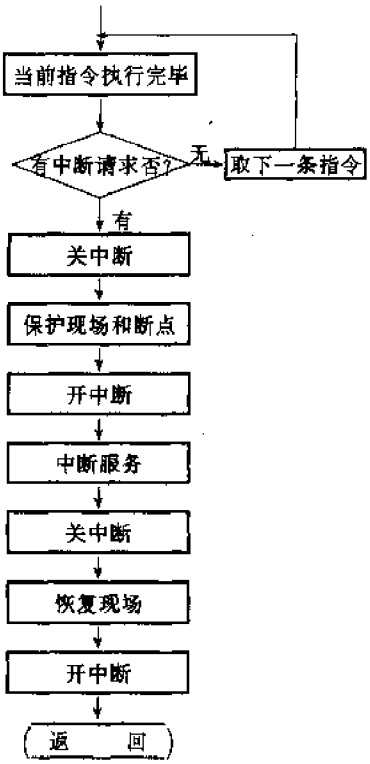


图 7.2 中断处理流程框图

在中断返回之前，执行出栈指令 POP 将保存的现场信息恢复。

第二节 IBM PC 机的中断系统

一、中断类型

IBM PC 机中的中断源按与 CPU 的相对位置关系分为外部中断和内部中断两种。外部中断也称硬件中断，是由 CPU 外部引脚触发的一种中断，分为不可屏蔽中断 NMI 和可屏蔽中断 INTR。NMI 中断不受中断允许控制位 IF 的控制，其请求触发方式为上升沿有效，当 CPU 在 NMI 引脚上采样到一个由低到高的跳变信号时，就自动进入 NMI 中断服务处理。INTR 中断受中断允许 IF 位的控制，其请求触发方式为高电平有效，只有在中断允许 IF＝1 时，CPU 才响应 INTR 的请求，而 IF＝0 时禁止响应。

内部中断也称为异常中断，是由处理器检测到异常情况或执行软件中断指令引起的一种中断，它们属于非屏蔽中断。通常有：除法出错中断、溢出中断、INT 中断和单步执行中断等。中断源的优先权级别为内部中断最高(除单步执行中断)，其次为 NMI 中断，再次为 INTR 中断、单步执行中断最低，中断处理优先顺序如图 7.3 所示。

80486 内部中断的种类在 8086 内部中断类型的基础上增加了许多。把许多执行指令过程中产生的错误情况都纳入内部中断处理的范围，大大丰富了内部中断的功能。在 80486 系统中，内部中断称之为异常中断，可分为下列三种类型。

(1)故障。若某条指令在启动之后真正执行之前被检测到异常，即产生异常中断，而且在中断服务完成后返回该指令，重新启动并执行完成。

(2)陷阱。产生陷阱的指令在执行后才被报告，在中断服务完成后返回到设置陷阱的下一条指令继续执行。

(3)终止。异常发生后无法确定造成异常指令的实际位置，程序无法继续执行，中断服务程序须重新启动系统。

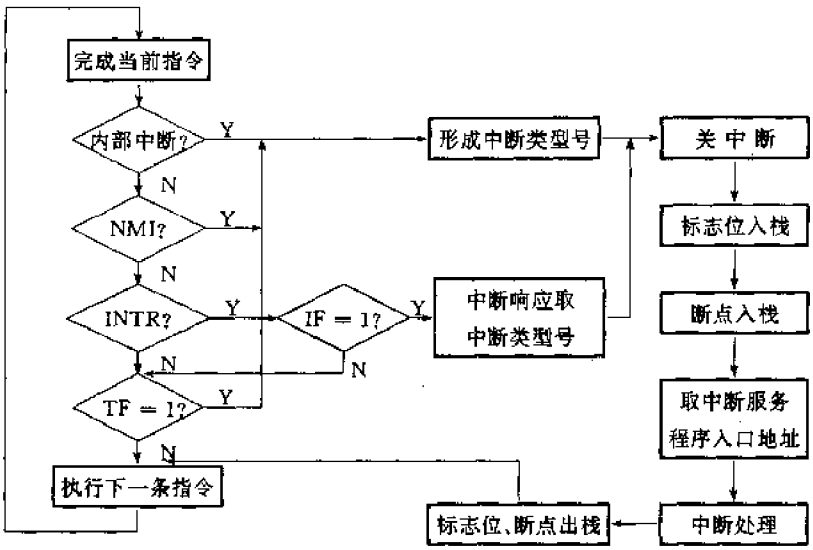


图 7.3 IBM PC 机中断处理流程图

外部可屏蔽中断 INTR 主要供外部设备使用，但由于 CPU 只提供了一个 INTR 引脚，处理中断的能力有限，因此在 IBM PC 系列微机中通过扩展中断控制器来协助 CPU 管理中断。如在 PC/XT 微机中使用了一片 82C59A 中断控制器管理中断；在 PC/AT 微机中使用了两片 82C59A 管理中断；在 386、486 等其它高档微机中通常使用多功能接口芯片管理中断。

二、中断矢量表

中断矢量表的存储结构如图 7.4 所示。它是中断类型号和中断服务程序入口地址之间的链接表。系统将中断源划分为 256 种类型，类型号由 0～255(0～FFH)表示。每个中断类型分配 4 个连续的字节单元，用于存放该中断服务程序的入口地址。两个高字节单元存放入口的段基址，两个低字节单元存放入口的偏移地址。256 种类型中断共需要 1K 字节的 RAM，矢量表地址范围为 0～3FFH。

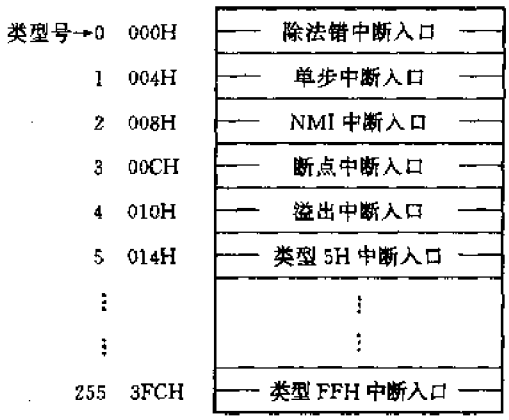


图 7.4 中断矢量表

中断矢量表分为三部分使用：专用中断、备用中断和用户中断。专用中断的中断服务程序的入口地址由系统负责装入，用户不能随意修改。备用中断为生产厂家开发软、硬件保留。用户中断的中断服务程序的入口地址由用户程序负责装入。由中断矢量表结构可以看出，中断类型号 n 与其对应的矢量表地址关系为 4n。即入口的偏移地址存放在由 4n 和 4n+1 指示的存储单元中，入口段基址存放在由 4n+2 和 4n+3 指示的存储单元中。只要确定了中断类型号 n，就可以很方便地计算出矢量表的地址。

为了避免直接读、写中断向量表而可能造成的错误，在 DOS 功能调用 INT 21H 中提供了 25H 功能设置中断矢量，35H 号功能获取中断矢量。

(1)25H 功能调用

系统功能 25H 调用提供的入口参数为：

(AH)＝25H

(DS:DX)＝中断服务程序入口地址

(AL)＝中断类型号

假设某中断源的中断类型号 n 为 40H，中断服务程序的入口地址为标号 INT-P，设置中断矢量的程序段如下：

```
        ⋮
        CLI                          ;关中断
        MOV AH,25H                   ;25H 功能调用
        MOV AL,40H                   ;中断类型号 40H 送入 AL
        MOV DX,SEG INT-P             ;入口段基址送入 DS
        MOV DS,DX
        MOV DX,OFFSET INT-P          ;入口偏移地址送入 DX
        INT 21H                      ;中断调用
        STI                          ;开中断
          ⋮
INT－P:……                            ;中断服务程序入口
          ⋮
        IRET                         ;中断返回
```

(2)35H 功能调用

系统功能 35H 调用提供的入口参数为：

(AH)＝35H

(AL)＝中断类型号

出口参数为：

(ES:BX)＝中断矢量

如：MOV AH,35H

MOV AL,中断类型号

INT 21H ;返回 ES:BX 中断矢量

三、中断响应时序

当 CPU 响应中断请求 INTR 后，CPU 向请求设备回送一个中断响应信号$\overline{\text{INTA}}$，自动进入中断响应总线周期，进行中断处理。

中断响应$\overline{\text{INTA}}$总线周期时序如图 7.5 所示。每个$\overline{\text{INTA}}$周期包含 4 个 T 状态。每响应一次 INTR 中断，CPU 执行两个连续的$\overline{\text{INTA}}$周期。在第一个$\overline{\text{INTA}}$周期，CPU 将地址/数据总线置为浮空状态，在 $T_2 \sim T_4$ 状态向 82C59A 发送第一个$\overline{\text{INTA}}$，表示响应 INTR 请求，并使$\overline{\text{LOCK}}$信号有效，以禁止其它总线主控者争用总线。在第二个$\overline{\text{INTA}}$期间，82C59A 将中断类型号 n 送上数据总线，CPU 读取 n 之后，完成 4 * n 运算，便可从中断矢量表中取得中断服务程序的入口地址，转去执行中断处理。同时还利用第二个$\overline{\text{INTA}}$的后沿复位 INT。

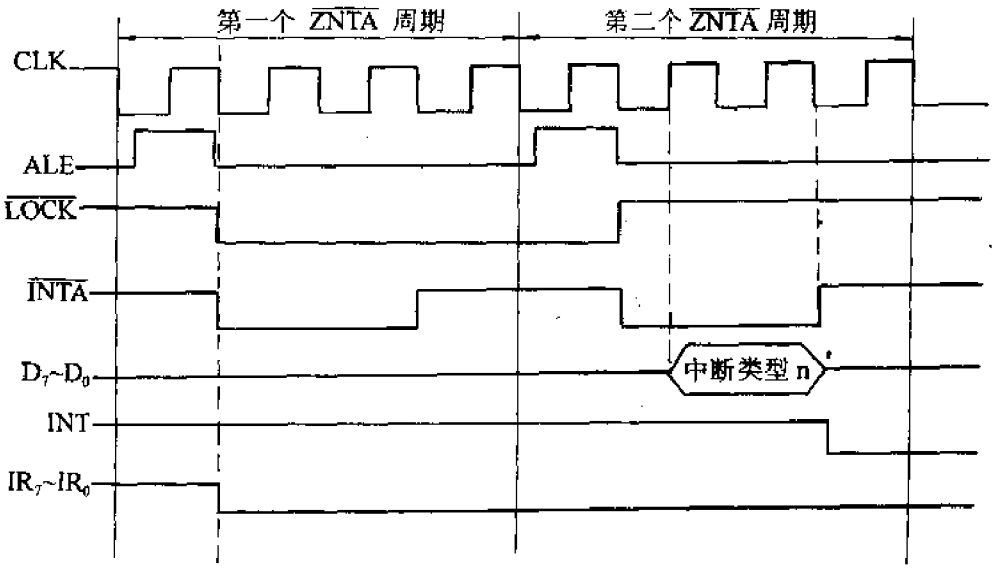


图 7.5

第三节　中断控制器 82C59A

82C59A 是由 Intel 公司生产的可编程中断控制器，可直接用来协助 CPU 管理中断。每片 82C59A 可以接收 8 级外部中断，具有中断屏蔽、中断优先权判优、中断嵌套等中断管理功能。在 PC/XT 和 PC/AT 系统中都是使用 82C59A 来管理中断。

一、82C59A 的结构与引脚信号

82C59A 为 28 个引脚 DIP 封装、CMOS 工艺接口芯片，其内部结构与引脚信号如图 7.6 和图 7.7 所示。

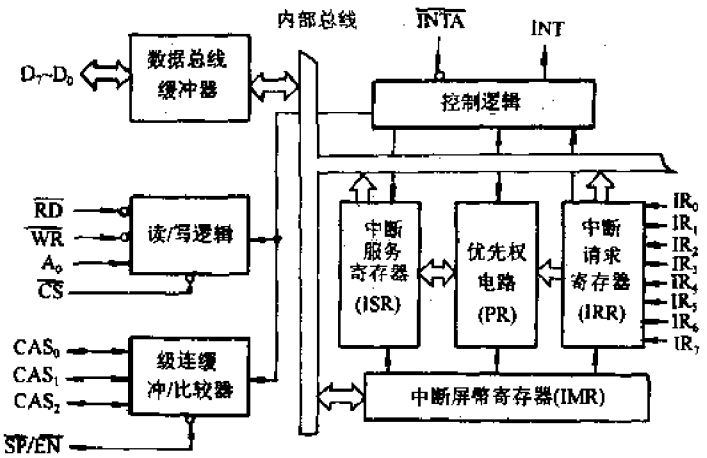


图 7.6　82C59A 结构框图

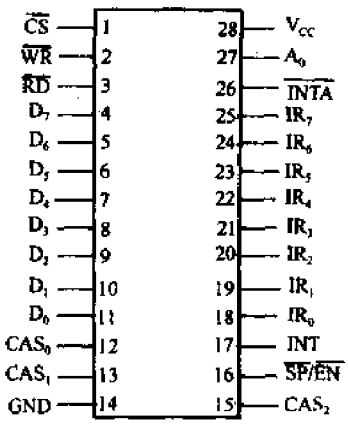


图 7.7　82C59A 引脚图

各部分功能与引脚信号说明如下：

1. 数据总线缓冲器

数据总线缓冲器为三态、双向 8 位寄存器。数据线 D_7～D_0 和 CPU 的数据总线连接，构成 CPU 和 82C59A 之间信息传送的通道。

2. 读/写控制逻辑

读/写控制逻辑用来接收来自系统总线的读、写控制信号和端口选择信号，用于控制 82C59A 内部寄存器的读写操作。

$\overline{WR}$为写信号，输入，低电平有效。

$\overline{RD}$为读信号，输入，低电平有效。

$\overline{CS}$为片选信号，输入，低电平有效。

A_0 为端口选择信号，输入。

有关寄存器的端口地址分配及读、写操作功能见表 7.1。

表 7.1 82C59A 读写操作功能

$\overline{CS}$	$\overline{WR}$	$\overline{RD}$	A_0	D_4	D_3	功能
0	0	1	0	1	×	写 ICW1 命令字
0	0	1	1	×	×	写 ICW2 命令字
0	0	1	1	×	×	写 ICW3 命令字
0	0	1	1	×	×	写 ICW4 命令字
0	0	1	1	×	×	写 OCW1 操作字
0	0	1	0	0	0	写 OCW2 操作字
0	0	1	0	0	1	写 OCW3 操作字
0	1	0	0	×	×	读 IRR 寄存器
0	1	0	0	×	×	读 ISR 寄存器
0	1	0	1	×	×	读 IMR 寄存器
0	1	0	1	×	×	读状态寄存器

注：D_4D_3 为对应寄存器中的标志位。

3. 级联缓冲/比较器

82C59A 可以工作于单片或多片(级联)、非缓冲或缓冲方式。在级联方式下，级联缓冲/比较器提供了对多片 82C59A 的管理和选择。其中一片为主片，其余的为从片，级联信号线 CAS_2～CAS。作为主片与从片的连接线，其传送方向是主片为输出，从片为输入。

$\overline{SP}/\overline{EN}$信号线用于主/从片选择线或驱动信号线。当 82C59A 工作于级联非缓冲方式时，$\overline{SP}/\overline{EN}$为主/从片选择输入线，主片$\overline{SP}/\overline{EN}$接高电平，从片$\overline{SP}/\overline{EN}$接低电平。当 82C59A 工作于级联缓冲方式时，$\overline{SP}/\overline{EN}$用作总线驱动器的启动控制输出信号。

4. 中断管理逻辑

中断管理逻辑主要由中断请求寄存器 IRR、中断服务寄存器 ISR、中断屏蔽寄存器 IMR 和中断优先权判别器 PR 组成。

(1)中断请求寄存器 IRR

IRR 是一个具有锁存功能的 8 位寄存器，用于存放外部中断请求信号线 IR_0～IR_7 的状态。当 IR_i($i=0$～7)有请求时，IRR 中的相应位 D_i 置 1，在中断响应信号$\overline{INTA}$有效时 D_i 位被清除。IRR 中的置位与清除是由硬件完成的。

(2)中断服务寄存器 ISR

ISR 是 8 位寄存器，用于存放 CPU 当前正在服务的中断标志。当外部中断 IR_i 请求进入服务时，ISR 中的相应位 D_i 置 1，中断服务结束 D_i 位被清除。ISR 中的置位是由硬件完成的，

而复位则有硬件和软件两种方式。

(3)中断屏蔽寄存器 IMR

IMR 是中断屏蔽寄存器,8 位,用于存放外部中断请求 IR_0～IR_7 的屏蔽信息,可以由软件置位或清除。当 IMR 中的 D_i 位为"1"时,对应的 IR_i 请求被屏蔽,禁止进入中断;当 D_i 为"0"时,允许 IR_i 进入中断。

(4)优先权判别器 PR

PR 是优先权判优逻辑,它可以对外部中断请求 IR_i 进行排队判优,以便找出当前中断请求最高级,向 CPU 发出中断请求 INT,请求为其服务。

5. 中断控制逻辑

中断控制逻辑按照编程设定的工作方式管理中断,负责向片内各部件发送控制信号;向 CPU 发送中断请求信号 INT 和接收 CPU 发送来的中断响应信号$\overline{INTA}$,控制 82C59A 进入中断服务状态。

二、82C59A 的工作方式

82C59A 的中断管理功能很强,有多种工作方式。

1. 中断嵌套方式

中断嵌套方式分为完全嵌套和特殊完全嵌套两种。

完全嵌套方式是 82C59A 在初始化时自动进入的一种最基本的优先权管理方式。其特点是:中断优先权级别固定。8 个中断请求输入 IR_0～IR_7 的优先级规定为 IR_0 最高,IR_1 次高,依次类推。在它们提出中断请求时,82C59A 按它们的优先权高低进行排队判优,选出优先权最高者请求 CPU 为之服务。在 CPU 中断服务期间,只允许比当前服务的中断的优先权高的中断请求进入,对于同级或低级中断请求禁止响应。

特殊完全嵌套方式是 82C59A 在多片级联方式下使用的一种优先权管理方式。其特点是:中断优先权级别固定,IR_0～IR_7 的优先顺序与完全嵌套规定相同,与完全嵌套方式不同之处是:在 CPU 中断服务期间,允许高级或同级中断请求进入,从而实现一种对同级中断的特殊嵌套方式。

在级联方式中,主片通常设置为特殊完全嵌套方式,从片设置为完全嵌套方式,当主片在为某一个从片的中断请求服务时,从片中的 IR_0～IR_7 的请求都是通过主片中的某个 IR_i 请求来进入的,因此从片的 IR_0～IR_7 对于主片 IR_i 来说,它们属于同级。当主片工作于特殊完全嵌套方式时,从片才能实现中断的完全嵌套。

2. 自动循环方式

自动循环是可以改变 IR_0～IR_7 优先权级别的一种优先权管理方式。分为普通自动循环方式和特殊自动循环方式。

普通自动循环方式是指当某一个中断请求 IR_i 服务结束后,该中断的优先权自动降为最低,而紧跟其后的中断请求 IR_{i+1} 的优先权自动变为最高。IR_0～IR_7 优先权级别按如下所示的右循环方式递增。

$$\rightarrow IR_7 \rightarrow \cdots \rightarrow IR_2 \rightarrow IR_1 \rightarrow IR_0 \rightarrow$$

设在初始状态 IR_0 优先权最高,当 IR_0 有请求且服务完毕,其优先权自动变为最低,排在 IR_7 之后,而 IR_1 的优先权升为最高,其余依次类推。这种优先权管理方式,可以使 8 个中断请求都拥有享受优先服务的权利。

特殊自动循环方式工作原理与普通循环方式类似，区别仅在于普通自动循环方式中的初始的最低优先级由系统指派，而特殊自动循环方式中初始的最低优先级可以由用户通过优先权命令指派。

3. 中断屏蔽方式

中断屏蔽方式是对 82C59A 的外部中断请求源 IR_0～IR_7 实现屏蔽的一种中断管理方式。分为普通屏蔽方式和特殊屏蔽方式。

普通屏蔽方式是通过编程写入 OCW_1 操作命令字将中断屏蔽寄存器 IMR 中的某些位置“1”，将对应的中断请求信号 IR_i 屏蔽掉。也就是说，当 IMR 中的 D_i 位为“1”时，IR_i 请求被禁止；当 D_i 为“0”时，IR_i 请求被允许。

特殊屏蔽方式只对同级中断请求实行屏蔽，对于高或低优先级中断请求均不能禁止。这种屏蔽方式通常用于级联方式或在同一个请求端 IR_i 上连接有多个中断源的场合。特殊屏蔽方式可以通过编程写入 OCW_3 操作命令字来设置或取消。

4. 中断结束方式

中断结束是指当 CPU 为某个中断请求设备服务结束后，应及时清除中断服务标志位，否则就意味着中断服务还在继续，致使其它中断请求信号无法得到响应。

我们在前面已经介绍过，中断服务标志位是存放在中断服务寄存器 ISR 中，当某个中断 IR_i 被响应后，ISR 中的对应位 D_i 为“1”，服务结束后，应将相应位 D_i 清除，表明该中断服务结束。如何将 ISR 中的置“1”位清除，82C59A 提供了以下三种中断服务结束方式。

(1)自动结束方式

自动结束方式是在中断响应信号$\overline{INTA}$的第二个负脉冲结束时，将 ISR 中的中断服务标志位清除。这种中断服务结束方式是由硬件自动完成的，但需要引起注意的是 ISR 中为“1”位的清除是在中断响应过程中完成的，并非中断服务程序的真正结束，若在中断服务程序的执行过程中再有一个比该中断优先级低的请求信号到来，82C59A 并没有保存任何标志表示当前服务尚未结束，致使低优先级中断请求进入，打乱正在服务的程序，这是不允许的。因此自动结束方式适合使用在系统中只有一个中断源无嵌套的场合。

(2)普通结束方式

普通结束方式是通过在中断服务程序中编程写入 OCW_2(格式见图 7.14)操作命令字向 82C59A 传送一个普通 EOI 命令来清除 ISR 中当前优先权级别最高位。由于这种结束方式只清除 ISR 中优先权级别最高的那一位，适合使用在完全嵌套方式下的中断结束。

(3)特殊结束方式

特殊结束方式与普通结束方式类似，也是通过在中断服务程序中编程写入 OCW_2 操作命令字向 82C59A 传送一个特殊 EOI 命令，清除 ISR 中的指定位。由于在特殊 EOI 命令中指出复位 ISR 中的那级中断，用法比较灵活，既可以用于完全嵌套方式下的中断结束，更适合于特殊完全嵌套方式下的中断结束。

5. 中断触发方式

82C59A 中断请求输入端 IR_0～IR_7 的触发方式有电平触发和边沿触发两种，可由软件设定。

电平触发是以 IR_i 端输入高电平信号作为中断请求信号。在这种触发方式中，要求触发电平必须保持到中断响应信号$\overline{INTA}$有效为止，并且在 CPU 响应中断后，及时撤消该请求信号，以防止 CPU 再次响应，出现重复中断现象。

边沿触发是以 IR_i 端上输入由低到高的正跳变信号作为中断请求信号，采用这种触发方式可避免出现重复中断现象。

三、82C59A 的连接方式

82C59A 与 CPU 的连接方式有单片与多片、缓冲与非缓冲结构。在单片连接方式中(见图 7.18)只使用一片 82C59A，至多可管理 8 级外部中断。在多片连接方式中(见图 7.19)由一片主片和多个从片构成，至多可管理 64 级外部中断。主片中的$\overline{SP}/\overline{EN}$引脚接高电平，从片中的$\overline{SP}/\overline{EN}$引脚接低电平。

缓冲方式是指在多片 82C59A 级联系统中，若从片使用较多时，为了增加系统的驱动能力，82C59A 可通过驱动器与系统总线连接，引脚信号$\overline{SP}/\overline{EN}$用于总线驱动器控制，若从片较少时，可以直接与系统总线连接。

四、82C59A 的编程

在 82C59A 内部有两组命令寄存器，一组用于存放初始化命令字 ICW，另一组用于存放操作命令字 OCW。ICW 用于设定 82C59A 的工作方式，在初始化编程中写入；OCW 用于中断操作管理，设置灵活，视需要写入。

(一)命令字 ICW 的格式

初始化编程中可供写入的命令字有四个，分别称为 ICW_1、ICW_2、ICW_3 和 ICW_4。其中，ICW_1 占用一个端口地址，约定端口选择线 A_0 为“0”；ICW_2～ICW_4 共用一个端口地址，约定 A_0 为“1”。

1. ICW_1

ICW_1 的格式如图 7.8 所示。

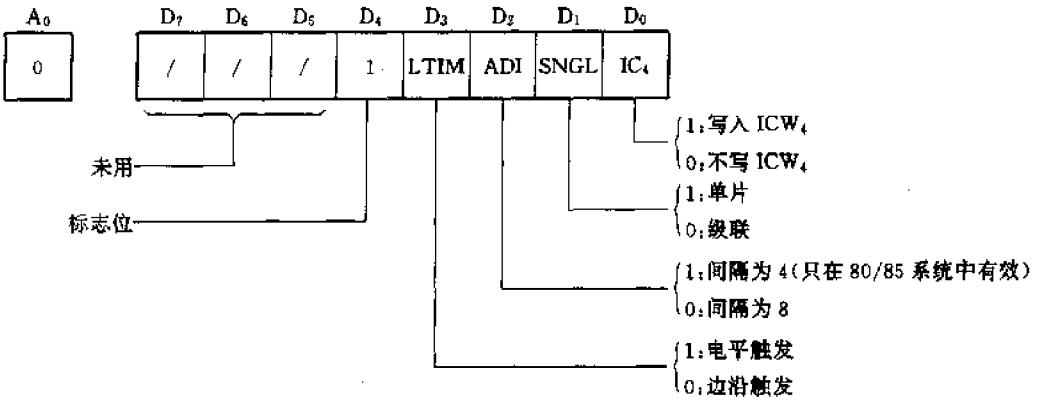


图 7.8 ICW_1 的格式

2. ICW_2

ICW_2 用于设置中断类型号，格式如图 7.9 所示。

ICW_2 中的低 3 位 ID_2～ID_0 由中断请求输入端 IR_i 编码自动引入，高 5 位 T_7～T_3 由用户编程写入。若 ICW_2 为 40H 时，则 IR_0～IR_7 对应的中断类型号为 40H～47H。

中断类型号 n 是在中断响应周期内的第二个$\overline{INTA}$信号有效期间，由 82C59A 通过数据总

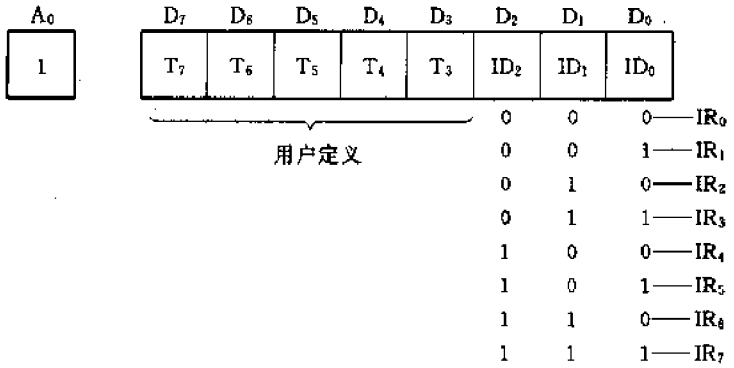


图 7.9 ICW_2 的格式

线 D_7～D_0 传送给 CPU，由 CPU 根据 4n 地址指针从中断矢量表中取出当前响应的中断服务程序的入口地址，继而转去执行中断服务程序。

3. ICW_3

ICW_3 是级联命令字，在级联方式下才需要写入。主片和从片对应的 ICW_3 的格式不同，主片 ICW_3 的格式如图 7.10 所示，S_7～S_0 与 IR_7～IR_0 相对应，若 IR_i 引脚上连接从片，则 $S_i=1$；否则，$S_i=0$。

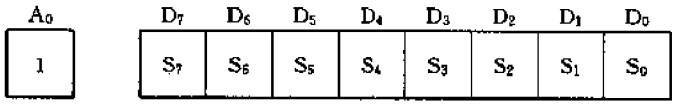


图 7.10 主片 ICW_3 的格式

从片 ICW_3 的格式如图 7.11 所示。

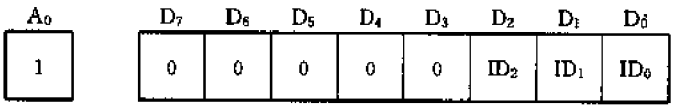


图 7.11 从片 ICW_3 的格式

ID_2～ID_0 为该从片接到主片 IR_i 上的标识码，若该从片的中断请求信号 INT 与主片的 IR_2 连接时，则 ID_2～ID_0 应设置为 010，D_7～D_3 未用，设置为 0。

4. ICW_4

ICW_4 用于设定 82C59A 的工作模式，其格式如图 7.12 所示。

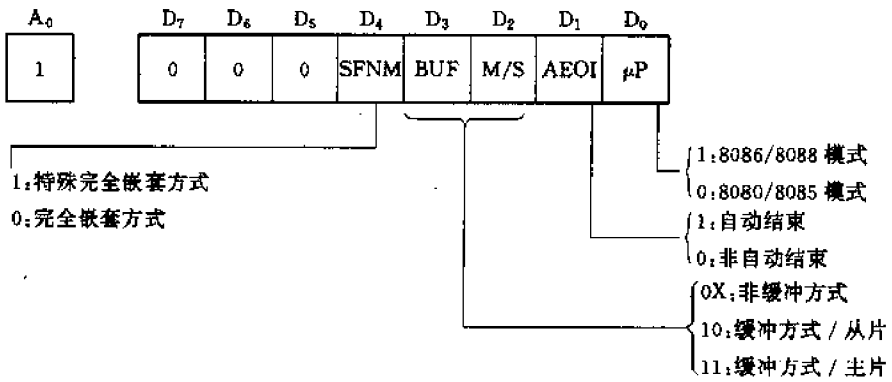


图 7.12 ICW_4 的格式

(二)操作命令字 OCW 的格式

操作命令字有 OCW_1、OCW_2 和 OCW_3 三个字，OCW_1 使用端口选择线 A_0 为"1"的口地址；OCW_2 与 OCW_3 共用一个 A_0 为"0"的口地址，由不同的特征位加以区分。

1. OCW_1

OCW_1 为中断屏蔽字，写入到中断屏蔽寄存器 IMR 中，对外部中断请求信号 IR_i 实行屏蔽。格式如图 7.13 所示。当某位 M_i 为"1"时，则对应的 IR_i 请求被禁止，当 M_i 为"0"时，则对应的 IR_i 请求被允许。IMR 中的内容初始状态为 0，在工作期间可根据需要随时写入或读出。

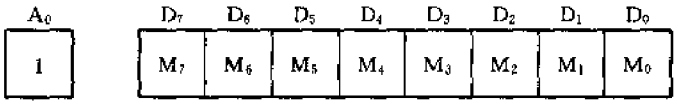


图 7.13 OCW_1 的格式

2. OCW_2

OCW_2 用于设置中断优先级方式和中断结束方式，其格式如图 7.14 所示。

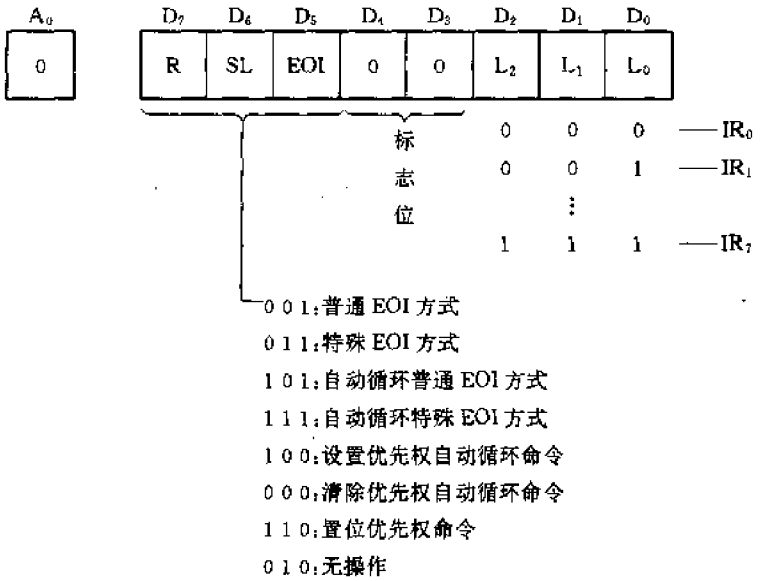


图 7.14 OCW_2 的格式

L_2～L_0 为 8 个中断请求输入端 IR_7～IR_0 的标识位，用来指定中断级别。L_2～L_0 指定的中断级别是否有效，由 SL 位控制。当 SL＝1 时，L_2～L_0 起作用；当 SL＝0 时，L_2～L_0 不起作用。

EOI 为中断结束命令位，若 EOI＝1 时，在中断服务程序结束时向 82C59A 回送中断结束命令 EOI，以便使中断服务寄存器 ISR 中当前最高优先权位复位(指普通 EOI 方式)或由 L_2～L_0 标识的优先权位复位(指特殊 EOI 方式)。

R 为优先权循环位。若 R＝1 时，为优先权自动循环方式。若 R＝0，为优先权固定方式。

D_4D_3 位为 OCW_2 标志位，当 A_0＝0 且 D_4D_3 位为 00 时，CPU 执行写入 OCW_2 操作。

3. OCW_3

OCW_3 用于设置或清除特殊屏蔽方式和读取寄存器的状态，格式如图 7.15 所示。

RR 为读 ISR 和 IRR 命令位，RIS 为寄存器标识位，当 RR＝1，RIS＝0 时，读取 IRR 命令；当 RR＝1，RIS＝1 时，读取 ISR 命令。在进行读 ISR 或 IRR 操作时，先写入读命令 OCW_3，然后紧接着执行读ISR或IRR的指令。例如设82C59A的两个端口地址为80H和

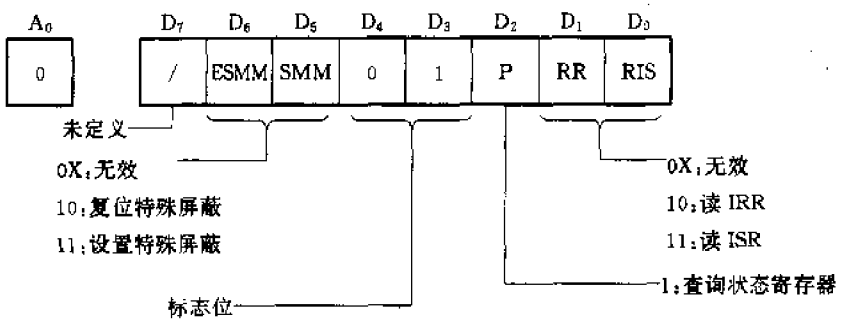


图 7.15 OCW_3 的格式

81H,OCW_3、ISR 和 IRR 共用一个地址 80H,读取 ISR 和 IRR 内容的指令段分别为:

```
MOV   AL,00001011B
OUT   80H,AL          ;读 ISR 命令写入 OCW3
IN    AL,80H          ;读 ISR 内容至 AL 中
 ⋮
MOV   AL,00001010B    ;读 IRR 命令写入 OCW3
OUT   80H,AL
IN    AL,80H          ;读 IRR 内容至 AL 中
```

P 为中断状态查询位,当 P=1 时,可通过读入状态寄存器的内容,查询是否有中断请求正在被处理。如有则给出当前处理中断的最高优先级。中断状态字格式如图 7.16 所示。

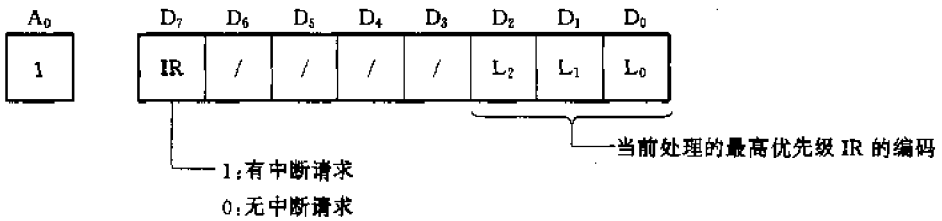


图 7.16 状态寄存器的格式

在读取中断状态字时,先写入中断查询命令,然后读取中断状态字,程序如下:

```
MOV   AL,00001100B
OUT   80H,AL          ;读中断状态字命令写入 OCW3
IN    AL,80H          ;读取中断状态字
```

ESMM 与 SMM 组合可用来设置或取消特殊屏蔽方式。当 ESMM=1,SMM=1 时,设置特殊屏蔽;当 ESMM=1,SMM=0 时,取消特殊屏蔽。

(三)82C59A 的初始化编程

82C59A 是一个可编程芯片,在使用时,需要对它的工作方式进行编程设置。初始化编程主要完成对 ICW_1~ICW_4 的设置。即:

(1)设置芯片的连接方式;

(2)设置中断请求的触发方式;

(3)设置中断请求源的中断类型号;

(4)设置中断管理方式。

由于 ICW_1~ICW_4 四个命令字使用两个端口地址,即 ICW_1 用 $A_0=0$ 的端口地址,ICW_2~

ICW_4 共用一个地址，使用 $A_0=1$ 的端口地址，因此，在写入命令字时须按图 7.17 所示的顺序写入。

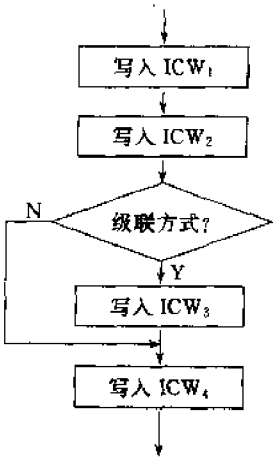


图 7.17　82C59A 初始化流程图

例如在 IBM PC/XT 系统中，只用一片 82C59A 管理中断，硬件连接如图 7.18 所示。8 个中断请求 IR_7～IR_0，除 IR_2 提供给用户使用外，其它均为系统使用。设定：IR_2 为边沿触发、完全嵌套和非自动结束方式；中断类型号为 08H；端口地址为 20H 和 21H。初始化程序段如下：

```
MOV   AL,00010011B
OUT   20H,AL          ;写入 ICW1
MOV   AL,00001000B
OUT   21H,AL          ;写入 ICW2
MOV   AL,00000001B
OUT   21H,AL          ;写入 ICW4
```

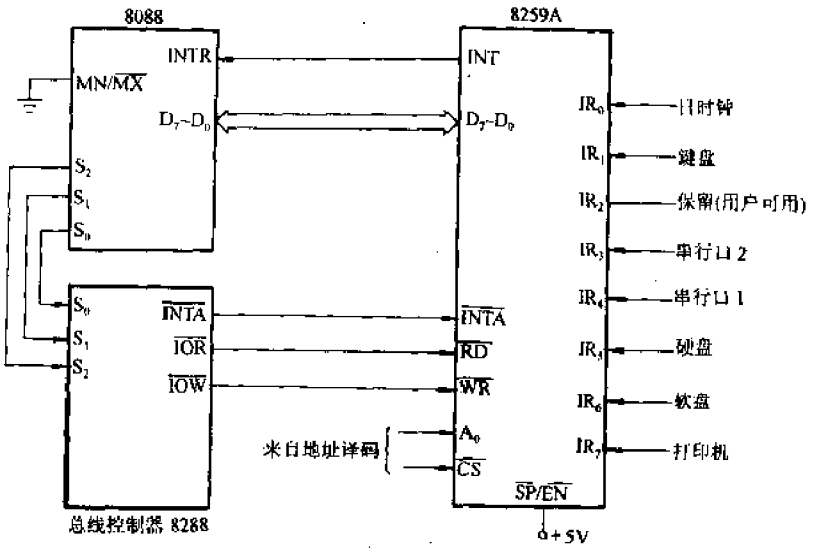


图 7.18　PC/XT 中断控制结构

在 IMB PC/AT 系统中，中断管理由主、从两片 82C59A 组成，硬件连接如图 7.19 所示。两片级联信号线 CAS_0～CAS_2 相连，主片$\overline{SP}/\overline{EN}$引脚接+5V，从片$\overline{SP}/\overline{EN}$引脚接地，从片中断请求 INT 引脚与主片的 IR_2 引脚相连。设定：主片端口地址为 20H 和 21H，中断类型号为 08H；从片端口地址为 A0H 和 A1H，中断类型号为 70H。主片为边沿触发、特殊完全嵌套、非自动结束和非缓冲方式。从片为边沿触发、完全嵌套和非自动结束方式。

主片初始化程序如下：

```
MOV   AL,00010001B
OUT   20H,AL          ;写入 ICW1
MOV   AL,00001000B
OUT   21H,AL          ;写入 ICW2
```

```
MOV   AL,00000100B
OUT   21H,AL          ;写入 ICW3
MOV   AL,00010001B
OUT   21H,AL          ;写入 ICW4
```

从片初始化程序如下：

```
MOV   AL,00010001B
OUT   0A0H,AL         ;写入 ICW1
MOV   AL,01110000B
OUT   0A1H,AL         ;写入 ICW2
MOV   AL,00000010B
OUT   0A1H,AL         ;写入 ICW3
MOV   AL,00000001B
OUT   0A1H,AL         ;写入 ICW4
```

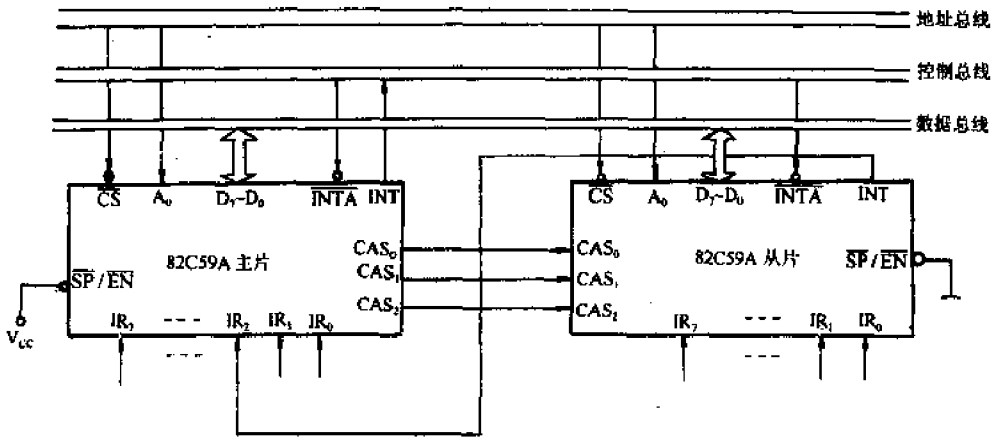


图 7.19　PC/AT 中断控制结构

五、82C59A 的应用

为了对中断管理功能建立一个完整的概念，以 IBM PC/XT 中断系统为例，下面给出了一个 82C59A 应用实例。

要求：中断请求信号以跳变方式由 IR_2 引入，(可为任一定时脉冲信号)，编写程序，当 CPU 响应 IR_2 请求时，输出显示字符串："THIS A 82C59A INTERRUPT!"，中断 10 次程序退出。

程序如下：

```
DATA   SEGMENT
MESS   DB 'THIS A 82C59A INTERRUPT!',0AH,0DH,'$'
INTA00  EQU  0020H
INTA01  EQU  0021H
DATA   ENDS
STACK  SEGMENT  STACK
STA   DB  100H  DUP(?)
```

```
TOP  EQU  LENGTH  STA
STACK  ENDS
CODE    SEGMENT
        ASSUME  CS:CODE,DS:DATA,SS:STACK
MAIN:MOV  AX,DATA
     MOV  DS,AX
     MOV  AX,STACK
     MOV  SS,AX
     MOV  SP,TOP
     MOV  DX,INTA00          ;8259A 初始化
     MOV  AL,13H             ;写 ICW1
     OUT  DX,AL
     MOV  DX,INTA01
     MOV  AL,08H             ;写 ICW2
     OUT  DX,AL
     MOV  AL,01H             ;写 ICW4
     OUT  DX,AL
     PUSH DS
     MOV  AX,SEG  INT-P      ;设置中断矢量
     MOV  DS,AX
     MOV  DX,OFFSET  INT-P
     MOV  AL,0AH
     MOV  AH,25H
     INT  21H
     POP  DS
     MOV  AL,0FBH            ;写中断屏蔽字 OCW1
     OUT  DX,AL
     MOV  DX,INTA00
     MOV  AL,20H             ;写中断结束方式 OCW2
     OUT  DX,AL
     MOV  BX,10
WAIT:STI                     ;开中断
     JMP  WAIT               ;等待中断
INT-P:MOV  AX,DATA           ;中断服务程序入口
     MOV  DS,AX
     MOV  DX,OFFSET  MESS    ;输出指定字符串
     MOV  AH,09H
     INT  21H
     MOV  DX,INTA00          ;写 OCW2,送中断结束命令 EOI
     MOV  AL,20H
```

```
        OUT   DX,AL
        DEC   BX                    ;控制 10 次循环
        JNZ   NEXT
        MOV   DX,INTA01             ;读屏蔽寄存器 IMR
        IN    AL,DX
        OR    AL,04H                ;屏蔽 IR2 请求
        OUT   DX,AL
        STI                         ;开中断
        MOV   AX,4C00H              ;返回操作系统
        INT   21H
NEXT:   IRET  ;中断返回
CODE:   ENDS
        END   MAIN
```

思考题与习题

1. 什么叫中断？中断的功能有哪些？CPU 响应中断时需要经过哪些处理过程？

2. IBM PC 机的中断源分为哪几种类型，它们各有什么特点？

3. 中断矢量表用来存放什么内容？它存放在内存的哪个区域？CPU 是如何设置和读取中断矢量表的内容？

4. 简述 82C59A 中断控制器的中断管理功能。

5. 82C59A 有哪几种中断结束方式？它们适合应用在什么场合？

6. 82C59A 初始化需要写入的命令字有哪些？写入的顺序是什么？

7. 在 82C59A 初始化时为什么要提供中断类型号？CPU 怎样读取它？

8. 外部中断 INTR 和 NMI 有何区别？CPU 在什么时候查询外部中断请求？响应 INTR 请求的条件是什么？

9. 设某系统使用一片 82C59A 管理中断，中断请求由 IR_2 引入，端口地址为 80H 和 81H，采用电平触发、完全嵌套、普通 EOI 结束方式，中断类型号为 40H，试编写初始化程序。

10. 设某系统使用两片 82C59A 管理中断。主片的端口地址为 80H 和 81H，工作于边沿触发、特殊完全嵌套、非自动结束和非缓冲方式；从片的端口地址为 20H 和 21H，工作于边沿触发、完全嵌套、非自动结束和非缓冲方式(中断类型号自设)。要求：

(1)画出主从片的级联图

(2)编写主从片初始化程序

11. 若要求 82C59A 的端口地址为 0FE20H 和 0FE21H，试画出 82C59A 与 CPU 的硬件连接图。若系统中由 82C59A 的 IR_2 引入外设中断请求(跳变信号有效)，要求当 CPU 响应 IR_2 请求时，输出显示字符串："＊＊＊＊＊"，并中断 10 次退出。试编写完成以上功能的完整程序。

第八章　输入输出接口技术

输入和输出设备是计算机系统的重要组成部分，称为外部设备或I/O设备。在微机应用系统中，被计算机处理的信息，如程序、原始数据和各种现场采集到的数据需要通过输入设备送入计算机处理，而计算机处理的结果又需要通过输出设备输出显示或打印。

随着计算机系统功能的不断增强，I/O设备的种类繁多，常用的输入设备有：键盘、鼠标、磁盘、光盘等。常用的输出设备有：显示器、打印机、绘图仪等。由于I/O设备的特性差别很大，如有机械式、电动式、电子式和光电式等；传送的信息也不同，如有数字量、模拟量和开关量等。因此CPU与I/O设备无法直接连接，需要通过"接口"把CPU与I/O设备连接起来。接口把来自外部设备的各种信号变换之后送给CPU，而CPU处理的结果再经接口变换之后送给外部设备，实现这一过程的处理技术称为输入/输出接口技术。

本章在介绍I/O接口基本概念的基础上，重点介绍几种通用的I/O接口芯片及其应用实例。

第一节　概　　述

一、I/O接口的构成及功能

(一)I/O接口的基本构成

I/O接口是CPU与外部设备进行信息交换的桥梁，三者之间的关系如图8.1所示。CPU与外部设备之间传送的信息流为：CPU ⟺ I/O接口 ⟺ 外部设备。CPU与I/O接口之间传送的信息主要有：

(1)命令信息。CPU送至接口，用于控制接口的工作方式；

(2)状态信息。接口送至CPU，供CPU了解接口的工作状态。

(3)数据。与外部设备交换的信息。

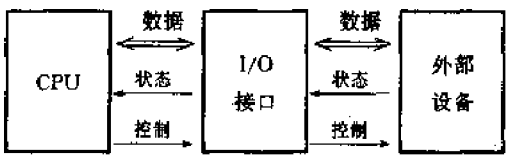


图8.1

为了实现以上信息的传送，I/O接口应具备信息存储和控制功能。通常由下列几部分构成：

(1)输入/输出数据缓冲单元。用来存放外设送入CPU的数据或CPU送往外设的数据。

(2)读/写控制电路。用来接收系统总线上的读/写控制信号，以便产生接口内部的读、写控制信号，完成接口内部寄存器的读出或写入。

(3)命令/状态单元。命令单元用来存放CPU发送给接口的命令信息，状态单元用来存放接口的工作状态信息，供CPU查询。每个存储单元称为寄存器或端口。

(二)I/O接口的功能

I/O接口的主要功能可概括为以下几点：

(1)具有缓冲数据的传送功能，以实现高速CPU与慢速I/O设备之间数据传送时取得同

步。

(2)具有转换数据格式的功能。如串行与并行数据格式之间的转换。

(3)具有信号量转换的功能。如数字量与模拟量之间的转换。

(4)具有定时/计数功能。以满足总线对数据传送的时序要求等。

随着计算机应用的日益广泛,I/O 接口芯片产品很多,大致可分为专用接口芯片和通用接口芯片两大类。专用接口是为某种专门用途或某种专用 I/O 设备而研制的。如 CRT 显示器接口、打印机接口、键盘和磁盘接口等。通用接口以用于多种用途为目标。如并行接口、串行接口、定时/计数器等。

二、I/O 接口的编址方式

把 I/O 接口中能被 CPU 访问的寄存器称为端口。为了使 CPU 能对端口进行正确的读写操作,要为每个端口分配一个地址,称为端口地址,简称端口。通常有下列两种端口编址方式。

1. 端口与存储器统一编址

在这种编址方式中,把 I/O 接口中的每个寄存器看成存储器的一个存储单元,纳入统一的存储器地址空间,为每一个端口分配一个存储器地址,CPU 可以用访问存储器的方式来访问端口。

这种编址方式的优点是不用专门设置访问 I/O 接口的指令,用于存储器操作的指令都可以用于端口的操作。其缺点是由于端口地址占用了存储器空间的地址,使存储器的实际存储容量减少。

2. 端口独立编址

在这种编址方式中,端口地址是独立的,CPU 使用专门的 I/O 指令来访问 I/O 端口。80X86CPU 指令系统就采用了这种编址方式。有如下两种寻址:

(1)直接寻址

I/O 端口使用 8 位地址 $A_7 \sim A_0$ 编码,即 00H~FFH,可寻址 256 个端口。在输入输出指令中,I/O 端口地址以 8 位立即数方式给出,指令格式为:

```
OUT nnH,AL   ;AL→端口
IN AL,nnH    ;AL←端口
```

(2)间接寻址

I/O 端口使用 16 位地址 $A_{15} \sim A_0$ 编码,即 0000H~FFFFH,可寻址 65536 个端口。在输入输出指令中,I/O 端口地址以间接方式给出,指令格式为:

```
MOV DX,nnnnH   ;端口地址存入 DX
OUT DX,AL      ;AL→端口
IN AL,DX       ;AL←端口
```

这种编址方式使 I/O 端口不占用存储器的地址,可提高存储器空间的利用率。

三、I/O 数据传送的控制方式

CPU 与外部设备之间的数据传送实际上是 CPU 与接口之间的数据传送。根据不同的外部设备的特点,I/O 数据传送通常采用程序控制方式、中断控制方式和 DMA(直接内存访问)控制方式。

1. 程序控制方式

程序控制方式的特点是:输入/输出操作完全在程序控制下进行的,用 IN 和 OUT 指令直

接访问 I/O 端口。在这种方式中，根据外设的特点可采用直接传送数据或查询方式传送数据。

图 8.2 所示流程图为程序查询方式的执行流程。因为 CPU 与外部设备往往不是同步工作，只有当外设准备就绪，CPU 才能传送数据。因此这类外设通常提供工作状态信息供 CPU 查询。

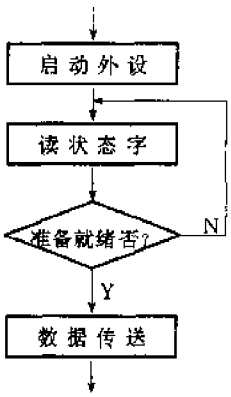


图 8.2 查询控制流程图

从查询控制方式的执行过程可以看出，查询传送实际上是程序循环等待。在数据传送之前，先从 I/O 状态口读取状态字进行测试，若外设准备好，进行数据传送；若外设未准备好，继续转去读状态字进行测试。由于在外设准备数据期间，CPU 只能循环等待而不能进行其它操作，致使 CPU 的利用率较低。因此，这种方式适合于工作不太繁忙的系统中。

2. 中断控制方式

中断控制方式的特点是：当外部设备需要进行输入/输出操作时，向 CPU 发出“中断”请求信号，请求 CPU 处理。中断处理过程已在第七章详细介绍过，这里不再重复。

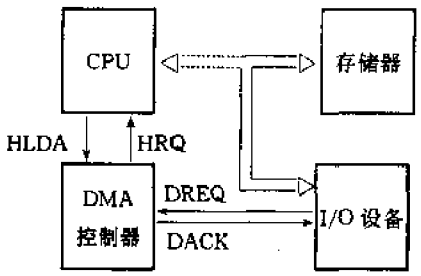


图 8.3 DMA 传送控制示意图

3. DMA 控制方式

DMA（直接存储器访问）控制方式是由 DMA 控制器控制存储器与高速 I/O 设备之间直接进行数据传送。DMA 传送控制示意图如图 8.3 所示。

DMA 控制方式的工作过程是：若 I/O 设备与存储器之间需要传送一批数据时，先由 I/O 设备向 DMA 控制器发出请求信号 DREQ，再由 DMA 控制器向 CPU 发送请求占用总线的信号 HRQ，CPU 响应 HRQ 后向 DMA 回送一个响应信号 HLDA，随后 CPU 让出总线控制权给 DMA 控制器，再由 DMA 控制器回送 I/O 设备应答信号 DACK。此时可以开始进行存储器与 I/O 设备之间直接数据传送。当一批数据传送完毕，DMA 把总线控制权退还给 CPU。DMA 传送控制方式的特点是：在数据传送过程中，以内存为中心，不需要 CPU 的干预，对批量数据传送效率很高，通常用于高速 I/O 设备（如磁盘）与内存之间的数据传送。详细工作过程见 DMA 控制器一节。

第二节 并行接口 8255A

并行通信是计算机与 I/O 设备进行数据传送的一种方式。其特点是：传送的数据以字节或字为单位，每个数据位占用一根数据传输线，各数据位同时发送或接收，传输效率高，但传输线的成本高。因此，这种通信方式合适于数据传送距离较近的系统中。

8255A 是 Intel 公司生产的通用可编程并行输入和输出接口芯片，片内有三个 8 位输入/输出口线，可作为 CPU 与 I/O 设备的接口。

一、8255A 的结构与引脚信号

8255A 采用单一＋5V 电源供电，40 引脚 DIP 封装，内部结构与引脚信号如图 8.4 和图 8.5 所示。

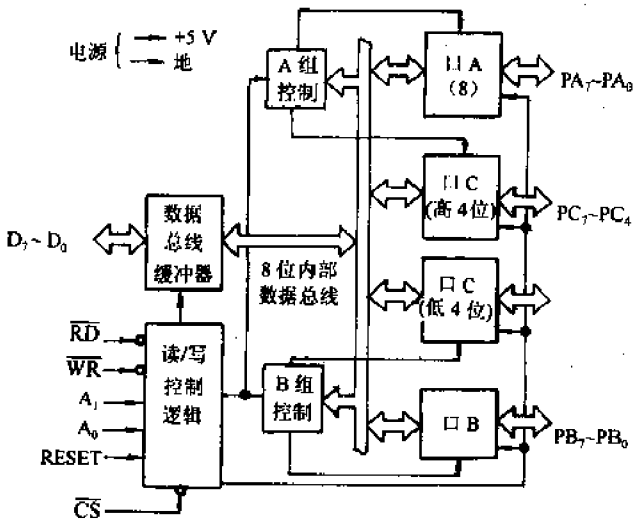


图 8.4　8255A 的内部结构框图

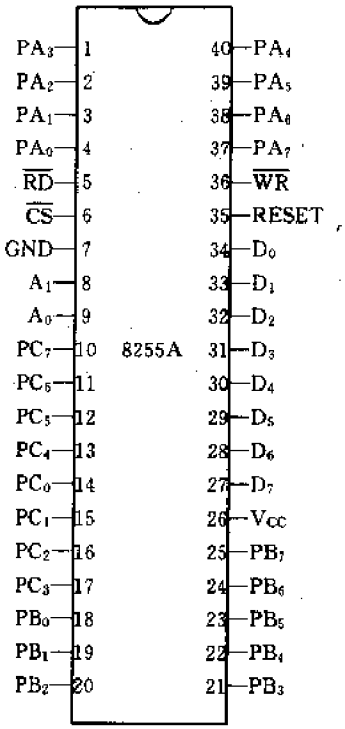


图 8.5　8255A 引脚图

各部分功能说明如下：

1. 数据总线缓冲器

数据总线缓冲器为三态、双向 8 位寄存器。数据线 $D_7 \sim D_0$ 与系统数据总线连接，用于 CPU 与 8255A 之间传递信息。CPU 通过数据总线缓冲器向 8255A 写入数据和命令或读取数据。

2. 读/写控制逻辑

读/写控制逻辑用来接收来自系统总线的读、写控制信号和端口选择信号，控制 8255A 内部的读、写操作。与读、写控制逻辑有关的引脚信号有：

$\overline{WR}$：写信号，输入，低电平有效。

$\overline{RD}$：读信号，输入，低电平有效。

$\overline{CS}$：片选信号，输入，低电平有效。

A_1、A_0：端口选择信号，输入。

RESET：复位信号，输入，高电平有效。

片选信号$\overline{CS}$通常由系统地址译码器产生。端口选择信号 A_0 和 A_1 通常与系统地址总线 A_0 和 A_1 连接，经 8255A 片内译码产生四个端口地址 00、01、10 和 11 以次分配给 A 口、B 口、C 口和控制寄存器。$\overline{RD}$和$\overline{WR}$引脚与系统总线的 I/O 读、写控制线$\overline{IOR}$和$\overline{IOW}$连接，以便决定 CPU 与 8255A 之间数据传送的方向。当 CPU 执行输入(IN)指令时，$\overline{RD}$信号有效；当 CPU 执行输出(OUT)指令时，$\overline{WR}$信号有效。读、写控制信号组合功能见表 8.1。

复位信号 RESET 与系统复位信号连接在一起，当该信号有效后，片内寄存器被清除，A、B、C 三个端口被置为输入方式。

表 8.1　8255A 读、写功能表

$\overline{CS}$	$\overline{WR}$	$\overline{RD}$	A_1	A_0	功　　能
0	0	1	0	0	数据写入 A 口
0	0	1	0	1	数据写入 B 口
0	0	1	1	0	数据写入 C 口
0	0	1	1	1	命令写入控制寄存器
0	1	0	0	0	读取 A 口数据
0	1	0	0	1	读取 B 口数据
0	1	0	1	0	读取 C 口数据
0	1	0	1	1	无操作

3. 输入/输出口

8255A 片内有三个 8 位的并行输入/输出口，称为 A 口、B 口和 C 口，用来与 I/O 设备连接。

A 口为 8 位 I/O 锁存/缓冲器，PA_7～PA_0 为 A 口 8 条 I/O 线。

B 口为 8 位 I/O 锁存/缓冲器，PB_7～PB_0 为 B 口 8 条 I/O 线。

C 口为 8 位输出锁存/缓冲器和 8 位输入缓冲器，PC_7～PC_0 为 C 口 8 条 I/O 线。C 口除了作为数据端口外，还有一个特殊的功能：当 A 口、B 口数据传送需要联络控制信号时，C 口的高 4 位(PC_7～PC_4)分配给 A 口使用，C 口的低 4 位(PC_3～PC_0)分配给 B 口使用。

8255A 读/写控制分为 A 组控制和 B 组控制。A 组控制 A 口和 C 口的高 4 位，B 组控制 B 口和 C 口的低 4 位。

二、8255A 的工作方式

8255A 有下列三种工作方式：

1. 方式 0——基本输入/输出

方式 0 是 8255A 的基本的 I/O 传送方式，A、B、C 三个端口都可以工作在方式 0。A 口和 B 口为两个独立的 8 位 I/O 口，C 口可分为两个 4 位 I/O 口，即 PC_7～PC_4 和 PC_3～PC_0 可以分别设置成输入或输出。

方式 0 适用于无条件数据传送或查询方式的数据传送，可利用 C 口的某一位作为查询状态位。

2. 方式 1——选通输入或输出

方式 1 是 8255A 需要联络信号(或称握手信号)的一种单方向数据传送方式。适合于中断或查询方式传送数据。

方式 1 分为 A 组控制和 B 组控制两部分。在 A 组控制中，A 口作为 8 位数据端口，C 口的高 4 位作为联络信号线。在 B 组控制中，B 口作为 8 位数据端口，C 口的低 4 位作为联络信号线。由于 C 口作为联络控制线使用，因此只允许 A 口和 B 口工作于方式 1。

(1)方式 1 输入

A、B 口方式 1 数据输入引脚信号功能如图 8.6 所示。A 口工作于方式 1 输入时，用 PC_3～PC_5 作联络控制线。B 口工作于方式 1 输入时，用 PC_0～PC_2 作联络控制线。联络信号功能如下：

$\overline{STB}$为选通输入，低电平有效。它是由外设提供的输入信号。当$\overline{STB}$为低电平时，把外设送来的数据送入 A 口或 B 口的输入锁存器中锁存。

IBF 为输入缓冲器满信号，输出，高电平有效。当输入锁存器中已接收到外设送来的数据后，IBF 置“1”，数据被 CPU 读取后，IBF 被清除。

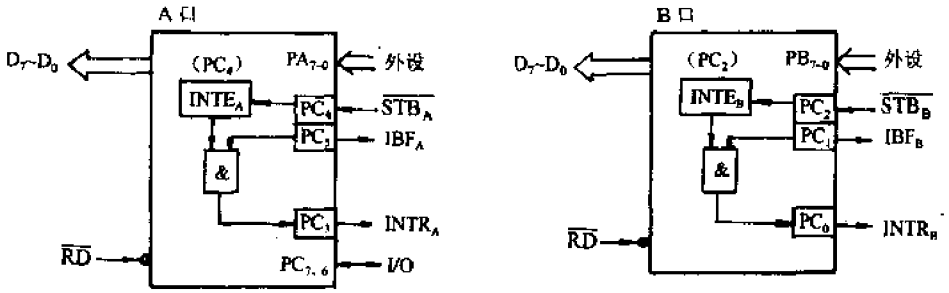


图 8.6　A、B 口方式 1 输入引脚信号

INTR 为中断请求信号，输出，高电平有效。在中断允许 INTE=1 且 IBF=1 的条件下，由 $\overline{STB}$ 的后沿可产生一个“高”有效的中断请求 INTR，用于请求用中断方式输入数据。方式 1 输入工作时序如图 8.7 所示。

INTE 为输入中断允许控制标志。INTE=1，中断允许；INTE=0，中断禁止。在 A 口方式 1 输入中，由 PC_4 控制；在 B 口方式 1 输入中。由 PC_2 控制，PC_4 与 PC_2 的状态可由置位/复位控制命令来设置。

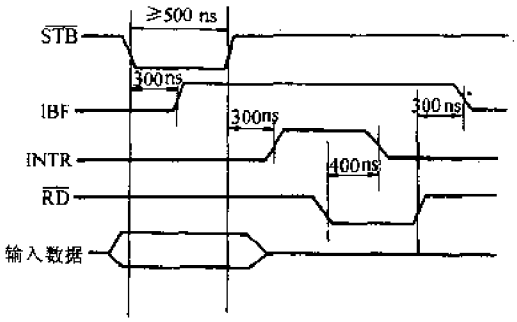


图 8.7　方式 1 输入工作时序

(2)方式 1 输出

A、B 口方式 1 数据输出引脚信号功能如图 8.8 所示。A 口工作于方式 1 输出时，用 PC_3、

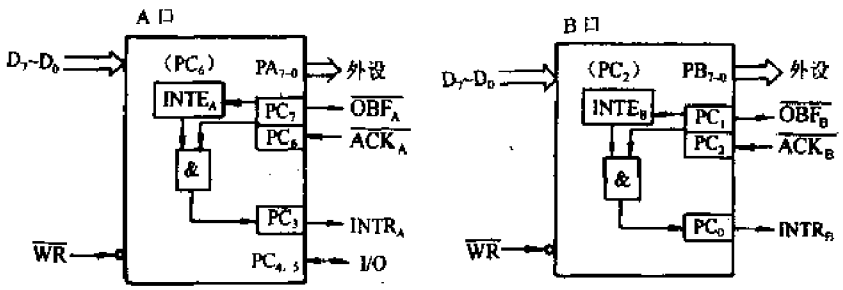


图 8.8　A、B 口方式 1 输出引脚信号

PC_6 和 PC_7 作联络控制线。B 口工作于方式 1 输出时，用 PC_0～PC_2 作联络控制线。联络信号功能如下：

$\overline{OBF}$ 为“输出缓冲器满”指示信号，输出，低电平有效。当 CPU 写入端口数据时，由 $\overline{WR}$ 信号上升沿使 $\overline{OBF}$ 变为低电平，用此信号通知外设读取端口中的数据。

$\overline{ACK}$ 为应答信号，输入，低电平有效。外设对 $\overline{OBF}$ 的应答信号，用于清除 $\overline{OBF}$。

INTR 为中断请求信号，输出，高电平有效。在中断允许 INTE=1 且 $\overline{OBF}$=1 的条件下，由 $\overline{ACK}$ 的后沿可产生一个“高”有效的中断请求信号 INTR，用于请求用中断方式输出数据。方式 1 输出工作时序如图 8.9 所示。

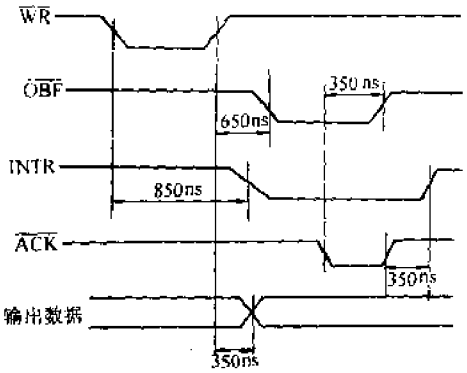
INTE 为输出中断允许控制标志。INTE=1，则中断允许；INTE=0，则中断禁止。在 A 口方式 1 输出中，由 PC_6 控制；在 B 口方式 1 输出中，由 PC_2 控制。PC_6 与 PC_2 的状态可由置位/复位控制命令设置。

3. 方式 2——双向选通输入输出

在方式 1 中，只允许 A 口或 B 口设定为带选通信号的单方向输入或输出数据传送方式。在方式 2 中，允许 A 口设定为带选通信号的双向输入输出数据传送方式。

A 口工作于方式 2 的引脚信号功能如图 8.10 所示。用 C 口的 PC_7～PC_3 作为双向数据传送的联络控制信号线，图中引脚信号定义与方式 1 类似。$\overline{STB_A}$和 IBF_A 作为数据输入的联络信号；$\overline{OBF_A}$和 $\overline{ACK_A}$为数据输出的联络信号；$INTR_A$ 作为数据输入或数据输出时的中断请求信号；$INTE_1$ 和 $INTE_2$ 为输入和输出中断允许控制标志，由 PC_6 和 PC_4 置位/复位控制。

需要指出的是由于 8255A 各端口的工作方式可以独立编程设定，允许各端口以不同的方式工作。



注：图中所标的参数为最大值。

图 8.9　方式 1 输出工作时序

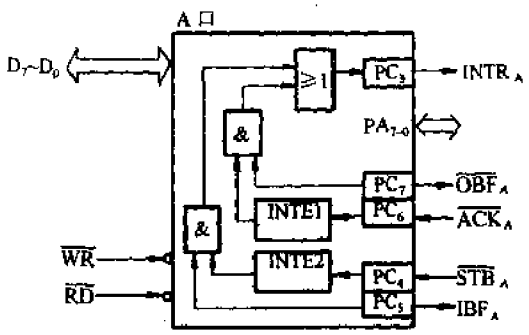


图 8.10　A 口方式 2 引脚信号

三、8255A 的编程

8255A 各端口的工作方式在初始化编程时通过写入工作方式控制字来设定。

(一)控制字格式

8255A 的控制字有如下两种：

1. 工作方式控制字

该字用于设置各端口的工作方式和数据传送的方向。其格式如图 8.11 所示。

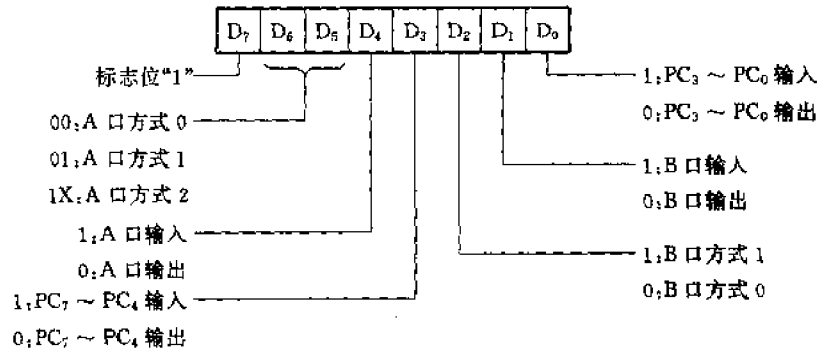


图 8.11　8255A 控制字格式

由工作方式控制字可知，A 组有三种方式(方式 0,1,2)，B 组有两种方式(方式 0,1)。C 口分为两部分，PC_3～PC_0 为 B 组，PC_7～PC_4 为 A 组。D_7 位为工作方式字标志位，恒为“1”。

2. 置位/复位控制字

该字用来设置 C 口某一位口线的状态为高电平或低电平。其格式如图 8.12 所示。

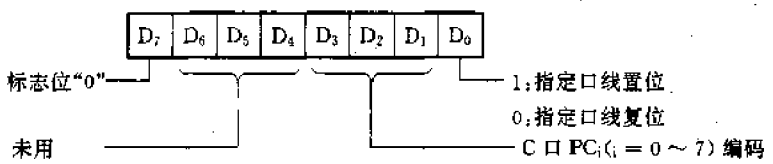


图 8.12　8255A 置位/复位控制字格式

D_7 位为该字的标志位恒为“0”；D_6～D_4 未定义，通常设为 0；D_3～D_1 为 C 口 8 条口线 PC_7～PC_0 的编码，即 000 为 PC_0，001 为 PC_1，依次类推；D_0 为置位或复位指示位，若 $D_0=1$，则指定口线状态为高电平，若 $D_0=0$，则指定口线状态为低电平。例如若把 PC_2 口线输出状态设置为高电平，则该字应为 05H。

置位/复位控制字只能对 C 口的某一位 I/O 线的输出状态进行控制，对各端口的工作方式无影响。

(二)8255A 的初始化编程

8255A 的初始化编程是指通过给 8255A 写入控制字来确定各端口的工作方式。由以上两个控制字的功能可以看出，工作方式控制字是用于设定端口的工作方式，因此在初始化编程中必须写入，而置位/复位控制字是用来控制 C 口某一位口线的状态，视需要写入，两个控制字共用一个端口地址，依据标志位加以区分。

例如：设 8255A 的 A 口工作于方式 0，输出；B 口工作于方式 1，输入，并设定 8255A 的四个端口地址为 FFE0H～FFE3H，试编写初始化程序。

由题意可知，工作方式控制字为 10000110B。初始化程序为：

```
MOV AL,10000110B
MOV DX,0FFE3H
OUT DX,AL                ;写入控制字
```

四、8255A 的应用

8255A 作为通用的 8 位并行通信接口，常用于显示器、键盘和打印机的接口。

1. 8255A 作为显示器接口

例如：利用 8255A 作为连接显示器的接口。A 口接有四个开关，为方式 0 输入；B 口接有一个数码管显示器(共阳极 LED)，为方式 0 输出，其硬件连接电路如图 8.13 所示。试编制程序，将 A 口输入的数据送 B 口输出显示。

本例是 8255A 方式 0 应用的一个例子。由图可知，8255A 的端口地址由 A_0～A_{15}提供，8255A 的端口选择信号线 A_0、A_1 与 CPU 地址线 A_0、A_1 相连，$\overline{CS}$与地址译码器输出相连，本例设 8255A 的四个端口地址为 FFC0H～FFC3H。A 口连接输入设备开关，其输入的数据为 K_3～K_0 组合的 16 种状态，即 0000～1111B(0～FH)，B 口连接输出设备 LED 显示器，可输出显示十六进制数 0～F。LED 为共阳极七段发光二极管，其七段显示代码如表 8.2 所示。当 A 口的 K_3～K_0 的状态为 0011B 时，B 口对应输出七段代码为 B0H，则 LED 显示数字 3。

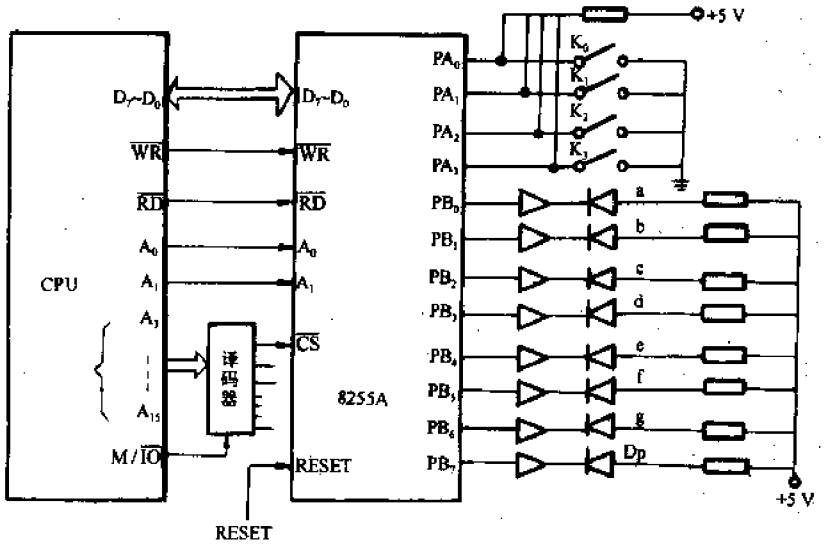


图 8.13　8255A 为显示器接口图

表 8.2

显示字符	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
七段代码(H)	C0	F9	A4	B0	99	92	82	F8	80	90	88	83	C6	A1	86	8E

显示程序如下：

```
DATA SEGMENT
    LIST DB C0H,F9H,A4H,…,8EH
    PORTA EQU FFC0H
    PORTB EQU FFC1H
    CONTR EQU FFC3H
DATA ENDS
CODE SEGMENT
ASSUME CS:CODE,DS:DATA
START:MOV AX,DATA
      MOV DS,AX
      MOV AL,90H          ;A 口方式 0 输入,B 口方式 0 输出
      MOV DX,CONTR
      OUT DX,AL           ;写控制字
L0:   MOV DX,PORTA        ;读 A 口,取开关状态
      IN  AL,DX
      AND AL,0FH          ;屏蔽 PA7~PA4
      MOV BX,0FFSET LIST
      XLAT                ;查七段代码表字符⇒AL
      MOV DX,PORTB
      OUT DX,AL           ;输出显示
      MOV CX,1000         ;显示延时
```

```
L1:     DEC CX
        JNZ  L1
        JMP  L0                    ;循环显示
CODE    ENDS
        END  START
```

此程序是循环显示程序，可由硬件强迫中断。

2. 8255A 作为打印机的接口

例如：利用 8255A 的 A 口作为 μ80 微型打印机的接口。A 口工作于方式 1 输出，其硬件连接电路如图 8.14 所示。试编制程序，将输出缓冲区 OBUF 内的 100H 个字节数据送打印机输出。

μ80 微型打印机的工作原理是：当数据选通信号$\overline{DSTB}$有效时，数据线 $D_7 \sim D_0$ 上的数据被锁存打印，同时输出应答信号$\overline{ACK}$，告知 8255A 数据已取走。由于打印机的速度比 CPU 的速度低许多，因此打印机提供了“忙”状态信号 BUSY，供 CPU 查询。当打印机正在执行打印时，BUSY 为“1”，此时 CPU 不能输出数据，当打印机“空闲”时，BUSY 为“0”，通知 CPU 可以输出数据。BUSY 信号供 CPU 用查询方式输出数据。

由图 8.14 可知，由于打印机需要用联络信号$\overline{DSTB}$和$\overline{ACK}$与 8255A 取得联系，故 8255A 的 A 口选择方式 1 输出工作方式。8255A 的端口地址由地址线 A_0、A_1 和 74LS138 译码器输入地址线 $A_2 \sim A_6$ 经译码器输出$\overline{Y_0}$的逻辑组合确定，若把未连接的地址线 $A_7 \sim A_{15}$ 的状态设定为“0”，则 8255A 的四个端口地址为：0400～0403H。

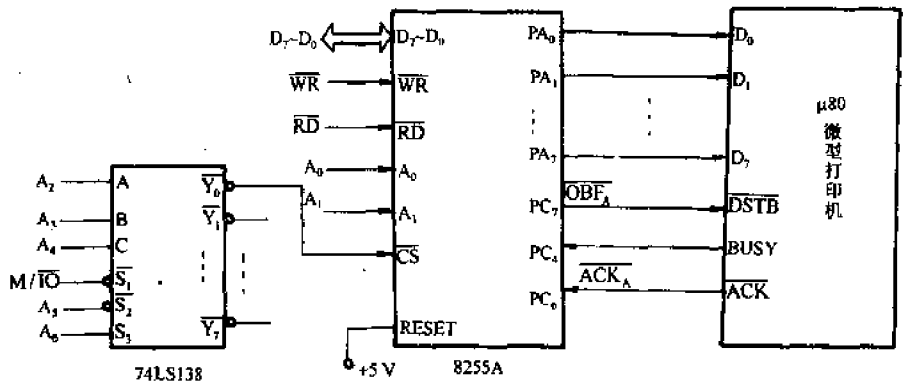


图 8.14　8255A 为打印机接口图

打印程序如下：

```
DATA SEGMENT
    OBUF DB 100H DUP(?)
    PORTA EQU 0400H
    PORTC EQU 0402H
    CONTR EQU 0403H
DATA ENDS
CODE SEGMENT
```

```
ASSUME CS:CODE,DS:DATA
START:MOV AX,DATA
      MOV DS,AX
      MOV AL,0A8H          ;A 口方式 1 输出,PC4 输入
      MOV DX,CONTR
      OUT DX,AL            ;写控制字
      MOV CX,100H
      MOV SI,OFFSET OBUF
L0:   MOV DX,PORTC
L1:   IN AL,DX
      AND AL,10H           ;查询 BUSY 的状态,打印机空闲否?
      JNZ L1
      MOV AL,[SI]
      MOV DX,PORTA
      OUT DX,AL            ;输出数据至 A 口
      INC SI
      LOOP L0              ;循环输出数据
      MOV AX,4C00H
      INT 21H
CODE ENDS
    END START
```

第三节　定时/计数器 8253

在微型计算机应用系统中,常常需要为处理器和外部设备提供时间基准或对外部事件进行计数。因此,定时/计数电路成为计算机外围电路的重要组成部分。

可编程定时/计数器接口芯片 8253 是 Intel 公司研制的 I/O 产品。它内部有三个独立的 16 位计数器,每个计数器有六种工作方式,可用于定时、计数和作为波特率发生器等,最高计数速率可达 2MHz。

一、8253 的结构与引脚信号

8253 采用单一+5V 电源供电,NMOS 工艺制造,24 引脚 DIP 封装,其内部结构与引脚信号如图 8.15 和图 8.16 所示。

各部分功能说明如下:

1. 数据总线缓冲器

数据总线缓冲器为三态、双向 8 位寄存器。数据线 $D_7 \sim D_0$ 与系统数据总线连接,用于 CPU 与 8253 之间传送信息。CPU 通过数据总线缓冲器向 8253 写入数据和命令或读出数据。

2. 读/写控制逻辑

读/写控制逻辑接收来自系统总线的读、写控制信号和 I/O 端口选择信号,选择读出或写入的寄存器,控制 8253 内部进行读、写操作。与读、写控制逻辑有关的引脚信号有:

$\overline{WR}$：写信号，输入，低电平有效。
$\overline{RD}$：读信号，输入，低电平有效。
$\overline{CS}$：片选信号，输入，低电平有效。

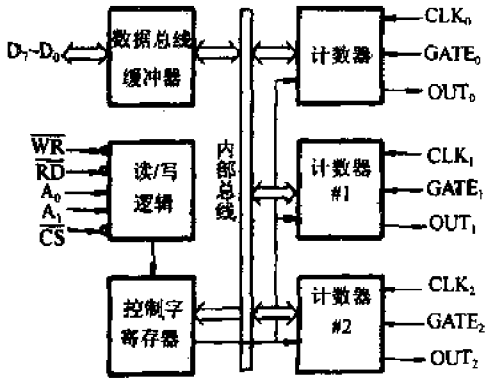


图 8.15　8253 结构框图

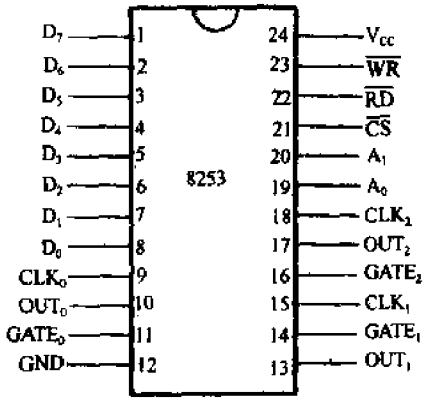


图 8.16　8253 引脚信号图

A_1、A_0：端口选择信号，输入。

片选信号$\overline{CS}$通常由系统地址译码器产生，端口选择信号 A_1 和 A_0 与系统地址总线的 A_1 和 A_0 连接，经 8253 内部译码器产生四个端口地址 00～11 以次分配给计数器 0、计数器 1、计数器 2 和控制寄存器。$\overline{RD}$和$\overline{WR}$与系统总线的 I/O 读写信号线$\overline{IOR}$和$\overline{IOW}$连接以便决定 CPU 与 8253 之间数据传送的方向。当 CPU 执行写(OUT 指令)操作时，$\overline{WR}$信号有效；当 CPU 执行读(IN 指令)操作时，$\overline{RD}$信号有效。读、写信号组合功能见表 8.3。

表 8.3　8253 内部寄存器的读写功能

$\overline{CS}$	$\overline{RD}$	$\overline{WR}$	A_1	A_0	功能
0	1	0	0	0	写计数器 0
0	1	0	0	1	写计数器 1
0	1	0	1	0	写计数器 2
0	1	0	1	1	写控制字寄存器
0	0	1	0	0	读计数器 0
0	0	1	0	1	读计数器 1
0	0	1	1	0	读计数器 2
0	0	1	1	1	无操作

3. 计数器

8253 内部有 3 个结构完全相同而又相互独立的计数器，每个计数器有六种工作方式，各自可按照编程设定的方式工作。计数器的逻辑结构如图 8.17 所示。每个计数器包含一个 16 位的计数初值寄存器，一个 16 位减 1 计数执行部件和一个具有锁存功能的 16 位锁存寄存器。对 8253 初始化时，将计数初值写入计数初值寄存器，在计数脉冲 CLK 和门控信号 GATE 的作用下，减"1"计数执行部件从计数初值寄存器中获取计数初值后便可以开始进行减 1 计数，当计数值减至"0"时，在 OUT 端输出相应的信号，用以指示一个计数过程的结束。

锁存寄存器中的内容与减"1"计数器中的内容保持一致，可供CPU读取。当有一个读锁存器的锁存命令到来时，减"1"计数器中的计数值被锁存在锁存器中，锁存的计数值被读取之后，锁存器又跟随减"1"计数执行部件的动作工作。

每个计数器有三条信号线：

CLK：计数脉冲输入信号。用来输入定时基准脉冲或计数脉冲。

GATE：门控输入信号，用来控制计数器的启动或停止。

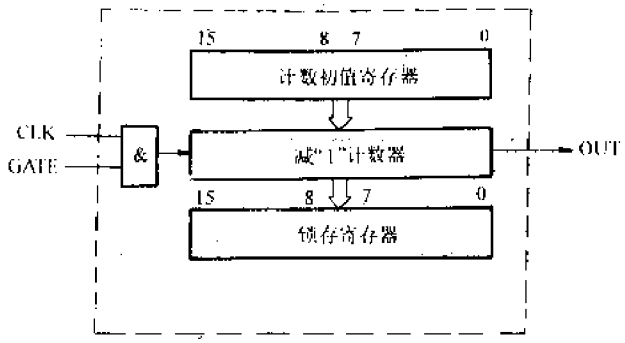


图 8.17 计数器逻辑结构图

OUT：计数器输出信号。对于六种不同的工作方式，OUT端输出信号形式不同，该信号既可以用于延时、计数控制，也可以作为中断请求信号。

二、8253的工作方式

8253的每个计数器有六种工作方式，可通过初始化编程设定。

1. 方式0——计数结束中断

方式0为计数结束中断，工作时序如图8.18所示。在写入方式0控制字(CW)之后，输出端OUT初始电平为低电平，在写入计数初值N之后的下一个计数脉冲信号CLK到来(下降沿)且门控信号GATE为高电平时开始启动计数，当计数值减为"0"时，OUT端由低变高并且保持高电平，到此完成一次计数过程。

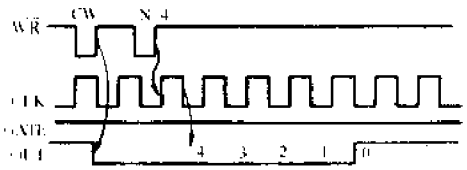


图 8.18 方式0波形

在方式0中，计数初值为一次性有效，若要继续计数，则须重新写入计数初值。门控信号GATE为电平触发计数方式，当GATE为高电平时，允许计数；当GATE为低电平时，禁止计数。由于在计数达"0"时，在OUT输出一个由低到高的跳变信号，因此可用此信号作为中断请求。

2. 方式1——单拍负脉冲

方式1是一种硬件触发启动，无自动重复计数方式。工作于此方式可在输出端OUT产生单拍负脉冲，其工作时序如图8.19所示。当方式1控制字(CW)写入之后，OUT初始电平为高电平，在写入计数初值N之后并不马上开始计数，启动计数是由门控信号GATE的上跳沿触发的，一旦开始计数，OUT由高变低，直到计数达"0"再变为高。一个计数过程结束，在OUT端输出一个宽度为$N\times T_{CLK}$的单拍负脉冲。

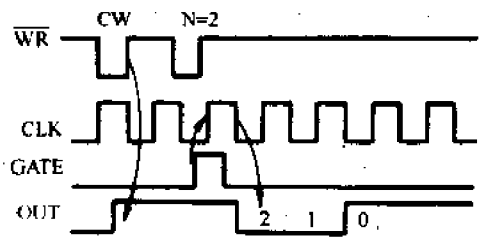


图 8.19 方式1波形

3. 方式2——频率发生器

方式2是一种具有自动加载计数常数、门控信号GATE为电平触发方式的频率发生器，工作时序如图8.20所示。当方式2控制字(CW)写入之后，OUT变为高电平，在计数初值N写入后的下一个CLK脉冲的下降沿到来且GATE为高电平时开始计数，当计数值减到"1"

时，OUT 由高变低；计数值减到“0”时，OUT 再由低变高，同时自动将计数初值送至计数执行部件，重新开始减“1”计数，形成循环计数过程。

由于方式 2 具有自动装入计数初值功能，可以实现连续不断地循环计数，因此在 OUT 端可以输出一个 N 分频的脉冲信号，输出脉冲的周期为 $N \times T_{CLK}$，其中负脉冲宽度为 1 个 T_{CLK}，正脉冲宽度为 $N-1$ 个 T_{CLK}。

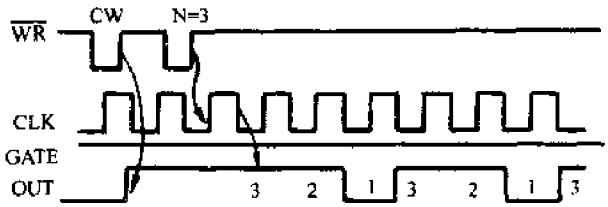


图 8.20　方式 2 波形

4. 方式 3——方波发生器

方式 3 与方式 2 类似，也是一种自动装入计数常数的 N 分频方波发生器，工作时序如图 8.21 所示。与方式 2 不同之处是：当计数初值 N 为偶数时，输出对称方波，正、负脉冲的宽度都为 N/2 个 T_{CLK}；当计数初值 N 为奇数时，输出不对称方波，正脉冲的宽度为 $(N+1)/2$ 个 T_{CLK}，负脉冲宽度为 $(N-1)/2$ 个 T_{CLK}。

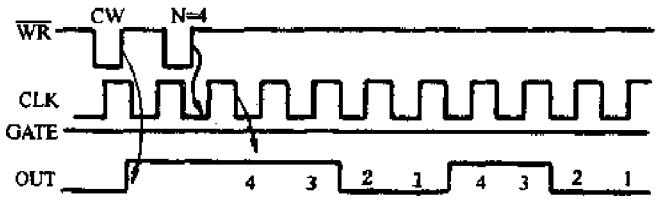


图 8.21　方式 3 波形

5. 方式 4——软件触发计数

方式 4 是一种软件触发启动，无自动重复计数功能的计数方式，工作时序如图 8.22 所示。在写入方式 4 控制字(CW)之后 OUT 变为高电平，若 GATE 信号为高电平，在写入计数初值 N 后的下一个 CLK 的下降沿开始启动减“1”计数，计数值减为“0”时，OUT 由高变低，经过 1 个 CLK 时钟后，OUT 由低变高，在 OUT 端输出一个 CLK 周期宽度的负脉冲信号。

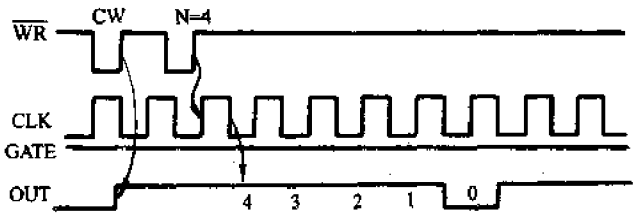


图 8.22　方式 4 波形

6. 方式 5——硬件触发计数

方式 5 为硬件触发启动、无自动重复计数功能的计数方式，工作时序如图 8.23 所示。在写入方式 5 控制字(CW)后，OUT 变为高电平，写入计数初值 N 后并不马上开始计数，只有 GATE 信号出现由低到高的跳变(硬件触发)时，才开始减“1”计数；当计数值减到“0”时，OUT 变为低电平，经过一个 CLK 时钟周期

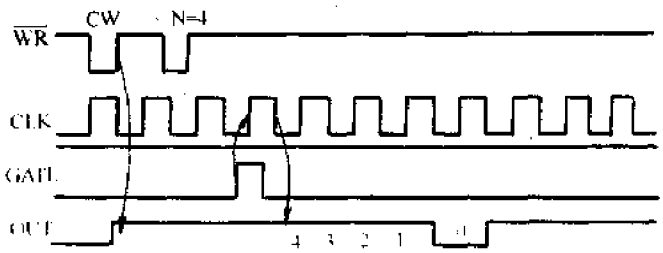


图 8.23　方式 5 波形

后，OUT 再变为高电平，一个计数过程结束，在 OUT 端输出一个宽度为一个 CLK 时钟周期的负脉冲信号。若要继续计数，须重新写入计数初值。

以上我们介绍了 8253 的六种工作方式的主要特点及工作时序波形，在不同的应用场合可选择不同的工作方式。

三、8253 的编程

(一)控制字格式

8253 的每个计数器内有一个 8 位控制寄存器，用来存放 CPU 写入的工作方式控制字，对该寄存器只能执行写入操作，不能执行读出操作。控制字的格式如图 8.24 所示。

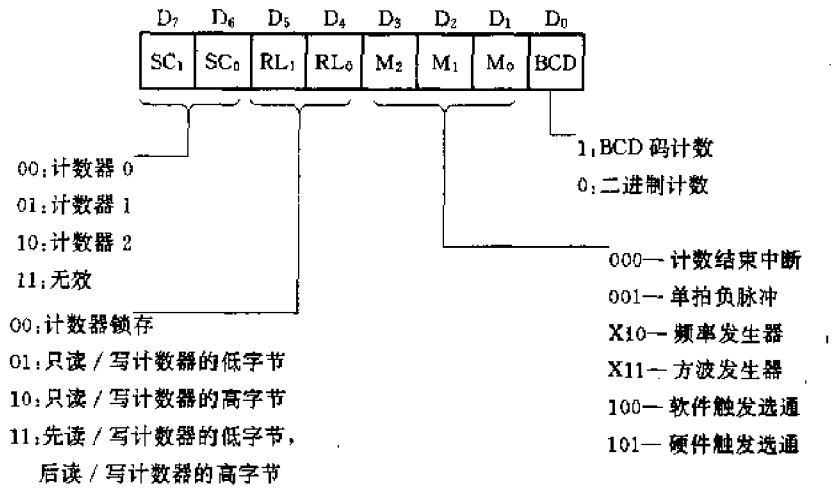


图 8.24　8253 控制字格式

其中：

BCD：计数值格式选择位。当 BCD=1 时，计数常数为二－十进制数即 BCD 码，最大计数值为 9999＋1，即为 0000，表示 10^4。当 BCD＝0 时，计数常数为二进制数，最大计数值为 FFFFH＋1，即为 0000H，表示 2^{16}(65536)。

M_2、M_1、M_0：工作方式选择位。因为 8 种编码对应六种工作方式，因此方式 2 和方式 3 的 M_2 为任意值。

RW_1、RW_0：读/写计数器控制位。计数值的写入或读出可按字节或字进行操作，RW_1、RW_0 两位编码组合用来控制写入或读出计数值的顺序与字节数。写入计数值是按照所确定的计数常数的大小和格式来选择字节写入或字写入，按字写入时，分为两步完成，即先写入低 8 位，后写入高 8 位。

若要读取当前计数值时，须先发锁存命令，后执行读出操作。当 8253 接收到锁存命令后，将当前的计数值锁定到计数锁存寄存器中，供 CPU 读取。例如读取计数器 0 的当前计数值保存在 AX 中，程序如下：

```
MOV     AL,00000000B
OUT     控制端口,AL        ;写入锁存命令
IN      AL,通道 0 端口     ;读取低字节
MOV     AH,AL
```

```
IN      AL,通道0端口    ;读取高字节
XCHG    AH,AL
```

SC_1、SC_0：计数器选择位。因为8253内部的3个计数器在结构上相互独立，在使用时须对指定的计数器进行初始化编程。

(二)8253的初始化

8253的初始化编程比较简单，只需对所使用的计数器写入工作方式控制字和计数初值。要求先写入控制字，后写入计数初值。

例如：设定计数器1工作于方式0，计数初值为1200，采用BCD码计数格式，试写出初始化程序。

根据题意，确定控制字为01110001B=71H，若设控制端口为83H，计数器1端口为81H，则初始化程序如下：

```
MOV AL,71H
OUT 83H,AL      ;写控制字
MOV AL, 00H
OUT 81H,AL      ;写计数初值低8位
MOV AL,12H
OUT 81H,AL      ;写计数初值高8位
```

在本例中，因为计数初值的低8位为00H，因此在写入计数初值时，也可以省略写入低8位的指令，而只写入高8位。请读者修改上述程序。

四、8253的应用

在IBM PC/XT系统中使用了一片8253实现定时和计数功能。通道0用作系统定时器，通道1用作系统的存储器刷新，通道2用作系统扬声器驱动信号，端口地址为40～43H。电路连接如图8.25所示。

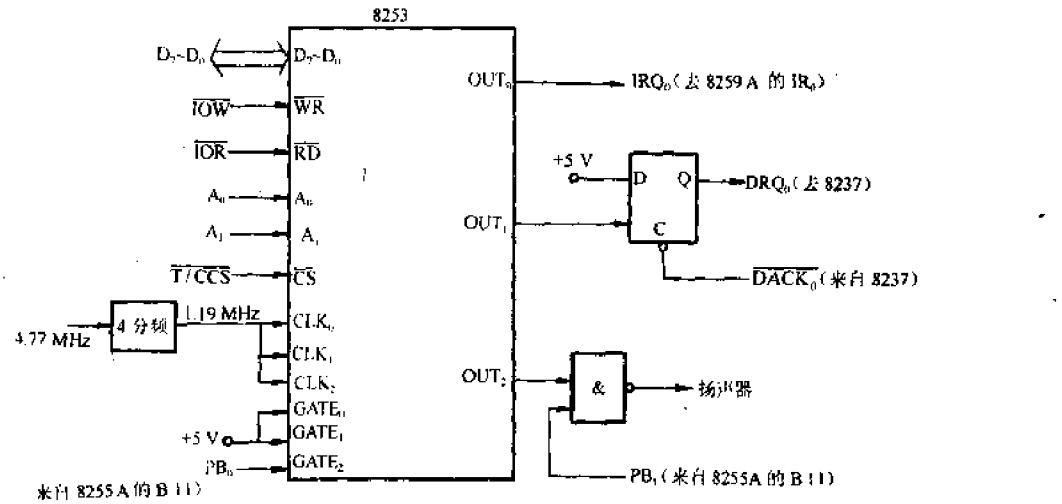


图8.25

三个通道的时钟信号CLK_0～CLK_2由系统时钟4.77MH_Z四分频之后得到1.1925MH_Z提供。

通道 0 工作于方式 3，即方波发生器，$GATE_0$ 固定为高电平，OUT_0 输出作为中断请求接至 82C59A 的 IR_0 端，每隔 55ms 秒产生一次中断请求，用于时钟报时和磁盘驱动器的马达定时。

通道 1 工作于方式 2，即频率发生器，$GATE_1$ 固定为高电平，OUT_1 输出经 D 触发器后作为对 8237ADMA 控制器通道 0 的 DMA 服务请求 DRQ，用于定时(约 15μs)启动刷新动态 RAM。

通道 2 工作于方式 3，即方波发生器，$GATE_2$ 由 8255A 芯片的 PB_0 控制，OUT_2 输出与 8255A 芯片的 PB_1 形成"与"之后去驱动扬声器鸣响。

系统中 8253 三个通道的初始化编程如下：

通道 0 用于定时(约 55ms)中断初始化：

```
MOV AL,00110110B   ;方式 3,二进制计数
OUT 43H,AL
MOV AL,0           ;计数初值为 0000H 即 65536
OUT 40H,AL
OUT 40H,AL         ;OUT 输出脉冲周期为 840ns×2^16=55ms
```

通道 1 用于刷新定时(约 15μs)DMA 请求初始化：

```
MOV AL,01010100B   ;方式 2,只装低字节,8 位计数
OUT 43H,AL
MOV AL,18          ;计数初值为 18
OUT 41H,AL         ;OUT 周期为 840ns×18=15μs
```

通道 2 用于产生 1kHz 方波驱动扬声器初始化：

```
MOV AL,10110110B    ;方式 3,二进制计数
OUT 43H,AL
MOV AX,1193         ;计数初值为 1193,即 f入/f出=1.1925MHz/1kHz
OUT 42H,AL          ;写入低字节
MOV AL,AH
OUT 42H,AL          ;写入高字节
```

第四节　串行接口 8250

一、串行通信

在计算机与外部设备相距较远时，通常采用串行通信方式。其特点是：传送的信息只占用一根传输线，信息按低位到高位的顺序依次一位一位传送。与并行通信比较，数据传输率较低，但传输线成本低。

由于串行通信只占用一根传输线，在串行通信方式中须解决以下几个问题：

(1)数据传送的方式

(2)数据传送的格式

(3)数据传送速率与距离

(4)信息的检错与纠错

(5)串行接口标准

下面就以上几个问题作简要介绍。

(一)数据传送的方式

在串行通信中,传送的信息是在两站之间进行的,按照线路的传送方向可分为以下三种传送方式。

1.单工传送

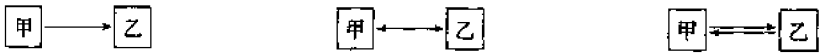
只允许数据按一个固定的方向传送。如图 8.26(a)所示。一方只能发送,另一方只能接收。

2.半双工传送

允许通信双方既可以发送,也可以接收,发/收数据在同一传输线上进行,如图 8.26(b)所示。虽然允许双方具有发送和接收功能,但不能同时进行,在同一时刻数据只能沿着一个方向传送。

3.全双工传送

通信双方可以同时发送和接收,发/收数据在两根传输线上进行,如图 8.26(c)所示。



(a)单工　　(b)半双工　　(c)全双工

图 8.26　串行数据传送方式

(二)数据传送的格式

串行通信分为同步通信与异步通信两种类型。

1.同步通信

同步通信的数据格式分为面向字符型和面向比特型两种。

面向字符型的数据格式有单同步、双同步和外同步三种,如图 8.27(a)、(b)、(c)所示。在单同步和双同步数据格式中,以一个或两个同步字符作为数据块的起始标志,以两个循环冗余校验码 CRC_1 和 CRC_2 作为数据块的结束标志,从开始标志到结束标志构成一个完整的信息单位,称为一帧信息。在传送过程中,收发端以同步字符实现同步操作。外同步方式是由一根专用的控制线来传送同步信号。

比特型的数据格式,是采用二进制位组合作为起始和结束标志。目前比特型的控制规程有同步数据链路控制规程 SDLC 和高级数据链路控制规程 HDLC。如图 8.27(d)、(e)所示。SDLC/HDLC 规程规定,一帧信息以同一个二进制标志字符 01111110 作为开始和结束标志,收发端以标志字符建立帧同步。

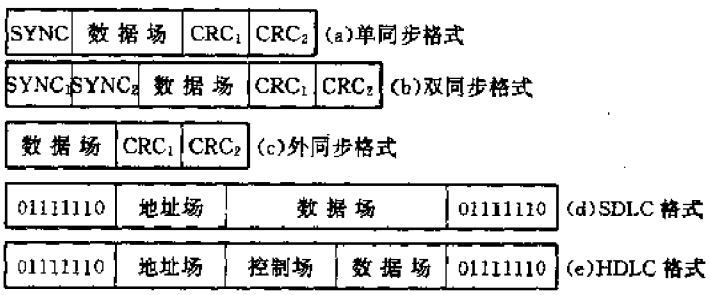


图 8.27　同步通信数据格式

2.异步通信

异步通信的数据格式如图 8.28 所示,传送的信息以帧为单位,一帧信息由 1 位起始位、5

～8 位数据位(为字符编码)、1 位奇偶校验位(可有可无)和 1 位或 1.5 位或 2 位停止位构成，在线路处于空闲状态时传送高电平。

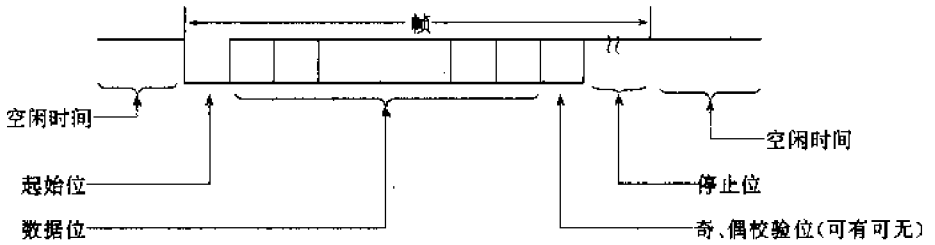


图 8.28　异步通信数据格式

在这种通信方式中，发送或接收端在发送或接收时钟的控制下发送或接收数据。接收端一旦在传输线上检测到一个低电平信号时，就认为是一帧信息的开始位，便开始装配一字节数据至 CPU。

从一个字符的结束到下一个字符的开始没有规定固定的间隔长度，即空闲时间是随机的，可长可短，但也不能太长，如果空闲时间超过一定范围，接收端就认为出现了超时错。

(三)数据传送的速率

数据传送的速率称为传输率，在串行通行中通常用比特率来表示。所谓比特率是指每秒钟传送二进制数的位数，即单位用 bit/s 表示。目前国际上规定一个标准的波特率系列：110，300，600，1200，1800，2400，4800，9600 和 19200 波特等。

(四)串行通信接口标准 RS－232C

在串行通信中，数据终端设备(DTE)中的数据要通过数据通信设备(DCE)传送。数据终端可以是计算机或 CRT 显示器；数据通信设备常指调制器和解调器，二者制作在一块板上称为 MODEM。

数据终端设备与数据通信设备之间常通过专门的标准信号连接，目前使用的标准比较多，微机中常用 RS－232C 标准。

RS－232C 标准设有 25 条信号线，其外型使用 25 针 D 型转接插座，如图 8.29 所示。在微机通信中最常用的信号有 9 个，如表 8.4 所示。这些信号分为基本的数据传输信号和用于 MODEM 的控制信号。

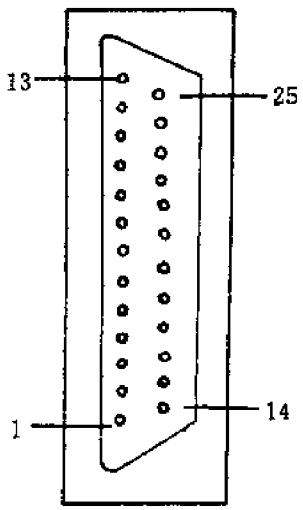


图 8.29

表 8.4

引脚	信号	方向	功能
2	TXD	出	发送数据
3	RXD	入	接收数据
4	RTS	出	请求传送
5	CTS	入	允许传送
6	DSR	入	数据装置就绪
7	GND		信号地
8	DCD	入	数据载波检测
20	DTR	出	数据终端就绪
22	RI	入	振铃指示

1. 基本数据传输信号

TXD：数据发送信号，用于数据输出。不传送时该引脚为高电平。

RXD：数据接收信号，用于数据输入。不传送时该引脚为高电平。

GND：信号地。

在近距离通信不需要 MODEM 的情况下，通信双方只需要将 TXD 与 RXD 对接，GND 直接连接即可，如图 8.30 所示。

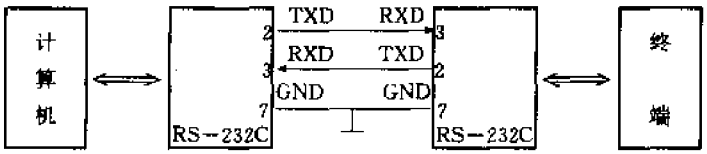


图 8.30　串行通信

2. MODEM 控制信号

DTR：数据终端就绪信号，输出。用来通知 MODEM，计算机已准备好，要求与终端设备进行通信。

DSR：数据装置就绪信号，输入。用来通知计算机，MODEM 已准备好。

RTS：请求发送，输出。用来通知 MODEM，计算机请求发送数据。

CTS：允许发送，输入。用来通知计算机，MODEM 可以接收数据。

DCD：线路信号检测，输入。用来通知计算机，通信线路连接好。

RI：振铃指示，输入。用来通知计算机，有来自通信线路的数据或振铃信号。

由于 RS－232C 的标准是按负逻辑定义的，即－15V～－5V 表示“1”，＋5V～＋15V 表示“0”，而计算机中使用的是 TTL 电平信号，因此在 RS－232C 与 TTL 电平电路连接时，需进行电平转换。转换电路如图 8.31 所示，其中 MC1488 将 TTL 电平转换为 RS－232C 电平，MC1489 将 RS－232C 电平转换为 TTL 电平。

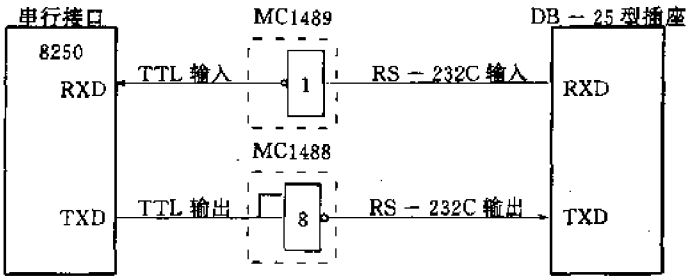


图 8.31　RS－232C 电平转换电路图

近距离通信只适合于约 50 米以内的通信，当通信双方距离较远时，需要使用 MODEM，RS－232C 与 MODEM 连接如图 8.32 所示。

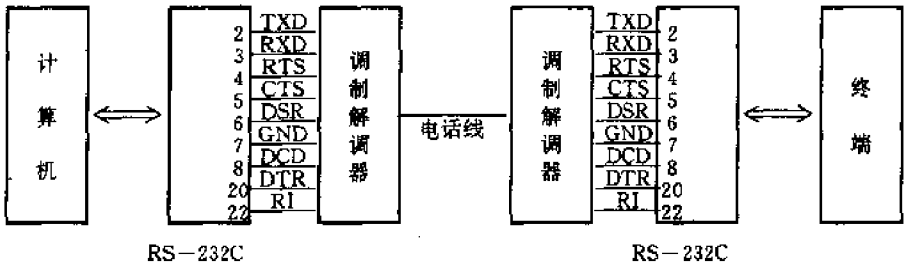


图8.32　带 MODEM 的串行通信

二、串行接口8250

INS8250是由国家半导体公司(National Semicoductor INC)研制的专用于支持异步通信的串行接口芯片。其特点是编程能力很强，主要体现在：

(1)传输速率可在50～19200bps 范围内选择；

(2)传送的数据格式可选择5、6、7或8位数据位，1位起始位(低电平)，1、1.5或2位停止位(高电平)和带奇校验、偶校验或无校验；

(3)具有控制 MODEM 功能和完整的状态报告功能。

(一)8250的结构与引脚信号

8250接口芯片采用单一+5V 电源供电，40引脚 DIP 封装，内部结构和引脚信号如图8.33和图8.34所示。除与系统总线连接的数据总线缓冲和读/写控制逻辑外，还可分为五个功能模块，每个模块内供 CPU 访问的寄存器有两个，总共为10个寄存器。

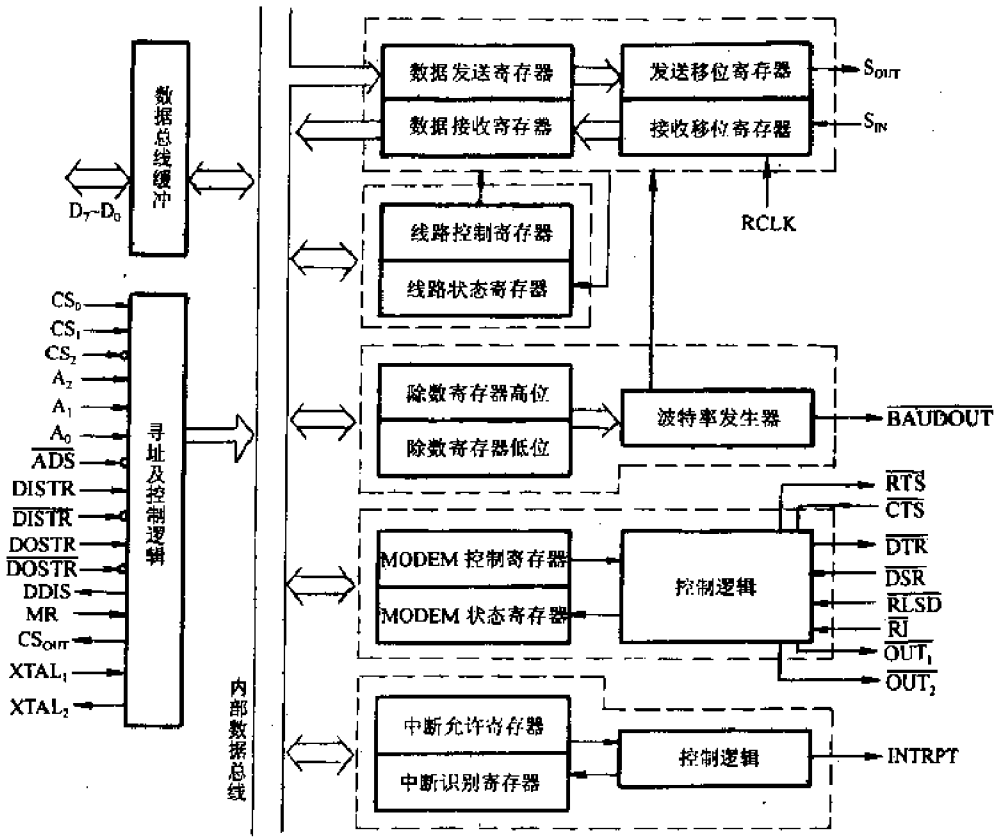


图8.33 INS 8250的结构框图

各模块的功能和引脚信号说明如下：

1. 数据总线缓冲器

数据总线缓冲器为三态、双向8位寄存器。数据线 D_7～D_0与 CPU 的数据线连接，构成 CPU 与8250之间信息传送的通道。

2. 寻址及控制逻辑

寻址及控制逻辑用来接收来自系统总线的读/写控制信号、寄存器选择信号和时钟信号等。用来控制8250内部的读、写操作。与寻址及控制逻辑有关的引脚信号有：

CS_0、CS_1、$\overline{CS_2}$为片选输入信号。当 $CS_0=1$，$CS_1=1$ $\overline{CS_2}=0$时，8250被选中。

CS_{OUT}为片选输出信号。当三个片选输入信号 CS_0、CS_1、$\overline{CS_2}$同时有效时，$CS_{OUT}=1$，作为选中8250芯片的指示信号；当芯片未选中时，$CS_{OUT}=0$，禁止数据传送。

A_2、A_1、A_0为寄存器选择信号。用来确定片内10个可访问寄存器的地址。

$\overline{ADS}$为地址选通输入信号。当$\overline{ADS}=0$时，片选信号 CS_2～CS_0和地址信号 A_2～A_0输入锁存。

DISTR/$\overline{DISTR}$为读控制信号。两个信号作用相同，极性相反。在片选信号有效时，当 DISTR＝1或者$\overline{DISTR}=0$时，CPU 对芯片进行读出操作。

DOSTR/$\overline{DOSTR}$为写控制信号。两个信号作用相同，极性相反。在片选信号有效时，当 DOSTR＝1或者$\overline{DOSTR}=0$时，CPU 对芯片进行写入操作。

DDIS 为禁止数据传送输出信号，高电平有效。当 DDIS＝1时，禁止对8250数据传送。

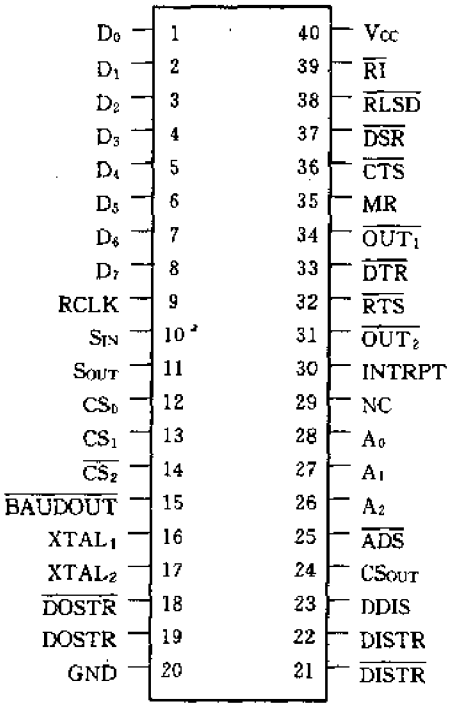


图8.34　INS 8250引脚图

MR 为复位信号，高电平有效。当 MR＝1时，芯片内部寄存器及控制逻辑复位初始化，其复位状态见表8.5。

$XTAL_1$、$XTAL_2$为时钟信号输入/输出端。当使用芯片内部时钟电路时，在 $XTAL_1$与 $XTAL_2$之间外接一个石英晶振和微调电容；当使用外部时钟电路时，$XTAL_2$端接外部振荡源，$XTAL_1$端接地。

表8.5　INS 8250内部寄存器的复位

寄存器或信号		复位控制	复位结果
中断允许寄存器		MR	D_7～D_0全为零
中断识别寄存器		MR	D_0＝1，其余位全为零
线路控制寄存器		MR	全为零
线路状态寄存器		MR	D_5＝D_6＝0，其余位全为1
MODEM 控制寄存器		MR	全为零
MODEM 状态寄存器		MR	D_3～D_0为零，其余取决于输入
中断识别寄存器的 D_2～D_0三位的状态	1 1 0	MR 或读线路状态寄存器	D_0＝1，其余位全为零
	1 0 0	MR 或读接收寄存器	
	0 1 0	MR 或写发送寄存器或读中断识别寄存器	
	0 0 0	MR 或读 MODEM 状态寄存器	
信号 S_{OUT}、$\overline{OUT_1}$、$\overline{OUT_2}$、$\overline{RTS}$、$\overline{DTR}$		MR	全为1

3. 数据发送和接收

数据发送是由数据发送寄存器和发送移位寄存器以及发送控制电路完成的。当 CPU 发送数据时，首先检查数据发送寄存器是否为空，若为空时，先将发送的数据（以字节为单位）并行输出到数据发送寄存器中，然后在发送时钟信号的控制下送入发送移位寄存器中，由发送移

位寄存器将并行数据转换为串行数据经发送端 S_{OUT} 输出。在输出过程中由发送控制电路按照约定的通信格式自动插入起始位、奇偶校验位和停止位，装配成完整的一帧发送信息。

数据接收是由接收移位寄存器、数据接收缓冲器和接收控制电路完成的。在接收串行输入的数据时，在接收时钟信号的控制下，首先搜寻起始位(即低电平)，然后将接收端 S_{IN} 输入的数据逐位送入接收移位寄存器中，当接收到停止位后，将接收移位寄存器中的数据送入接收数据缓冲器中，供 CPU 读取。

S_{OUT}：串行数据输出端。

S_{IN}：串行数据输入端。

RCLK：输入数据检测时钟脉冲，用于检测 S_{IN} 端输入的数据，其频率为接收波特率的16倍。使用时可与$\overline{BAUDOUT}$端连接。

表8.6

SPB	EPS	PEN	功能
0	0	0	无校验
0	0	1	奇校验
0	1	1	偶校验
1	0	1	附加位为1
1	1	1	附加位为0

4. 线路控制和状态寄存器

线路是指串行输入输出线。线路控制寄存器主要用于指定串行异步通信的数据格式；线路状态寄存器为 CPU 提供8250内部的状态信息。

(1)线路控制寄存器

线路控制寄存器格式如图8.35所示。

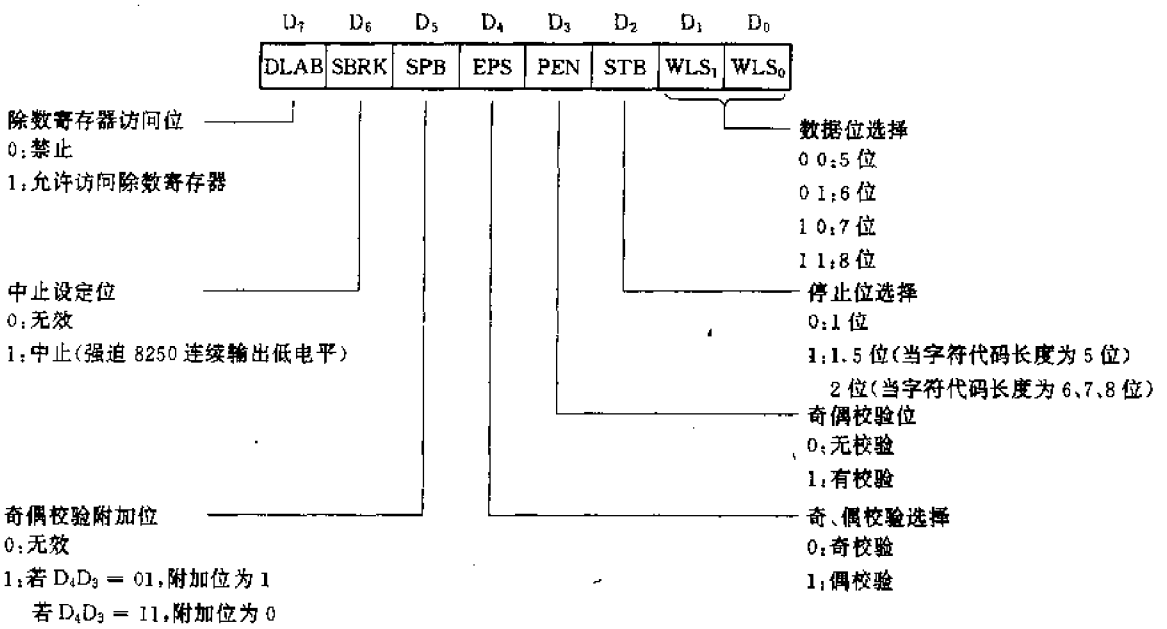


图8.35 线路控制寄存器格式

其中：

DLAB 为访问除数寄存器标志位。当 DLAB=1时，执行 I/O 指令访问的是波特率控制部分的除数寄存器；当 DLAB=0时，为正常寻址。寻址情况见表8.7。

SBRK 为中止设定位。当正常发送、接收时，SBRK=0；当 SBRK=1时，中止正常发送，使输出端 S_{OUT} 保持低电平。

SPB 为奇偶校验附加位。由该位和 EPS 与 PEN 三位共同规定通信数据的奇偶校验规则(见表8.6)。SPB 的作用可以把发送方校验的奇偶性规定告诉接收方。即当 SPB=1时，在发送

数据的奇偶校验位和停止位之间附加一个标志位：若采用偶校验，则附加位为“0”；若采用奇校验，则附加位为“1”。接收方收到数据后，只要将附加位分离出来，便可知道发送方采用的是奇校验，还是偶校验。

STB 为停止位选择位。当 STB＝0时，则只有一位停止位；当 STB＝1时，若数据位长度为5位，则有一位半停止位；若数据位长度为6、7或8位，则停止位为两位。

WLS_1和 WLS_0为数据位长度选择位。

(2)线路状态寄存器

线路状态寄存器格式如图8.36所示。

以上各位在该位状态存在时置“1”，CPU 读取线路状态寄存器，便可了解到数据发送和接收的情况。

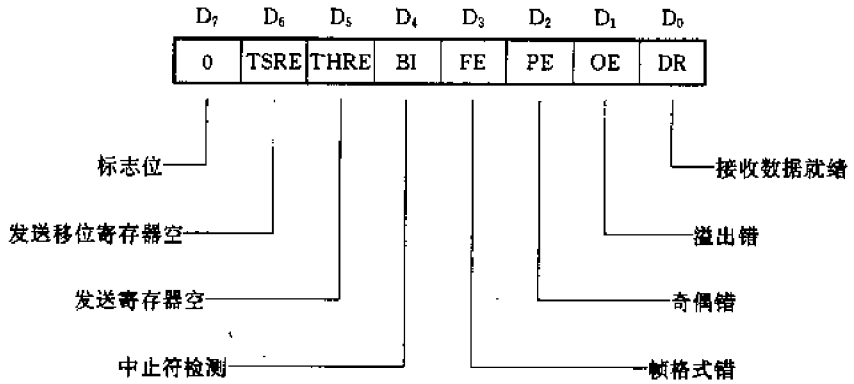


图8.36 线路状态寄存器

5. 波特率控制

串行数据传送速率的控制是由8250内部的波特率发生器完成的。它是一个由软件控制的分频器，输出的工作时钟频率为16倍的波特率。在基准时钟频率确定后，通过改变除数寄存器的值来选择所需的波特率。即：

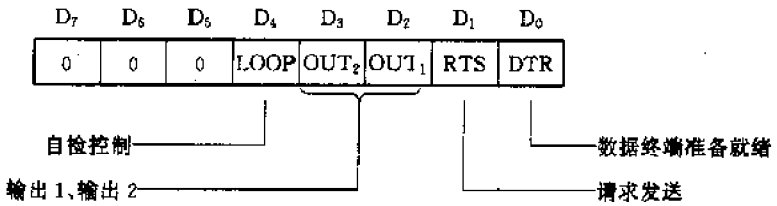
除数＝基准时钟频率/(波特率×16)

$\overline{BAUDOUT}$：波特率输出信号，其频率为发送波特率的16倍。通常与 RCLK 端连接。

6. MODEM 控制和状态寄存器

数据在远距离通信时，为避免传送的波形发生畸变，通常在发送与接收端加调制解调器(即 MODEM)。此模块用于与通讯设备的联络功能。MODEM 控制寄存器控制芯片的4个引脚信号的输出和芯片的环路检测。MODEM 状态寄存器检测芯片的4个引脚的输入状态。

MODEM 控制寄存器格式如图8.37所示。



8.37 MODEM 控制寄存器格式

LOOP 为自检控制信号，决定8250的工作方式。当 LOOP＝0时，8250正常工作，当 LOOP

=1时，8250处于自检状态。D_3～D_0的信号功能与相应引脚信号功能相同，而极性相反。

MODEM 状态寄存器格式如图8.38所示。

其中高4位为外部输入信号的状态，低4位记录高4位的变化。即当高4位中某一位的状态发生改变(由"0"变到"1"或由"1"变到"0")时，则低4位中的相应位置"1"。这些状态位的变化，除了允许 CPU 执行输入指令进行查询外，也可以引起中断。

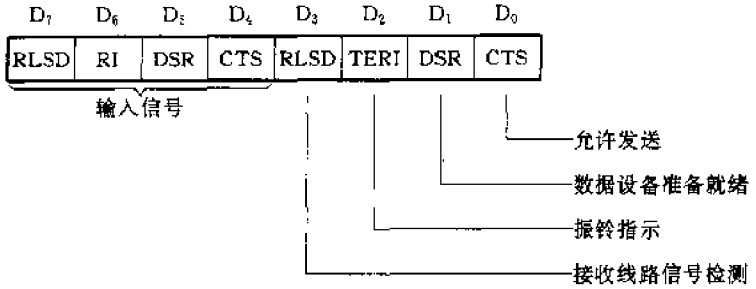


图8.38 MODEM 状态寄存器格式

与 MODEM 控制逻辑有关的引脚信号有：

$\overline{RTS}$和$\overline{CTS}$为请求发送(输出)和允许发送(输入)，是一对低电平有效的握手信号。当8250准备好发送时，将请求发送信号$\overline{RTS}$输出至对方的数据设备，对方收到该信号后，若同意发送，则向8250回送一个允许发送信号$\overline{CTS}$作为对$\overline{RTS}$的回答，于是握手成功，可以开始传送。

$\overline{DTR}$和$\overline{DSR}$为数据终端准备好(输出)和数据设备准备好(输入)，也是一对低电平有效的握手信号。其工作过程与上述两个信号类似。当$\overline{DTR}$有效时，指示8250已作好通信准备，当数据设备准备好，向8250回送一个准备就绪信号$\overline{DSR}$，以示握手成功。

$\overline{RLSD}$为接收线路信号检测输入，低电平有效。表示通信设备已检测到数据串。

$\overline{RI}$为振铃信号，输入，低电平有效。表示通信设备已收到振铃信号，即电话交换台的拨号呼叫。

OUT_1和 OUT_2为芯片内 MODEM 控制寄存器的 D_2D_3两位的输出信号，可以通过编程指定输出引脚，作为对外部的控制信号。例如在 PC/XT 系统中，指定 OUT_2作为控制8250的中断请求 INTRP 信号，OUT_1未用。

7. 中断控制

8250芯片具有很强的中断功能。共有4级中断，按其优先级别从高到低的顺序为：接收出错中断；接收缓冲器"满"中断，发送保持器"空"中断；MODEM 输入状态改变中断。

中断控制逻辑主要由中断允许寄存器、中断识别寄存器和中断控制逻辑电路组成。

(1)中断允许寄存器

中断允许寄存器是可以进行读、写操作的寄存器，其作用是对四类中断源实现允许或禁止控制，格式如图8.39所示。

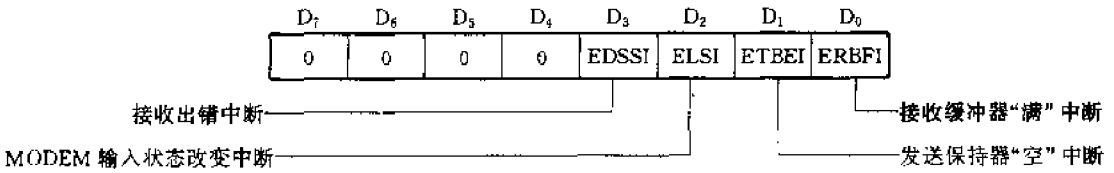


图8.39 中断允许寄存器格式

D_0~D_2位决定线路状态寄存器引起的中断是否允许。若为"1"，允许中断；若为"0"，禁止中断。D_3位决定由 MODEM 状态寄存器的高四位状态发生改变时引起的中断是否允许。若 D_3 为"1"，则中断允许；若 D_3为"0"，则中断禁止。

需要指出的是：中断允许寄存器的相应位为"1"，只允许中断源产生一个高电平有效的中断请求信号 INTRPT，在 PC/XT 系统中，INTRPT 输出后还需要经过$\overline{OUT_2}$信号的控制（$\overline{OUT_2}=0$），才能最终对系统形成中断请求 IRQ 信号。

(2)中断识别寄存器

中断识别寄存器的作用是：当8250处于中断工作方式时，该寄存器指出有无待处理的中断发生及其类型，并且封锁比此类型优先权级别低的所有类型中断，当现行中断请求被 CPU 响应服务后，才释放被封锁的中断。中断识别寄存器的格式如图8.40所示。

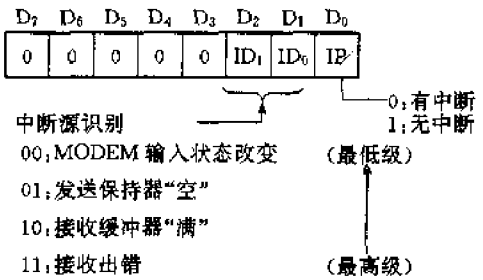


图8.40　中断识别寄存器格式

D_0~D_2记录中断发生的情况，只允许读出。

以上我们按照8250内部的功能结构框图分别介绍了各功能模块的功能及其寄存器的格式，表8.7给出了芯片内部10个寄存器的端口分配与读/写方式。

表8.7　8250内部寄存器寻址与读/写方式

A_2	A_1	A_0	DLAB 标志	读/写	寄存器
0	0	0	0	写	写发送保持寄存器
0	0	0	0	读	读接收缓冲寄存器
0	0	0	1	写	写除数寄存器低字节
0	0	1	1	写	写除数寄存器高字节
0	0	1	0	写	写中断允许寄存器
0	1	0	×	读	读中断识别寄存器
0	1	1	×	写	写线路控制寄存器
1	0	0	×	写	写 MODEM 控制寄存器
1	0	1	×	读	读线路状态寄存器
1	1	0	×	读	读 MODEM 状态寄存器

注：DLAB 为线路控制寄存器的 D_7位，为允许访问除数寄存器标志位。

(二)8250的编程

Ins8250的编程分为初始化和工作两部分，初始化编程主要用来设置8250的工作方式、数据的传送格式等。需要编程的寄存器有：除数寄存器、线路控制寄存器、MODEM 控制寄存器和中断允许寄存器。由于写入除数寄存器时要求线路控制寄存器的 D_7(DLAB)＝1，建立访问

除数寄存器标志，因此在写除数寄存器之前先将线路控制寄存器中的 D_7 位置“1”。8250的初始化流程图如图8.41所示。

例如 PC/XT 异步通信适配器8250芯片的输入的基准频率为1.8432MHz，端口地址为3F8H～3FEH 编写初始化程序，要求：波特率为1200；数据长度7位，1位停止位，偶校验；屏蔽全部中断。初始化程序如下：

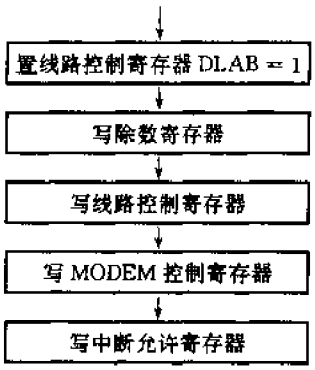


图8.41　8250初始化流程图

```
MOV DX,3FBH
MOV AL,80H
OUT DX,AL     ;置线路控制寄存器 DLAB 为“1”
MOV DX,3F8H
MOV AL,60H    ;除数=1.8432MHz/(16×1200)
OUT DX,AL     ;写除数寄存器低8位
INC DX
MOV AL,0
OUT DX,AL     ;写除数寄存器高8位
MOV DX,3FBH
MOV AL,1AH
OUT DX,AL     ;写线路控制寄存器
MOV DX,3FCH
MOV AL,03H    ;OUT2=1,DTR=0,RTS=0
OUT DX,AL     ;写 MODEM 控制寄存器
MOV DX,3F9H
MOV AL,0
OUT DX,AL     ;全部中断被屏蔽
```

第五节　DMA 控制器8237A

DMA(Direct Memory Accecc)是指在外部设备与存储器之间直接进行数据传送的一种I/O 控制方式。DMA 控制器可以获得总线控制权，由它控制高速 I/O 设备(如磁盘)和存储器之间直接传送数据。

典型的 DMA 控制器有8237和8257等芯片。在 PC/XT 系统中使用了一片8237A－5 DMA 控制器，提供了四个 DMA 通道供系统使用。在 PC/AT 系统中使用了两片8237A－5，提供了七个 DMA 通道。PC 386/486微机中所使用的 DMA 与 PC/AT 完全兼容，因此本节以8237A DMA 控制器为例，介绍 DMA 数据传送控制方式的工作原理。

一、8237A 的结构与引脚信号

8237A 是一个高性能可编程 DMA 控制器。它内部有四个独立的 DMA 通道，每一个通道有64K 字节的寻址和字计数能力；有单字节传送、数据块传送、请求传送和级联传送四种方式；数据传送速率最高可达1.5MB/s。

8237A 是一个具有40个引脚信号的双列直插式接口芯片，其内部结构如图8.42所示。

(一)8237A 的内部结构

8237A 内部有四个独立的 DMA 通道，每个通道包含四个16位的寄存器。它们是基地址寄存器和当前地址寄存器，基字节计数器和当前字节计数器，还包含一个6位的方式寄存器、1位请求触发器和1位屏蔽触发器。四个通道共用一个命令寄存器和一个状态寄存器。

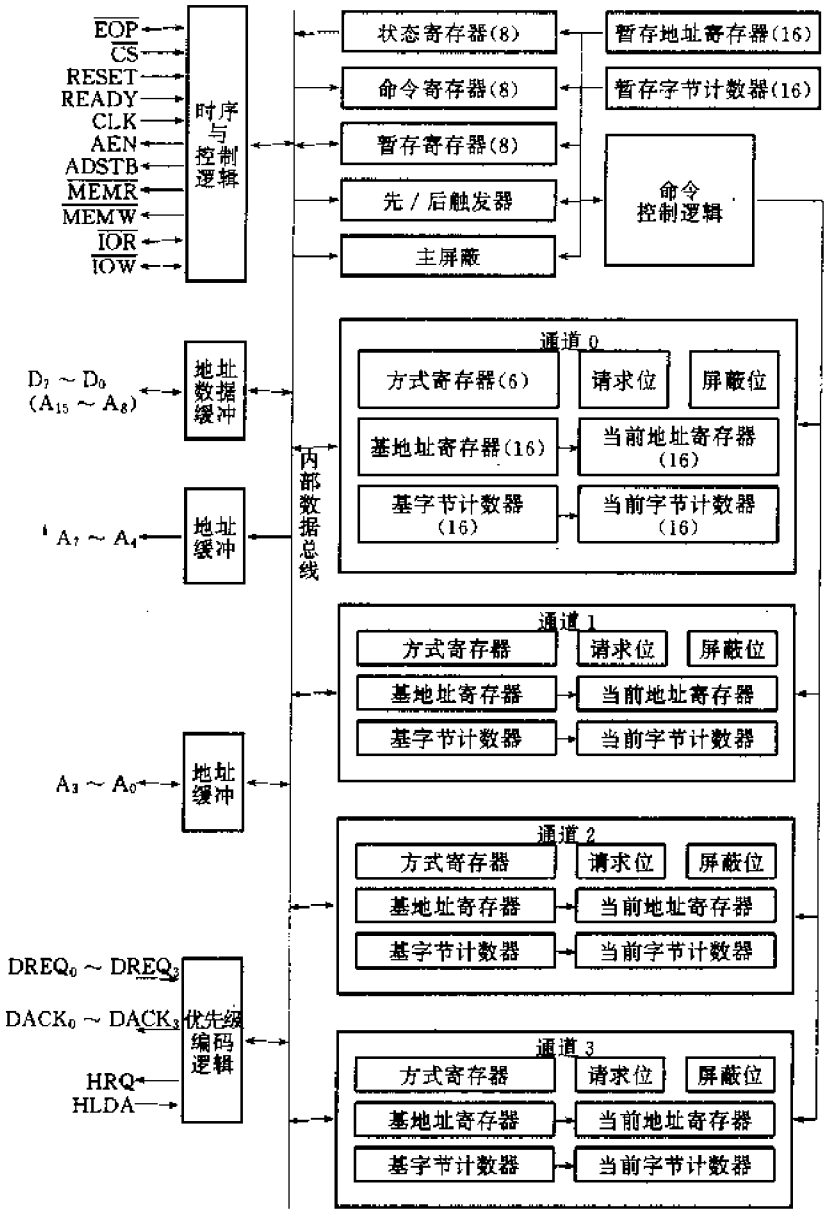


图8.42　8237A 内部结构

基地址寄存器用来存放本通道 DMA 传送时的地址初值，在初始化编程时写入。当前地址寄存器用来存放当前正在传送的地址值。它的初始状态为地址初值，在每执行一次 DMA 传送后自动加1或减1。

基字节计数器用来存放 DMA 传送数据块的字节数的初值，初始化编程时写入。当前字节计数器的初值为传送的字节数，每传送一个字节，当前字节计数器内容减1，当减至"0"时，产生 DMA 传送结束信号$\overline{EOP}$，至此一个 DMA 传送过程结束。

(二)8237A 的引脚信号

8237A 的引脚信号如图8.43所示。

各引脚信号说明如下：

CLK：时钟输入。

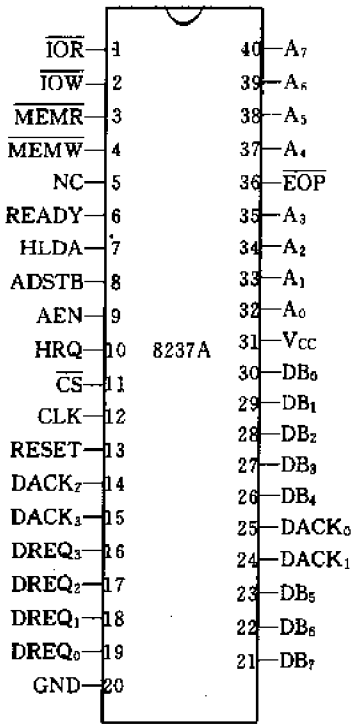


图8.43　8237A 引脚图

$\overline{CS}$：片选输入，低电平有效。

RESET：复位输入，高电平有效。芯片复位后，屏蔽寄存器被置“1”，其它寄存器被清“0”。

READY：准备就绪输入，高电平有效。当 READY＝1时，表示外部设备已准备好读/写；当 READY＝0时，表示外设未准备好，需在总线周期中插入等待周期 S_w。

AEN：DMA 地址允许输出，高电平有效。当 AEN 有效时，允许 DMA 控制器送出地址信号，而禁止 CPU 送出地址信号。

ADSTB：地址选通输出，高电平有效。当 ADSTB 有效时，地址信号 A_{15}～A_8被锁存到地址锁存器中。

$\overline{MEMR}$：DMA 存储器读，输出，低电平有效。

$\overline{MEMW}$：DMA 存储器写，输出，低电平有效。

$\overline{IOR}$：I/O 读，双向，低电平有效。当8237A 作为从设备时，$\overline{IOR}$为输入信号，CPU 读8237A 内部寄存器；当8237A 作为主设备时，$\overline{IOR}$为输出信号，8237A 读外部设备的数据送入存储器中。

$\overline{IOW}$：I/O 写，双向，低电平有效。输入时 CPU 写8237A 内部寄存器；输出时8237A 将存储器的数据写入外部设备。

$\overline{EOP}$：DMA 传送结束，双向，低电平有效。当$\overline{EOP}$有效时，DMA 传送停止，并且复位内部

寄存器。

$DREQ_3$～$DREQ_0$：四个通道的 DMA 请求信号，由外设输入，其有效极性和优先级可以通过编程设定。

$DACK_3$～$DACK_0$：四个通道的 DMA 应答信号，输出至外设，其有效极性由编程设定，作为对 DREQ 请求的回答。

HRQ：总线请求，输出，高电平有效。与 CPU 的总线请求信号引脚 HOLD 连接，以示请求总线控制权。8237A 接收了任何一个通道的 DREQ 请求之后，使 HRQ 变为有效。

HLDA：总线响应，输入，高电平有效。与 CPU 的总线响应信号引脚 HLDA 连接，当 HLDA 有效后，8237A 获得总线的控制权。

A_3～A_0：双向地址线。CPU 访问8237A 时为输入，寻址内部寄存器；DMA 控制数据传送时为输出地址信号的低4位。

A_7～A_4：地址线。DMA 控制数据传送时为输出地址信号的高4位。

DB_7～DB_0：双向数据/地址线。当8237A 作为从设备时，CPU 由 DB_7～DB_0读、写芯片内部的寄存器；当8237A 作为主设备时，由 DB_7～DB_0输出高8位地址信号 A_{15}～A_8，由 ADSTB 信号选通锁存。

二、8237A 的工作方式

8237A 有主控和从属两种工作状态，当它没有获得总线控制权时，作为从设备由 CPU 控制（如初始化编程）；一旦获得总线控制权，由从态变为主态进行 DMA 控制传送，数据传送完毕，将总线控制权退还给 CPU，又由主态变为从态。

在8237A 获得总线控制权后，作为主控设备执行 DMA 传送时，有以下四种数据传送方式。

1. 单字节传送方式

这种传送方式是每次 DMA 传送请求只传送一个字节的数据。数据传送后当前字节计数器自动减1，当前地址寄存器作加1或减1修改，撤消 HRQ 信号，释放系统总线控制权。若传送使当前字节计数器减为“0”或由外部产生$\overline{EOP}$信号时，终止 DMA 过程。

2. 块传送方式

这种传送方式是一旦8237A 获得总线控制权，连续地传送数据（即一个数据块），直到当前字节计数器减为“0”或由外部生产$\overline{EOP}$信号时，终止 DMA 传送，释放总线。在块传送时，HRQ 信号一直保持有效。

3. 请求传送方式

这种传送方式也是一种连续传送数据方式。当在出现以下三种情况之一时就会终止 DMA 传送。

(1)当前字节计数器减至“0”；

(2)由外部产生一个有效的$\overline{EOP}$信号；

(3)外设的 DREQ 请求信号变为无效。

与块传送方式相比，只增加了第3个功能：一般情况下，请求信号 DREQ 保持到响应信号 DACK 有效后即可撤消，而数据块的传送仍会继续下去。但在请求传送方式下，8237A 每传送一个字节就要检查一次 DREQ 信号是否有效，若有效则继续传送下一个字节；若无效则停止传送，结束 DMA 过程。但 DMA 的传送现场全部保留，待请求信号 DREQ 再次有效时，8237A

接着原来的计数和地址值继续进行传送，直到计数值减为“0”或外部输入$\overline{EOP}$信号有效，才终止 DMA 传送，释放总线。

4. 级联方式

当一片8237A 通道不够使用时，可通过多片级联的方式增加 DMA 通道。级联方式如图8.44所示。

由主、从两级构成，从片8237A 的 HRQ 和 HLDA 信号和主片8237A 的 DREQ 和 DACK 信号连接，一个主片至多可连接4个从片。在这种工作方式下，从片作 DMA 传送，而主片除了在从片与 CPU 之间传递联络信号外，还对从片的优先级进行管理。

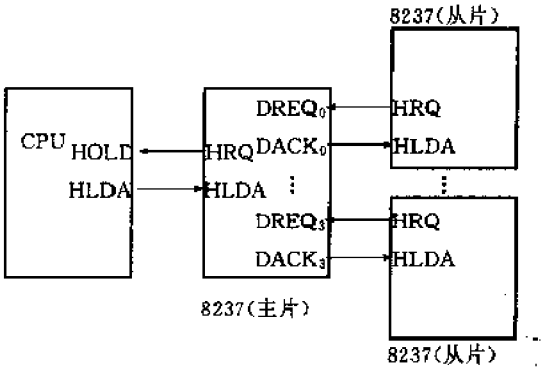


图8.44　8237A 级联方式

三、8237A 的工作时序

8237A 的工作时序如图8.45所示。

为了区别于 CPU 的时钟周期，DMA 的每一个时钟周期称为一个 S 状态。

(1)S_1状态是空闲状态。在进入 DMA 传输之前，8237A 一直处在连续的 S_1状态。这时8237A 作为从设备，可以接受 CPU 的编程写入或读出。在 S_1状态中8237A 要不断地采样各 DREQ 信号。若有DREQ 信号(一个或多个)产生，且经过屏蔽逻辑及优先级排队后仍有效，在S1的上升沿产生 HRQ 信号，向 CPU 发出总线请求，同时结束 S_1状态，进入 S_0状态。

(2)S_0状态中8237A 等待 CPU 的总线响应信号 HLDA，在 HLDA 信号有效之前，8237A 一直重复 S_0状态。S_0状态中的8237A 还是从属状态，可以接收 CPU 的读写。在 S_0的上升沿检测到 HLDA 信号有效，则下一状态进入 S_1。真正的 DMA 传送是从 S_1状态开始。

(3)S_1状态首先产生 AEN 信号，使 CPU 等其它总线器件的地址线和系统总线的地址线断开，而使8237A 的地址线 A_{15}～A_0接通。AEN 信号一旦产生后在整个 DMA 过程中一直有效。S_1状态还有一个作用是产生 DMA 地址选通信号 ADSTB，将 DB_7～DB_0上送出的地址信号 A_{15}～A_8用 ADSTB 的下降沿锁存到外部锁存器中。考虑到在块传送方式中，相邻字节的高位地址往往是相同的。最极端的情况下，连续传送256字节，地址 A_{15}～A_8才变化一次。因此不必每次都用 ADSTB 信号将一个不变的 A_{15}～A_8锁存一次。这种情况下，连续下去的 DMA 时序中省去 S_1状态，不产生 ADSTB 信号，直接从 S_2状态开始，如图8.45中后一部分所示。

(4)S_2状态中8237A 产生 DMA 响应信号$\overline{DACK}$给外部设备。得到响应的外部设备，可用$\overline{DACK}$信号代替 CPU 控制总线时的片选信号，使自己在整个 DMA 期间都处于选中状态，同

时地址总线上出现所要访问的存储器地址 A_{15}～A_0。

(5)S_3状态产生$\overline{MEMR}$或$\overline{IOR}$读信号，于是数据线 DB_7～DB_0上出现被传送的字节。正常的读信号从 S_3状态一直持续到 S_4状态，以便使 DB_7～DB_0上的数据稳定至 S_4状态写入目的处。另一种压缩读方式取消 S_3状态，读信号和写信号同时在 S_4状态时产生，适用于高速电路。相反若 DMA 传送数据的源或目的电路速度较慢，不能在 S_4状态前使读出数据稳定，则可以将8237的准备好 Ready 端变低，使 S_3和 S_4之间插入等待状态 S_W 直到准备好后，Ready 变高才结束 S_W 进入 S_4状态。在 PC/XT 机中除通道0以外都固定地插入一个 S_W。

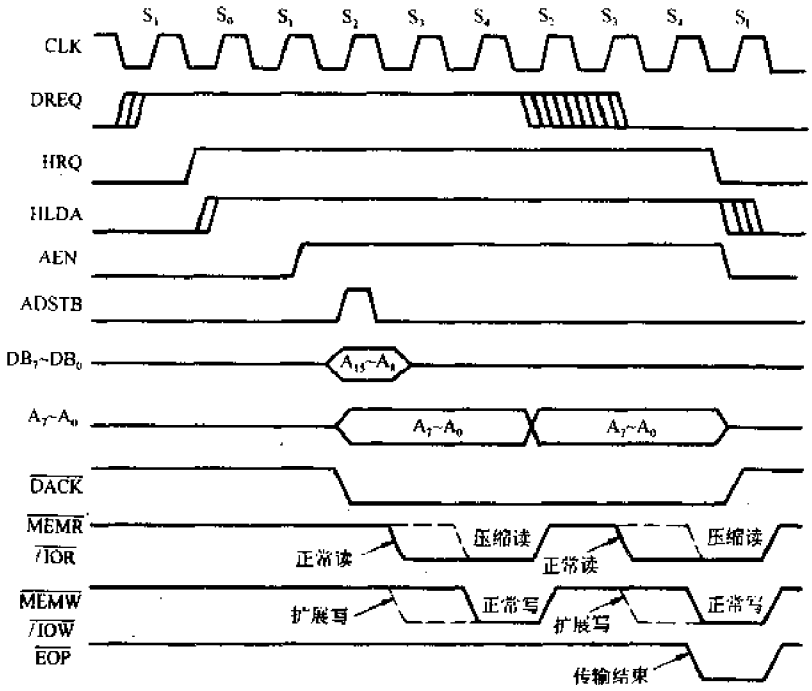


图8.45　8237A 的时序

(6)S_4状态产生$\overline{MEMW}$或$\overline{IOW}$写信号，将 DB_7～DB_0上的数据写入目的单元，写信号也可以提前到 S_3状态时产生，这就是所谓的扩展写(超前写)。若是块传输，则在 S_4结束后又进入 S_1(或 S_2)，继续传输下一字节。若是单字节传输或是块传输的最后一个字节传输完成，则产生传输结束信号$\overline{EOP}$，并撤消总线请求信号 HRQ 释放总线。外部输入的$\overline{EOP}$信号强制8237A 在完成传输当前字节的 S_4后结束 DMA 过程。

四、8237A 的内部寄存器

8237A 的内部寄存器可分为两类：一类称为通道寄存器，每个通道有四个：基地址寄存器、基字节计数器、当前地址寄存器和当前字节计数器。这四个寄存器都是16位的，其内容初始化编程时写入。另一类为控制和状态寄存器，这类寄存器是四个通道共用的。控制寄存器用来设置8237A 的工作方式和请求控制等，初始化编程时写入；状态寄存器用来存放8237A 的工作状态信息，供 CPU 读取查询。各寄存器的端口地址分配见表8.8。

1. 命令寄存器

命令寄存器格式如图8.46所示。

表8.8　8237A 内部寄存器寻址及读/写方式

通道号	A_3	A_2	A_1	A_0	读操作($\overline{IOR}=0$)	写操作($\overline{IOW}=0$)
0	0	0	0	0	读当前地址寄存器	写基(当前)地址寄存器
	0	0	0	1	读当前字节计数器	写基(当前)字节计数器
1	0	0	1	0	读当前地址寄存器	写基(当前)地址寄存器
	0	0	1	1	读当前字节计数器	写基(当前)字节计数器
2	0	1	0	0	读当前地址寄存器	写基(当前)地址寄存器
	0	1	0	1	读当前字节计数器	写基(当前)字节计数器
3	0	1	1	0	读当前地址寄存器	写基(当前)地址寄存器
	0	1	1	1	读当前字节计数器	写基(当前)字节计数器
公用	1	0	0	0	读状态寄存器	写命令寄存器
	1	0	0	1		写请求寄存器
	1	0	1	0		写单通道屏蔽寄存器
	1	0	1	1		写方式寄存器
	1	1	0	0		清除高/低触发器
	1	1	0	1	读暂存寄存器	主清除(软件复位)
	1	1	1	0		清除屏蔽寄存器
	1	1	1	1		写四通道屏蔽寄存器

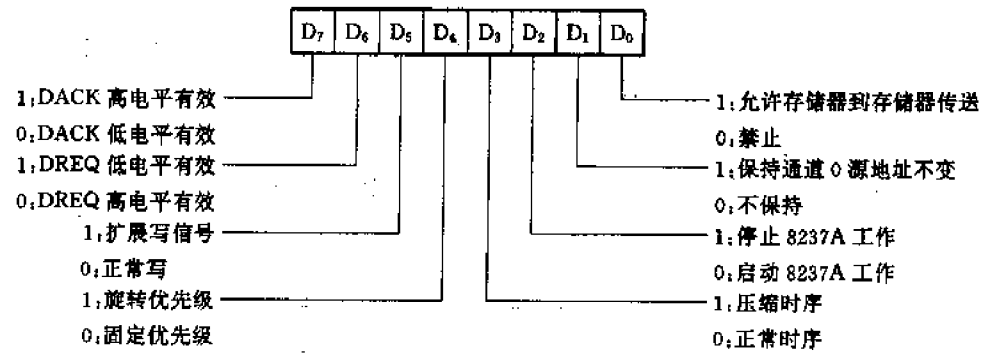


图8.46　8237A 命令寄存器格式

命令寄存器用来控制8237A 的操作。其内容由 CPU 写入，由复位信号 RESET 或清除命令清除。对有关位定义说明如下：

D_1位控制通道源地址在存储器到存储器传送过程中保持不变。其作用是可以使源地址内的同一个数据传送到一组目标存储单元中去。当 $D_0=1$时，$D_1=1$才有意义。

D_3和 D_5位选择8237A 的工作时序，$D_3=0$时为正常时序，即每进行一次 DMA 传送时，通常用 S_2、S_3和 S_4三个状态；$D_3=1$时为压缩时序，通常用 S_2和 S_4两个状态。当 $D_0=1$时，D_3定义无意义。$D_5=1$时为扩展写信号操作，当 $D_3=1$时，D_5定义无意义。

D_4位控制通道的优先权。8237A 通道的优先权有固定和旋转两种方式。当 $D_4=0$时为固定优先级，规定四个通道从高到低的优先级顺序为：$DREQ_0 \longrightarrow DREQ_1 \longrightarrow DREQ_2 \longrightarrow DREQ_3$。当 $D_4=1$时为旋转优先级，即通道的优先权随着 DMA 服务的结束而变化。当前正在服务的

DMA 通道传送结束后，其优先级变为最低，而其后的 DMA 通道的优先级变为最高，如此循环，使得每一个通道都有取得最高优先权的机会。

2. 工作方式寄存器

8237A 的每一个通道有一个工作方式寄存器，用于设置通道的工作方式，其格式如图8.47所示。

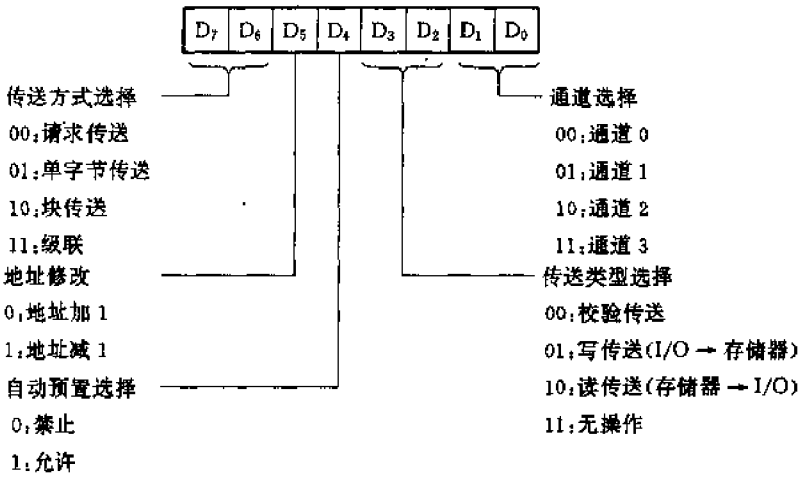


图8.47　工作方式寄存器格式

D_3、D_2位选择8237A 的传送类型。写传送是把外设的数据写入存储器；读传送是把存储器中的数据读出传送到外部设备；校验传送是一种空操作，存储器和 I/O 的读写控制线无效，只是产生 DMA 时序，外设可以利用这个时序进行校验。

D_4位选择自动预置功能。所谓预置是指当计数值减为“0”时，当前字节计数器和当前地址寄存器会自动从基字节计数器和基地址寄存器中获取初值，又从头开始相同的操作。

3. 请求寄存器

8237A 的每一个通道有一个 DMA 请求触发器，可以通过硬件(DREQ 有效)或软件(设置请求寄存器)使该触发器置“1”，表示有 DMA 请求发生。请求寄存器格式如图8.48所示。

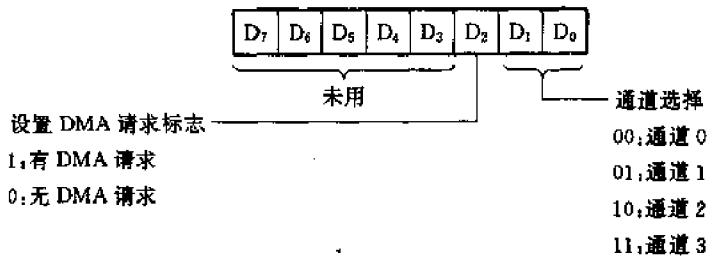


图8.48　请求寄存器的格式

4. 屏蔽寄存器

8237A 的每一个通道有一个屏蔽触发器，当该触发器为“1”时，屏蔽该通道的 DMA 请求。屏蔽位可用下列三种命令字来置位或清除。

(1)单通道屏蔽字

单通道屏蔽字格式如图8.49所示。

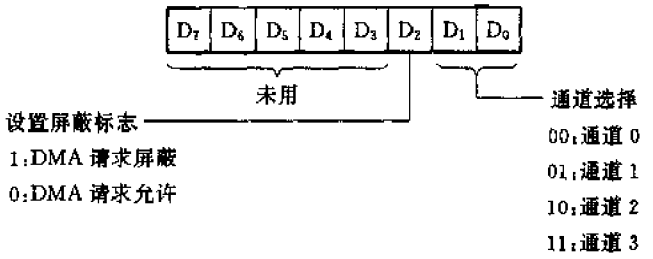


图8.49 单通道屏蔽字的格式

(2)四通道屏蔽字

四通道屏蔽字用综合屏蔽命令来设置通道的屏蔽触发器,其格式如图8.50所示。

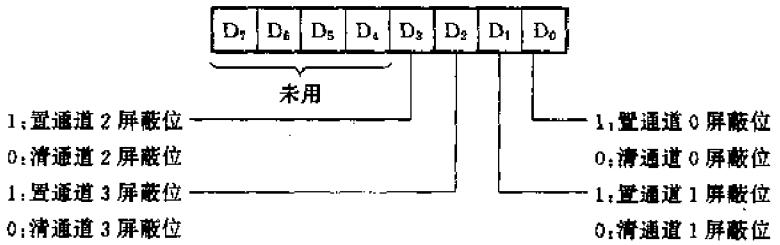


图8.50 四通道屏蔽字的格式

(3)清除屏蔽寄存器

在8237A 复位之后,所有的屏蔽位都置"1",即禁止所有的 DMA 请求,在非自动预置方式下,一旦某通道 DMA 传送结束,该通道的屏蔽位也被置"1"。因此,在 DMA 通道初始化时,为了开放通道的 DMA 请求,必须清除屏蔽位。对端口 DMA+14(指屏蔽寄存器)进行一次写操作,即可清除4个通道的屏蔽位,开放全部通道的 DMA 请求。如:

```
MOV   AL,0
MOV   DX,DMA+14
OUT   DX,AL          ;清除四通道的屏蔽位
```

5. 状态寄存器

状态寄存器用来存放各通道的工作状态与请求标志。其格式如图8.51所示。

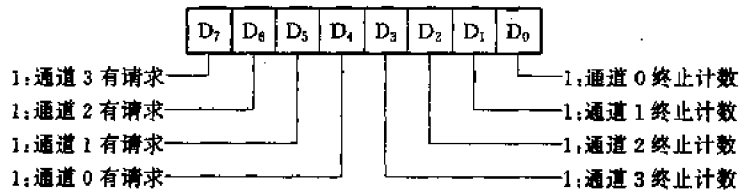


图8.51 状态字的格式

6. 暂存寄存器

8237A 进行从存储器到存储器传送操作时,通道0先把由源存储单元中存储的数据读出送到暂存寄存器中暂存,然后由通道1把暂存寄存器中的数据读出传送至目标存储单元中。

7. 清除命令

清除命令不需要通过写入控制寄存器来执行,而只是对特定的地址执行一次写操作即可执行清除。8237A 共有三种清除命令:

(1)主清除命令

主清除命令的功能与复位信号 RESET 类似，除屏蔽寄存器各位置“1”外，其余各寄存器的内容均为“0”。使8237A 进入空闲周期，以便进行初始化编程。地址为：$A_3 \sim A_0 = 1101$，$\overline{CS}=0$。

(2)清除先/后触发器命令

8237A 通道内的寄存器为16位，而数据线是8位，先/后触发器用来控制读、写16位寄存器的高字节还是低字节。若触发器为“0”，则对低字节操作；若触发器为“1”，则对高字节操作。执行 RESET 或清除命令后，该触发器为“0”。地址为：$A_3 \sim A_0 = 1100$，$\overline{CS}=0$。

(3)清除屏蔽寄存器命令

清除屏蔽寄存器的作用在上面已经介绍过，这里不再重复。地址为：$A_3 \sim A_0 = 1110$，$\overline{CS}=0$。

五、8237A 的编程

8237A 是高性能的可编程器件，在使用时对其工作方式、操作类型及有关寄存器的初始状态进行编程设定。其初始化流程图如图8.52所示。

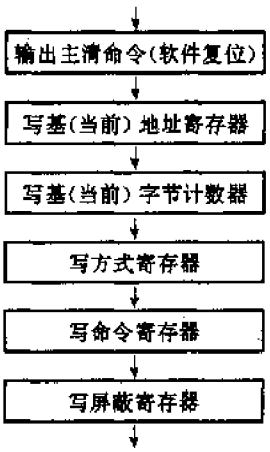


图8.52 8237A 初始化流程图

例如利用8237A 通道1，将外设长度为100H 字节的数据块传送到内存2000H 开始的连续的存储单元中。要求：采用块连续传送，外设的 DREQ 和 DACK 为高电平有效，允许请求，试编写初始化程序。

设8237A 的端口地址为00～0FH，初始化程序如下：

```
START:MOV  AL,0
      OUT  0DH,AL  ;输出主清除命令，软件复位。
      OUT  02H,AL  ;写低位地址
      MOV  AL,20H
      OUT  02H,AL  ;写高位地址
      MOV  AX,100H ;传送的字节数
      OUT  03H,AL  ;写低位计数值
      MOV  AL,AH
      OUT  03H,AL  ;写高位计数值
      MOV  AL,85H
      OUT  0BH,AL  ;写方式字：块传送，地址增1
      MOV  AL,80H
      OUT  08H,AL  ;写命令字：DACK1、DREQ1为高电平有效
      MOV  AL,01H
      OUT  0AH,AL  ;写屏蔽字：允许通道1请求
```

第六节 多功能 I/O 接口82380

随着半导体器件制造业的发展，微型计算机 I/O 接口电路的集成度越来越高，功能越来越强。原来多片单一功能的 I/O 电路，现在已可能做到一只超大规模集成芯片中。限于篇幅，本节以80386/80486微型计算机系统中广泛使用的82380为例，对多功能 I/O 接口电路的构

成、工作原理及应用作简要介绍。

一、82380的结构与功能

82380的结构框图如图8.53所示。它包括了中断控制器、定时器、DMA 控制器、等待状态控制、DRAM 刷新控制及系统复位控制等。

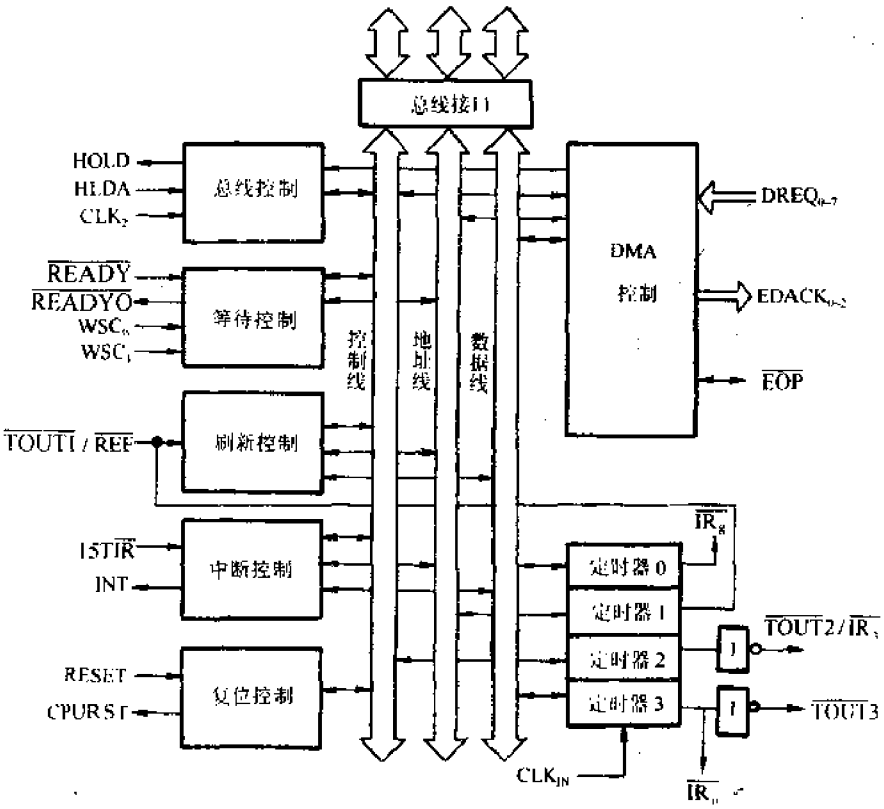


图8.53　82380结构框图

(一)82380的中断控制器

1. 中断控制器的功能

82380内部设有3个与82C59A 功能等效的中断控制器。这三个中断控制器彼此级联，如图8.54所示。三个中断控制器具有15个外部和5个内部中断请求输入线。每一个外部中断请求输入端又可与一个82C59A 级联，因此82380最多可以处理120个外部中断请求。

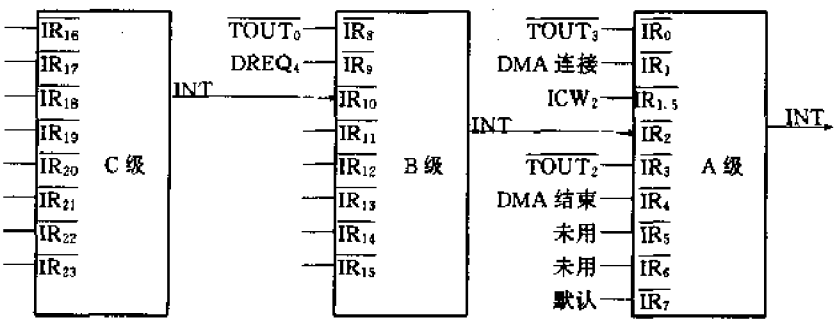


图8.54　82380内部中断控制器结构

在图8.54所示内部结构中共有A、B、C三级82C59A，C级的INT接B级$\overline{IR_{10}}$，B级的INT接A级$\overline{IR_2}$。三级控制器基本相同，唯一的区别是A级多了一个$\overline{IR_{1.5}}$，用于处理特殊的ICW_2中断请求输入。这样共有15个外部中断输入：$\overline{IR_3}$、$\overline{IR_9}$、$\overline{IR_{11}}$～$\overline{IR_{23}}$，5个内部中断输入：$\overline{IR_0}$～$\overline{IR_{1.5}}$、$\overline{IR_4}$、$\overline{IR_8}$。5个内部中断请求的作用如下：

$\overline{IR_0}$：计数器3输出$\overline{TOUT_3}$

$\overline{IR_1}$：DMA连接请求

$\overline{IR_{1.5}}$：写ICW_2

$\overline{IR_4}$：DMA服务结束

$\overline{IR_8}$：计数器0输出$\overline{TOUT_0}$

图中默认的含义是：中断控制器要求在接收到第一个$\overline{INTA}$负脉冲跳变之前，请求信号一直保持为有效电平，如果不能保持，82380内部中断控制器自动假设该请求信号来自$\overline{IR_7}$。因此在系统设计时，可用$\overline{IR_7}$的中断服务程序指示或处理中断请求信号出错。

另外，在15个外部中断请求输入中，有2个属于共享输入端，一个是$\overline{IR_3}$，另一个是$\overline{IR_9}$。其中$\overline{IR_3}$可作为外部请求输入，或者用$\overline{TOUT_2}$作为$\overline{IR_3}$的输入。$\overline{IR_9}$可作为外部中断请求输入，或者DMA请求($DREQ_4$)输入，但在同一时间只能用其中的一个。

82380内部每一级中断控制逻辑由以下五部分组成：

(1)中断请求寄存器IRR；

(2)优先级分析器和控制逻辑PR；

(3)中断服务寄存器ISR；

(4)中断屏蔽寄存器IMR；

(5)中断向量寄存器VR。

以上IRR、ISR、IMR和优先级分析器和控制逻辑PR与82C59A完全相同，区别在于82380每一级中断控制器中都增设了8个中断向量寄存器，每个中断的类型码均可在相应的向量寄存器中独立设置，而82C59A的8个中断类型码是连续的。

2. 中断控制器的编程

82380内部中断控制器的编程方法与82C59A基本相同，由ICW_1～ICW_4进行初始化，由OCW_1～OCW_3进行随机操作。所不同的是ICW_2主要是与82C59A相容，其内容不影响每一级的中断操作，在任一级写入ICW_2时，A级的$IR_{1.5}$产生中断，CPU读取ICW_2后，该中断请求自动清除，每一个中断源的类型码必须在相应的中断向量寄存器中设置。

例如：假设在80486微机系统中用一片82380进行中断控制，中断请求采用边沿触发方式，命令结束。在对82380内部中断控制器初始化时先由ICW_1设置边沿触发方式和非级联方式，由于没有级联，所以不写入ICW_3，最后由ICW_4设置完全嵌套方式、非缓冲方式和命令结束方式。其B级初始化程序如下：

```
CLI
MOV   AL,08H
OUT   28H,AL          ;IR8类型码为08H
 ⋮                    ;IR9～IR15的类型码用同样方法设置
MOV   AL,00010011B
OUT   20H,AL          ;写入ICW1
```

```
MOV   AL,n
OUT   21H,AL          ;写入 ICW2
MOV   AL,00001001B
OUT   21H,AL          ;写入 ICW4
STI                   ;开中断
```

(二)82380的定时器

1.定时器的功能

82380内部有四个16位可编程定时器0～定时器3。每个定时器的功能与8253功能类似。四个定时器共用时钟信号 CLKIN。各定时器作用如下：

定时器0输出信号 $TOUT_0$ 的上升沿产生中断请求 IR_8。在系统中，定时器0常用作24小时的时钟。

定时器1用作 DRAM 刷新控制。其输出端 $TOUT_1$ 的上升沿到刷新控制电路产生刷新请求信号。

定时器2的输出端 $TOUT_2$ 和中断请求线 IR_3 相联引出。当它作为定时器输出时可驱动扬声器发声。

定时器3一方面在82380内部产生 IR_0 中断请求，另一方面通过 $TOUT_3$ 引出作为一个通用的外部信号。

2.定时器的编程

82380内部定时器有两个控制字寄存器，CPU 可通过向这两个寄存器写控制字来设定4个定时器的工作方式。其中方式控制字1用来设定定时器0、1、2的工作方式和读回，其格式如图8.55所示。

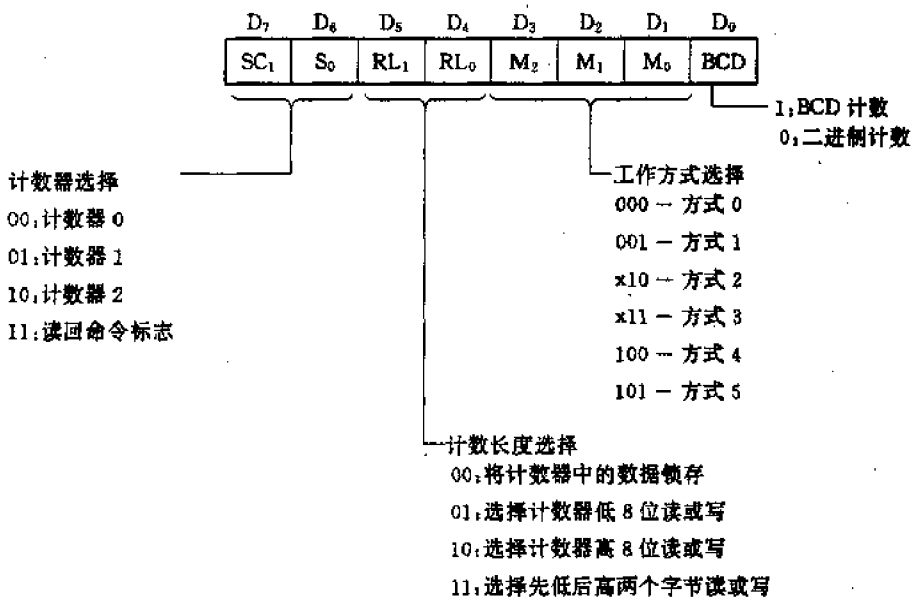


图8.55 方式控制字1格式

方式控制字2用来确定定时器3的工作方式和读回，其格式如图8.56所示。

对82380内部定时器初始化时与8253初始化类似，只需写入方式控制字与计数常数即可。

(三)82380的 DMA 控制器

求 IR_1(见图8.54),CPU 执行中断服务程序将下一缓冲区的地址及字节数置入地址寄存器和字节计数器,然后继续传送。这一过程重复进行,直至 CPU 将通道设置成单一缓冲区方式或者全部缓冲区传送完毕。

(3)总线操作方式

总线操作方式有两种,一种是单周期方式,另一种是双周期方式。

①单周期方式

单周期方式传送是82380内部 DMA 数据传送最迅速、最有效的方式。它在一个总线周期内把数据从源设备取出,同时写入到目标设备中。数据传送时,DMA 响应信号选取请求设备,同时 DMA 控制器将目标地址置于地址总线上,然后由 M/$\overline{IO}$和 W/$\overline{R}$组合完成一次数据传送。

单周期方式不能用于存储器到存储器传送,且要求请求设备与目标设备的数据线宽度相同,而且直接联接在一起。

②双周期方式

双周期方式是指进行一次传送需要两个总线周期。数据在第一总线周期读入到暂存器,在第二总线周期写入目标单元中。在双周期方式中可以传送地址不在字或字组边界上的数据,82380可以自动识别这种数据,并以字节群的方式读/写,使数据存至正确(即对正)的地址上,这一过程称为字节组合,这样的数据称为对正数据。反之,也可以读取对正的数据,写在不对正的地址上,这一过程称为字节拆分。数据组合、拆分过程程序设计者看不到,只能以双周期方式传送来实现。

综上所述,82380内部 DMA 控制器包含了8237A 的所有寄存器及功能,而且有很大的改进,增强了总体功能,表现如下:

(1)82380内部 DMA 控制器有8个通道,仅一个通道就能实现存储器到存储器的 DMA 传送,也可实现 I/O 设备与 I/O 设备之间的 DMA 传送。

(2)可按8位、16位、32位传送数据。

(3)两个地址寄存器32位,字节计数器24位,增大了数据块的范围,而且还可以使缓冲区链接,进一步增大了数据块的范围。

二、82380与微处理器的连接

82380可以直接与80486的系统总线连接使用,如图8.57所示。在82380的总线接口模块中32位数据总线和地址总线直接与80486的对应信号线连接,部分控制信号用于支持系统中不同的总线操作。由于82380可与80486共享局部总线,所以对82380访问时,它有与80486相同的地址、数据和控制信号。

(1)地址总线

地址总线 A_{31}~A_0是双向三态门输出的地址线,直接与80486地址总线连接。在主控方式下,82380输出地址,对 I/O 设备或存储器寻址。I/O 地址范围为64KB,存储器地址范围为4GB。在从属方式下接收地址,对82380内部寄存器寻址。$\overline{BE_3}$~$\overline{BE_0}$用于多字节数据选择。

(2)数据总线

数据总线 D_{31}~D_0是32位双向三态数据线。在主控方式下,用于82380与 I/O 接口或存储器之间32位、16位或8位数据传送。在从属方式下 CPU 与82380传送数据时,只能按字节操作,写入时 D_7~D_0和 D_{15}~D_8数据相同,忽略 D_{31}~D_{16}。读出时 D_7~D_0、D_{15}~D_8、D_{23}~D_{16}和 D_{31}~D_{24}数据相同。传送的字节由$\overline{BE_3}$~$\overline{BE_0}$选择。

(3)时钟信号

82380和80486之间使用82384的 CLK_2 输入进行同步。CLK_2 在82380内部进行二分频，产生内部时钟信号。内部时钟信号分为两个部分，表示为 PHI_1 和 PHI_2。一般情况下，PHI_1 用于锁存数据，PHI_2 用于输入采样。

(4)RESET 信号

是一个同步输入信号，用于82380复位，复位后82380处于从属状态，等待 CPU 对其初始化。RESET 信号应大于15个 CLK_2 周期。

(5)总线周期定义信号

总线周期定义信号有3个，即 $W/\overline{R}$，$D/\overline{C}$ 和 $M/\overline{IO}$，都是双向三态门输入输出信号。在主控方式下，这些信号由82380驱动；在从属方式下由80486驱动。在从属方式下，$D/\overline{C}$ 总是高电平。

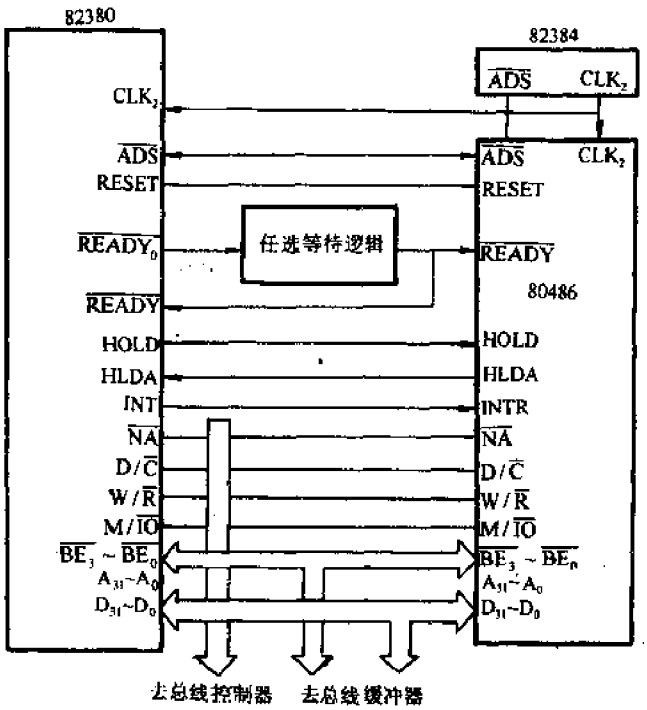


图8.57　82380与80486的连接

(6)地址状态信号

地址状态信号 $\overline{ADS}$ 是一个双向信号，用于检测总线上的有效地址（A_{31}～A_2 和 $\overline{BE_3}$～$\overline{BE_0}$）及总线周期定义信号（$W/\overline{R}$，$D/\overline{C}$ 和 $M/\overline{IO}$）。在从属方式下，82380监视 $\overline{ADS}$ 和 READY 输入信号，以确定下一个总线周期是不是一个流水线地址总线周期。在主控方式下，$\overline{ADS}$ 由82380输出驱动。

(7)下一地址请求信号

下一地址请求信号 $\overline{NA}$ 在主控方式下由82380监视，以确定是否进入地址流水线总线周期。

(8)中断请求信号

中断请求信号 INT 由82380输出，与80486的 INTR 连接，用来请求中断。

思考题与习题

1. CPU 与 I/O 设备之间传送的信息有哪些?各表示什么含义?

2. I/O 接口具有哪些功能?

3. CPU 与 I/O 设备数据传送的控制方式有几种?其特点是什么?

4. I/O 端口的编址方式有哪两种?它们各有什么特点?

5. 8255A 有哪几种工作方式?各适合使用在什么场合?

6. 设8255A 的 A 口工作于方式1输出,B 口工作于方式0输入,试编写初始化程序(端口地址自设)。

7. 使用8255A 作为开关和显示器的接口,设8255A 的 A 口连接8个开关,B 口连接8个指示灯,要求将 A 口的开关状态读入,然后送至 B 口控制指示灯亮、灭,试画出接口电路图并编写程序实现之。

8. 8253定时/计数器有哪几种工作方式?它们适合应用在什么场合?

9. 编制程序。设8253的输入时钟频率 $CLK_1=2.5MHz$,利用通道1产生500Hz 的方波信号(设端口地址为 FFA0H~FFA3H)。

10. 某系统使用8253的通道1作为计数器,设计数值为1000,计数达"0"向 CPU 发中断请求,试编写初始化程序(端口地址自设)。

11. 利用8253的通道0产生周期为10ms 的连续方波,设输入时钟的频率为100kHz,试编写8253初始化程序(设8253端口地址为38H~3BH)。

12. 串行通信的特点是什么?有哪几种通信方式?

13. 试简述串行接口 Ins 8250的基本组成与功能。

14. 设某系统使用一片8250进行串行通信,要求波特率为2400,8位数据位,2位停止位,偶校验,对接收缓冲器满开中断,试编写初始化程序。

15. 一个异步串行发送器,发送的字符格式为:7位数据位,一个奇偶校验位和两位停止位,若每秒发送100个字符,则其波特率为多少?

16. 利用8250实现远距离甲、乙两站之间的数据通信,要求:

(1)设计通信电路,画出系统硬件连接图;

(2)完成系统中芯片的初始化;

(3)编写数据块发送和接收程序。

17. DMA 传送特点是什么?

18. 8237有几个通道?其工作方式有哪几种?通道的优先级如何确定?

19. 某系统使用一片8237A 完成从存储器到存储器的数据传送,已知源数据块的首地址为2000:0000H,目标数据的首地址为2000:1050H,数据块长度为100个字节,试编写初始化程序。

20. 利用8237的通道1进行 DMA 传送,把软驱中2K 字节的数据块传送至内存2000H 开始的区域内,传送完毕停止通道工作,试编写初始化程序。

21. 82380内部结构由哪几部分功能模块组成?各部分的功能是什么?

22. 若要求在 I/O 设备与存储器之间进行 DMA 传送,试画出82380与80486 CPU 的连接图。

第九章　数/模和模/数转换接口技术

第一节　模拟输入输出通道

在微机应用系统中，模拟输入输出通道是计算机与控制对象之间的一种重要接口，是微机测控系统中的重要组成部分。

在工业生产过程中，一般被测试和控制的物理量多数为模拟量。如：温度、压强、位移、流量和速度等。在计算机处理过程中，需要将测试到的物理量经过传感器变换为电信号(通常是模拟信号)，再经过模/数转换器变换为数字信号送计算机处理；而计算机处理后的结果还需经数/模转换器变换成模拟信号去驱动模拟调节执行机构工作。一个典型的计算机控制系统如图9.1所示。它由执行机构、模拟输入通道、模拟输出通道和计算机组成。输入通道由传感器、多路模拟开关、整形放大、采样保持和模/数(A/D)转换器等部件组成，完成由模拟信号变换成数字信号的数据采集功能。输出通道由数/模(D/A)转换器、放大驱动电路等部件组成，完成由数字信号变换为模拟信号去驱动受控设备的功能。各部件的功能简要介绍如下：

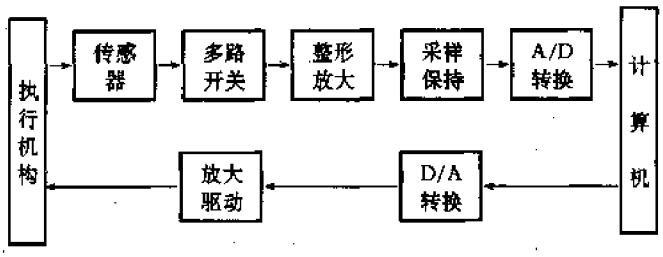


图 9.1　计算机控制系统

1. 传感器

传感器是采集或传感最原始物理量的部件。它能够把非电量的模拟信号转换成电量(如电流或电压)的模拟信号。常用的传感器有：温度传感器、压力传感器、流量传感器等。

2. 多路模拟开关

在一个测控系统中，通常被检测的物理量有多个，如果每一个输入通道都连接放大器、采样保持器和 A/D 转换器件，显然很不经济。在实际电路中，一般是采用多路模拟开关使多个通道共用一个放大器、采样保持器和 A/D 转换器，使用时间分割法使多个模拟通道轮流接通，致使系统既简洁，又经济实用。

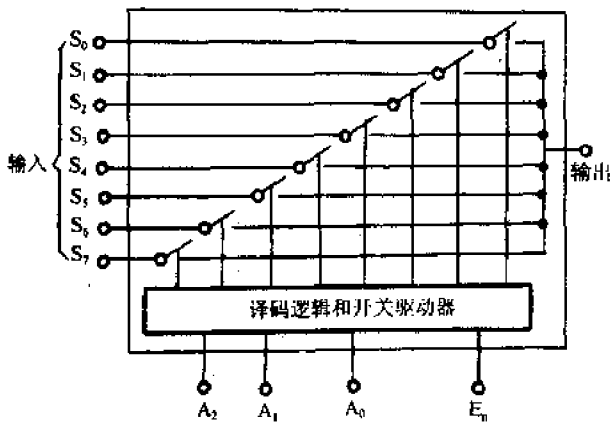


图 9.2　多路模拟转接器

多路模拟转接器由模拟通道开关组合而成，图 9.2 所示为一个 8 通道的 CMOS 模拟开关电路。每个开关的状态由驱动控制逻辑控制。当启动输入控制信号 E_n 为高电平时，8 个输入通

道 S_0～S_7 是否接通由通道选择线 A_0～A_2 的状态决定。当 $A_2A_1A_0=000$ 时 S_0 开关接通，当 $A_2A_1A_0=001$ 时，S_1 开关接通，其它依次类推。因此，可以通过控制多路模拟开关的状态来接通模拟输入通道。

3. 整形放大

由于多路模拟开关送入的信号往往很微弱，并且还需要抑制某些干扰，因此，要对多路模拟开关输入的信号使用测量放大器进行整形放大。

4. 采样保持

在许多类型的模/数转换电路中，如果在转换期间输入电压是变化的，那么就可能造成所转换的数字量误差太大，影响转换的精确度。若能在转换过程中使输入电压(即采样信号)保持不变，就可以避免上述问题。采样保持器的功能就是为了这个目的而设计的电路，其电路的基本组成如图 9.3 所示。它由输入输出高增益放大器 A_1 和 A_2、保持电容 C_H 和模式控制开关 K 组成，可由模式控制信号选择工作于采样或保持两种模式。在采样期间，开关 K 闭合，A_1 的输出通过 K 给保持电容 C_H 快速充电，使采样保持器的输出随输入变化。K 接通时，要求充电时间越短越好，以使 u_c 迅速达到输入电压值。在保持期间，开关 K 断开，由于运放 A_2 的输入阻抗高，可使电容器 C_H 将保持充电时的最高值。

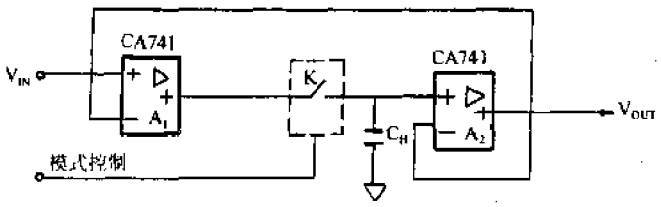


图 9.3　采样保持基本电路

目前，采样保持电路多数为集成电路芯片，种类很多，常用的芯片有 LF198、LF398 和 AD582 等。

5. A/D 转换器

A/D 转换器的功能是把模拟量变换成数字量。由于实现转换的工作原理和工艺技术不同，目前市场上 A/D 转换芯片种类繁多。按分辨率可分为 4 位、6 位、8 位、10 位、12 位、14 位、16 位等。按转换速度可分为超高速(转换时间≤330ns)、次超高速(转换时间为 330～3.3μs)、高速(转换时间为 3.3～33μs)、中速(转换时间为 33～333μs)和低速(转换时间＞330μs)等。按转换工作原理可分为逐次逼近型和积分型等。

6. D/A 转换器

D/A 转换器的功能是把数字量变换成模拟量。它将 CPU 送来的数字信号转换为成比例的模拟信号输出。经过 D/A 转换器转换的模拟信号通常需要进行低通滤波和功率放大等处理。目前市场上集成 D/A 芯片类型很多。按其制造工艺分类有双极型、MOS 型等；按字长分类有 8 位、10 位、12 位等；按输出形式分类有电流型和电压型。

第二节　D/A 转换器

一、D/A 转换原理

D/A 转换器的基本功能是将一个用二进制表示的数字量转换成相应的模拟量。实现这种

转换的基本方法是对应于二进制的每一位，产生一个相应的电压(电流)，而这个电压(电流)的大小正比于相应的二进制位的权。

图 9.4 为 D/A 转换器的原理图，由图可见，它主要由三部分构成，即加权电阻解码网；受输入数字量控制的电子开关组和由运算放大器构成的电流-电压转换器。电子开关组受输入二进制数据 $D_{7\sim0}$控制，当某一位为"1"，则电子开关闭合，基准电压 V_{REF}接电阻解码网络，使某一支路电阻上有电流流过。当某一位为"0"时，则电子开关断开，该支路电阻上无电流流过。加权电阻解码网络各支路的电阻值与输入二进制数据 $D_{7\sim0}$的"权"相对应，"权"大的电阻值小，"权"小的电阻值大。因此各支路的电流不仅决定于输入数字量的值(0 或 1)，还决定于"权"。各支路的电流如下：

$$I_0=\frac{V_{REF}}{2^8\cdot R}\cdot D_0=2^0\frac{V_{REF}}{2^8\cdot R}\cdot D_0$$

$$I_1=\frac{V_{REF}}{2^7\cdot R}\cdot D_1=2^1\frac{V_{REF}}{2^8\cdot R}\cdot D_1$$

$$\vdots$$

$$I_7=\frac{V_{REF}}{2R}\cdot D_7=2^7\cdot\frac{V_{REF}}{2^8\cdot R}\cdot D_7$$

因此，总电流为 I_{OUT}：

$$I_{OUT}=I_0+I_1+\cdots+I_7=\frac{V_{REF}}{2^8\cdot R}\cdot\sum_{i=0}^{7}2^iD_i$$

该总电流经电流-电压转换器(运算放大器)后，电压 V_{OUT}为：

$$V_{OUT}=-R_fI_{OUT}=-\frac{V_{REF}}{2^8}\cdot\frac{R_f}{R}\cdot\sum_{i=0}^{7}2^iD_i=-E_0\cdot N$$

其中 $E_0=\frac{V_{REF}}{2^8}\cdot\frac{R_f}{R}$，$N=\sum_{i=0}^{7}2^iD_i$。$D_i$ 为第 i 位输入数字量，其值为"0"或"1"。

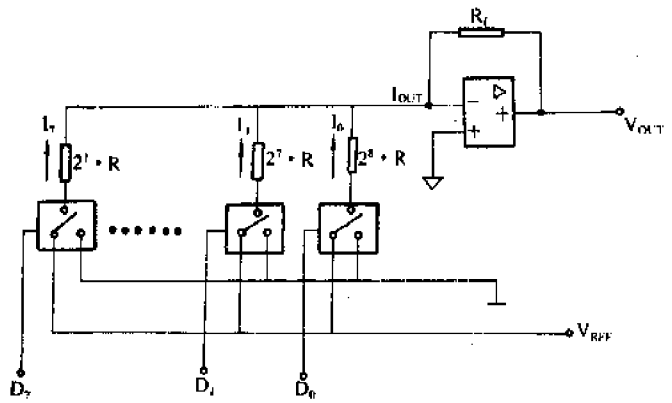


图 9.4　D/A 转换原理图

由上式看出，输出电压 V_{OUT}正比于输入数字量 N，其幅度的大小可通过选择基准电压 V_{REF}和 R_f/R 的比值来调整。尽管使用的网络结构不同，但对于 D/A 转换器的输入输出来说是等效的。图 9.4 所示的 8 位 D/A 转换器，每一数字输入位所代表的输出模拟量是其相邻的两倍，这样就组成二进制数字量至模拟量的转换器。

二、D/A 转换器的特性

D/A 转换器的性能特点主要反映在它的性能指标和外部特性两个方面。

1. D/A 转换器的性能指标

衡量一个 D/A 转换器性能的主要指标有：

(1)分辨率。表征 D/A 转换器对微小输入量变化的敏感程度，通常用数字量的数位表示，如 8 位，12 位，14 位等。分辨率为 10 位的 D/A 转换器，表示它可以对满量程的$\frac{1}{2^{10}}=\frac{1}{1024}$的增量作出反应。

(2)相对精度。在满刻度已校准的前提下，在整个刻度范围内，对应于任一数码的模拟量输出与它的理论值之差。通常用偏差几个 LSB 来表示和该偏差相对满刻度的百分比表示。

(3)转换时间。数字变化量是满刻度时，达到终值±LSB/2 时所需的时间，通常为几十纳秒至几微秒。

(4)线性误差。通常给出在一定温度下的最大非线性度，一般为 0.01%～0.03%。

2. D/A 转换器的外部特性

DAC 芯片是微机应用系统中的一个功能单元，对于不同的应用场合，对其输入输出特性的要求有所不同，芯片的外部特性通常指下面几个方面。

(1)输入缓冲能力。DAC 是否带有输入寄存器或锁存器来保存输入的数字信号，对于不能长时间保持输出数据的微机系统十分重要，在微机应用系统中一般选择具有缓冲功能的 D/A 转换器。

(2)输入数据的宽度。DAC 芯片有 8 位、10 位、12 位、16 位之分，当它的位数多于微机的数据总线宽度时，需选择带有两个输入寄存器的 DAC 芯片。

(3)电流型与电压型。对于电流输出型，其电流在几到十几 mA，对于电压输出型，其电压一般为 5V～10V 之间。

(4)单极性与双极性。对于一些需要正负电压控制的系统中，就需要选用双极性 DAC 芯片，使输出电压有极性变化。

(5)数字输入码制。一般来说，对于单极性输出的 DAC 芯片能接收二进制码或 BCD 码；对于双极性输出的 DAC 芯片只能接收补码。

三、典型的 D/A 转换器

(一)DAC 0832

DAC 0832 是美国国家半导体公司生产的 8 位电流型 CMOS D/A 转换芯片，其内部逻辑结构框图如图 9.5 所示。该芯片内部有两个具有锁存功能的 8 位数据缓冲寄存器和一个 8 位 D/A 转换器。两个数据缓冲器的工作状态分别受锁存信号$\overline{LE_1}$和$\overline{LE_2}$的控制。当$\overline{LE_1}$为高电平时，8 位输入寄存器的输出跟随输入而变化，当$\overline{LE_1}$由高电平变为低电平时，输入数据被锁存。8 位 DAC 寄存器的工作状态受$\overline{LE_2}$的控制，其控制方式和$\overline{LE_1}$类似。下面介绍 DAC 0832 的性能特点、引脚信号和工作方式。

1. DAC 0832 的主要特性

(1)分辨率为 8 位；

(2)转换时间为 1μs；

(3)单缓冲、双缓冲和直通方式输入；

(4)只需在满量程下调整其线性度；

(5)逻辑电平输入与 TTL 兼容；

(6)单一电源供电(+5V～+15V)；

(7)功耗为 20mV。

2. DAC 0832 的引脚功能

DAC 0832 芯片的引脚信号见图 9.5。其各引脚信号功能如下：

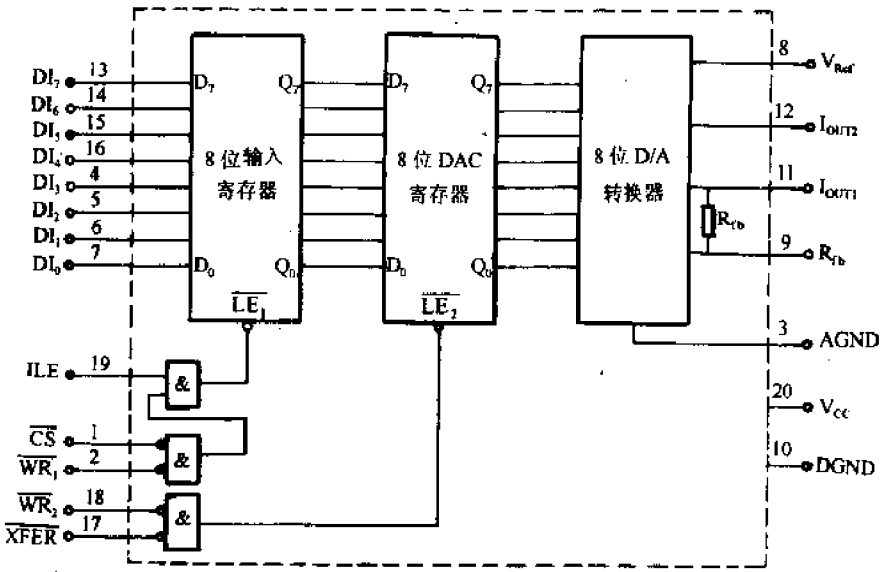


图 9.5 DAC 0832 逻辑结构框图

DI_7～DI_0：8 位数字信号输入线。

$\overline{CS}$：片选，输入，低电平有效。

$\overline{WR_1}$：写信号 1，输入，低电平有效。

$\overline{WR_2}$：写信号 2，输入，低电平有效。

ILE：允许输入锁存信号，高电平有效，用来产生 8 位输入寄存器的锁存信号$\overline{LE_1}$。当 ILE=1，$\overline{CS}=0$，$\overline{WR_1}=0$ 时，则$\overline{LE_1}=1$，此时输入寄存器的状态与输入线 DI_7～DI_0 的状态保持一致，当$\overline{LE_1}$ 信号由“1”变为“0”时，即$\overline{WR_1}$由低电平变成高电平时，DI_7～DI_0 的状态被锁存在 8 位输入寄存器中。

$\overline{XFER}$：数据传送控制信号，输入，低电平有效。当$\overline{XFER}=0$，$\overline{WR_2}=0$ 时，则$\overline{LE_2}=1$，使 DAC 寄存器的状态与输入信号状态保持一致；当$\overline{LE_2}$信号由“1”变为“0”时，即$\overline{WR_2}$由低电平变成高电平时，输入信号被锁存在 8 位 DAC 寄存器中，并同时将输入的数据送 8 位 D/A 转换器进行转换输出。

V_{REF}：基准电源输入端。它与 DAC 芯片内的权电阻解码网络相接，可在－10V～+10V 范围内选择。

I_{OUT1}：DAC 电流输出端 1。它为电流输出端为“1”的各位权电流汇集输出端，当 DAC 寄存

器全为"1"时，I_{OUT1}最大；当 DAC 寄存器全为"0"时，I_{OUT1}为"0"。

I_{OUT2}：DAC 电流输出端 2。它为电流输出端为"0"的各位权电流汇集输出端，当 DAC 寄存器各位为"0"时，I_{OUT2}输出最大，各位为"1"时，I_{OUT2}输出为"0"。因而有：$I_{OUT1}+I_{OUT2}=$常数。

R_{fb}：反馈电阻输入端。反馈电阻在芯片内部，可用作外部运算放大器的分路反馈电阻。

V_{CC}：电源输入端。工作电压为+5～+15V。

AGND：模拟信号地。

DGND：数字信号地。

3. DAC 0832 的工作方式

由于 DAC 0832 内部有两个具有锁存功能的输入数据寄存器和 D/A 转换器，它有以下三种工作方式。

(1)直通方式

将$\overline{CS}$、$\overline{WR_1}$、$\overline{WR_2}$和$\overline{XFER}$引脚与 DGND 连接，ILE 引脚与 V_{CC}连接，芯片便处于直通状态，一旦 DI_7～DI_0 有信号输入，就立即直通进行 D/A 转换并输出。此方式适合于不受 CPU 直接控制的场合。

(2)单缓冲方式

此方式是使具有锁存功能的两个寄存器，其中一个处于直通状态，而另一个处于受控的锁存状态。一般在使用时，使 DAC 寄存器处于直通，即将$\overline{WR_2}$和$\overline{XFER}$引脚接地，而使输入寄存器为受 CPU 控制的锁存，将$\overline{WR_1}$与 CPU 写信号$\overline{WR}$连接(见图 9.7)。当 CPU 执行写操作时，使$\overline{WR_1}$变为有效，若此时$\overline{CS}=0$，ILE＝1，输入寄存器的状态与输入线 DI_7～DI_0 的状态一致，当$\overline{WR_1}$由低变高时输入数据被锁存。被锁存的信号经 DAC 寄存器直通输入到 D/A 转换器中转换输出。此方式适合于多个 DAC 芯片转换的模拟信号，不需要同步输出的场合。

(3)双缓冲方式

此种方式是使芯片内的两个寄存器都处于受控的锁存状态。其连接方法是：把 ILE 固定为高电平，$\overline{CS}$和$\overline{XFER}$分别接两个端口的地址译码输出，$\overline{WR_1}$和$\overline{WR_2}$与 CPU 的写信号$\overline{WR}$连接。在这种工作方式下，CPU 对芯片需进行两次写操作，第一次写操作时，使$\overline{WR_1}=0$，$\overline{CS}=0$且 ILE＝1，将 DI_7～DI_0 的状态输入数据寄存器中并锁存，第二次写操作时，使$\overline{WR_2}=0$，$\overline{XFER}=0$，将输入寄存器中的数据输入锁存到 DAC 寄存器中并转换输出。此方式适合于要求多个 DAC 芯片转换的模拟信号同步输出的场合。

(二)DAC 1210

DAC 1210 芯片是一个 12 位的 D/A 转换器，其性能特点与工作方式和 DAC 0832 基本相同，区别仅是分辨率不同。其内部逻辑结构和引脚信号如图 9.6 所示。输入寄存器由一个 8 位寄存器和一个 4 位的寄存器组成，被转换的 12 位数字信号需要经过两次写入，由引脚信号 $B_1/\overline{B_2}$ 控制。当 $B_1/\overline{B_2}$ 为高电平时，且$\overline{CS}=0$，$\overline{WR_1}=0$ 时，将高 8 位数字信号 DI_{11}～DI_4 写入 8 位输入寄存器中，将低 4 位数字信号 DI_3～DI_0 写入 4 位输入寄存器中，12 位数字信号同时输入并锁存器。当 $B_1/\overline{B_2}$ 为低电平时，只将 12 位数字信号的低 4 位 DI_3～DI_0 送入 4 位输入寄存器中，以保护 8 位输入寄存器中的内容不变。

DAC 1210 的工作方式与 DAC 0832 相同，这里就不再重复。

四、DAC 芯片的应用

(一)DAC 芯片与 CPU 接口电路的设计

DAC 芯片的种类繁多,型号各异,它们与 CPU 的连接方法通常有下面三种:

(1)直接与 CPU 连接;

(2)通过三态门和数据寄存器与 CPU 连接;

(3)通过并行 I/O 接口芯片与 CPU 连接。

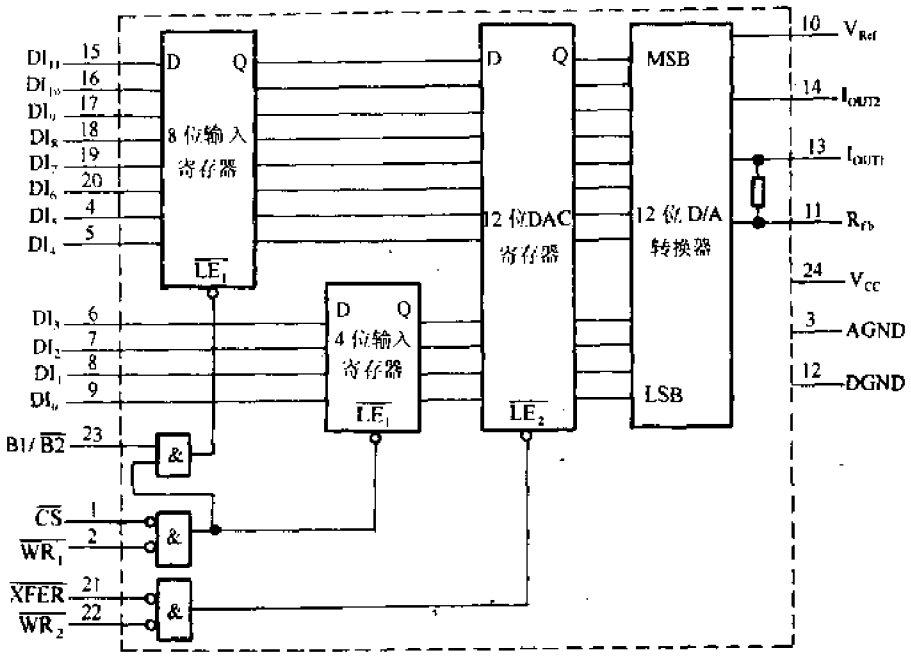


图 9.6 DAC 1210 逻辑结构框图

若 DAC 芯片内部包含有三态门输入数据锁存器时,可采用第一种接口形式与 CPU 直接连接;若无三态门输入数据锁存器时,则可采用第二或第三种接口形式,需外加锁存器来保存 CPU 输出的数据;如果 CPU 的数据总线宽度小于 DAC 芯片的转换分辨率时,也采用第二或第三种接口形式,并且需要两级缓冲,以消除由于两次传送数据而产生的毛刺。

1. DAC 0832 与 CPU 的连接

由于 DAC 0832 内部具有两级缓冲,因此它可以直接与 CPU 连接。其连接方法如图 9.7 所示。

此图为 DAC 0832 单缓冲方式连接。ILE 引脚固定接高电平,$\overline{WR_2}$和$\overline{XFER}$引脚固定接低电平,$\overline{WR_1}$与 CPU 写控制信号$\overline{WR}$连接,$\overline{CS}$由地址译码器生产,输入数据线 DI_7~DI_0 与 CPU 数据线 D_7~D_0 连接,输出端 I_{OUT1}和 I_{OUT2}外接一个运算放大器,将电流型输出转换为电压型输出。CPU 对芯片执行一次写操作,就可以把输出的数字量转换成模拟量输出。若设 DAC 0832 的端口地址为 FFF8H,执行下列指令就可以把 AL 中的 8 位数字量转换输出。

```
MOV   DX,0FFF8H
OUT   DX,AL
```

2. DAC 1210 与 CPU 的连接

由于 DAC 1210 芯片内有两级缓冲锁存器，故可以直接与 CPU 连接，其连接方法如图9.8所示。DAC 1210 的分辨率为 12 位，而 CPU 的数据线为 8 位，待转换的数据分两次送入芯片内，即先送高 8 位(使 $B_1/\overline{B}_2=1$)，后送低 4 位(使 $B_1/\overline{B}_2=0$)。若设定 8 位输入寄存器的地址为 FFE0H，4 位输入寄存器的地址为 FFE1H，12 位 DAC 寄存器的地址为 FFE2H，将 BX 寄存器中的低 12 位数字量转换输出。程序如下：

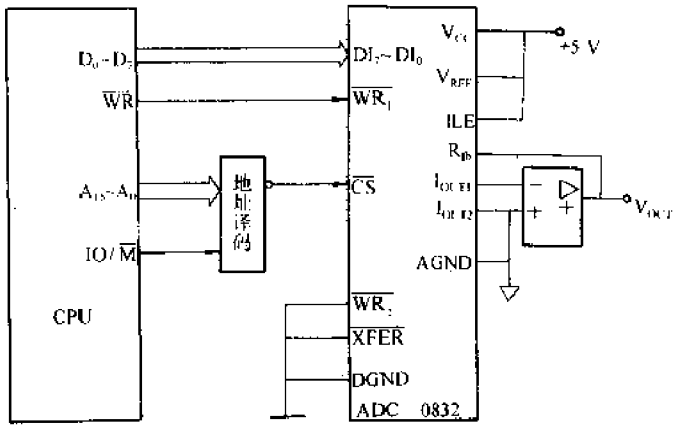


图 9.7　DAC 0832 与 CPU 连接图

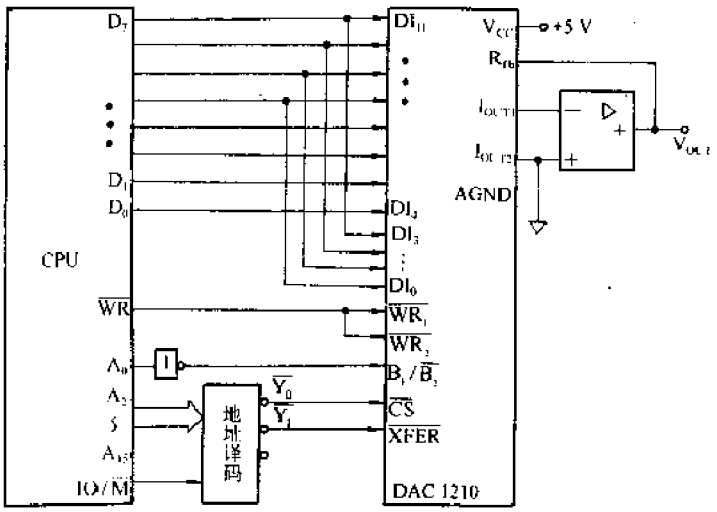


图 9.8　DAC 1210 与 CPU 的连接图

```
START:MOV  DX,0FFE0H     ;送地址
      MOV  CL,4
      SHL  BX,CL
      MOV  AL,BH
      OUT  DX,AL         ;写高 8 位数字至 8 位输入寄存器
      INC  DX
      MOV  AL,BL
      OUT  DX,AL         ;写低 4 位数字至 4 位输入寄存器
```

```
    INC   DX
    OUT   DX,AL          ;启动 D/A 转换
```

(二)D/A 转换器的应用举例

利用 D/A 转换器可产生各种电压波形。如三角波、矩形波和锯齿波等。下面是一个采用 DAC 0832 产生三角波的例子。硬件连接如图 9.9 所示。图中利用并行 I/O 接口 8255A 作为 80486 CPU 与 DAC 0832 之间的接口电路,8255A 的 A 口作为数据端口,B 口作为控制端口。如此连接的目的是在时序配合上和驱动能力上容易与 CPU 一致。如图可知,8255A 端口地址为 FF80～FF83H。

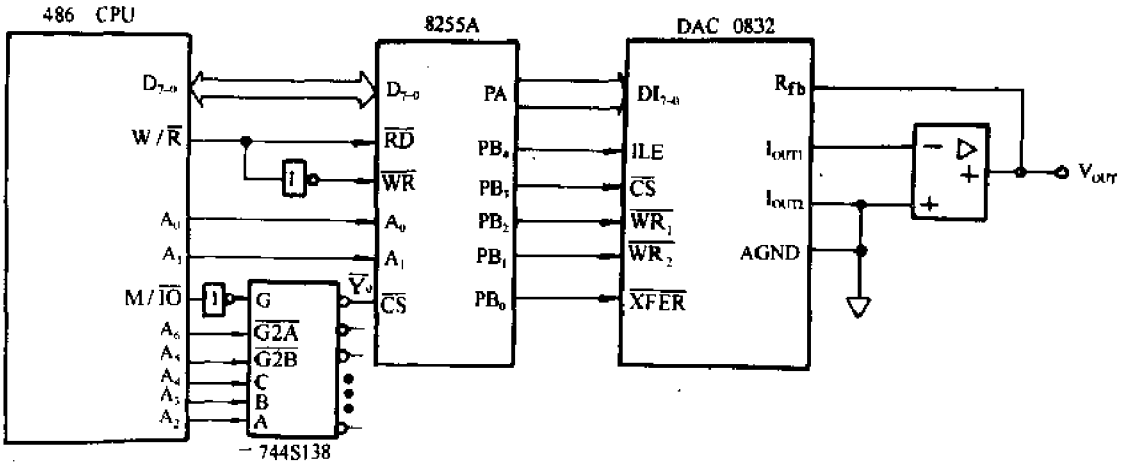


图 9.9

编写程序如下:

```
    MOV   DX,0FF83H
    MOV   AL,10000000B
    OUT   DX,AL          ;8225A 的 A、B 口方式 0 输出
    MOV   DX,0FF81H      ;置 B 口地址
    MOV   AL,00010000B
    OUT   DX,AL          ;置 0832 为直通方式
    DEC   DX
    MOV   AL,0
L2: OUT   DX,AL          ;转换输出
    INC   AL
    JNZ   L2
    HLT
```

此程序在 DAC 0832 输出端只能输出一个三角波形,若要求输出连续的三角波形,请考虑程序应如何修改。

第三节　A/D 转换器

一、A/D 转换原理

A/D 转换器的种类很多，但目前应用较广泛的主要有三种类型。逐次逼近式 A/D 转换器、双积分式 A/D 转换器和 V/F 变换式 A/D 转换器。下面简要介绍前两种 A/D 转换器的转换工作原理。

1. 逐次逼近式 A/D 转换器的工作原理

图 9.10 是逐次逼近式 A/D 转换器的工作原理图。它由比较器、A/D 转换器、输出锁存器和控制逻辑组成。其转换原理为：将输入模拟电压 U_{IN} 与 D/A 转换器的输出电压 U_i 分别输入比较器中的 A、B 端进行比较，根据 U_i 大于还是小于输入模拟信号 U_{IN} 来决定增大还是减小 U_i，以便向模拟输入信号逼近。当 U_i 信号与输入模拟信号 U_{IN} 相等时，向 D/A 转换器输入的数字就是对应模拟输入信号的数字量。

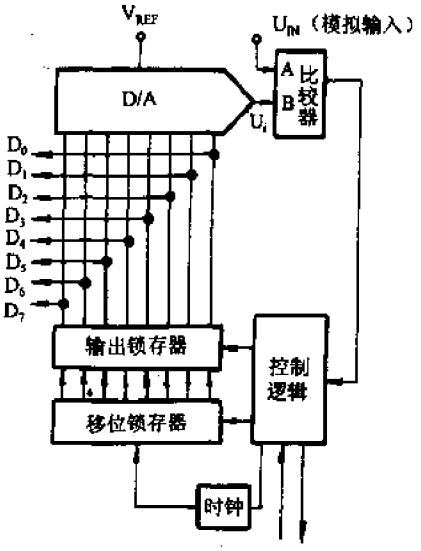


图 9.10　逐次逼近式 A/D 转换器工作原理图

其工作过程是：比较开始时，首先对二进制计数器（输出锁存器）的最高位置“1”，然后进行转换、比较、判断。若模拟输入 U_{IN} 大于 U_i，比较器输出为 1，则使输出锁存器的最高位保持为 1。然后对较低的位依次按照该办法进行比较和调整，无论哪种情况，均应继续比较下一位，直到最末位为止。此时 D/A 转换器的数字输入（输出锁存器内容）即为对应模拟输入信号的数字量。将此数字量输出就完成了 A/D 转换过程。这种方法好比有天平称一个物体的重量，第一次放最大的砝码，若不合适，就改放小一号的，依次类推。一旦天平指示砝码太重说明刚才放进去的那个应当取走，显然对于 n 位的转换器，总共需要重复这种过程 n 次。

逐次逼次 A/D 转换器的主要优点是转换速度比较快，此外与有同样分辨率的双积分型转换器比较，它不需要高精度的运算放大器，而且成本也较低。这种形式的 A/D 转换器在微机系统中被广泛采用。

2. 双积分式 A/D 转换器的工作原理

双积分式 A/D 转换器，一般具有精度高，抗干扰性好，价格便宜等优点，但转换速度较慢。

图 9.11 为双积分式 A/D 转换器工作原理图。它由电子开关、积分器、比较器、计数器和控制逻辑等组成。它通过对输入电压 U_{IN} 和对基准电压 U_R 两个阶段的积分来完成 A/D 转换。其电路的工作原理如下：在第一阶段，开关 K_1 将输入模拟电压 U_{IN} 加到积分电路进行固定时间的积分，同时计数器开始对时钟脉冲进行计数。这时计数器是作为定时器来使用的。当计满一个预先规定的固定值之后（通常这段时间约为整个转换周期的三分之一），K_1 断开，K_2 或 K_3 则将极性相反的标准电压 U_R 加到积分电路上，积分电路从刚才积分的终值开始进行反向积分。与此同时，计数器清零并重新开始对时钟脉冲计数，直至积分器输出达零，使计数器停止计数。这时控制逻辑电路向 CPU 发出“数据有效”的状态信号。CPU 可从计数器的输出端得到转

换结果。输入模拟电压 U_{IN} 幅度越大，在固定时间积分的终了时，积分器的输出电压值也越大，因而反向积分所需的时间也越长。如图 9.12 所示。

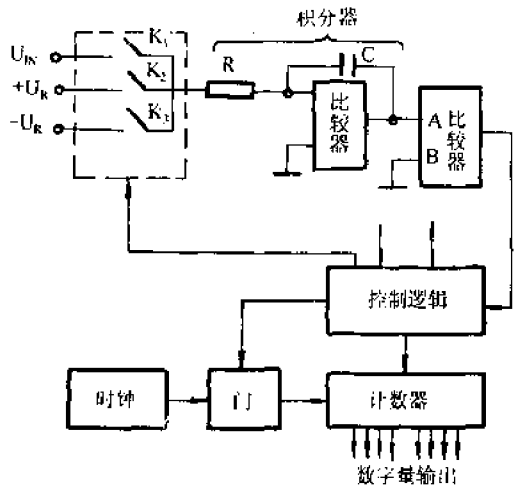


图 9.11　双积分式 A/D 转换工作原理图

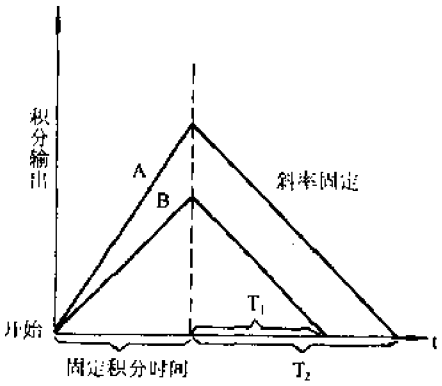


图 9.12　双积分式 A/D 转换原理示意图

用高频率标准时钟脉冲来测量积分时间，即可以得到相应模拟电压的数字量。

二、A/D 转换器的特性

在 A/D 转换器接口电路设计时，需了解 A/D 转换器(简称 ADC)的主要性能指标。

(1)分辨率。通常用数字量的位数表示，如 8 位，10 位，12 位，16 位分辨率等。若分辨率为 8 位，表示它可以对全量程的 $1/2^8=1/256$ 的增量作出反应。分辨率越高，转换时对输入量的微小变化的反应越灵敏。

(2)量程。即所能转换的电压范围，如 5V，10V 等。

(3)精度。有绝对精度和相对精度两种表示方法，常用数字量的位数作为度量精度的单位。如精度为±1/2LSB，而用百分比来表示满量程时的相对误差，如±0.05%。精度和分辨率是不同的概念。精度指的是转换后所得结果相对于实际值的准确度，而分辨率是指对转换结果发生影响的最小输入量。分辨率很高者可能由于温度漂移、线性不良等原因而并不具有很高的精度。

(4)转换时间。完成一次 A/D 转换所需的时间。不同型号、不同分辨率的器件，其转换时间的长短相差很大，可为几微秒至几百毫秒。在选择器件时要根据应用的需要和成本来具体地考虑。

(5)输出逻辑电平。多数与 TTL 电平兼容。在考虑数字输出量与微机数据总线的关系时，还要对其它一些问题加以考虑，如是否要用三态逻辑输出，采用何种编码制式等。

(6)工作温度范围。由于温度会对运算放大器和电阻解码网络等产生影响，所以只有在一定的温度范围内才能保证额定的温度指标。较好的转换器件的工作温度为－40℃～85℃，较差者为 0℃～70℃。

三、典型 A/D 转换器

ADC 0809 是逐次逼近型 8 位单片 A/D 转换芯片。片内有 8 路模拟开关，可输入八个模拟

量，单极性，量程为 0V～5V。典型的转换速度约 100μs。片内带有三态输出缓冲器，可直接与 CPU 总线接口。其性能价格比有明显的优势，是目前广泛采用的芯片之一，可应用于对精度和采样速度要求不高的场合或一般的工业控制领域。

1. ADC 0809 的主要特性

(1)分辨率为 8 位；

(2)转换误差为 $\pm\frac{1}{2}$LSB～±1LSB；

(3)转换时间为 100μs；

(4)单极性，量程为 0V～5V；

(5)温度范围为－40℃～＋85℃；

(6)5V 电源供电；

(7)具有锁存控制功能的 8 路模拟开关，可对 8 路模拟电压分时进行转换。

2. ADC 0809 的逻辑结构框图

ADC 0809 的逻辑结构框图如图 9.13 所示，其内部结构分为四部分。

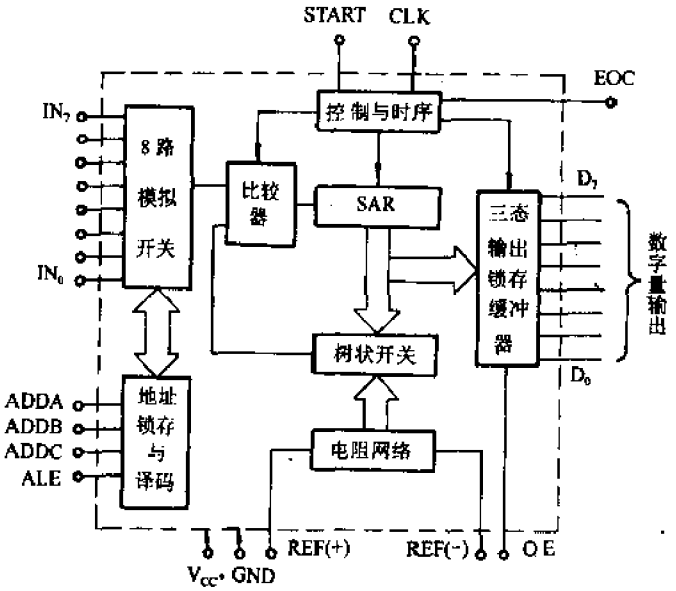


图 9.13　ADC 0809 逻辑结构框图

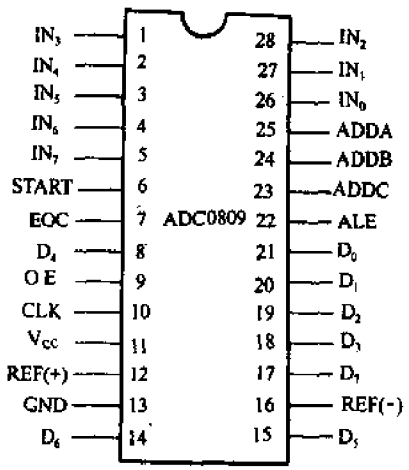


图 9.14　ADC 0809 引脚图

(1)模拟输入部分

有 8 路单端输入的多路开关和地址锁存与译码逻辑，可由三位地址输入 ADDA、ADDB、ADDC 编码选择 8 路中的一路输入(这三个地址输入信号可锁存)。其地址译码与输入选通的关系见表 9.1。

(2)变换器部分

主要由电阻网络、比较器逐位逼近寄存器 SAR 和控制逻辑组成。

控制逻辑提供转换器的时钟 CLK 和起动信号 START。转换完成时，发出转换结束信号 EOC，高电平有效。

(3)三态输出锁存缓冲器

该寄存器为 8 位，存放转换后的数字信号。

(4)基准电压输入端 REF(＋)和 REF(－)

它们决定了输入模拟电压的最大值和最小值。通常把 REF(+)接到 V_{CC}(+5V)电源上,REF(−)接到地端 GND。

表 9.1

ADDC	ADDB	ADDA	模拟通道
0	0	0	IN_0
0	0	1	IN_1
0	1	0	IN_2
0	1	1	IN_3
1	0	0	IN_4
1	0	1	IN_5
1	1	0	IN_6
1	1	1	IN_7

3. ADC 0809 的引脚功能

ADC 0809 的引脚信号如图 9.14 所示,各引脚功能说明如下:

IN_0～IN_7: 8 路模拟输入。

ADDA、ADDB、ADDC:模拟通道选择地址信号。

ALE:地址锁存允许信号。当此信号有效时,送入的通道选择地址被锁存。在使用时该信号常和启动转换信号 START 连在一起,以便同时锁存地址并启动 A/D 转换。

START:A/D 转换启动信号。要求持续时间在 200ns 以上。在地址锁存许允信号 ALE 的上升沿,被选中的通道地址锁存到地址锁存器中,在启动信号 START 的下降沿开始启动 A/D 转换,在转换结束后,EOC 信号由低变高,一次转换时间为 100μs。

CLOCK:时钟输入信号。要求频率范围为 10kHz～1.2MHz,典型值为 640kHz。可由 CPU 时钟分频得到。

EOC:转换结束信号。当 A/D 转换结束时,该信号输出高电平。

OE:允许输出信号。当该信号有效时,8 位三态门打开,可以把转换数据输出。

D_7～D_0:A/D 转换后的数字量输出数据线,可直接和系统数据总线连接,向 CPU 传送转换的结果。

REF(+)、REF(−):模拟参考电压输入端。与微机接口时,REF(+)为 5V,REF(−)为 0V。

4. ADC 0809 的工作时序

ADC 0809 的工作时序图如图 9.15 所示。

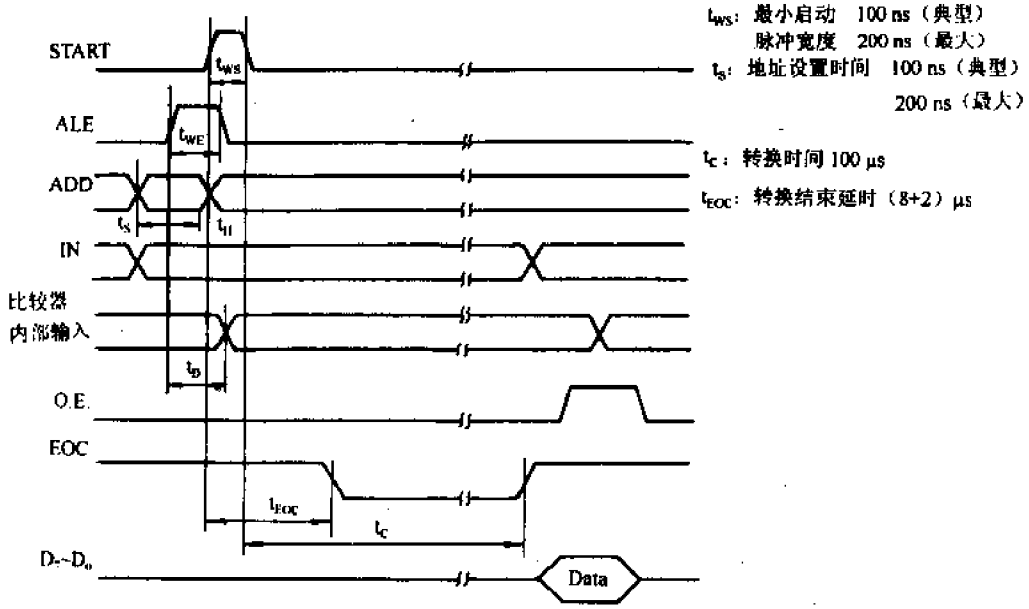


图 9.15 ADC 0809 的时序

从时序图可看出 ADC 0809 的启动信号 START 是脉冲信号,也即此芯片是靠脉冲启动

的。当模拟量送至某一输入通道后，由三位地址信号译码选择，地址信号由地址锁存允许 ALE 锁存，高电平有效。当转换完成后，输出转换结束信号 EOC 由低电平变为高电平有效信号。外界的输出允许信号 OE(高电平有效)打开三态缓冲器的门，把转换结果送到数据总线上。使用时可利用 EOC 信号短接到 OE 端，也可利用 EOC 信号向 CPU 申请中断。

四、ADC 芯片的应用

(一)ADC 芯片与 CPU 接口电路的设计

ADC 芯片与 CPU 的连接方式通常有下列三种：

(1)与 CPU 直接连接。

若 ADC 芯片内部包含有数据输出寄存器和三态门时，它们的输出端可以直接与 CPU 的数据总线连接。

(2)通过三态门与 CPU 连接。

若 ADC 芯片内部不包含有数据输出寄存器和三态门时，需外接三态门锁存器后才能与 CPU 数据线连接。

(3)利用 I/O 接口芯片与 CPU 连接

这是较常用的一种连接方式，利用 I/O 接口芯片与 CPU 连接，不仅使用方便，而且在时序配合上和驱动能力上容易与 CPU 一致。

在采样速度要求较高的场合，可以利用 DMA 控制器传送数据，将外设采样到的数据直接传送至存储器，无需 CPU 的干预。

(二)A/D 转换器应用举例

ADC 0809 为 8 通道 A/D 转换器，适合应用于多通道的数据采集系统。图 9.16 给出了数据采集系统的硬件连接图。由于 ADC 0809 芯片内部有数据输出寄存器和三态门，故可以与 CPU 直接连接。

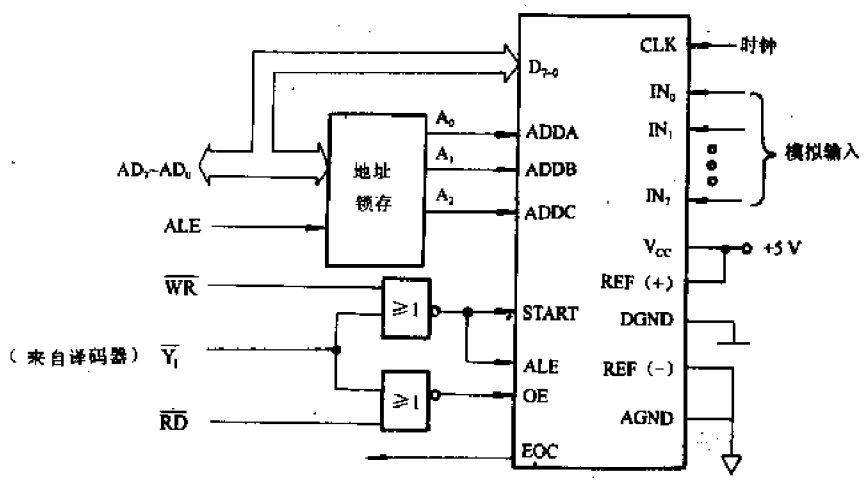


图 9.16 ADC 0809 数据采集系统硬件连接图

图中将地址锁存信号 ALE 和启动转换信号 START 连接在一起，由写信号$\overline{WR}$和片选信号$\overline{Y}_i$产生，只要向该芯片进行一次写入(执行 OUT 指令)操作，就可以使启动信号 START 有效，开始启动转换，待转换结束时，转换结束信号 EOC 由低变为高电平，此信号可用于查询或

请求中断读取数据。当 CPU 进行一次读出(执行 IN 指令)操作时,OE 信号有效,开启内部的三态门,将输出寄存器中的数据送到数据总线上供 CPU 读取。时钟 CLK 由 CPU 时钟经分频后提供。

设 ADC 0809 的 8 个输入通道 IN_0～IN_7 的端口地址为 FE80～FE87H,若把 8 路模拟输入转换的数字量以中断方式循环读入内存输入缓冲区 IBUF 中,A/D 转换程序设计如下:

```
START:  LEA  DI,IBUF
        MOV  DX,0FE80H
        MOV  CX,08H
        STI
NEXT:   OUT  DX,AL          ;产生WR和Yi启动转换
        HLT                 ;等待中断
        MOV  [DI],AL        ;转换后的数据存 IBUF 缓冲区
        INC  DI
        INC  DX             ;下一通道
        LOOP NEXT           ;循环启动转换并读入转换的结果
          ⋮
        中断处理程序
INT_P:  IN AL,DX            ;产生RD和Yi读转换结果至 AL 中
        STI                 ;开中断
        IRET                ;中断返回
```

除了用中断方式读取 ADC 芯片转换的数据外,还可用程序查询和延时等待方式读取数据。

下面再举一个 ADC 0809 通过并行 I/O 接口 8255A 与 32 位 CPU 连接,并使用程序查询方式读取转换结果的例子。硬件连接如图 9.17 所示。

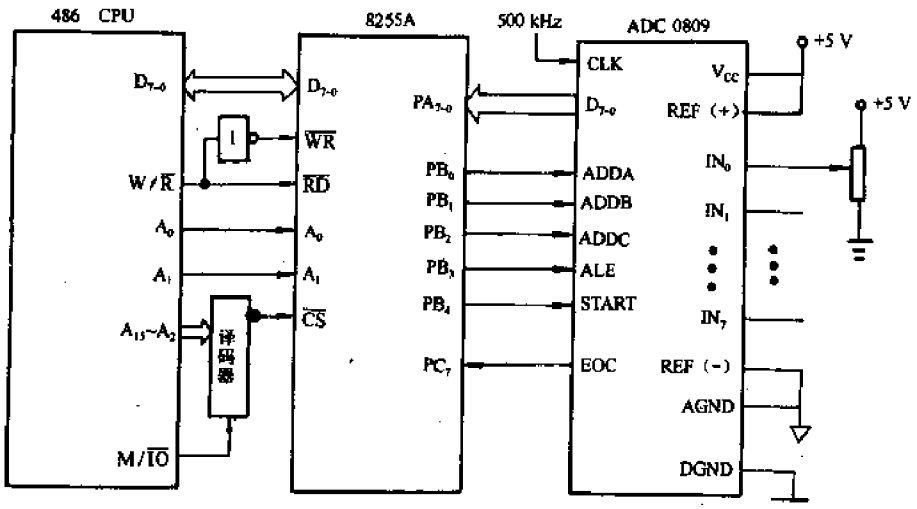


图 9.17

ADC 0809 转换的数据由 8255A 的 A 口传送给 CPU，端口地址、地址锁存和启动转换信号 CPU 通过 8255A 的 B 口送 ADC 0809，转换结束信号 EOC 由 8255A PC_7 输入，供 CPU 查询。模拟信号由 IN_0 输入。设 8255A 的端口地址为 FFE0～FFE3H　A/D 转换程序设计如下：

```
START:MOV   AL,98H          ;A 口方式 0 输入,B 口方式 0 输出,PC7 输入
      MOV   DX,0FFE3H
      OUT   DX,AL           ;写 8255A 控制字
      MOV   AL,08H
      MOV   DX,0FFE1H
      OUT   DX,AL           ;送 IN0 地址和 ALE 信号
      MOV   AL,18H
      OUT   DX,AL           ;送启动转换信号 START,启动 A/D 转换
      INC   DX
TTEST:IN    AL,DX           ;读 PC7,查询 EOC 是否有效
      TEST  AL,80H
      JZ    TTEST
      MOV   DX,0FFE0H
      IN    AL,DX           ;读转换结果至 AL 中
      HLT
```

第四节　模拟通道的抗干扰措施

在微型计算机和模拟器件接口的实际应用中，一般都有相互干扰的问题，这是应用中的实际问题，处理不当的话，有时极为扰人，不仅精度和灵敏度不可能实现，甚至无法稳定可靠地工作。解决干扰问题，主要采用光电隔离、正确接地和其它抗干扰措施，并且需要一定的实际经验。

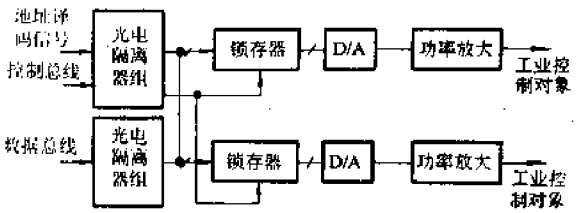


图 9.18　高抗干扰 D/A 系统

(一)计算机总线光电隔离

图 9.18 为一高抗干扰工业控制系统框图。图中被控对象是强电系统，输出线上常有很强的干扰，地线中也常出现较大的干扰电流。若无隔离措施，它们将通过器件耦合和地线回路干扰计算机的正常工作。在计算机总线上加入光电隔离器即光耦合器，便使计算机与被控系统在电气上完全隔离，消除了强电干扰。

(二)正确的地线连接

集成电路生产厂家在设计 ADC 和 DAC 芯片时，已将模拟地和数字地分别用不同的引脚引出。在线路设计中，应将一个系统中所有模拟地和数字地各自分别相连，然后全部电路中的模拟和数字两地最后仅在一点 G 处连接，见图 9.19。这样便可消除模拟电路和数字电路通过地线回路的相互干扰。

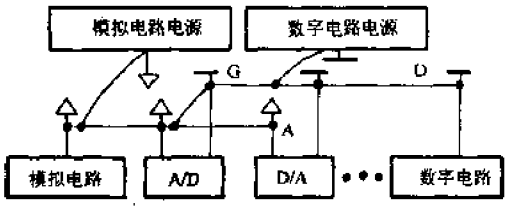


图 9.19　正确的地线连接

(三)其它抗干扰设计方法

除上述计算机总线光电隔离和正确接地两项重要的抗干扰措施之外，还有各种面向实际问题而针对性很强的抗干扰设计，它往往成为过程通道(或控制接口)设计的重要内容，而且往往成为微机系统可靠应用的重要条件。下面对有关方面作简要介绍。

1. 干扰的主要渠道

干扰渠道有三方面：空间电磁场，配电系统，以及主机与外部设备之间的过程通道。前者用良好的屏蔽和正确地接"地"即可解决。配电系统则只要把传统的"交流稳压→低通滤波→直流稳压"三级配电设置改成先进的"分布参数低通滤波(把一捆几十米长的绞扭导线装进屏蔽箱)→非磁饱和低通滤波(DL－3A)→正弦波恒压变压器→直流稳压"四级配电设置即可得到"干净的"电源系统。最后，过程通道的抗干扰设计成为重点。

2. 抗干扰器件的选择

就过程通道而言，采控设备与主机相连，强电与弱电交接，无论开关量输入/输出或模拟量输入/输出，都是干扰的渠道。要截断干扰渠道，就要去掉强弱电之间的公共地线，以实现"电隔离"。可选的隔离器件视功率不同有光电耦合器、继电器和变压器。小功率情况的隔离，则光电耦合器因其密封耦合，输入阻抗低(10^2 级)及绝缘电阻大(10^{12}级)的优良隔离性能成为首选(有的也可选用微型继电器)。图 9.20 示出了一个二极管-三级管型光电耦合器。使用时只需接上适当的 R_D 以提供驱动发光二极管的电流和接上适当的 R_C 以在限定的工作电流下获得所需的集电极电平即可。前述的计算机总线光电隔离以及过程通道的许多抗干扰设计都要用到它。

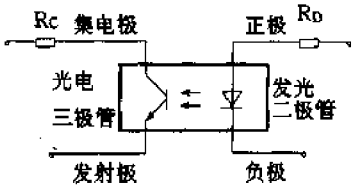


图 9.20　二极管-三极管型光电耦合器

3. 过程通道的抗干扰设计方法

因篇幅所限，此处只把有关过程通道的各种抗干扰设计方法概列如下，至于其中的细节，读者可在实际应用时再查阅有关详细资料。

(1)开关量输入/输出抗干扰设计

①小负荷、低响应：OC 门驱动微小型继电器隔离等；

②大负荷、低速度：小型继电器隔离触发大功率可控硅无触点控制；

③快速驱动直流负载：光电隔离驱动晶体管控制可控硅等；

④快速驱动交流负载：光隔离驱动晶体管控制双向可控硅等；

(2)模拟量输入/输出抗干扰设计

①抗干扰 A/D：光隔离设在 A/D 输出数据与 CPU 接口的数据总线缓冲器及控制总线缓冲器之间；

②抗干扰 D/A：光隔离设在 CPU 数据总线与 D/A 输入数据缓冲器及控制总线缓冲器之

间；

③遥控接口传输长线抗干扰设计：两端隔离的完全“隔、浮”处理（中间隔浮线必要时采用差分接收、发送）等。

思考题与习题

1. 在一个典型的微机控制系统中，输入和输出通道主要由哪些部件组成？试分别说明它们在通道中的作用。

2. D/A 转换器的转换工作原理是什么？性能指标和外部特性有哪些？

3. D/A 转换器与 CPU 连接时，通常有哪几种接口形式？

4. 试简述 DAC0832 三种工作方式的特点及其应用场合。

5. 试分别说明逐次逼近式和双积分式 A/D 转换原理的工作过程。

6. ADC 0809 芯片的主要性能特点有哪些？

7. 通常可通过哪几种编程方式读取 A/D 转换结果？试各举一例。

8. 设某系统采用 DAC 0832 产生连续的锯齿波形，试画出 DAC 0832 与 CPU 的连接图，并编写 D/A 转换程序。

9. 试设计一个由两片 DAC 0832 组成的两路模拟量同步输出的接口电路，要求两路模拟量的输出分别接至示波器的 X、Y 输入端。编写程序，能在示波器上输出显示字母“W”的图象。

10. 利用 DAC 1210 设计一个 D/A 转换电路，能将给定的 32 位数据转换输出，并写出 D/A 转换程序。

11. 使用 ADC 0809 的 IN_3 通道采集模拟信号，要求每隔 100ms 采集一次，共采集 100 次，将采集的数据存入内存首地址为 2000:0000H 开始的数据区。试编写数据采集程序。

12. 利用 PC 机标准扩展槽设计一个 8 通道 A/D 采集板。A/D 转换器采用 ADC 0809，将其转换为数字量后，通过 8255A 的端口 PA 传送给 CPU，并存入存储器。

已知条件如下：

(1)8255A 的口 A 工作于方式 0，用于接收 ADC 0809 的转换结果；

(2)通过 8255A 的 PC 口的低 4 位实现 ADC 0809 的 8 路输入端口的切换；

(3)通过 8255A 的 PC4 来判断 A/D 转换是否结束；

(4)8255A 与 CPU 间的数据传送采用查询方式；

(5)PC 机标准扩展槽 I/O 端口地址是 10 位、8255A 的控制口地址为 243H　地址译码采用 74LS138 完成或自己选用标准 74 系列芯片完成；

(6)PC 扩展槽中，OSC 引脚的频率为 14.31818MHz，SYSCLK 引脚的频率为 4.77MHz，ADC 0809 的时钟信号可由 SYSCLK 经 8253 分频或选用标准 74 系列芯片完成。

要求：

(1)完成系统的硬件连接设计，可根据需要自行选择接口芯片；

(2)根据硬件连接，编写初始化程序；

(3)编写循环采集 8 路模拟量 5 次，并存入内存首地址为 4000:0000H 中的采集服务程序。

第十章　Pentium 微处理器

90 年代初，Intel 公司推出了 Pentium 微处理器，由它构成的计算机称为 586 微型计算机。

Pentium 是一种 64 位微处理器，其兼容性，数据完整性和灵活的升级能力，使它性能表现卓越。Pentiam 不仅继承了 80486 的优点，而且对 80486 作了许多重大的改进。

(1)Pentium 采用超标量体系结构，内含三个独立的处理单元：一个浮点处理器和两个整型处理单元。在一个时钟周期内可执行两条整数运算指令和一条浮点运算指令。

(2)增加了分支预测逻辑以减少分支导致的时间消耗。

(3)含有指令和数据两个独立的 8KBCache，使预取指令和数据分别独立，提高了微处理器的执行速度。

(4)数据总线从 32 位增加到 64 位，存储系统分为 8 个存储块，使经总线访问内存数据的速度大大提高。

(5)提供了灵活的存储器页面管理，可支持 4KB 和 4MB 的存储页面。

(6)引入大型计算机中采用的错误检测、功能冗余校验和错误报告等自诊断功能；并提供了性能监察功能，以利于软、硬件产品的优化和升级。

(7)增加了用于控制存储器管理模式特性和串行化指令。

Pentium 采用 66MHz 的时钟，由于作了以上的主要改进，其执行指令的速度比采用同样时钟的 80486 快 5 倍左右。

第一节　Pentium 微处理器的芯片引脚

图 10.1 是 Pentium 芯片的外部引脚图，从图中可看到，该芯片共有 237 个引脚。目前，Pentium 微处理器有两种型号：全功能型和 Pentium P24T 型。后者具有 32 位数据总线，可以插在具有 P24 插槽的 80486 机器上。下面对 Pentium 芯片引脚作一简要介绍。Pentium 引脚的最大特点在于它具有 64 个数据总线连接点而不是 32

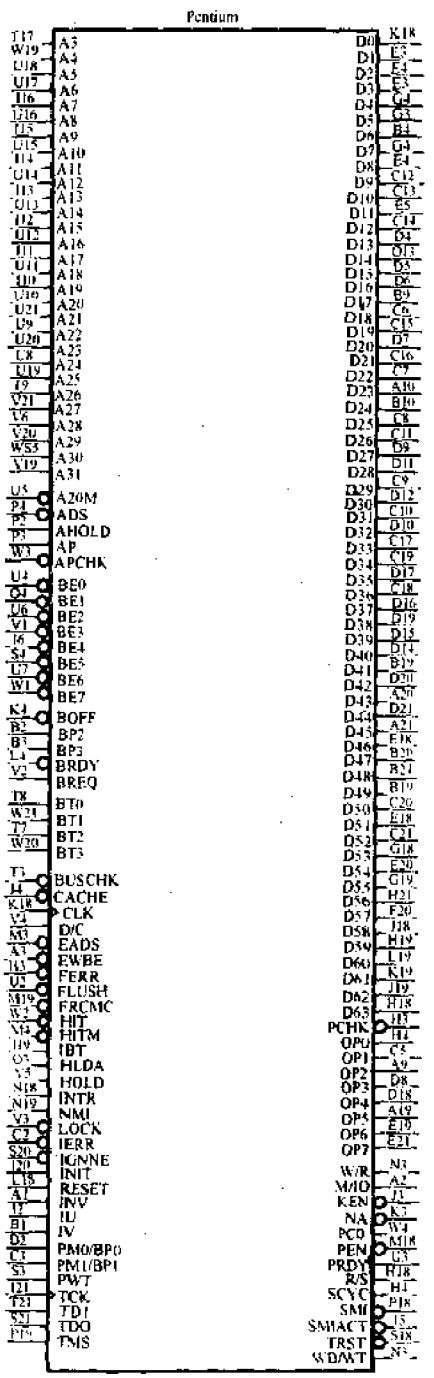


图 10. 1　Pentium 微处理器的外部引脚

个，这就要求 Pentium 有更大的物理空间来放置这些引脚。

与早期的 Intel 微处理器家族成员一样，Pentium 的早期型号需要一个单一的＋5V 电源，电源电流平均值：66MHz Pentium 为 3.3A，60MHz Pentium 为 2.91A。由于电流较大，所以这些微处理器的耗电量也不小：66MHz Pentium 的功率是 13W，而 60MHzPentium 的功率11.9 W。当前型号的 Pentium，90MHz 及以上的 Pentium，采用 3.3V 电源供电，因而减少了电能消耗。目前 Pentium 需用一个具有良好通风的散热器来保证 Pentium 芯片不致过热。Pentium 具有多个 Vcc 和 Vss 连接点，这些连接点必须根据情况连到＋5.0V 或＋3.3V 电源上，并且正确接地。最新型 Pentium 已就功耗问题作了改进，比如 133MHz 的 Pentium 只要求 3.4A 电流，仅比 66MHz Pentium 所需的 3.3A 大一点。

每个 Pentium 输出引脚在逻辑 0 时提供 4.0mA 电流，而在逻辑 1 时提供 2.0mA 电流。与早期的 8086、8088 和 80286 相比，Pentium 的驱动电流增加了。每个 Pentium 输入引脚仅需要 15μA 的负载电流。在一些系统中，除了最小系统，这些电平需要总线缓冲。

每个 Pentium 引脚组的功能如下：

$\overline{A20}$	地址 A20 屏蔽(address A20 mask)是一个输入引脚，用于在实模式中给 Pentium 发信号进行地址保护，就像在 8086 微处理器中一样，它供 HIMEM.SYS 驱动程序使用。
A31—A3	地址总线(addressbus)连接引脚用于寻址 Pentium 存储系统中任何 512KB×64 存储空间，注意 A0、A1 和 A2 在总线允许($\overline{BE7}$—$\overline{BE0}$)中被编码用于选择 64 位宽存储空间中的任意一个或全部 8 个字节。
$\overline{ADS}$	当 Pentium 发出一个有效的存储器地址或 I/O 地址，地址数据选通(address data strobe)就变为有效。这个信号与 W/$\overline{R}$ 和 M/$\overline{IO}$信号一起产生在早期 8086～80286 系统中有过的独立的读信号和写信号。
AHOLD	地址保持(address hold)是个输入引脚，用于使 Pentium 为下一个时钟周期保持地址和 AP 信号。
AP	地址校验(address parity)引脚为所有的 Pentium 存储器和 I/O 传送器提供偶校验。在所有的请求周期中 AP 引脚必须在与$\overline{EADS}$信号相同的时钟周期内由偶校验信息驱动。
$\overline{APCHK}$	地址校验检查(address parity check)引脚，当 Pentium 检查到一个地址校验错时变为逻辑 0。
$\overline{BE7}$—$\overline{BE0}$	体允许信号(bank enable signals)用于选择访问字节、字、双字还是四字数据。这些信号在微处理器内由地址值 A0、A1 和 A2 产生。
$\overline{BOFF}$	Back-off 输入信号中止所有总线周期并使总线悬浮直到$\overline{BOFF}$信号变为负，当$\overline{BOFF}$为负，Pentium 就重新启动所有中止的总线周期。
BP[3：2]和 PM/BP[1：0]	当排错寄存器被编程来监测寻求匹配时，断点引脚(breakpoint pins)组 BP3—BP0 用以指示一个断点。性能监控(performance monitoring)引脚 PM1 和 PM0 用以指示排错模式控制寄存器的性能监控位的设置。
$\overline{BRDY}$	猝发(burst)就绪信号告知 Pentium 处理器外部系统已从数据总线连接中取得数据。此信号用于向 Pentium 时序中插入等待状态。
BREQ	总线请求输出信号说明 Pentium 已产生了一个总线请求。

BT3—BT0	分支跟踪(branch trace)输出引脚提供分支目标的线性地址 2~0 位,并在 BT3 位上提供缺省操作数长度。这些输出信号在一个分支跟踪特殊消息周期中变为有效。
$\overline{\text{BUSCHK}}$	总线检查(bus check)输入引脚允许系统向 Pentium 发送信号告知总线传送失败。
$\overline{\text{CACHE}}$	cache 输出信号指示当前 Pentium 周期可以对数据进行缓存。
CLK	时钟引脚由具有当前 Pentium 操作频率的时钟信号驱动,例如,为了使 Pentium 工作于 66MHz,我们将 66MHz 时钟加于此引脚。
D63—D0	数据总线(data bus)连接引脚在微处理器、内存和 I/O 系统间进行字节、字、双字和四字数据的传送。
D/$\overline{\text{C}}$	数据/控制(data/control)信号为逻辑 1 时表明数据总线中包含从存储器或 I/O 中取得的数据或将送往这二者的数据。如果 D/$\overline{\text{C}}$ 为逻辑 0,则表明微处理器或者处于暂停态或者正在执行一个中断请求。
OP7—DP0	数据校验(data parity)信号由 Pentium 产生,用于检查它的 8 个存储块的数据。
$\overline{\text{EADS}}$	外部地址选通(external address strobe)输入信号指示地址总线包含一个请求周期的地址。
$\overline{\text{EWBE}}$	外部写缓存空(external write bufer empty)输入信号指示写周期在外部系统中处于悬空状态。
$\overline{\text{FERR}}$	浮点错(floating-point error)信号与 80386 中的$\overline{\text{ERROR}}$信号兼容,它用来指示内部协处理器出现错误。
$\overline{\text{FLUSH}}$	清 cache 输入信号使缓存清除所有回写行并使内部 cache 无效。如果在进行重启操作时。$\overline{\text{FLUSH}}$为 0,则 Pentium 进入到测试模式。
$\overline{\text{FRCMC}}$	功能性冗余检查(functional redundancy check)信号在重启时配置 Pentium 为主模式(0)或检查模式(1)。
$\overline{\text{HIT}}$	命中信号指示在查询方式中内部 cache 包含合法数据。
$\overline{\text{HITM}}$	命中修改信号表明在查询周期中发现了一个修改过的 cache 行,此输出信号用于在已修改过的 cache 行写回到存储器之前禁止其他单元访问数据。
HOLD	保持信号请求一个 PMA 操作。
HLDA	保持响应(hold acknowledge)信号指示 Pentium 当前处于保持态。
IBT	指令分支采用(instruction branch taken)信号表示 Pentium 采用了一个指令分支。
$\overline{\text{IERR}}$	内部错(internalerror)输出信号表明 Pentium 已检测到一个校验错或功能性冗余错。
$\overline{\text{IGNNE}}$	忽略数字错(ignore numeric error)输入信号使 Pentium 忽略协处理器错误。
INIT	初始化(initialization)输入信号用于执行重启,但并不初始化 cache、回写缓冲区和浮点寄存器,此信号在加电后不能用于代替 RESET 信号进行处理器的重启。
INTR	中断请求(interrupt request)信号用于外部电路进行中断请求。

INV	使无效(invalidation)输入信号决定一个请求后的 cache 行状态。
IU	U-流水线指令完成(U-pipe instruction complete)输出信号表明 U-流水线中的指令已完成。
IV	V-流水线指令完成(V-pipe instruction complete)输出信号表明 V-流水线中的指令已完成。
$\overline{\text{KEN}}$	caehe 允许输入信号使内部缓存可用。
$\overline{\text{LOCK}}$	锁定信号在指令前置 LOCK：前辍时变为逻辑 0，这在 DMA 访问中常用。
M/$\overline{\text{IO}}$	存储器/IO 信号为逻辑 1 时选择一个存储器设备，而在为逻辑 0 时选择一个 I/O设备，I/O 操作过程中，地址总线在 A15～A3 上包含 16 位的 I/O 地址。
$\overline{\text{NA}}$	下一地址(next address)信号表明外部存储器系统已准备好接收一个新的总线周期。
NMI	非屏蔽中断(non－maskable interrupt)信号请求一个非屏蔽中断，这与早期版本的微处理器相同。
PCD	页缓存禁止(page cache disable)输出信号通过 CR3 PCD 位的状态表示页缓存被禁止。
$\overline{\text{PCHK}}$	奇偶校验检查(parity check)输出信号指示从存储器或 I/O 读数据出现校验检查错误。
$\overline{\text{PEN}}$	奇偶校验允许(parity enable)输入信号使机器检查中断或异常。
PRDY	探针(probe)就绪输出信号表明已为排绪准备好探针方式。
PWT	页写通输出信号在 CR3 上显示 PWT 位的状态。
R/$\overline{\text{S}}$	此引脚与 Intel 查错端口一起使用并产生一个中断。
RESET	重启信号初始化 Pentium，使其在内存 FFFFFFF0H 处开始执行软件，Pentium 被置为实模式，最左边的 12 条地址线保持逻辑 1(FFFH)，直至执行一个远程调用或跳转，这需要与早期微处理器具有兼容性，执行一个硬重启后 Pentium 的状态请参考表 10.1。
SCYC	分割周期(split cycle)输出信号指示一个误排的锁定总线周期。
$\overline{\text{SMI}}$	系统管理中断(system management interrupt)输入信号使 Pentium 进入系统管理操作模式。
$\overline{\text{SMIACT}}$	系统管理中断激活(system management interrupt active)输出信号表明 Pentium 正工作在系统管理模式。
TCK	可测试性时钟(testability clock)输出信号根据 IEEE1149.1 边界检测接口选择时钟操作。
TDI	测试数据(test data)输入信号用来测试由 TCK 信号打入 Pentium 的数据。
TDO	测试数据(test data)输出信号用来获得由 TCK 移出 Pentium 的测试数据和指令。
TMS	测试方式选择(test mode select)输入信号在测试模式中控制 Pentium 操作。
$\overline{\text{TRST}}$	测试重启(test reset)输入信号允许测试模式被重启。

WB/$\overline{WT}$　　写回/写道(write-back/write-through)信号为 Pentium 数据 cache 选择相应操作。

表 10.1　RESET 后 Pentium 的状态

寄　存　器	RESET 值	RESET+BIST 值
EAX	0	0(如果测试通过)
EDX	0500XXXXH	0500XXXXH
EBX,ECX,ESP,EBP,ESI,and EDI	0	0
EFLAGS	2	2
EIP	0000FFF0H	0000FFF0H
CS	F000H	F000H
DS,ES,FS,GS,and SS	0	0
GDTR and TSS	0	0
CR0	60000010H	60000010H
CR2,CR3 and CR4	0	0
DR0－DR3	0	0
DR6	FFFF0FF0H	FFFF0FF0H
DR7	00000400H	00000040H

注:BIST=内置自检;XXXX=Pentium 型号。

第二节　Pentium 微处理器的存储器系统和管理

一、Pentium 微处理器的存储器系统

Pentium 微处理器的存储器系统大小为 4GB,与 80486 的差别在于存储器数据总线的宽度为 64 位。4GB 的存储器分成 8 个存储块,每块 512MB,每个存储块都有一个含有一位校验位的字节数据。Pentium 采用内部校验发生和检查逻辑来获得存储器系统的数据总线信息。64 位宽的存储器对于双精度浮点型数据的传输是非常方便的,Pentium 存储器系统的字节计数从 00000000H 到 FFFFFFFFH。

存储块的选择由块允许信号(BE7～BF0)完成,这些单独的存储块允许 Pentium 在一个存储器传送周期中存取任何一个字节、字、双字或四字数据。

对于 32 位的存储器系统,Pentium 可通过一个多路器将 64 位数据总线转换为 32 位数据总线,从而可以同 32 位宽的存储器系统一起工作。

Pentium 的另一个功能是能对地址总线 A_{31}～A_5 进行检验,AP3 脚为系统提供校验信息,$\overline{APCHK}$引脚指示地址总线的错误,对于地址校验中的错误,若需要系统可采取中断方式处理。

二、Pentium 存储器的管理

Pentium 存储器管理单元与 80486 的存储单元兼容。其改变在于分页单元和系统的存储器管理模式

1. 分页单元

Pentium 的页管理机制可以工作于 4KB 或 4MB。在 80486 微机系统中,为了对 4GB 存储器完全重新分页,大约需要 4MB 的内存来存效页表。在 Pentium 中,由于使用了 4MB 页特性,只需要单一的页表,从而减少了内存用量。新的 4MB 页的尺寸由控制寄存器 0 的 PSE 位选

择。

4MB 页和 4KB 页的主要区别是 4MB 页方式中没有存储线性地址的页表入口，而是通过页目录来寻址每个包含 4MB 内存的 1024 个页，从而节省了内存。

2. 存储器管理模式

Pentium 增加了存储器管理模式(SMM)。SMM 不是用作应用程序的管理，而是用于主级系统中，如电源管理和安全性等。

对 SMM 的访问是通过接在 $\overline{SMI}$ 引脚的一个外部硬件中断来实现的。当硬件中断激活时，微处理器将处理系统管理 RAM 或 SRAM 内存区域的系统级软件。SMM 中断使用$\overline{CS}$=3000H 和 EIP=8000H 来调用最初处于 38000H 存储单元开始的软件，这个初始状态可以通过一个跳转指令转到第一个 1MB 内存中任何处加以改变。管理方式中断进入的环境类似于实地址方式，但又有所不同，因为 SMM 不仅能寻地 1MB 内存，而且是将存储器系统作为 4GB 系统来处理的。

在执行始于 38000H 处的软件时，SMM 中断将把 Pentium 的状态存储于一个转储记录中(dump record)，转储记录存储在 3FFA8H～3FFFFH 处，其中 3FEOOH～3FEF7H 由 Intel 保留，如表 10.2 所示。

表 10.2 Pentium SMM 状态转储记录

地址	寄存器	地址	寄存器
3FE00－3FEF7H	停留	3FFC8H	DR7
3FEF8H	状态转储基地址	3FFCCH	DR6
3FEFCH	SMM 修正标志	3FFDOH	EAX
3FF00H	I/O 陷阱重启动	3FFD4H	ECX
3FF02H	暂停自动重启动	3FFD8H	EDX
3FF04H～3FFA7H	保留	3FFDCH	EBX
3FFA8H	ES	3FFEOH	ESP
3FFACH	CS	3FFE4H	EBP
3FFBOH	SS	3FFE8H	ESI
3FFB4H	DS	3FFECH	EDI
3FFB8H	FS	3FFF0H	EIP
3FFBCH	GS	3FFF4H	EFLAGS
3FFC0H	LDTR	3FFF8H	CR3
3FFC4H	TR	3FFFCH	CR0

暂停自动重启和 I/O 陷阱重启用于执行 RSM 指令退出 SMM 模式时，这些数据使 RSM 指令回到暂停状态或返回到中断 I/O 指令。I/O 在 SMM 模式下，暂停操作和 I/O 操作都没有发生，则 RSM 指令从状态 dump 中重新装入机点状态并返回到中断点。

SMM 方式可在通常的操作系统中装入内存，并在执行之前使用。

第三节 Pentium 对 80486 的几项重大改进

一、超标量体系结构

Pentium 微处理器由一个浮点部件部件、两个整型指令执行部件(U 流水线和 V 流水线)组成。这意味着 Pentium 可同时执行 3 条指令。例如：当协处理器执行 FADDST，ST(2)时，U 流水线可执行 MOV EAX，10H，而同时 V 流水线可执行 MOVEBX，12H 指令。在编写软件时应当充分利用这个特性。如果应用得当，Pentium 可提高 40%的执行速度。

二、Cache 结构

Pentium 的 Cache 结构与 80486 不同，它采用两个独立的 8KB Cache，一个 8KB Cache 只存储指令，而另一个 8KB Cache 只存储指令所需的数据。在 80486 中，由于只有一个 8KB 的 Cache，很快就被程序的数据占满，而没有空间存放指令，从而降低了微机的执行速度。而在 Pentium 中，由于有了单独的指令 Cache，不会发生 80486 中出现的情况，提高了微机的执行速度。

三、分支预测逻辑

Pentium 微处理器采用分支逻辑来减少由分支程序分支导致的时间消耗。当程序中遇到分支指令时，Pentium 微处理器可在分支地址处进行指令的预取，这些指令被装入指令 cache 中，因而当执行分支时，指令 2 在 cache 中，这样可使分支在一个时钟同期内执行完毕。当分支预测逻辑由于某些特殊的原因发生错误时，分支需再增加 3 个时钟周期来执行，但大多数情况下，分支预测逻辑不会发生这样的错误，故不会有时间的延误发生。

第四节　Pentium 的寄存器和新指令

一、寄存器

Pentium 除了控制寄存器和标志寄存器和 80486 相比有了改变外，其余的寄存器与 80486 基本相同。

(一)Pentium 微处理器新增加了 CR_4 控制寄存器，并添加到控制器陈列中。

CR_4 是 32 位的控制寄存器，共用了六位，下面加以介绍。

- VME(D_0 位)　虚拟方式扩展位(Virtnal made extension)，在保护方式下允许对虚拟中断标志的支持。当 VME＝0 时，虚拟中断支持被禁止。
- PVI(D_1 位)　保护方式虚拟中断位(protected mode virtual interropt)，在保护方式下允许对虚拟中断标志的支持。
- TSD(D_2 位)　时间标记禁止(time stamp disable)，控制 RDTSC 指令。
- DE(D_3 位)　排错扩展(debugging extension)，设置时允许 I/O 断点。排错扩展
- PSE(D_4 位)　页尺寸扩展(page size extension)，设置时允许 4MB 存储页。
- MCE(D_6 位)　机器检查允许(machine chock enable)，激活机器检查中断。

(二)Pentium 在 80486 的 EFLAG 标志寄存器中增加了以下几位

- ID(D_{21}位)　用于 CPUID 指令的检测。
- VIP(D_{20}位)　虚似中断挂起位。
- VIF(D_{19}位)　虚拟中断位，与 VIP 一起使用。

二、Pentium 新增加的指令

Pentium 的新指令用于控制存储管理模式特性和串行化指令。

1. CMPXCHG8B 指令

功能：比较和交换 8 个字节。

例：CMPXCHG8B DATA1

将存储于内存地址 DATA1 处的 8 个字节和存于 EDX:EAX 中的 64 位数作比较，若 DATA1 中的数等于 EDX:EAX 中的数，则将存于 ECX:EBX 的 64 位数存到内存地址 DATX1 处；若不等，则将 DATA1 地址处的数存到 EDX:EAX 中。

2. CPUID

功能：从 Pentium 中读出 CPU 标识码和其它一些信息。

执行 CPUID 指令前，先将输入值装入 EAX，执行后将返回信息存入表 10.3 所示的寄存器中。

3. RDTSC

功能：将时间标记计数读入 EDX:EAX，时间标计计数器 TSC 从微处理器重新启动时开始计数 CPU 的时钟。

TSC 是 64 位计数器，一个 100MHZ 的 Pentium 处理器在重新开始计数前可计数 5800 年。

RDTSC 指令只在实地址方式或保护方式特权级 0 下运行。

4. RDMSR 和 WRMSR

功能：读或写特殊格式寄存器。

特殊格式寄存器是 Pentium 特有的，其用途是跟踪、检查性能、测试和查找机器的错误。RDMSR 和 WRMSR 使用 ECX 给微处理器传递寄存器号，使用 EDX:EAX 为 64 位数据的读或写服务。

表 10.4 列出了 Pentium 的特殊格式寄存器和它们的内容。RDMSR 和 WRMSR 指令只能在实地址方式和保护方式特权级 0 下操作。

表 10.3 CPUID 指令执行结果

输入值	CPUID 执行的结果
0	EA=1 用于所有微处理器
	EBX－EDX－ECX＝销售商标识
1	EAX(位 3～0)＝步进标识
	EAX(位 7～4)＝模型
	EAX(位 11～8)＝家族
	EAX(位 13～12)＝类型
	EAX(位 31～14)＝保留
	EDX(位 0)＝CPU 包含 FPU
	EDX(位 1)＝支持增强 8086 虚拟模式
	EDX(位 2)＝支持 I/O 断点
	EDX(位 3)＝支持页尺寸扩展
	EDX(位 4)＝支持 TSC 计数器
	EDX(位 5)＝支持 MSR
	EDX(位 7)＝支持机器检查异常
	EDX(位 8)＝支持 CMPXCHG8B
	EDX(位 9)＝3.3V 微处理器
	EDX(其它位)＝保留

表 10.4 Pentium 特殊格式寄存器

地址(ECX)	尺寸	功　能
00H	64 位	机器检查异常地址
01	5 位	机器检查异常类型
02	14 位	TR1 校验测试寄存器
04	4 位	TR2 指令 Cache 结束位
05H	32 位	TR3 cache 数据
06H	32 位	TR4 cache 标记
07H	15 位	TR4 cache 控制
08H	32 位	TR6 TLB 命令
09H	32 位	TR7 TLB 数据
OBH	32 位	TR9 BTB 标记
OCH	32 位	TR10 BTB 目标
ODH	12 位	TR11 BTB 控制
OEH	10 位	TR12 新特性控制
10H	64 位	时间标记计数器
11H	26 位	事件计数器选择和控制
12H	40 位	事件计数器 0
13H	40 位	事件计数器 1

使用这两条指令前，必须先对 ECX 赋值。例如，若 ECX＝OEH，则测试寄存器 TR12 被访问。注意 IR_1～IR_{12}是用于内部测试的内部寄存器，这些寄存器不能用在通常的编程中。

5. RMS

功能：从一个存储器管理模式中断中返回。有关存储总管理模式 SMM 前面已经简要介绍，这里不再重复。

附录 A　80486 指令系统表

类别	指令	功能
数据传送指令	MOX	传送
	XCHG	交换
	IN	字节和字输入
	OUT	字节和字输出
	LEA	装入有效地址
	LDS	指针装入 DS
	LES	指针装入 ES
	LFS	指针装入 FS
	LGS	指针装入 GS
	LSS	指针装入 SS
	XLAT	完成一个字节的查表转换
	PUSH	将操作数压栈
	POP	将操作数弹出栈
	PUSHA	将全部 16 位通用寄存器压栈
	PUSHAD	将全部 32 位通用寄存器压栈
	POPA	将全部保存的 16 位寄存器值弹出栈
	POPAD	将全部保存的 32 位寄存器值弹出栈
	MOVSX	带符号扩展传送
	MOVZX	带零扩展传送
	BSWAP	字节交换
算术运算指令	ADD	整数加法
	SUB	整数减法
	ADC	带进位整数加法
	SBB	带借位整数减法
	INC	加 1
	DEC	减 1
	NEG	整数变反
	CMP	比较
	CHPXCHG	比较并交换
	MUL	无符号数乘法
	IMUL	有符号数乘法
	DIV	无符号数除法
	IDIV	有符号数除法
	AAA	未组合 BCD 码加后调整
	AAS	未组合 BCD 码减后调整
	AAM	未组合 BCD 码乘后调整
	AAD	未组合 BCD 码除后调整
	DAA	组合 BCD 码加后调整
	DAS	组合 BCD 码减后调整
	CBW	将字节带符号扩展成方
	CWD	将字带符号扩展为双字，存 DX，AX 中
	CWDE	将字带符号扩展为双字，存 EAX 中
	CDQ	将双字带符号扩展为 64 位，存 EDX，EAX 中

类别	指令	功能
逻辑运算与移位指令	AND	逻辑“与”
	OR	逻辑“或”
	XOR	逻辑“异或”
	NOT	逻辑“非”
	TEST	测试
	SAL	算术左移
	SAR	算术右移
	SHL	逻辑左移
	SHR	逻辑右移
	SHLD	双精度左移
	SHRD	双精度右移
	ROL	循环左移
	ROR	循环右移
	RCL	带进位循环左移
	RCR	带进位循环右移
串操作指令	MOVS	字节/字/双字串传送
	MOVSB	字节串传送
	MOVSW	字串传送
	MOVSD	双字串传送
	LODS	字节/字/双字串装入
	LODSB	字节串装入
	LODSW	字串装入
	LODSD	双字串装入
	STOS	字节/字/双字串存储
	STOSB	字节串存储
	STOSW	字串存储
	STOSD	双字串存储
	CMPS	字节/字/双字串比较
	CMPSB	字节串比较
	CMPSW	字串比较
	CMPSD	双字串比较
	SCAS	字串/字/双字串扫描
	SCASB	字节串扫描
	SCASW	字串扫描
	SCASD	双字串扫描
	INSB	字节串输入
	INSW	字串输入
	INSD	双字串输入
	OUTSB	字节串输出
	OUTSW	字串输出
	OUTSD	双字串输出
	REP	ECX≠0 重复
	REPZ/REPE	相等时重复
	REPNZ/REPEZ	不等时重复

续上表

类别	指令	功能
位操作指令	BT	位测试
	BTS	位测试并置 1
	BTR	位测试并置 0
	BTC	位测试并求补
	BSF	向前位扫描
	BSR	向后位扫描
控制转移指令	JMP	无条件转移
	CALL	调用过程
	RET	从过程返回
	JC	有进位转移
	JNC	无进位转移
	JP/JPE	偶转移
	JNP/JPO	奇转移
	JZ/JE	为零/等于转移
	JNZ/JNE	非零/不等于转移
	JS	负数转移
	JNS	正数转移
	JO	溢出转移
	JNO	无溢出转移
	JA/JNBE	无符号数大于/不小于或等于转移
	JAE/JNB	无符号数大于或等于/不小于转移
	JB/JNAE	无符号数小于/不大于或等于转移
	JBE/JNA	无符号数小于或等于/不大于转移
	JG/JNLE	带符号数大于/不小于或等于转移
	JGE/JNL	带符号数大于或等于/不小于转移
	JL/JNGE	带符号数小于/不大于或等于转移
	JLE/JNG	带符号数小于或等于/不大于转移
	LOOP	ECX≠0 时循环
	LOOPZ/LOOPE	ECX≠0 且 ZF=1 时循环
	LOOPNZ/LOOPNE	ECX≠0 且 ZF=0 时循环
	JCXZ	ECX=0 时循环
	INTn	软件中断
	INTO	溢出中断
	IRET	从中断返回
处理器控制指令	HLT	处理器暂停
	WAIT	处理器等待
	ESC	处理器脱离
	LOCK	总线锁定前缀
	NOP	空操作

类别	指令	功能
标志位操作指令	LAHF	读标志寄存器
	SAHF	写标志寄存器
	PUSHF	标志寄存器低 16 位压栈
	PUSHFD	标志寄存器 32 位全部压栈
	POPF	16 位压栈标志位弹出栈
	POPFD	32 位压栈标志位弹出栈
	CLC	清进位标志
	STC	置进位标志
	CMC	进位标志取反
	CLD	清方向标志
	STD	置方向标志
	CLI	清中断允许标志
	STI	置中断允许标志
高级语言指令	BOUND	数组边界检查
	ENTER	进入过程
	LEAVE	退出过程
系统寄存器存取指令	MOV CR0,OP2	将 OP2 的 32 位操作数装入 CRn (n=0,2,3)
	MOV OP1,CRn	将CR0 内容存储到OP1(n=0,2,3)
	MOV DRn,OP2	装入调试寄存器 DR_0～DR_7
	MOV OP1,DRn	存储调试寄存器 DR0～DR7
	MOV TRn,OP2	装入测试寄存器 TR3～TR7
	MOV OP1,TRn	存储测试寄存器 TR3～TR7
	LMSW OP1	装入机器状态字(CR0 的低 16 位)
	SMSW OP1	存储机器状态字
	LGDT/LIDT OP1	装入全局/中断描述符表寄存器
	SGDT/SIDT OP1	存储全局/中断描述符表寄存器
	LLDT OP1	装局部描述符表寄存器
	SLDT OP1	存局部描述符表寄存器
	LTR OP1	装任务寄存器
	STR OP1	存任务寄存器
保护属性检查指令	CLTS	清除机器状态字中任务转换标志
	LAROP1,OP2	装入访问权限
	LSL OP1,OP2	装入段界限
	VERR OP1	验证段的可读性
	VERW OP1	验证段的可写性
	ARPL OP1,OP2	调整选择符的特权级
Cache 管理指令	INVD	作废片内 Cache
	WBINVD	写回和作废 Cache
	INVLPG	使 TLB 中某一项作废

附录B　数字处理指令一览表

	指令助记符	功　　能
①数据传送指令	FBLD	装压缩BCD数入栈(FPU堆栈)
	FBSTP	存压缩十进制数(BCD)并弹出栈
	FILD	装整数入栈
	FIST	整数存储
	FISTP	整数存储并弹出栈
	FLD	装实数入栈
	FST	存实数
	FSTP	存实数并弹出栈
	FXCH	交换寄存器内容
②算术运算指令	FABS	取绝对值
	FADD/FIADD/FADDP	实数加法/整数加/实数加并弹出
	FCHS	变符号
	FDIV/FIDIV/FDIVP	实数除法/整数除/整数除并弹出
	FDIVR/FIDIVR/FDIVRP	反向实数除法/反向整数除/反向实数除并弹出
	FMUL/FIMUL/FMULP	实数乘法/整数乘/实数乘并弹出
	FPREM	部分余数
	FPREMI	部分余数
	FRNDINT	舍入成整数
	FSCALE	比例
	FSUB/FISUB/FSUBP	实数减法/整数减/实数减并弹出
	FSUBR/FISUBR/FSUBRP	反向实数减法/反向整数减/反向实数减并弹出
	FSQRT	平方根
	FXTRACT	析取指数和有效数
③比较指令	FCOM/FCOMP/FCOMPP	实数比较/比较并弹出/比较并弹出两次
	FICOM/FICOMP	整数比较/比较并弹出
	FTST	测试
	FUCOM/FUCOMP/FUCOMPP	无序实数比较/比较并弹出/比较并弹出两次
	FXAM	检验
④超越函数指令	F2XM	计算 2^x-1
	FCOS	余弦
	FPATAN	局部反正切
	FPTAN	局部正切
	FSIN	正弦
	FSINCOS	正弦/余弦
	FYL2X	计算 $Y*\log_2 X$
	FYL2XP1	计算 $Y*\log_2(X+1)$
⑤装常数指令	FLD1	装1.0
	FLDL2E	装 $\log_2 e$
	FLDL2T	装 $\log_2 10$
	FLDLG2	装 $\log_{10} 2$
	FLDLN2	装 $\ln 2$
	FLDP1	装 π
	FLDZ	装0
⑥控制指令	FCLEX/FNCLEX	清除异常(清所有事故标志、中断请求标志和忙标志)
	FDECSTP	堆栈指针减1
	FFREE	释放寄存器
	FINCSTP	堆栈指针增1
	FINIT/FNINIT	初始化处理器
	FLDCW	装控制字
	FLDENV	装环境
	FNOP	空操作
	FRSTOR	恢复状态
	FSAVE/FNSAVE	存状态
	STCW/FNSTCW	存控制字
	FSTENV/FNSTENV	存环境
	FSTSW/FNSTSW	存状态字
	FSTSW AX/FNSTSW AX	把状态字存入AX
	FWAIT	等待

附录C　ASCⅡ码表

低4位 \ 高3位	000	001	010	011	100	101	110	111
0000	NUL	DLE	SP	0	@	P	`	p
0001	SOH	DC_1	!	1	A	Q	a	q
0010	STX	DC_2	"	2	B	R	b	r
0011	ETX	DC_3	#	3	C	S	c	s
0100	EOT	DC_4	$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	'or'	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	\	l	/
1101	CR	GS	-	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	-	o	DEL

上表中，各控制符号的定义如下表所示：

控制符号的定义

控制符号	定　义	控制符号	定　义
NUL	Null 空白	DLE	Data link escape 转义
SOH	Start of heading 序始	DC_1	Device control 1 机控 1
STX	Start of text 文始	DC_2	Device control 2 机控 2
ETX	End of text 文终	DC_3	Device control 3 机控 3
EOT	End of tape 送毕	DC_4	Device control 4 机控 4
ENQ	Enquiry 询问	NAK	Negative acknowledge 未应答
ACK	Acknowledge 应答	SYN	Synchronize 同步
BEL	Bell 响铃	ETB	End of transmitted block 组终
BS	Backspace 退格	CAN	Cancel 作废
HT	Horizontal tab 横表	EM	End of medium 载终
LF	Line feed 换行	SUB	Substitute 取代
VT	Vertical tab 纵表	ESC	Escape 换码
FF	Form feed 换页	FS	File separator 文件隔离符
CR	Carriage return 回车	GS	Group separator 组隔离符
SO	Shift out 移出	RS	Record separator 记录隔离符
SI	Shift in 移入	US	Unit separator 单元隔离符
SP	Space 空格	DEL	Delete 删除